



HIBERNATE

COURSE MATERIAL



NARESH

TM

technologies

Opp. Satyam Theatre, Ameerpet, Hyderabad - 16
Ph: 040-23746666, 23734842 Cell: 9000994007 / 08

SOFTWARE TRAINING & DEVELOPMENT

B.O. Kaba Plaza, 3rd Floor, LB Road, Adyar, Chennai - 20. Phone : 044-6555 5557, 6555 5558

 info@nareshit.com  www.nareshit.com  [nareshit](https://www.facebook.com/nareshit)  [nareshitech](https://www.twitter.com/nareshitech)

HIBERNATE

By

Mr.Nataraj

HIBERNATE

What is Hibernate?

- Hibernate is the ORM tool given to transfer the data between a java (object) application and a database (Relational) in the form of the objects.
- Hibernate is the open source light weight tool given by Gavin King, actually JBoss server is also created by this person only.
- Hibernate is a non-invasive framework, means it won't force the programmers to extend/implement any class/interface, and in hibernate we have all POJO classes so its light weight.
- Hibernate can run with in or without server, it will be suitable for all types of java applications (stand alone or desktop or any servlets etc....)
- Hibernate is purely for persistence (to store/retrieve data from Database).
- Hibernate goal is to relieve the developer from 95 percent of common data persistence related programming tasks.

⇒ Type--ORM S/W (or) ORM Persistence Tech

⇒ Vendor--SoftTree(RedHat)

⇒ OpenSource S/w

⇒ Creator--Mr.Gavin King

⇒ Versions – 2.X, 3.X, 4.X

What is ORM (Object Relational Mapping)?

ORM stands for Object-Relational Mapping (ORM) is a programming technique for converting data between relational databases and object oriented programming languages such as Java, C# etc.

ORM tools provide a host of services thereby allowing developers to focus on the business logic of the application rather than repetitive CRUD (Create Read Update Delete) logic

ORM s/w's:-

EJB Entity beans-----from sun

Hibernate-----from softtree(RedHat)

JPA---from Apache foundation

JDO---from sun

OJB--from Apache foundation

Toplink--- from Oracle s/w

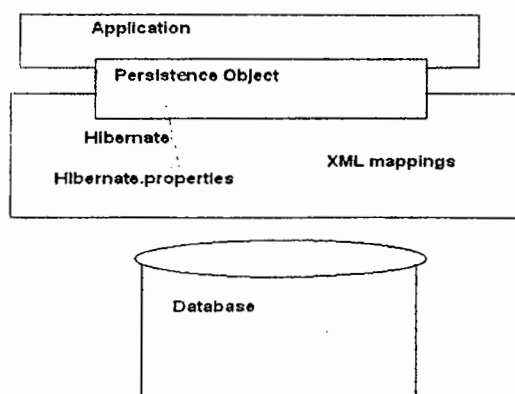
All the ORM S/w internally generates Under Lying DB s/w specific sql queries Dynamically when programmer performs his persistence operations on object's.

Q.) Features of Hibernate?

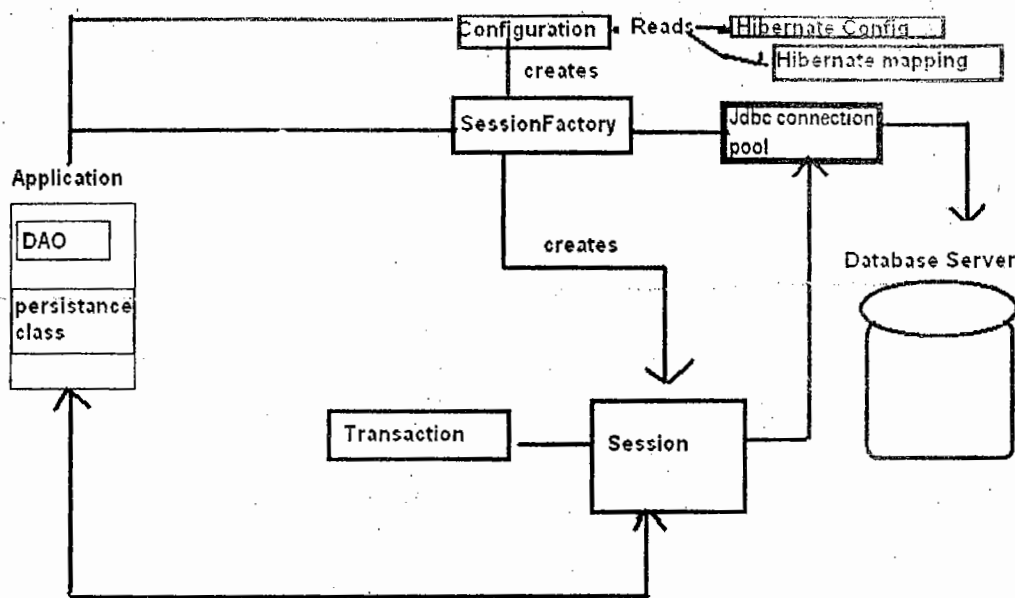
- Hibernate persists java objects into database (Instead of primitives)
- It provides Database services in Database vendor independent Manner, so that java applications become portable across the multiple databases.
- Hibernate generates efficient queries for java application to communicate with Database.
- It provides fine-grained exception handling mechanism. In hibernate we only have Un-checked
- It supports synchronization between in-memory java objects and relational records
- Hibernate provides implicit connection pooling mechanism
- Hibernate supports Inheritance, Associations, Collections
- Hibernate supports a special query language(HQL) which is Database vendor independent
- Hibernate has capability to generate primary keys automatically while we are storing the records into database
- Hibernate addresses the mismatches between java and database
- Hibernate provides automatic change detection Hibernate often reduces the amount of code needed to be written, so it Improves the productivity
- Database objects (tables, views, procedures, cursors, functions ... etc) name changes will not affect hibernate code
- Supports over 30 dialects
- Hibernate provides caching mechanism for efficient data retrieval
- Lazy loading concept is also included in hibernate so you can easily load objects on start up time
- Getting pagination in hibernate is quite simple.
- Hibernate Supports automatic versioning of rows
- Hibernate provides mechanisms for implementing both types of locking(Optimistic and Pessimistic) in the applications.
- Hibernate has data filter mechanism to filter the retrieve data from database
- Hibernate has feature of full text search. This feature is provided from Hibernate 4.

Hibernate Architecture

The Hibernate architecture includes many objects persistent object, session factory, sessions, mapping files, configuration files .



The following diagram shows the high level architecture of Hibernate with mapping file and configuration file.



Hibernate framework uses many objects i.e session factory, session, transaction etc. Along with existing Java API such as JDBC (Java Database Connectivity), JTA (Java Transaction API) and JNDI (Java Naming Directory Interface).

Required Files in Hibernate Application :

- Any hibernate application must contains the files .
 - ⇒ POJO class/Persistence class/Entity class/Domain class
 - ⇒ Mapping File
 - ⇒ Configuration File
 - ⇒ Client App (One java file to write our logic(main class))
- Actually these are the minimum requirement to run any hibernate application.
- Whether the java application will run in the server or without server, and the application may be desktop or stand alone, swing, awt, servlet. Whatever, but the steps are common to all.
- In order to work with hibernate we don't require any server as mandatory but we need hibernate library files (.jar(s) files).
- No Framework is installable Software ,it Means we does n't contain any setup.exe
When we download any framework,we will get a zip file and we need to unzip it,to get the required jar files,actually all frameworks will follow same common principles like ...
Framework will be in the form of a set of jar files. Every Framework s/w contains two types jar files

- 1) Main jar files
- 2) Dependent jar files

Each Framework contain one configuration file ,but multiple configuration files also allowed.

We can download hibernate jar files from the following links.Based on requirement we can download the corresponding version

For version 3.x <http://sourceforge.net/projects/hibernate/files/hibernate3/>

For Version 4.x <http://sourceforge.net/projects/hibernate/files/hibernate4/>

Note : Along with Hibernate jar's we must include one more jar file,which is nothingbut related to our database,this is depending on your database.For example we are working with Oracle we need to ojdbc6.jar.

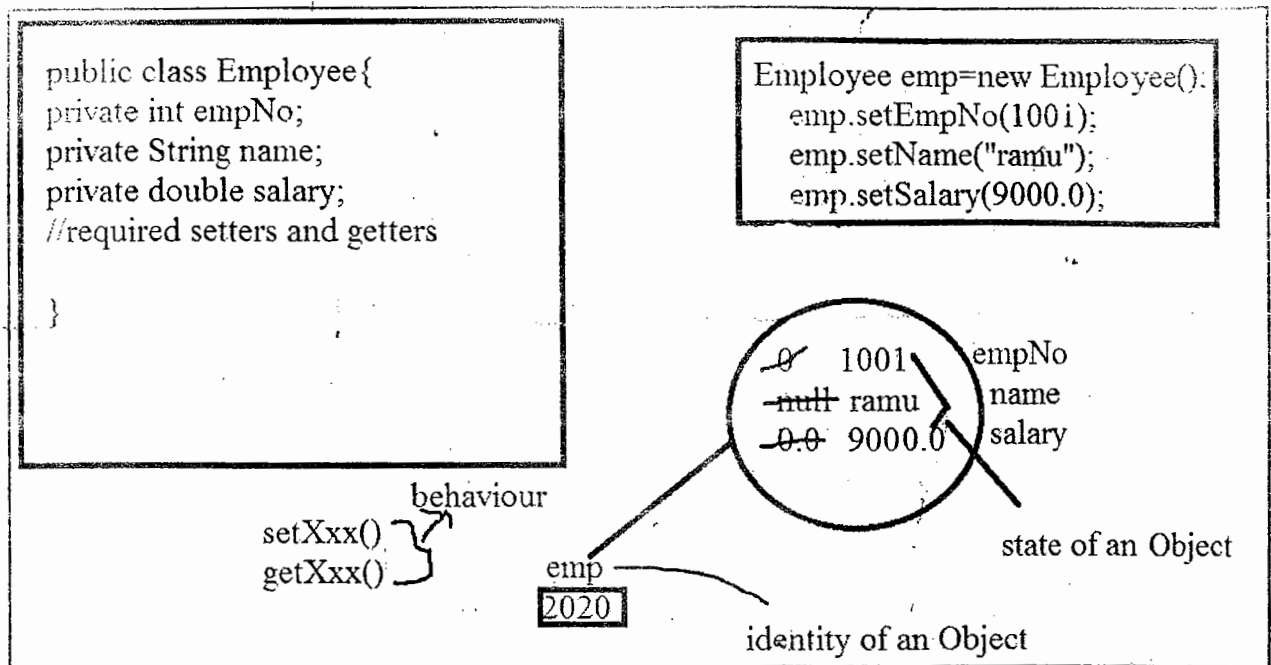
Persistence class/Entity class/pojo class

- It should use java bean mapped with Database table Client App uses this class Object to develop (OR) mapping persistence logic.
- This class Object represents database table record having synchronization b/w them.

Mapping file :

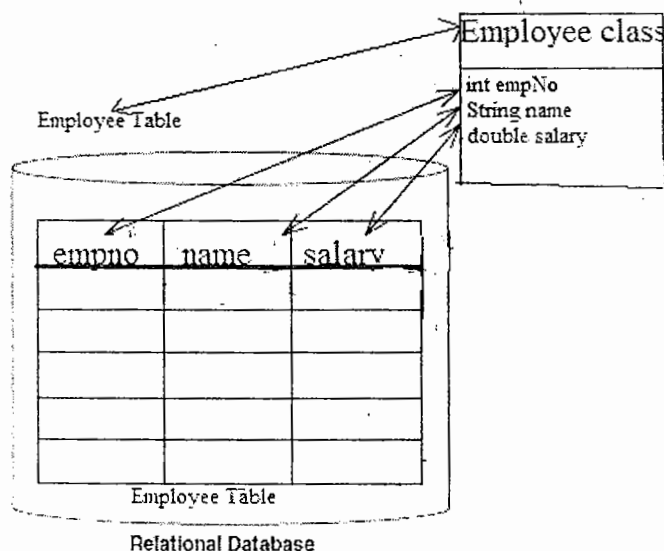
- In this file,hibernate application developer specify the mapping from entity class name to table name and entity properties names to table column names. i.e. mapping of an object oriented data to relational data is done in this file.
- Standard name for this file is<domain-object-name.hbm.xml>
- Mapping can be done using annotations also. If we use annotations for mapping then we no need to write mapping file.
- From hibernate 3.x version on wards it provides support for annotations,So mapping can be done in two ways
 - ❖ XML
 - ❖ Annotations
- Mapping is a meta data ,but not the data. Through mapping meta data hibernate stores (OR) reads the data from database.
- Every ORM tool needs mapping, mapping is the mechanism of placing an object properties(state) into columns of a table.
- Generally an object contains 3 properties like Identity (Object Name) State (Object values) Behavior (Object Methods).
- In ORM, Storing of an Object is nothing but storing the state of an Object, but not the Identity and behavior

For Example:-



An ORM tool cannot understand what class Object is needed to stored in what table. To give this information programmer must create/construct mapping.

Mapping between Employee class to Employee Table will be like



While constructing a mapping file in hibernate , It is possible to write multiple java classes mapping in a single mapping file.

Syntax Of Mapping xml:

```
<!DOCTYPE ..... >
<hibernate-mapping>
  <class name="Fully qualified name of the class" table="database table Name">
    <id name="variable name" column="column name" type="java/hibernate type" />
    <property name="variable1 name" column="column name" type="java/hibernate type" />
```

```
<property name="variable2 name" column="column name" type="java/hibernate type" />
<class>
</hibernate-mapping>
```

- Each hibernate mapping file must contain one <id> tag. java object identified uniquely by the <id> tag property.
- <id> tag property corresponding column can be primary key(or)non-primary key in the DataBase

Note:-

- In Mapping file class names and Property names are Case-sensitive .
- But Table Names and Column Names are not case sensitive. When the Property name and column name are same we no need to give Column attribute.
- When the Persistence class name and table name are same we no need to give table attribute.
- In the Mapping file it is not required to map all the properties of the entity (pojo class) and all the columns of table.
- As for our requirement we can configure required properties of the entity with required columns of the table.

Configuration file:

- Configuration is the file loaded into hibernate application when working with hibernate, this configuration file contains 3 types of information.
 - ⇒ Connection Properties
 - ⇒ Hibernate Properties
 - ⇒ Mapping file name(s)
- We can write this configuration in 2 ways...
- Xml and annotations(from hibernate 3)

Syntax Of Configuration xml:

```
<hibernate-configuration>
<session-factory>
<!-- Related to the connection START -->
<property name="connection.driver_class">Driver Class Name </property>
<property name="connection.url">URL </property>
<property name="connection.username">user </property>
<property name="connection.password">password</property>
<!-- Related to the connection END -->
<!-- Related to hibernate properties START -->
<property name="show_sql">true/false</property>
<property name="dialect">Database dialect class</property>
<property name="hbm2ddl.auto">create/update/create-drop</property>
<!-- Related to hibernate properties END-->
<!-- Related to mapping START-->
<mapping resource="hbm file 1 name .xml" / >
<mapping resource="hbm file 2 name .xml" / >
<!-- Related to the mapping END -->
</session-factory>
</hibernate-configuration>
```

Dialect in Hibernate:-

- It is a simple java class, which contains mapping b/w java language data type and database data type.
- This class contains queries format for predefined hibernate methods.
- Hibernate generates queries for the specific database based on the Dialect class. If you want to shift from one data base to another just change the Dialect class name and connection Details in hibernate.cfg.xml file
- All Dialect classes must extend Dialect(Abstract class)
- Hibernate supports almost 30 dialect classes.
- All the Dialect classes are present in org.hibernate.dialect package

- We can write our own Dialect by extending Dialect class.
- Dialect class is used to convert HQL queries into Database specific queries

Note :-

By using Hibernate to perform the basic Persistent operations in the client application we need to implement some basic steps.

The basic steps involved in client Application are

Step 1: *Prepare Configuration Object*

Step 2 : *Build SessionFactory object*

Step 3: *Obtain a Session*

Step 4: *Perform the Persistence operations*

Step 5: *close the Session*

Configuration:**Configuration Object**

- The org.hibernate.cfg.Configuration is a class and is the basic element of the Hibernate Api.
- An Object Oriented Representation of hibernate configuration file along with mapping file is known as Configuration object.
- By default Hibernate reads configuration file with the name "hibernate.cfg.xml" which is located in classes folder.
- If we want to change the file name(OR) if we want change the location of

Ex:-

```
Configuration cfg =new Configuration();  
cfg.configure("/com/nareshit/xml/hibernate.cfg.xml");
```

"hibernate.cfg.xml" then we need to pass user given configuration file name(along with path) to "configure()" method of Configuration class.

The Configuration class configure() method described in the below table

<code>configure()</code>	Use the mappings and properties specified in an application resource named <code>hibernate.cfg.xml</code> .
<code>configure(Document document)</code>	Use the mappings and properties specified in the given XML document.
<code>configure(File configFile)</code>	Use the mappings and properties specified in the given application file.
<code>configure(String resource)</code>	Use the mappings and properties specified in the given application resource.
<code>configure(URL url)</code>	Use the mappings and properties specified in the given document.

All the `configure()` methods described in the above table returns Configuration object.

Configuration object stores the Configuration file data in different variables. Finally all these variables are grouped and created one high level hibernate object called as SessionFactory object.

So configuration object only meant for creating SessionFactory object.

We can also provide the configuration information programmatically, without writing configuration file. (But it will become HardCoding, So not advisable)

Configuration in a programmatic manner

```
Configuration configuration = new Configuration();
```

```
//Adding the Connection details to Configuration object in Programmatically
```

```
configuration.setProperty("connection.driver_class", "oracle.jdbc.driver.OracleDriver");
```

```
configuration.setProperty("hibernate.connection.url", "jdbc:oracle:thin:@localhost:1521:XE");
```

```
configuration.setProperty("hibernate.connection.username", "system");
```

```
configuration.setProperty("hibernate.connection.password", "manager");
```

```
//Adding the hibernate properties to Configuration object in Programmatically
```

```
configuration.setProperty("hibernate.dialect", "org.hibernate.dialect.Oracle9Dialect")
```

```
//Adding the mapping files to configuration object programmatically
```

```
configuration.addResource("Employee.hbm.xml");
```

```
configuration.addResource("PersonalDetails.hbm.xml");
```

Note : *The Configuration object we can create only once for an application, at startup-time, that is while initializing the application.*

After successfully preparing the configuration object by setting all the configuration parameters and mapping documents location, the configuration object is used to create the SessionFactory object. to do this we can use the buildSessionFactory() method of configuration.

After creating the SessionFactory the configuration object can be discarded.

SessionFactory :

SessionFactory is an interface, present in "org.hibernate" package.

SessionFactoryImpl is an implemented class of SessionFactory interface

SessionFactory is a heavy weight object that has to be created only once per application.

SessionFactory is not a singleton.

SessionFactory object provides lightweight session object's.

Session Factory object is the factory for session objects.

Generally one sessionfactory should be created for one database.

When you have multiple databases in your application you should create multiple SessionFactory objects.

If you are using two databases called mysql and oracle in your hibernate application then you need to build 2 SessionFactory object's. Sessionfactory is long live multithreaded object.

SessionFactory is a ThreadSafe object.

SessionFactory is an ImmutableObject.

To build the SessionFactory object we can call buildSessionFactory() method from Configuration class

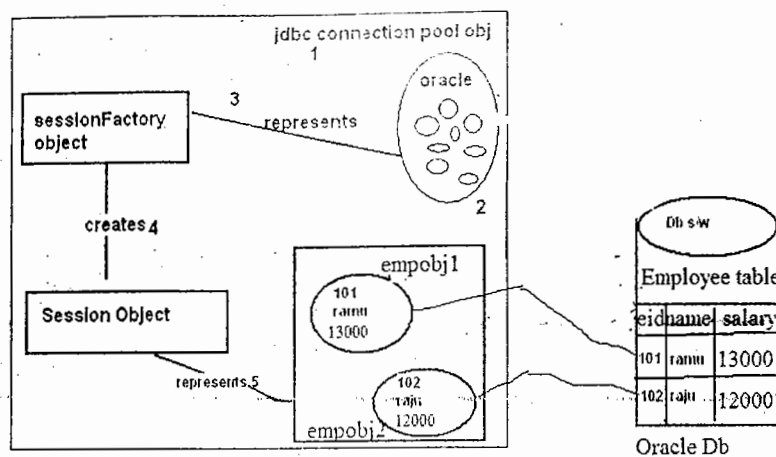
```
public SessionFactory buildSessionFactory()
```

Ex:-

```
Configuration cfg=new Configuration();
```

```
cfg.configure("/hibernate.cfg.xml");
```

```
SessionFactory sessionFactory =cfg.buildSessionFactory();
```



Hibernate software creates Built in Jdbc connection pool having the set of Jdbc connection object's. This connection pool created based on the details given in the Configuration File and will be represented with Hibernate SessionFactory object.

Hibernate Session Object represents Persistence context

Check next chapter for more details about connection pooling

Why Session factory is a heavy weight?

Session factory encapsulates session object, connections, Hibernate properties caching and mappings.

How To Create Session Factory in Hibernate 4

In our earlier versions we have created the session factory as below: -

```
SessionFactory sessionFactory=cfg.buildSessionFactory();
```

since Hibernate 4.x, this approach is deprecated. According to Hibernate 4.0 API docs, the Configuration class' buildSessionFactory() method is deprecated and it recommends developers to use the buildSessionFactory(ServiceRegistry) instead.

Here is the new recommended code that is used to builds the SessionFactory based on a ServiceRegistry and obtains the Session:

```
Configuration configuration = new Configuration().configure();
```

```
ServiceRegistryBuilder registry = new ServiceRegistryBuilder();
```

```
registry.applySettings(configuration.getProperties());
```

```
ServiceRegistry serviceRegistry = registry.buildServiceRegistry();
```

```
SessionFactory sessionFactory = configuration.buildSessionFactory(serviceRegistry);
```

```
Session session = sessionFactory.openSession();
```

In hibernate 4.3 ServiceRegistryBuilder is deprecated so in the new Version's it is recommended to StandardServiceRegistryBuilder instead of ServiceRegitryBuilder.

Example code :

```
Configuration configuration = new Configuration().configure("/hibernate.cfg.xml");
ServiceRegistry serviceRegistry = new StandardServiceRegistryBuilder();
    serviceRegistry .applySettings(configuration.getProperties()).build();

    // builds a session factory from the service registry
```

```
SessionFactory sessionFactory = configuration.buildSessionFactory(serviceRegistry);
```

Session :-

Session is an interface present in org.hibernate package and SessionImpl is an implemented class.

Session object is called persistent manager for the hibernate application.

It is a single-threaded (not-thread safe), short-lived object

It encapsulates the functionality of JDBC connection, statement obj

It has convenience methods to perform persistent operations.

When a session is opened then internally a connection with the database will be established.

It is a factory for Transaction objects.

Session Holds a mandatory (first-level) cache of persistent objects

Note: After we complete the use of the Sesson, It has to be closed, to release all the resources such as associated objects and jdbc connection.

The SessionFactory object is used to open a Hibernate Session .To do this we can use any one of the openSession() methods of SessionFactory.

Methods of SessionFactory interface to open a new Session :

public Session openSession()	This method Prepares a new session that obtains a jdbc connection object by using the connection Manager
public Session openSession(Connection con)	This method prepares a new session that uses the given Connection
public Session openSession(Interceptor icr)	This method prepares a new session that obtains a Jdbc connection object by using the connection manager.The session is registered with the given Interceptor

```
public Session openSession(Connection  
con,Interceptor icr)
```

This method prepares a new session that uses the given Jdbc connection object. The session is registered with the given Interceptor

The following code shows for creating a new session :

```
Configuration cfg=new Configuration();  
cfg.configure("/hibernate.cfg.xml");  
SessionFactory sessionFactory=cfg.buildSessionFactory();  
Session session=sessionFactory.openSession();
```

The Session interface has various convenience methods to perform the persistence operations.

After obtaining the session we can use the session methods to perform CRUD(Create,Read,Update,Delete) operations on the instances of mapped persisten classes.

```
//get SessionFactory object reference  
  
// After getting the sessionFactory object we can obtain the session from  
sessionFactory  
  
Session session=sessionFactory.openSession();  
  
//Use the session for performing persistence operations  
  
//After the use of session,close the session  
  
session.close();
```

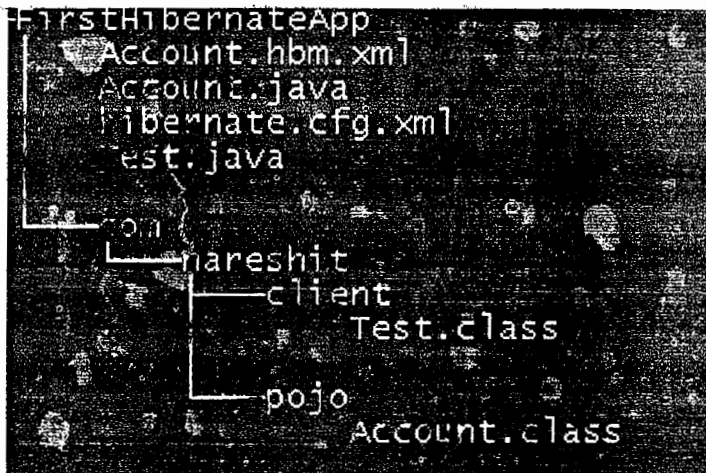
After we complete the use of session, it has to be closed to release all the resources such as cached entity object's ,collections,proxies,and a jdbc connection.

But it is recomnded to close the session explicitly by calling close().

Q. Develop Hibernate Application,In which we can perform account retrieve operation from Account table ?

Account Table

accountNumber	name	balance
1001	ramu	9000.0
1002	raju	5000.0

Account.java

```

package com.nareshit.pojo;
public class Account{
    private int accNum;
    private String name;
    private double balance;
    public void setAccNum(int accNum){
        this.accNum=accNum;
    }
    public void setName(String name){
        this.name=name;
    }
    public void setBalance(double balance){
        this.balance=balance;
    }
    public int getAccNum(){
        return accNum;
    }
    public String getName(){
        return name;
    }
    public double getBalance(){
        return balance;
    }
}
  
```

The Account Persistence class shown in the preceding code declares three persistence fields accNum,name,balance of types int,String,double respectively.

As discussed earlier ,Hibernate requires a small amount of meta data description for persistent class , which is secified using the Account.hbm.xml file .

Account.hbm.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
<class name="com.nareshit.pojo.Account" table="account1">
<id name="accNum">
<column name="accountNumber"/>
</id>
<property name="name">
<column name="name" />
</property>
    <property name="balance" column="balance"/>
</class>
</hibernate-mapping>
```

The Account.hbm.xml file describes the mappings for Account persistence class. As discussed earlier, It is recommended to save the mapping file name <persistent class name>.hbm.xml.However, it is not mandatory to follow this convention. We can use any filename.

Note :-

- Xml is a case –sensitive mark-up language.
- Every Xml file contains the Root tag. And whatever tag we are opened in the xml compulsory we can close.
- Xml file Contains Either Dtd(Document Type Definition) (OR) XSD(Xml Schema Definition)
- DTD and XSD are techniques used to frame the rules that are required to construct an xml document

Now ,let us configure the Hibernate configuration file by specifying the configuration parameters such as connection details,hibernate properties and mapping information.

hibernate.cfg.xml

```
<?xml version='1.0' encoding='UTF-8'?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
```

```
<session-factory>
<property name="connection.url">jdbc:oracle:thin:@localhost:1521:XE
</property>
<property name="connection.username">system</property>
<property name="connection.password">manager</property>
<property name="connection.driver_class">oracle.jdbc.driver.OracleDriver
</property>
<property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
<property name="show_sql">true</property>
<property name="hbm2ddl.auto">update</property>
<mapping resource="Account.hbm.xml" />
</session-factory>
</hibernate-configuration>
```

The hibernate.cfg.xml configuration file is using oracle Jdbc thin driver to connect to database for persistence operation. Here we have configured Hibernate properties dialect, show_sql and hbm2ddl.auto properties.

- **show-sql:** if the 'show_sql' value is true we can view all the hibernate generated sql queries in the console.
- **hbm2ddl.auto:** It has two values
 - a. create or create-drop
 - b. update

a. create: If its value is create while running the application

Case 1: table does not exist

Hibernate creates a new table based on the mapping file configurations

Case 2: table exists

Hibernate Drops the existing table and creates a new table based on the mapping file configurations

b. Update: If its value is update while running the application

Case 1 : table does n't exist

Create a new table based on the mapping file configurations

Case 2 : table exists

Hibernate Uses the existing table

Test.java

```
package com.nareshit.client;
import com.nareshit.pojo.Account;
import org.hibernate.cfg.Configuration;
import org.hibernate.SessionFactory;
import org.hibernate.Session;
public class Test{
    public static void main(String[] args) {
        Configuration cfg=new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory factory=cfg.buildSessionFactory();
        Session session=factory.openSession();
        Account acc=(Account)session.get("com.nareshit.pojo.Account",1001);
        if(acc!=null){
            System.out.println("Account Number :"+acc.getAccNum());
            System.out.println("Holder's Name:"+acc.getName());
            System.out.println("Balance : "+acc.getBalance());
        }
        else{
            System.out.println("Account Not Found");
        }
        session.close();
    }
}
```

To execute the above application we need to set the following jar files in the classpath:

```
hibernate3.jar
antlr-2.7.6.jar
asm.jar
cglib-2.1.3.jar
commons-collections-2.1.1.jar
commons-logging-1.0.4.jar
ehcache-1.1.jar
dom4j-1.6.1.jar
jta.jar
log4j-1.2.9.jar
```

Remember: Along with the hibernate jars we must include one more jar file, which is nothing but related to our database, this is depending on your database.

ojdbc14.jar/classes12.jar file

load operation/select operation/read operation :-

load/select/read operation is nothing but reading an object from a database into a java application.

In order to perform a load operation/select operation a java programmer has to call any one of the following method from session interface

1. public Object get(String className, Serializable id)
2. public Object get(Class clz, Serializable id)
3. public Object load(String className, Serializable id)
4. public Object load(Class clz, Serializable id)

While loading, Hibernate creates internally an Object of a pojo class for storing the selected data from a database. For creating an Object of pojo class, hibernate internally calls `newInstance()` of `java.lang.Class`.

When `session.get(className, id)` method is called for loading an Object from database. Hibernate first verifies whether the given id record is exist in the database (OR) not. If exist then immediately reads it from the database and creates persistence Object internally and finally returns the persistence class Object back to java app.

If the given id record does not exist in the database then `get(-,-)` returns null value back to the java app.

In the case of `load(-,-)` method, first hibernate creates a pojo class Object with empty state(data) and returns that Object back to a java program. here the Object returned by hibernate is not a real Object. It means the Object does not contain the data of a database. And it is called as a proxy Object

When we try to access the non-identifier properties from proxy object, at that time Hibernate will hit the database and reads the data from database and put's it into the pojo class Object this mechanism is called lazy-loading

If the given id does not exist in the database then hibernate throws `org.hibernate.ObjectNotFoundException`

What is the difference between `load()` and `get()` methods?

--> If the given id does not exist in the database then load method throws an Exception but `get()` method returns null.

--> `load()` loads the data from database on demand. It is supporting lazy loading.

But `get()` immediately loads the data from database so it is early loading.

Example to call the `get(-,-)` method.:

```
Session session=sessionFactory.openSession();
Account account=(Account)session.get(Account.class,1001); -
//At this line put a break point.... Now observe the console ,After this line executed,
//We can find select statement .
//And the account object initialized with database data
System.out.println(account.getName());
```

Example to call the load (-,-) method:

```
Session session=sessionFactory.openSession();
try{
Account account=(Account)session.load(Account.class,1001);
//At this line put a break point.... Now observe the console ,After this line executed, //We
can't find select statement .
//And the account object not initialized with the database data And the Account is proxy
object
System.out.println(account.getName());
//After this line you can find select query on the console ,And now account object is
//initialized with database data
}catch(ObjectNotFoundException e){
e.printStackTrace();
}
```

When we use session.get(-,-) method if object is present we have to implement some logic, if not we need to implement some other logic.

Example :

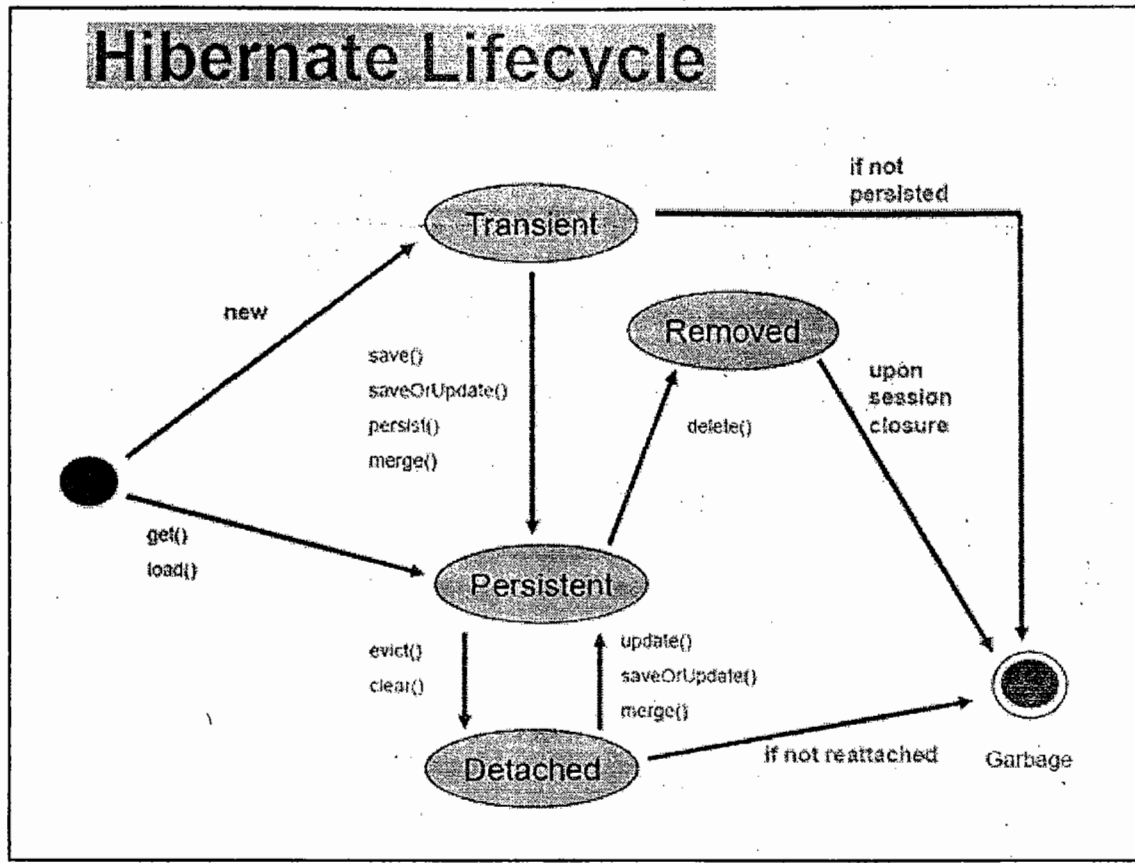
```
Session session=sessionFactory.openSession();
Account account=(Account)session.get(Account.class,1001);
if(account!=null){
System.out.println(account.getName());
System.out.println(account.getBalance());
}else{
System.out.println("Account Not Found ");
}
```

But in the case of load(-,-) method if the given id record is not exist load(-,-) method will rise org.hibernate.ObjectNotFoundException. So we cannot implement logic like above code .

Persistent Object Life Cycle

Persistent Object has three life cycle states

1. Transient state
2. Persistent state
3. Detached state



Transient state: An object is said to be in transient state, when it is not associated with session and that object data not present in data base.

Example: Table ACCOUNT

ACCNO	NAME	BALANCE
1001	Pavan	15000.0
1002	Ramu	20000.0

Application code:

```

Account account = new Account();
account.setAccNum(1003);
account.setName("raju");
account.setBalance(21000);

```

In the above example account object is not associated with session and there is no matching record in with name as "raju" the ACCOUNT table. So we can say that **account object is in transient state**.

In this state, object is non-transactional. i.e. object is not synchronized with record. i.e. Modifications done to entity, doesn't save into database.

Persistent state: An object is said to be in persistence state, when it is associated with session as well as object present in database.

Example: Table ACCOUNT

ACCNO	NAME	BALANCE
1001	Pavan	15000.0
1002	Ramu	20000.0

Application Code:

```
Account account = (Account) session.get(Account.class,1002);
```

In the above example account object is associated with session and there is a matching record in ACCOUNT table. So we can say that account object is in persistent state.

In this state object is transactional. i.e. the object is synchronized with database record.

Changes made to objects in a persistent state are automatically saved to the database without invoking session persistence methods.

Detached State: An object is said to be in detached state, when the object is not associated with session but present in database.

Example: Table Account

ACCNO	NAME	BALANCE
1001	Pavan	15000.0
1002	Ramu	20000.0
1003	Raju	210000.0

Application code :

```
Account account = new Account();
account.setAccNum(1003);
account.setName("Raju");
account.setBalance(21000);
//Now the account object is said to be in transient state
```

```

session.save(account);
session.beginTransaction().commit();
// Now the account object is said to be in persistence state
session.close(); // Now the account object is said to be in detached state.

```

In the above example after calling `session.close()` method, account object is moved to Detached state from persistent state. As session is garbage collected, if we try to perform some modifications to entity object, those changes will not be stored into database.

Session interface methods

save(-) :-

public Serializable save(Object obj) throws HibernateException

save method stores an object into the database. That means it insert an object state into the database. the `save()` method returns the serializable identifier back to java application.

Example Code :

```

public void createAccount(){
    Session session = sessionFactory.openSession();
    Transaction tx=session.beginTransaction();
    Account account = new Account();
    account.setAccNum(1002);
    account.setName("raja");
    account.setBalance(9000.0);
    Integer id=(Integer) session.save (account);
    tx.commit();
    System.out.println("Account object is created with id :"+id);
    //The record with 1002 will be inserted into the database by generating the insert query
}

```

What is Transaction ?

- ❖ Transaction used by the application to specify atomic units of work (Transaction management).
- ❖ Using Session Object we can create Transaction object in two ways.
 - Transaction transaction =session.getTransaction();
 - Transaction transaction = session.beginTransaction();
- ❖ Transaction object is unique per session object.
- ❖ Transaction interface defines following methods to deal with transactions.
 - transaction.begin() {Transaction beginning}
 - transaction.commit(); { successful transaction ending }
 - transactin.rollback(); {un successful transaction ending}
- ❖ Default auto commit value is false in Hibernate.
- ❖ Default auto commit value is true in JDBC

- ❖ In hibernate even to execute one DML operation, we need to implement Transactions.
- ❖ Hibernate supports
 - JDBC Transaction.
 - JTA Transaction.
 - Spring Transaction

Sample Transaction Code is as Follows ...

```
Session session= sessionFactory.openSession();
Transaction tx = null;
try {
    tx = session.beginTransaction();
    // DML operations
    tx.commit();
} catch (Exception e) {
    tx.rollback();
}
finally{
    session.close();
}
}
```

saveOrUpdate(-) method:-

public void saveOrUpdate(Object obj) throws HibernateException

- saveOrUpdate(Object obj) method can be used to either INSERT or UPDATE based upon existence of record. Clearly saveOrUpdate(-) is more flexible in terms of use but it involves an extra processing to find out whether record already exists in table or not.
- saveOrUpdate() method either INSERT or UPDATE based upon existence of object in database. If persistence object already exists in database then UPDATE SQL will execute and if there is no corresponding object in database then INSERT will run.

Account Table :

AccNum	Name	Balance

Application Code:-**Example 1**

```
Session session = sessionFactory.openSession();
Transaction tx=session.beginTransaction();
Account account = new Account();
account.setAccNum(1001);
account.setName("raja");
account.setBalance(6800.00);
session.saveOrUpdate(account);
tx.commit();
//Now saveOrUpdate() performs the insertion operation
```

Because record with 1001 id is not available in the database.

After executing the above program table will be updated as follows

Account Table :

AccNum	Name	Balance
1001	raja	6800.0

Example 2

```
Session session = sessionFactory.openSession();
Transaction tx=session.beginTransaction();
Account account = new Account();
account.setAccNum(1001);
account.setName("Raja.Rao");
account.setBalance(8800.00);
session.saveOrUpdate(account);
session.getTransaction().commit();
//Now saveOrUpdate() performs the update operation
```

Because record with 1001 id available in the database.

After executing the above program table will be updated as follows

AccNum	Name	Balance
1001	Rajarao	8800.0

update(Object obj)

public void update(Object obj)throws HibernateException

update method is used for updating the object using identifier. If the identifier is missing or doesn't exist, it will throw exception.

Account Table :

AccNum	Name	Balance
1001	raja	5000

Application code:-

```
Session session = sessionFactory.openSession();
Transaction tx=session.beginTransaction();
Account account = new Account();
account.setAccNum(1001);
account.setName("rajarao");
account.setBalance(6800.00);
session.update(account);
tx.commit();
```

//Now update() performs the update

record with 1001 id available in the database.

After executing the above program table will be updated as follows

AccNum	Name	Balance
1001	Rajaroo	6800.0

SessionInterfaceMethodsDemo

src

- com.nareshit.client
 - Test.java
- com.nareshit.config
 - hibernate.cfg.xml
- com.nareshit.dao
 - AccountDao.java
 - AccountDaoImpl.java
- com.nareshit.mapping
 - Account.hbm.xml
- com.nareshit.pojo
 - Account.java
- com.nareshit.utility
 - HibernateUtility.java

Account.java

```
package com.nareshit.pojo;
public class Account {
    private int accNum;
    private String name;
    private double balance;
    public String toString() {
        return "\naccNumber :"+accNum+"\nname :"+name+"\nbalance :"+balance;
    }
    public int getAccNum() {
        return accNum;
    }
    public void setAccNum(int accNum) {
        this.accNum = accNum;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public double getBalance() {
        return balance;
    }
    public void setBalance(double balance) {
        this.balance = balance;
    }
}
```

Account.hbm.xml

```
<!DOCTYPE hibernate-mapping PUBLIC
"-//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
    <class name="com.nareshit.pojo.Account" table="AccountDetails">
        <id name="accNum" length="12">
            <generator class="increment" />
        </id>
        <property name="name" length="15" />
        <property name="balance" column="bal" length="10" />
    </class>
</hibernate-mapping>
```

hibernate.cfg.xml

```
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver
        </property>
        <property name="connection.url">jdbc:oracle:thin:@localhost:1521:XE
        </property>
        <property name="connection.username">system</property>
        <property name="connection.password">manager</property>
        <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
        <property name="show_sql">>true</property>
        <property name="hbm2ddl.auto">update</property>
        <mapping resource="com/nareshit/mapping/Account.hbm.xml" />
    </session-factory>
</hibernate-configuration>
```

AccountDao.java

```
package com.nareshit.dao;
import com.nareshit.pojo.Account;
public interface AccountDao {
    public int createAccount(Account acc);
    public void updateAccountHolderName(int accNum,String newName);
    public void deleteAccount(int accNum);
    public Account searchAccount(int accNum);
}
```

AccountDaoImpl.java

```
package com.nareshit.dao;
import org.hibernate.Transaction;
import org.hibernate.Session;
import com.nareshit.pojo.Account;
import com.nareshit.utility.HibernateUtility;
public class AccountDaoImpl implements AccountDao{
    public int createAccount(Account acc){
        int id=0;
        Session session=HibernateUtility.getSession();
        if(session!=null){
            Transaction tx=session.beginTransaction();
            id=(Integer)session.save(acc);
        }
    }
}
```

```
        tx.commit();
        session.close();
    } //end of if block
    return id;
} //end of createAccount(-) method
public void updateAccountHolderName(int accNum, String newName) {
    Session session = HibernateUtility.getSession();
    if (session != null) {
        Account acc = (Account) session.get(Account.class, accNum);
        if (acc != null) {
            acc.setName(newName);
            //update the record
            session.update(acc);
            session.beginTransaction().commit();
            System.out.println(accNum + " : Record Updated Successfully");
            session.close();
        }
        else {
            System.out.println("Account Not Found ");
        }
    } //end of outer if-block
} //end of updateAccountHolderName(-) method
public void deleteAccount(int accNum) {
    Session session = HibernateUtility.getSession();
    if (session != null) {
        Account acc = (Account) session.get(Account.class, accNum);
        if (acc != null) {
            //delete the record
            session.delete(acc);
            session.beginTransaction().commit();
            System.out.println(accNum + " : Record deleted Successfully");
            session.close();
        }
        else {
            System.out.println("Account Not Found ");
        }
    } //end of Outer if-block
} //end of deleteAccount(-) method
public Account searchAccount(int accNum) {
    Account acc = null;
    Session session = HibernateUtility.getSession();
    if (session != null) {
        acc = (Account) session.get(Account.class, accNum);
    }
    return acc;
} //end of searchAccount(-) method
}
```

Test.java

```

package com.nareshit.client;
import com.nareshit.dao.AccountDao;
import com.nareshit.dao.AccountDaoImpl;
import com.nareshit.pojo.Account;
public class Test {
    public static void main(String[] args){
        AccountDao accDao=new AccountDaoImpl();
        System.out.println("Testing of createAccount(-) :");
        //create the account obj
        Account acc=new Account();
        acc.setName("raja hib");
        acc.setBalance(99999999);
        int id=accDao.createAccount(acc);
        if(id>0){
            System.out.println("User Account created With Accnum :"+id);
        }
        else{
            System.out.println("User Account Not created ");
        }
        System.out.println("=====");
        System.out.println("Testing of updateAccountHolderName(1002,raja) :");
        accDao.updateAccountHolderName(1002,"raja");
        System.out.println("=====");
        System.out.println("Testing of deleteAccount(1003) ");
        accDao.deleteAccount(1003);
        System.out.println("=====");
        System.out.println("Testing of searchAccount(1001) ");
        Account account=accDao.searchAccount(1001);
        if(account!=null){
            System.out.println(account);
        }
        else{
            System.out.println("Account Not Found ");
        }
    }
}

```

merge() method :

Suppose we create a session and load an object. Now object is in session cache. If we close the session at this point and we edit state of object and tried to save using update() it will throw exception. To make object persistent we need to open another session. Now we load same object again in current session. So if we want to update present object with previous object changes we have to use merge() method. Merge method will merge changes of both states of object and will save in database.

Let us take an example

```
-----  
-----  
  
SessionFactory factory = cfg.buildSessionFactory();  
  
Session session1 = factory.openSession();  
  
Student s1 = null;  
  
Object o = session1.get(Student.class, new Integer(101));  
  
s1 = (Student)o;  
  
session1.close();  
  
s1.setMarks(97);  
  
Session session2 = factory.openSession();  
  
Student s2 = null;  
  
Object o1 = session2.get(Student.class, new Integer(101));  
  
s2 = (Student)o1;  
  
Transaction tx=session2.beginTransaction();  
  
session2.merge(s1);
```

From the above example,

- We just loaded one object s1 into session1 cache and closed session1(session1.close()), so now object s1 in the session1 cache will be destroyed as session1 cache will expire when ever we say session1.close()
- Now s1 object will be in some RAM location, not in the session1 cache
- Here s1 is in detached state, and at line (s1.setMarks(97)) we modified that detached object s1, now if we call update() method then hibernate will throw an error, because we update the object in the session only
- So we opened another session [session2], again loaded the same student object from the database, but with name s2
- So in this session2, we called session2.merge(s1); now into s2 object s1 changes will be merged and saved into the database

persist() method :

persist method is similar to save

- First difference between save and persist is there return type. Similar to save method persist also INSERT records into database but return type of persist is void while return type of save is Serializable object.
- Another difference between persist and save is that both methods make a transient instance persistent. However, persist() method doesn't guarantee that the identifier value will be assigned to the persistent instance immediately, the assignment might happen at flush time.
- One more thing which differentiate persist and save method in Hibernate is that is there behavior on outside of transaction boundaries. persist() method guarantees that it will not execute an INSERT statement if it is called outside of transaction boundaries. save() method does not guarantee the same, it returns an identifier, and if an INSERT has to be executed to get the identifier (e.g. "identity" generator), this INSERT happens immediately, no matter if you are inside or outside of a transaction.

Providing an ID for Persistence

An id element (<id>) is used to create a mapping in the Hibernate XML file. The id element has attributes such as column, type, and generator that you use to generate the identifier. The JPA specification requires that every entity must have a primary key. From JPA's perspective, an @id is used to define how an identifier is generated.

When you use inheritance mapping (will be discussed in coming sections), more than one class can be mapped to a table. These classes (subclass and superclass) are said to be in an entity hierarchy. The primary key must be defined exactly once in an entity hierarchy, either on the entity that is the root of the entity hierarchy or on a mapped superclass of the entity hierarchy. Attributes such as @GeneratedValue and @column are used to define column mapping and strategy.

Generator's

Generator classes are used to generate the 'identifier' for a persistent object. i.e. While saving an object into the database, the generator informs to the hibernate that, how the primary key value for the new record is going to generate.

Hibernate using different primary key generator algorithms, for each algorithm internally a java class is there for its implementation.

All Generator classes has to implement 'org.hibernate.id.IdentifierGenerator' interface, And has to override generate (-,-) method. The logic to generate 'identifier' has to write in this method.

In built-in generator classes, identifier generation logic has implemented by using JDBC.

If we want to write the user defined generator class and it should implement 'org.hibernate.id.IdentifierGenerator' interface and has to override generate(-,-) method.

To configure generator class we can use <generator /> tag, which is sub element of <id/> tag <generator/> tag has one attribute called "class" with which we can be used to specify generator class-name.

While configuring <generator /> tag in mapping file, if we need to pass any parameters to generator class then we can use <param /> tag, which is the sub element of <generator/> tag,

Example: In HBM file

```
<hibernate-mapping>
  <class name="ClassName" table="tableName" >
    <id name="identifierVariableName" column="columnName"/>
    <generator class="generator-class-name">
      <param name="param-name">param-value </param>
    </generator>
  </id>
  <property name="var2" column="column2"/>
</class>
</hibernate-mapping>
```

The following are the list of main generators we are using in the hibernate framework

1. sequence
2. assigned
3. increment
4. hilo
5. seqhilo
6. identity
7. native
8. uuid
9. guid
10. select
11. foregin

1. sequence (org.hibernate.id.SequenceGenerator)

This generator class is database dependent it means, we cannot use this generator class for all the databases, we should know whether the database supports sequence or not before we are working with it - It is not supporting by MYSQL db.

Here we write a sequence and it should be configured in HBM file and while persisting the object in the database, sequence is going to generate the identifier and it will assign to the Id property of persistent object, then it will store the persistent object into Database.

Steps:-

1. Create a Sequence in Database

SQL> create sequence empno_sequence start with 1000 increment by 1; to get the next value
select empno_sequence.nextval from dual

2. Configure Sequence in HBM File.

```
<hibernate-mapping>
<class name="com.nareshit.pojo.Employee" table="employee">
<id name="empno">
<generator class="sequence">
<param name="sequence">Empno_SEQUENCE</param>
</generator>
</id>
<property name="name"></property>
<property name="salary"></property>
</class>
</hibernate-mapping>
```

Note: If we don't configure any sequence name then it will take default sequence name(HIBERNATE-SEQUENCE). But hibernate won't create the sequence, if already sequence is available with name "HIBERNATE-SEQUENCE", then it is used by hibernate. Otherwise hibernate raises the exception.

Note: But remember, if we enable hbrn2ddl.auto property in hibernate configuration file, then hibernate will create the database tables,sequences if they do not exist.

Note: It is not advisable to use the default sequence, always prefer to create a different sequence to each entity separately.

3. Create the entity and save it without assigning identifier.

1. Session session = SessionUtil.getSession();
2. session.getTransaction ().begin ();
3. Employee emp=new Employee();
4. emp.setName ("raja") ;
5. emp.setSalary (5000) ;
6. Serializable id = session. save (emp) ;
7. System.out.println ("Employee is created with No : "+id) ;
8. session.getTransaction () . commit () ;

Note: When we execute the above code, we can find the sequence execution query on the console, which is used to get identifier for the saving entity.

Internal Code

```
public class SequenceGenerator implements PersistentIdentifierGenerator, Configurable {
    public static final String SEQUENCE = "sequence";
    public void configure( . . ){
        sequenceName = PropertiesHelper.getString(SEQUENCE, params,"hibernate-sequence");// default
        sequence name
        parameters = params.getProperty(PARAMETERS);

        public Serializable generate( . . ) {
            PreparedStatement st = . . .prepareSelectStatement(sql);
            ResultSet rs = st.executeQuery();
            rs.next();
```

```

Serializable result = . . iterate results ...
return result;
}

```

Note: Now onwards for the following generator classes just give the HBM configuration, you can use the same entity saving logic(which we used in the above example as part of step-3) to test them.

2. Assigned (org.hibernate.id.Assigned)

This generator supports in all the databases

This is the default generator class used by the hibernate, if we do not specify <generator/> element under <id/> element, then hibernate by default assumes it as "assigned" generator class.

If generator class is assigned, then the programmer is responsible for assigning the identifier value to entity before saving into the database.

Example in code Mapping File:-

```

<id name=" empNo " column="empNo">
<generator class="org.hibernate.id.Assigned" />
</id >
(OR)
<id name=" empNo " column="EmpNO">
<generator class="assigned" />
</id >

```

Internal Code:

```

public class Assigned implements IdentifierGenerator,Configurable{
public Serializable generate(-,- ){
final Serializable id = . . . it will gets the developer given id.
if (id==null){
throw new IdentifierGenerationException("ids for this class must be manually assigned before
calling save(): " entityName);
return id;
}
}

```

3. increment (org.hibernate.id.IncrementGenerator)

This generator supports in all the databases, so it is database independent generator class.

This generator is used for generating the id value for the new record by using the formula

Max of id value in Database + 1

If there is no record initially in the database, then for the first time this will saves primary key value as 1.

Example:-

```

<id name=" empNo " column="empNo">
<generator class="increment" />
</id >
(or)
<id name=" empNo " column="empNo">
<generator class=" org.hibernate.id.IncrementGenerator " /></id>

```

4. High-low alg (org.hibernate.id.TableHiLoGenerator)

This generator is database independent it uses a high/low algorithm to generate Identifiers.

This alg uses High-Low Alg to Generate identity values

- This alg uses specified table, related column value as the source of high value
- uses the max_lo parameter value as the source of low value.
- uses following formula to generate identity value. (helper table column value)*(max_lo value+1)

Example:-

create helper table having helper column with value.

SQL>create table mytable(mycolumn number(10));

SQL>insert into mytable value(10);

SQL>commit;

Ex:-

<id name="empNo" column="empNo">

<generator class="hilo">

<param name="table">mytable</param>

<param name="column">mycolumn</param>

<param name="max_lo">5</param>

</generator>

</id>

It is used the following formula to generate the Identifier (helpercolval)*(max_lo value+1)

⇒ (10)*(5+1)

60-->11

next

11*(5+1)

==>66→ 12

If we do configure helper table name ,helper column value and max_lo value, by default hibernate will use helper table name as **hibernate_unique_key** and helper column value as **next_hi**

- And for the first record, the id value will be inserted as 1
- For the second record the id value will be inserted as 32768
- For the next records, the id value will be incremented by 32768 and will store into the database (that means adds to the previous)
- Actually this hibernate stores the count of id values generated in a column of separated table, with name "**hibernate_unique_key**" by default with the column name "**next_hi**"

5. seqhilo(org.hibernate.id.SequenceHiLoGenerator):-

It is just like hilo generator class, But hilo generator stores its high value in table, where as seqhilo generator stores its high value in sequence.

This alg generates long,short,int type identity value by using given sequence generated value as high value& max_lo.

This alg using the following formula to generate the identity value:-

$(\text{sequence generated value}) * (\text{max_lo param value} + 1)$

$1 * (5 + 1) = 6$

$11 * (5 + 1) = 66$

Example

```
<id name="empNo">
  <generator class="seqhilo">
    <param name="sequence">EmpNo_SEQUENCE</param>
    <param name="max_lo">5</param>
  </id>
```

6. identity(org.hibernate.id.IdentityGenerator)

This is database dependent, actually it's not working in oracle.

Identity columns are support by DB2, MYSQL, SQL SERVER, SYBASE and HYPERSYNCSQL databases.

This identity generator doesn't needs any parameters to pass

Syntax to create identity columns in MYSQL database:

```
CREATE TABLE Employee(
EmpNO INT(10) NOT NULL AUTO-INCREMENT,
Name CHAR(20),
salary FLOAT,
PRIMARY KEY (EMPNO)
```

Example :

```
<id name=" empNo " column="EMPNO">
  <generator class="identity" />
</id>
```

7. native:-

native is not having any generator class because, it uses internally identity (or) sequence (or) hilo generator classes.

when we use this generator class, it first checks whether the database supports identity or not, if the database not supporting the identity then it checks for sequence. if the database not supporting sequence then hilo will be used finally .

so the order will be..

```
identity
sequence
hilo
```

For example, if we are connecting with oracle, if we use generator class as native then it is equal to the generator class as sequence.

8. uuid(universalUniqueid)

uuid uses a 128-bit id algorithm to generate identifiers of type string.

uuid generated identifier is unique within a network.

uuid algorithm generates identifier using IP address.

uuid algorithm codes identifier as a string(hexadecimal digits) of length 32.

Generally uuid is used to generate passwords.

Example:-

```
<id name="empNo" column="empNO">  
<generator class="uuid" >  
</generator>  
</id>
```

9. guid(global unique identifier):

same as the uuid alg. It is database dependent.

it works on SQLServer and MYSQL.

It generates a string based identity value.

10. select:

select retrieves a primary key assigned by a database trigger by selecting the row by some unique key and retrieving the primary key value.

11. foreign:

This alg generates identity value for current obj by collecting the identity value of associated obj. This alg is useful in 1-1 association mapping.

User Defined Generator class

When we feel the existing generator classes are not fit for our requirement, then we will go for user defined generator class.

Steps to implement user defined generator class.

Step 1:-

Take any java class and implement org.hibernate.id.IdentifierGenerator and override generate() method.

In this method implement the identifier generation logic as per the requirement.

Step 2:-

Give the Generator class entity in the <generator> tag of HBM file.

Step1 : EmpNoGenerator.java

```
package com.nareshit.id;  
import java.io.Serializable;  
import java.sql.Connection;  
import java.sql.PreparedStatement;
```

```
import java.sql.ResultSet;
import java.sql.SQLException;
import org.hibernate.HibernateException;
import org.hibernate.engine.spi.SessionImplementor;
import org.hibernate.id.IdentifierGenerator;
public class EmpNoGenerator implements IdentifierGenerator{
public Serializable generate(SessionImplementor session, Object obj)
        throws HibernateException {
    String empNo="Emp";
    try{
        Connection connection=session.connection();
        String query="select emp_no_sequence.nextval from dual";
        PreparedStatement pst=connection.prepareStatement(query);
        ResultSet resultSet=pst.executeQuery();
        if(resultSet.next()){
            empNo=empNo+resultSet.getString(1);
        }
    }catch(SQLException se){
        se.printStackTrace();
    }
    return empNo;
}
}
```

In the above user defined Generator class i used database sequence to generate next value... And I got the connection from session object.

Step 2: Employee Mapping file :

```
<!DOCTYPE hibernate-mapping PUBLIC
"-//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
<class name="com.nareshit.pojo.Employee" table="emp">
<id name="empNo">
<generator class="com.nareshit.id.EmpNoGenerator">
</generator>
</id>
<property name="name" length="12"></property>
<property name="salary" length="12"></property>
</class>
</hibernate-mapping>
```

Employee.java

```
package com.nareshit.pojo;  
public class Employee {  
    private String empNo;  
    private String name;  
    private double salary;  
    //required setters and getters  
  
}
```

Application Code :

```
Session session=sessionFactory.openSession();  
Transaction tx=session.beginTransaction();  
Employee emp=new Employee();  
    emp.setName("ajay");  
    emp.setSalary(22000.0);  
String id=(String)session.save(emp);  
    tx.commit();  
System.out.println("Emp Record Inserted with the following :"+id);
```

JPA Supported Identifier Generators

The @Id annotation lets you define which property is the identifier of your entity bean.

This property can be set by the application itself (OR) be generated by Hibernate(preferred). You can define the Identifier Generation strategy using @GeneratedValue annotation.

The @GeneratedValue annotation is used to specify the primary key generationstrategy to use. If the strategy is not specified by default AUTO will be used.

Different GenerationType Strategies :

AUTO - either identity column, sequence or table depending on the underlying DB

it will work like native.

TABLE - hilo algorithm

IDENTITY - identity column

SEQUENCE - sequence

Note :-

Hibernate provides more id generators than the basic JPA

@GeneratedValue annotation will allow the following two attributes

strategy: It is used to set the generation strategy as defined in the `javax.persistence.GenerationType`

this can be :- AUTO, TABLE, SEQUENCE, IDENTITY

generator –It is used to specify a name of a generator the configurations are donated using the `@TableGenerator`, `@SequenceGenerator`,

or `@GenericGenerator` (Hibernate specific)

Example 1:- With `@GeneratedValue` annotation :

Employee.java

```
package com.nareshit.pojo;
@Entity
@Table(name="employee")
public class Employee {
    @Id
    @GeneratedValue
    @Column(name="eno")
    private int empNo;
    @Column(name="ename",length=12)
    private String name;
    @Column(name="esal")
    private double salary;
    //required setters and getters
}
```

Note : As we did n't specify any strategy, by default it will take `strategy=GenerationType.AUTO`. As we know AUTO works like "native" generator, so it will take either "identity" (OR) "sequence"(OR)"hilo".

Example 2:- `@GeneratedValue(strategy=GenerationType.AUTO)`

Employee.java

```
package com.nareshit.pojo;
@Entity
@Table(name="employee")
public class Employee {
    @Id
    @GeneratedValue(strategy=GenerationType.AUTO)
    @Column(name="eno")
    private int empNo;
    @Column(name="ename",length=12)
    private String name;
    @Column(name="esal")
```



```
private double salary;  
//required setters and getters  
}
```

Example 3:- With @GeneratedValue(strategy=GenerationType.IDENTITY)

Note : This is not supported by Oracle, But supported by MySql database

Employee.java

```
package com.nareshit.pojo;  
@Entity  
@Table(name="employee")  
public class Employee {  
    @Id  
    @GeneratedValue(strategy=GenerationType.IDENTITY)  
    @Column(name="eno")  
    private int empNo;  
    @Column(name="ename",length=12)  
    private String name;  
    @Column(name="esal")  
    private double salary;  
    //required setters and getters  
}
```

Example 4:- With @GeneratedValue(strategy=GenerationType.TABLE)

Employee.java

```
package com.nareshit.pojo;  
@Entity  
@Table(name="employee")  
public class Employee {  
    @Id  
    @GeneratedValue(strategy=GenerationType.TABLE,generator="myGenerator")  
    @TableGenerator(name="myGenerator",initialValue=1000,allocationSize=1,table="PK_VAL  
UE_TAB",pkColumnName="PK_COLUMN",pkColumnValue="PK_VALUE",valueColumnName=  
"PK_VALUE_COLUMN")  
    @Column(name="eno")  
    private int empNo;  
    @Column(name="ename",length=12)  
    private String name;  
    @Column(name="esal")  
    private double salary;  
    //required setters and getters  
}
```

Hibernate Supported Identifier Generators :

To Integrate hibernate specific generators into JPA we can use either

@org.hibernate.annotations.GenericGenerator(OR)

@org.hibernate.annotations.GenericGenerators annotations

Example 1:- assigned(org.hibernate.id.Assigned)

Employee.java

```
package com.nareshit.pojo;
@Entity
@Table(name="employee")
public class Employee {
    @Id
    @GenericGenerator(name="myGenerator",strategy="assigned")
    @GeneratedValue(generator="myGenerator")
    @Column(name="eno")
    private int empNo;
    @Column(name="ename",length=12)
    private String name;
    @Column(name="esal")
    private double salary;
    //required setters and getters
}
```

Example 2:- increment(org.hibernate.id.IncrementGenerator)

Employee.java

```
package com.nareshit.pojo;
@Entity
@Table(name="employee")
public class Employee {
    @Id
    @GenericGenerator(name="myGenerator",strategy="increment")
    @GeneratedValue(generator="myGenerator")
    @Column(name="eno")
    private int empNo;
    @Column(name="ename",length=12)
    private String name;
    @Column(name="esal")
    private double salary;
    //required setters and getters
}
```

Example 3:- sequence(org.hibernate.id.SequenceGenerator)**Employee.java**

```
package com.nareshit.pojo;
@Entity
@Table(name="employee")
public class Employee {
    @Id
    @GenericGenerator(name="myGenerator",strategy="sequence")
    @GeneratedValue(generator="myGenerator")
    @Column(name="eno")
    private int empNo;
    @Column(name="ename",length=12)
    private String name;
    @Column(name="esal")
    private double salary;
    //required setters and getters
}
```

Note : As there are no parameters so it will take default sequence name(Hibernate_Sequence)

Example 4:- sequence(org.hibernate.id.SequenceGenerator)

```
package com.nareshit.pojo;
@Entity
@Table(name="employee")
public class Employee {
    @Id
    @GenericGenerator(name="myGenerator",
strategy="sequence",
parameters={@Parameter(name="sequence",value="EMP_NO_SEQ")})
    @GeneratedValue(generator="myGenerator")
    @Column(name="eno")
    private int empNo;
    @Column(name="ename",length=12)
    private String name;
    @Column(name="esal")
    private double salary;
    //required setters and getters
}
```

Example 5:- UserDefined Generator (com.nareshit.id.EmpNoGenerator)

```
package com.nareshit.pojo;
@Entity
@Table(name="employee")
public class Employee {
    @Id
    @GenericGenerator(name="myGenerator",
        strategy="com.nareshit.id.EmpNoGenerator")
    @GeneratedValue(generator="myGenerator")
    @Column(name="eno")
    private int empNo;
    @Column(name="ename",length=12)
    private String name;
    @Column(name="esal")
    private double salary;
    //required setters and getters
}
```

Composite ID:-

Process of defining primary key on more than one column is called composite id. Usage is combination of values should not be repeated

➤ Means

ID1	ID2
1001	2001

Into the above columns it won't allow again 1001-2001 pair. but it allows 1001-2002, 1003-2001.

It means combination should not be repeated.

Generally this concept is implemented in link tables (many-many-relationship), we can implement this concept id in two ways.

1st way:-

Writing primary column related properties and other properties in the same entity.

2nd way:-

Writing primary column related properties in one entity class and other property in the other entity class.

Example Program

Files required....

Product.java (Pojo)

Test.java (main class)

hibernate.cfg.xml

Product.hbm.xml

Product.java

```
package com.nareshit;
public class Product implements java.io.Serializable{

    private int productId;
    private String productName;
    private double price;

    public void setProductId(int productId){
        this.productId = productId;
    }
    public int getProductId(){
        return productId;
    }

    public void setProductName(String productName){
        this.proName = productName;
    }
    public String getProductName(){
        return proName;
    }

    public void setPrice(double price){
        this.price = price;
    }
    public double getPrice(){
        return price;
    }
}
```

hibernate.cfg.xml

```
<?xml version='1.0' encoding='UTF-8'?>
<!DOCTYPE hibernate-configuration PUBLIC
"-//Hibernate/Hibernate Configuration DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">

<hibernate-configuration>
<session-factory>
<property name="connection.driver_class">oracle.jdbc.driver.OracleDriver
</property>
<property name="connection.url">jdbc:oracle:thin:@localhost:1521:XE</property>
<property name="connection.username">system</property>
<property name="connection.password">manager</property>

<property name="dialect">org.hibernate.dialect.OracleDialect</property>
<property name="show_sql">>true</property>
<property name="hbm2ddl.auto">update</property>

<mapping resource="Product.hbm.xml"></mapping>
```

```
</session-factory>
</hibernate-configuration>
```

Product.hbm.xml

```
<?xml version="1.0"?>
<!DOCTYPE hibernate-mapping PUBLIC
"-//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">

<hibernate-mapping>
<class name="com.nareshit.Product" table="products">

<composite-id>
<key-property name="productId" column="pid" />
<key-property name="productName" column="pname" length="10" />
</composite-id>

<property name="price"/>
</class>
</hibernate-mapping>
```

Test.java

```
package com.nareshit;
import org.hibernate.*;
import org.hibernate.cfg.*;

public class Test{

    public static void main(String[] args) {

        Configuration cfg = new Configuration();
        cfg.configure("hibernate.cfg.xml");

        SessionFactory factory = cfg.buildSessionFactory();
        Session session = factory.openSession();

        Product p=new Product();

        p.setProductId(101);
        p.setProductName("keyboard");
        p.setPrice(250);

        Transaction tx=session.beginTransaction();
        session.save(p);
        System.out.println("Object Loaded successfully.....!!");
        tx.commit();
    }
}
```

```

    session.close();
    factory.close();
}
}

```

Note:-

- Product.java pojo class implements java.io.Serializable
- <composite-id> element is defined in hibernate.cfg.xml
- Actually if we have a single primary key, we need to use <id> element, but this time we have multiple primary keys, so we need to use this new element <composite-id>

Persistent class Mapping :-

Configurations in hibernate mapping file are called as Object Relational mapping configurations. we can perform the following ORM configurations.

1. Basic ORM(in all the above examples we are using Basic ORM)
2. Component Mapping
3. Inheritance Mapping
4. Collection Mapping
5. Association Mapping

Component-mapping:-

The property of hibernate persistence class that is mapped with a single column of database table is called as simple property.

Example :-

```
<property name="empName" column="EMP_Name"/>
```

The property of hibernate persistence class that is mapped with multiple columns of database table is called as component property. To use component - mapping we can work with <component> tag.

Example :-

```

<component name="address" class="com.nareshit.pojo.Address">
<property name="h_No" column="h_No" length="15"/>

<property name="city" column="city"/>
<property name="state" column="state"/>
<property name="country" column="country"/>
</component>

```

A component property holds represents multiple column values of a record. The data type of component property is always an user-defined data types.

To configure simple property in hibernate mapping file we can use <property> tag. Similarly to configure component property in hibernate mapping use <component> tag.

To demonstrate the component type we map Address as value type and Employee as Persistence class.

In the Below example, Employee persistence class with four properties → one of them being of component type. i.e. Address class type and other three properties are empNo, name, salary.

Address component type is implemented representing four properties such as h_No, state, city, country.

Example : Employee Table

EMPNO	NAME	SALARY	H_NO	CITY	STATE	COUNTRY
1001	ramu	-	3-2-1/a	hyd	TS	INDIA

ComponentMappingExample

- src
 - com.nareshit.client
 - Test.java
 - com.nareshit.config
 - hibernate.cfg.xml
 - com.nareshit.dao
 - EmployeeDAO.java
 - EmployeeDAOImpl.java
 - com.nareshit.mapping
 - Employee.hbm.xml
 - com.nareshit.pojo
 - Address.java
 - Employee.java
 - com.nareshit.utility
 - HibernateUtility.java

Address.java

```
package com.nareshit.pojo;

public class Address {
    private String h_No, city, state, country;
    public Address(){
    }
    public Address(String h_No, String city, String state, String country){
        this.h_No=h_No;
        this.state=state;
        this.city=city;
        this.country=country;
    }
    //required setters and getters
}
```


Employee.java

```
package com.nareshit.pojo;
public class Employee {
    private int empNo;
    private String name;
    private double salary;
    private Address address;
    //required setters and getters
}
```

Employee.hbm.xml

```
<!DOCTYPE hibernate-mapping PUBLIC
"-//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
<hibernate-mapping package="com.nareshit.pojo">
    <class name="Employee" table="employee">
        <id name="empNo">
            <generator class="increment"/>
        </id>
        <property name="name" length="15"/>
        <property name="salary" length="10"/>
        <component name="address" class="Address">
            <property name="h_No" column="h_No" length="15"/>
            <property name="city" column="city" length="15"/>
            <property name="state" column="state" length="15"/>
            <property name="country" column="country" length="15"/>
        </component>
    </class>
</hibernate-mapping>
```

hibernate.cfg.xml

same as the above applications

HibernateUtility.java

Same as the above applications

EmployeeDAO.java

```
package com.nareshit.dao;
import com.nareshit.pojo.Employee;
public interface EmployeeDAO {
    public void createEmployee(Employee emp);
    public void getEmployee(int empno);
}
```

EmployeeDAOImpl.java

```
package com.nareshit.dao;
import org.hibernate.Session;
import com.nareshit.pojo.Employee;
import com.nareshit.utility.HibernateUtility;
public class EmployeeDAOImpl implements EmployeeDAO {
    public void createEmployee(Employee emp) {
        Session session=HibernateUtility.getSession();
        if(session!=null){
            Integer id=(Integer)session.save(emp);
            session.beginTransaction().commit();
            System.out.println("Employee inserted with Id :"+id);
            session.close();
        }
        } //end of createEmployee()
    public void getEmployee(int empno) {
        Session session=HibernateUtility.getSession();
        if(session!=null){
            Employee emp=(Employee)session.get(Employee.class,empno);
            System.out.println("Emp No : "+emp.getEmpNo());
            System.out.println("Name :"+emp.getName());
            System.out.println("Salary :"+emp.getSalary());
            System.out.println("H_No :"+emp.getAddress().getH_No());
            System.out.println("City :"+emp.getAddress().getCity());
            System.out.println("State :"+emp.getAddress().getState());
            System.out.println("Country :"+emp.getAddress().getCountry());
        }
    }
}
```

Test.java

```
package com.nareshit.client;
import com.nareshit.dao.EmployeeDAO;
import com.nareshit.dao.EmployeeDAOImpl;
import com.nareshit.pojo.Address;
import com.nareshit.pojo.Employee;
public class Test {
    public static void main(String[] args) {
        EmployeeDAO empDao=new EmployeeDAOImpl();
        Address address=new Address("3-2-1/a","hyd","TS","INDIA");
        Employee emp=new Employee();
```

```
emp.setName("ramu");
emp.setSalary(8000.0);
emp.setAddress(address);
System.out.println("Testing of createEmployee(-)");
empDao.createEmployee(emp);
System.out.println("Testing of getEmployee(1001)");
empDao.getEmployee(1001);
}
}
```

Inheritance Mapping :

Hibernate supports three different approaches of Inheritance hierarchy mapping:

1. Table Per Class Hierarchy
2. Table per subclass
3. Table per Concrete class

Now, let us discuss each of these approaches.

Table Per Class Hierarchy

In this approach all the Hibernate persistence classes of Inheritance will use single database table to store and manage their data.

By this inheritance strategy, we can map the whole hierarchy by single table only. Here, an extra column (also known as discriminator column) is created in the table to identify the class.

In this approach database table column have special column called discriminator column & this column contains some logical names which indicates each record in this table is inserted by using specific Hibernate persistence class of Inheritance Hierarchy.

In case of table per class hierarchy an discriminator column is added by the hibernate framework that specifies the type of the record. It is mainly used to distinguish the record. To specify this, discriminator subelement of class must be specified.

The subclass subelement of class, specifies the subclass.

This approach is easier to use and efficient compared to the other two approaches.

But this approach suffers from one major problem the columns for properties declared by sub classes must be declared to accept null values, that is, these columns cannot be declared with not null constraint. This limitation may cause serious problems for data integrity.

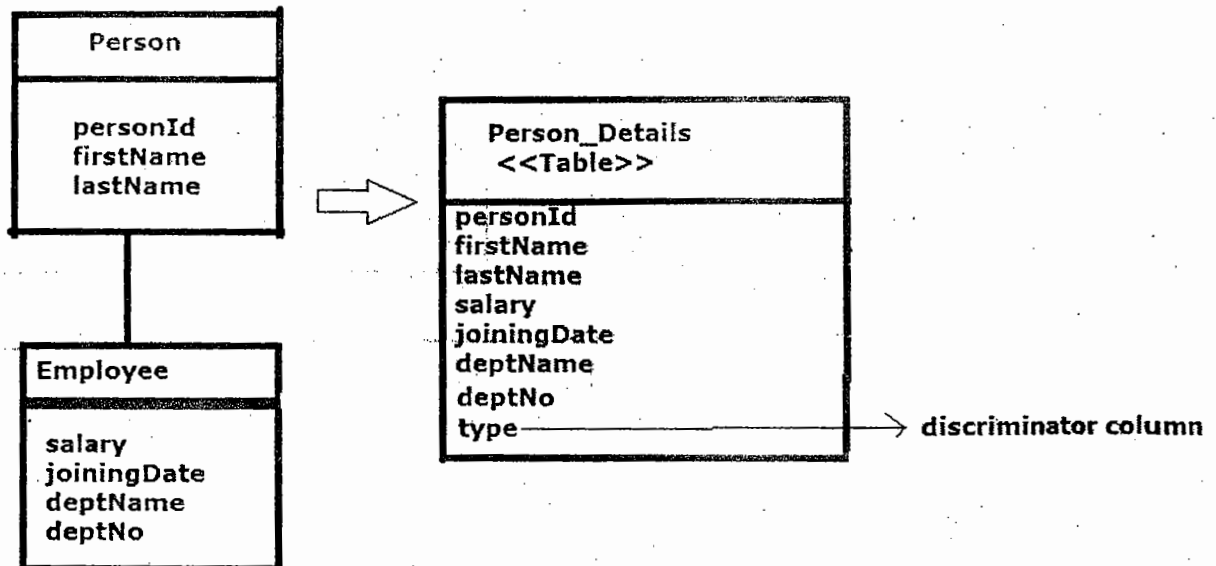


Table Per Class Hierarchy mapping

<discriminator> tag is used to define the discriminator column.

In this approach sub classes will be configured by using <subclass> tag.

SQL QUERY :

create table Person_Details (Person_ID number(10,0) not null, usertype varchar2(255 char) not null, firstName varchar2(15 char), lastName varchar2(15 char), salary double precision, deptNo number(10,0), deptName varchar2(20 char), joiningDate timestamp, primary key (Person_ID))

Example 1 :

- TablePerClassXMLMappingExample
 - src
 - com.nareshit.client
 - Test.java
 - com.nareshit.config
 - hibernate.cfg.xml
 - com.nareshit.mapping
 - Person_Details.hbm.xml
 - com.nareshit.pojo
 - Employee.java
 - Person.java
 - com.nareshit.utility
 - HibernateUtility.java

Person.java

```

package com.nareshit.pojo;
public class Person {
    private int personId;
    private String firstName,lastName;
    public int getPersonId() {
  
```

```
        return personId;
    }
    public void setPersonId(int personId) {
        this.personId = personId;
    }
    public String getFirstName() {
        return firstName;
    }
    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }
    public String getLastName() {
        return lastName;
    }
    public void setLastName(String lastName) {
        this.lastName = lastName;
    }
}
```

Employee.java

```
package com.nareshit.pojo;

import java.util.Date;

public class Employee extends Person {
    private double salary;
    private Date joiningDate;
    private int deptNo;
    private String deptName;
    public double getSalary() {
        return salary;
    }
    public void setSalary(double salary) {
        this.salary = salary;
    }
    public Date getJoiningDate() {
        return joiningDate;
    }
    public void setJoiningDate(Date joiningDate) {
        this.joiningDate = joiningDate;
    }
    public int getDeptNo() {
        return deptNo;
    }
    public void setDeptNo(int deptNo) {
        this.deptNo = deptNo;
    }
}
```

```

public String getDeptName() {
    return deptName;
}
public void setDeptName(String deptName) {
    this.deptName = deptName;
}
}

```

Person Details.hbm.xml

```

<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
<hibernate-mapping package="com.nareshit.pojo">
<class name="Person" table="Person_Details" discriminator-value="person">
<id name="personId" column="Person_ID">
<generator class="increment" /></id>
<discriminator column="usertype" type="string" />
<property name="firstName" length="15"/>
<property name="lastName" length="15"/>
<subclass name="Employee" extends="Person" discriminator-value="emp">
<property name="salary" length="10"/>
<property name="deptNo" length="10"/>
<property name="deptName" length="20"/>
<property name="joiningDate" type="java.util.Date" />
</subclass>
</class>
</hibernate-mapping>

```

hibernate.cfg.xml

same as the above applications

HibernateUtility.java

Same as the applications

Test.java

```

package com.nareshit.client;
import java.util.Date;
import org.hibernate.Session;
import com.nareshit.pojo.Employee;
import com.nareshit.utility.HibernateUtility;
public class Test {
public static void main(String[] args){
    Test test=new Test();
    Employee emp=new Employee();
    emp.setFirstName("ramu");
}
}

```

```

emp.setLastName("A");
emp.setSalary(12000.0);
emp.setDeptName("Sales");
emp.setDeptNo(12);
emp.setJoiningDate(new Date("12/12/2012"));
System.out.println("Testing of createEmployee(-) ");
test.createEmployee(emp);
System.out.println("Testing of getEmployee(1) :");
test.getEmployee(1);
}
public void createEmployee(Employee emp){
    Session session=HibernateUtility.getSession();
    Integer id=(Integer)session.save(emp);
    session.beginTransaction().commit();
    System.out.println("Employee created with id : "+id);
}
public void getEmployee(int empld){
    Session session=HibernateUtility.getSession();
    Employee emp=(Employee)session.get(Employee.class,empld);
    System.out.println(emp.getFirstName());
    System.out.println(emp.getLastName());
    System.out.println(emp.getDeptName());
    System.out.println(emp.getJoiningDate());
    System.out.println(emp.getSalary());
}
}

```

Table :

PERSON_DETAILS

PERSON_ID	USERTYPE	FIRSTNAME	LASTNAME	SALARY	DEPTNO	DEPTNAME	JOININGDATE
1	emp	ramu	A	12000	12	Sales	12-DEC-12 12.00.00.000000 AM

The discriminator-value for Employee is defined "emp", Thus, when Hibernate will persist the data for person or employee it will accordingly populate this value.

Example 2:-

TablePerClassAnnotationMappingExample

- src
 - com.nareshit.client
 - Test.java
 - com.nareshit.config
 - hibernate.cfg.xml
 - com.nareshit.pojo
 - Employee.java
 - Person.java
 - com.nareshit.utility
 - HibernateUtility.java

Person.java

```
package com.nareshit.pojo;
import javax.persistence.*;
import org.hibernate.annotations.GenericGenerator;
@Entity
@Table(name="Person_Details")
@Inheritance(strategy=InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name="usertype", discriminatorType=DiscriminatorType.STRING)
@DiscriminatorValue(value="Person")
public class Person {
    @Id
    @GeneratedValue(generator="myGenerator")
    @GenericGenerator(name="myGenerator",strategy="increment")
    private int personId;
    @Column(name="firstName",length=20)
    private String firstName;
    @Column(name="lastName",length=20)
    private String lastName;
    public int getPersonId() {
        return personId;
    }
    public void setPersonId(int personId) {
        this.personId = personId;
    }
    public String getFirstName() {
        return firstName;
    }
    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }
    public String getLastName() {
        return lastName;
    }
    public void setLastName(String lastName) {
        this.lastName = lastName;
    }
}}
```

Employee.java

```
package com.nareshit.pojo;
import java.util.Date;
import javax.persistence.*;
@Entity
@Table(name="Person_Details")
@DiscriminatorValue("emp")
public class Employee extends Person {
    @Column(name="salary")
```



```
private double salary;
@Column(name="joiningDate")
private Date joiningDate;
@Column(name="deptNo")
private int deptNo;
@Column(name="deptName")
private String deptName;
public double getSalary() {
    return salary;
}
public void setSalary(double salary) {
    this.salary = salary;
}
public Date getJoiningDate() {
    return joiningDate;
}
public void setJoiningDate(Date joiningDate) {
    this.joiningDate = joiningDate;
}
public int getDeptNo() {
    return deptNo;
}
public void setDeptNo(int deptNo) {
    this.deptNo = deptNo;
}
public String getDeptName() {
    return deptName;
}
public void setDeptName(String deptName) {
    this.deptName = deptName;
}
}
```

hibernate.cfg.xml

```
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
<session-factory>
<property name="connection.driver_class">
oracle.jdbc.driver.OracleDriver</property>
<property name="connection.url">
jdbc:oracle:thin:@localhost:1521:XE</property>
<property name="connection.username">system</property>
<property name="connection.password">manager</property>
<property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
<property name="show_sql">true</property>
```

```
<property name="hbm2ddl.auto">update</property>
<mapping class="com.nareshit.pojo.Person"/>
<mapping class="com.nareshit.pojo.Employee"/>
</session-factory>
</hibernate-configuration>
```

HibernateUtility.java(same as the above application)

Test.java(same as the above application)

The Person class is the root of hierarchy. Hence we have used some annotations to make it as the root.

@Inheritance - Defines the inheritance strategy to be used for an entity class hierarchy. It is specified on the entity/persistence class that is the root of the entity class hierarchy.

@DiscriminatorColumn - Is used to define the discriminator column for the SINGLE_TABLE inheritance mapping strategies. The strategy and the discriminator column are only specified in the root of an entity class. The name of the discriminator column defaults to "DTYPE" and the discriminator type to DiscriminatorType.STRING.

@DiscriminatorValue - Is used to specify the value of the discriminator column for entities of the given type. The Discriminator Value annotation can only be specified on a concrete entity class. If the Discriminator Value annotation is not specified and a discriminator column is used, a provider-specific function will be used to generate a value representing the entity type. If the Discriminator Type is STRING, the discriminator value default is the entity name.

Table Per Subclass:

In this approach, every class of Inheritance hierarchy contains its own database table. But the tables of Sub classes will maintain one-to-one relationship with table of Super class.

To configure Super class we can use <class> tag and to configure Sub class we can use <joined-subclass> tags in Hibernate mapping file.

The <joined-subclass> tag is used to map the Sub class with Super class

Using the primary key and foreign key relation

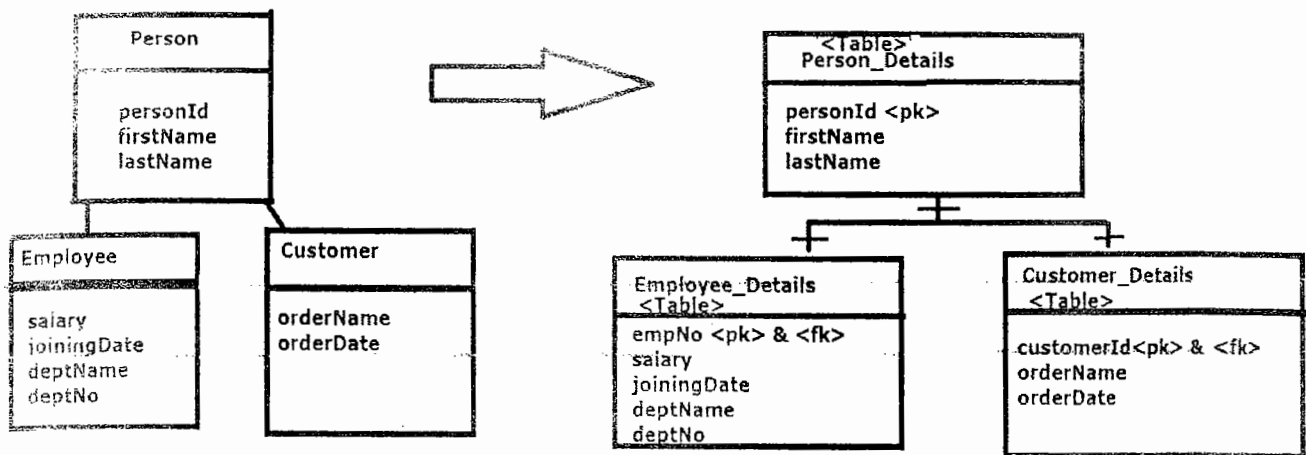


Table Per subclass Hierarchy mapping

Note that a foreign key relationship exists between the subclass tables and super class table. Thus the common data is stored in Person_Details table and subclass specific fields are stored in Employee_Details and Customer_Details tables.

SQL Script :

Create Database Tables

```
SQL> CREATE TABLE person_details (
    personid Number NOT NULL,
    firstname VARCHAR(20) NOT NULL,
    lastname VARCHAR(15) NOT NULL,
    PRIMARY KEY (personid)
);

SQL> CREATE TABLE employee_details(
    empNo Number NOT NULL,
    joiningDate DATE NULL,
    deptName VARCHAR(20),
    deptNo Number NOT NULL,
    salary Number NOT NULL,
    PRIMARY KEY (empNo),
    CONSTRAINT FK_PERSON1 FOREIGN KEY (empNo) REFERENCES person_details (personid)
);

SQL> CREATE TABLE customer_details(
    customerId Number NOT NULL,
    orderdate DATE NULL,
    orderName VARCHAR(20),
    PRIMARY KEY (customerId),
    CONSTRAINT FK_PERSON2 FOREIGN KEY (customerId) REFERENCES person_details
(personid)
)
```

Example :

- ▾ TablePerSubclassExampleWithXMLMapping
 - ▾ src
 - ▾ com.nareshit.client
 - ▾ Test.java
 - ▾ com.nareshit.config
 - ▾ hibernate.cfg.xml
 - ▾ com.nareshit.mapping
 - ▾ PersonDetails.hbm.xml
 - ▾ com.nareshit.pojo
 - ▾ Customer.java
 - ▾ Employee.java
 - ▾ Person.java

Person.java

```
package com.nareshit.pojo;
```

```
public class Person {
    private int personId;
    private String firstName, lastName;
    public int getPersonId() {
        return personId;
    }
    public void setPersonId(int personId) {
        this.personId = personId;
    }
    public String getFirstName() {
        return firstName;
    }
    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }
    public String getLastName() {
        return lastName;
    }
    public void setLastName(String lastName) {
        this.lastName = lastName;
    }
}
```

Employee.java

```
package com.nareshit.pojo;
```

```
import java.util.Date;
```

```
public class Employee extends Person{
```

```
private int deptNo;
private double salary;
private String deptName;
private Date joiningDate;
public int getDeptNo() {
    return deptNo;
}
public void setDeptNo(int deptNo) {
    this.deptNo = deptNo;
}
public double getSalary() {
    return salary;
}
public void setSalary(double salary) {
    this.salary = salary;
}
public String getDeptName() {
    return deptName;
}
public void setDeptName(String deptName) {
    this.deptName = deptName;
}
public Date getJoiningDate() {
    return joiningDate;
}
public void setJoiningDate(Date joiningDate) {
    this.joiningDate = joiningDate;
}
}
```

Customer.java

```
package com.nareshit.pojo;

import java.util.Date;

public class Customer extends Person {
    private String orderName;
    private Date orderDate;
    public String getOrderName() {
        return orderName;
    }
    public void setOrderName(String orderName) {
        this.orderName = orderName;
    }
    public Date getOrderDate() {
        return orderDate;
    }
}
```

```
}  
public void setOrderDate(Date orderDate) {  
    this.orderDate = orderDate;  
}  
}
```

PersonDetails.hbm.xml

```
<!DOCTYPE hibernate-mapping PUBLIC  
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"  
    "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">  
<hibernate-mapping package="com.nareshit.pojo">  
    <class name="Person" table="Person_Details">  
        <id name="personId">  
            <generator class="increment"/>  
        </id>  
        <property name="firstName" length="15"/>  
        <property name="lastName" length="15"/>  
        <joined-subclass name="Employee"  
            table="Employee_Details">  
            <key column="empNo"/>  
            <property name="deptNo" length="10"/>  
            <property name="deptName" length="15"/>  
            <property name="salary" length="15"/>  
            <property name="joiningDate" />  
        </joined-subclass>  
        <joined-subclass name="Customer"  
            table="Customer_Details">  
            <key column="customerId" />  
            <property name="orderName" length="15"/>  
            <property name="orderDate" length="15"/>  
        </joined-subclass>  
    </class>  
</hibernate-mapping>
```

The **joined-subclass** subelement of class, specifies the subclass. The **key** subelement of joined-subclass is used to generate the foreign key in the subclass mapped table. This foreign key will be associated with the primary key of parent class mapped table.

hibernate.cfg.xml

```
<!DOCTYPE hibernate-configuration PUBLIC  
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"  
    "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">  
<hibernate-configuration>  
    <session-factory>  
        <property name="connection.driver_class">  
            oracle.jdbc.driver.OracleDriver</property>
```

```
<property name="connection.url">
jdbc:oracle:thin:@localhost:1521:XE</property>
<property name="connection.username">system</property>
<property name="connection.password">manager</property>
<property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
<property name="show_sql">true</property>
<property name="hbm2ddl.auto">update</property>
<mapping resource="com/nareshit/mapping/PersonDetails.hbm.xml"/>
</session-factory>
</hibernate-configuration>
```

Test.java

```
package com.nareshit.client;
import java.util.Date;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;
import com.nareshit.pojo.Customer;
import com.nareshit.pojo.Employee;
public class Test {
    public static void main(String[] args) {
        Test test=new Test();
        Configuration cfg=new Configuration();
        cfg.configure("/com/nareshit/config/hibernate.cfg.xml");
        SessionFactory factory=cfg.buildSessionFactory();
        Session session=factory.openSession();
        System.out.println("Testing of Create Employee ");
        test.createEmployee(session);
        System.out.println("Testing of Create Customer ");
        test.createCustomer(session);
        System.out.println("Testing of Display Customer ");
        test.displayCustomer(session);
        System.out.println("Testing of Display Employee ");
        test.displayEmployee(session);
    }
    public void displayEmployee(Session session){
        Employee emp=(Employee)session.get(Employee.class,new Integer(1));
        System.out.println(emp.getFirstName());
        System.out.println(emp.getDeptName());
        System.out.println(emp.getSalary());
    }
    public void displayCustomer(Session session)
    {
        Customer c=(Customer)session.get(Customer.class,new Integer(2));
        System.out.println(c.getFirstName());
        System.out.println(c.getOrderName());
    }
}
```

```

System.out.println(c.getOrderDate());
}
public void createEmployee(Session session){
    Employee emp=new Employee();
    emp.setFirstName("Ramu");
    emp.setLastName("A");
    emp.setDeptNo(12);
    emp.setDeptName("sales");
    emp.setSalary(12000);
    emp.setJoiningDate(new Date("12/12/2012"));

    session.save(emp);
    session.beginTransaction().commit();
}
public void createCustomer(Session session){
    Transaction tx=session.beginTransaction();
    Customer c=new Customer();
    c.setFirstName("B");
    c.setLastName("Raja");
    c.setOrderName("Keyboard");
    c.setOrderDate(new Date("11/12/2012"));
    session.save(c);
    tx.commit();
}
}
}

```

Example 2: With Annotation mapping

TablePerSubClassExampleWithAnnotationMapping

- src
 - com.nareshit.client
 - Test.java
 - com.nareshit.pojo
 - Customer.java
 - Employee.java
 - Person.java
 - com.nareshit.xml
 - hibernate.cfg.xml

Person.java

```

package com.nareshit.pojo;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.Id;
import javax.persistence.Inheritance;
import javax.persistence.InheritanceType;
import javax.persistence.Table;
import org.hibernate.annotations.GenericGenerator;

```



```
@Entity
@Table(name="person_details")
@Inheritance(strategy=InheritanceType.JOINED)
public class Person {
    @Id
    @GenericGenerator(name="myGenerator",strategy="increment")
    @GeneratedValue(generator="myGenerator")
    private Integer personId;
    private String firstName,lastName;
    public Integer getPersonId() {
        return personId;
    }
    public void setPersonId(Integer personId) {
        this.personId = personId;
    }
    public String getFirstName() {
        return firstName;
    }
    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }
    public String getLastName() {
        return lastName;
    }
    public void setLastName(String lastName) {
        this.lastName = lastName;
    }
}
```

Employee.java

```
package com.nareshit.pojo;
import java.util.Date;
import javax.persistence.Entity;
import javax.persistence.PrimaryKeyJoinColumn;
import javax.persistence.Table;
@Entity
@Table(name="employee_details")
@PrimaryKeyJoinColumn(name="empNo")
public class Employee extends Person {
    private Integer deptNo;
    private String deptName;
    private Double salary;
    private Date joiningDate;
    public Integer getDeptNo() {
        return deptNo;
    }
    public void setDeptNo(Integer deptNo) {
        this.deptNo = deptNo;
    }
}
```

```
}  
public String getDeptName() {  
    return deptName;  
}  
public void setDeptName(String deptName) {  
    this.deptName = deptName;  
}  
public Double getSalary() {  
    return salary;  
}  
public void setSalary(Double salary) {  
    this.salary = salary;  
}  
public Date getJoiningDate() {  
    return joiningDate;  
}  
public void setJoiningDate(Date joiningDate) {  
    this.joiningDate = joiningDate;  
}}  

```

Customer.java

```
package com.nareshit.pojo;  
  
import java.util.Date;  
  
import javax.persistence.Entity;  
import javax.persistence.PrimaryKeyJoinColumn;  
import javax.persistence.Table;  
  
@Entity  
@Table(name="customer_details")  
@PrimaryKeyJoinColumn(name="customerId")  
public class Customer extends Person {  
    private String orderName;  
    private Date orderDate;  
    public String getOrderName() {  
        return orderName;  
    }  
    public void setOrderName(String orderName) {  
        this.orderName = orderName;  
    }  
    public Date getOrderDate() {  
        return orderDate;  
    }  
    public void setOrderDate(Date orderDate) {  
        this.orderDate = orderDate;  
    }  
}
```

Both Employee and Customer classes are child of Person class. Thus while specifying the mappings, we used `@PrimaryKeyJoinColumn` to map it to parent table.

@PrimaryKeyJoinColumn – This annotation specifies a primary key column that is used as a foreign key to join to another table.

It is used to join the primary table of an entity subclass in the JOINED mapping strategy to the primary table of its superclass; it is used within a `SecondaryTable` annotation to join a secondary table to a primary table; and it may be used in a OneToOne mapping in which the primary key of the referencing entity is used as a foreign key to the referenced entity.

If no `PrimaryKeyJoinColumn` annotation is specified for a subclass in the JOINED mapping strategy, the foreign key columns are assumed to have the same names as the primary key columns of the primary table of the superclass

Test.java

```
package com.nareshit.client;
import java.util.Date;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;
import com.nareshit.pojo.Customer;
import com.nareshit.pojo.Employee;
public class Test {
    public static void main(String[] args) {
        Test test=new Test();
        Configuration cfg=new Configuration();
        cfg.configure("/com/nareshit/config/hibernate.cfg.xml");
        SessionFactory factory=cfg.buildSessionFactory();
        Session session=factory.openSession();
        System.out.println("Testing of Create Employee ");
        test.createEmployee(session);
        System.out.println("Testing of Create Customer ");
        test.createCustomer(session);
        System.out.println("Testing of Display Customer ");
        test.displayCustomer(session);
        System.out.println("Testing of Display Employee ");
        test.displayEmployee(session);
    }
    public void displayEmployee(Session session){
        Employee emp=(Employee)session.get(Employee.class,new Integer(1));
```

```
        System.out.println(emp.getFirstName());
        System.out.println(emp.getDeptName());
        System.out.println(emp.getSalary());
    }
    public void displayCustomer(Session session)
    {
        Customer c=(Customer)session.get(Customer.class,new Integer(2));
        System.out.println(c.getFirstName());
        System.out.println(c.getOrderName());
        System.out.println(c.getOrderDate());
    }
    public void createEmployee(Session session){
        Employee emp=new Employee();
        emp.setFirstName("Ramu");
        emp.setLastName("A");
        emp.setDeptNo(12);
        emp.setDeptName("sales");
        emp.setSalary(12000);
        emp.setJoiningDate(new Date("12/12/2012"));
        session.save(emp);
        session.beginTransaction().commit();
    }
    public void createCustomer(Session session){
        Transaction tx=session.beginTransaction();
        Customer c=new Customer();
        c.setFirstName("B");
        c.setLastName("Raja");
        c.setOrderName("Keyboard");
        c.setOrderDate(new Date("11/12/2012"));
        session.save(c);
        tx.commit();
    }
}
```

In Table per class hierarchy we design table with huge no.of columns. This is against of database designing. But In Table Per sub class approach Multiple table will be there in shorter form & having the relationship along them .

so this is good database designing. In Real world the Regularly used Inheritance mapping approach is Table-per sub class.

In Table per sub class of Inheritance mapping each record of Sub tables maintains

One to one relationship with Super table.

Table per Concrete class:-

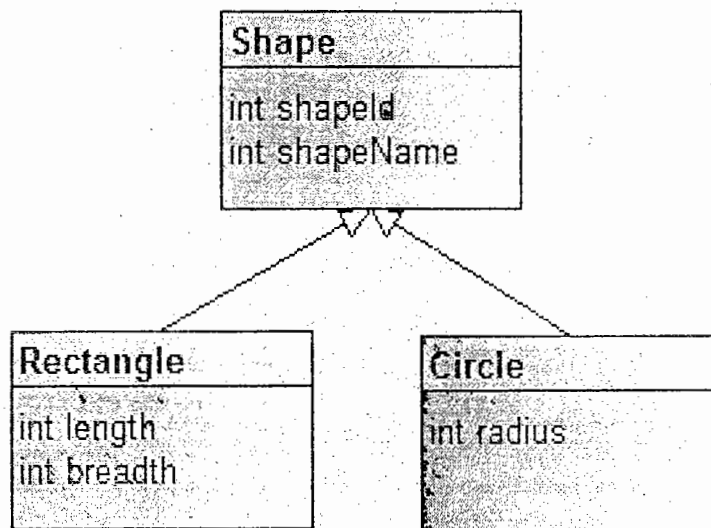
Table per Concrete class on one per pojo class basis.

In the case of Table Per Concrete class, there will be multiple tables in the database having no relations to each other. There are two ways to map the table with table per concrete class strategy.

- By union-subclass element
- By Self creating the table for each class

Let us illustrate Table per concrete class by creating table for each class

Lets say we have following class hierarchy. We have shape class as base class and Rectangle and Circle inherit from Shape class



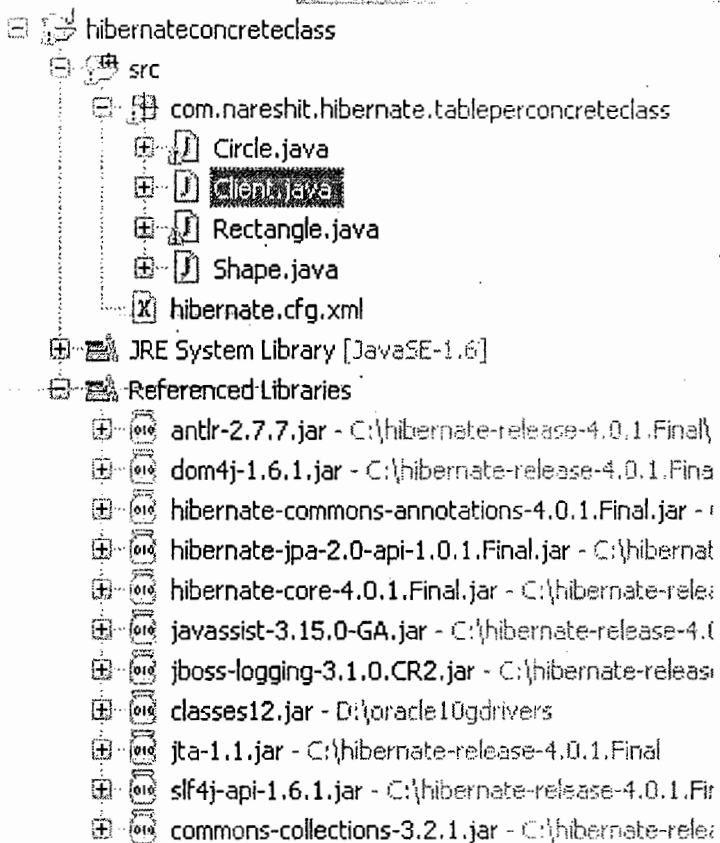
In table per concrete class, One table will be created for each concrete class..So there table will be created SHAPE, RECTANGLE and CIRCLE and subclass repeats property of parent class.

SHAPE
<u>Shape_Id</u>
Shape_Name

RECTANGLE
<u>Shape_Id</u>
Shape_Name
Rectangle_Length
Rectangle_Breadth

CIRCLE
<u>Shape_Id</u>
Shape_Name
Circle_Radius

Here is the eclipse directory structure



```
//Cirde.java
```

```
package com.nareshit.hibernate.tableperconcreteclass;
```

```
import javax.persistence.Column;
```

```
import javax.persistence.Entity;
```

```
import javax.persistence.Table;
```

```
@Entity
```

```
@Table(name = "CIRCLE")
```

```
public class Circle extends Shape {
```

```
    @Column(name = "Circle__Radius")
```

```
    intradius;
```

```
    public int getRadius() {
```

```
        returnradius;
```

```
    }
```

```
    public void setRadius(int radius) {
```

```
        this.radius = radius;
```

```
    }
```

```
    public Circle() {
```

```
}

    public Circle(String shapeName, int radius) {
        super(shapeName);
        this.radius = radius;
    }

}

//Rectangle.java

package com.nareshit.hibernate.tableperconcreteclass;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Table;

@Entity
@Table(name = "RECTANGLE")
public class Rectangle extends Shape {

    @Column(name = "Rectangle_Length")
    int length;

    public int getLength() {
        return length;
    }

    public void setLength(int length) {
        this.length = length;
    }

    public int getBreadth() {
        return breadth;
    }

    public void setBreadth(int breadth) {
        this.breadth = breadth;
    }

    @Column(name = "Rectangle_Breadth")
    int breadth;

    public Rectangle() {
    }
}
```

```
    public Rectangle(String shapeName, int length, int breadth) {  
        super(shapeName);  
        this.length = length;  
        this.breadth = breadth;  
    }  
}
```

//Shape.java

```
package com.nareshit.hibernate.tableperconcreteclass;
```

```
import javax.persistence.Column;  
import javax.persistence.Entity;  
import javax.persistence.GeneratedValue;  
import javax.persistence.GenerationType;  
import javax.persistence.Id;  
import javax.persistence.Inheritance;  
import javax.persistence.InheritanceType;  
import javax.persistence.Table;
```

```
@Entity  
@Table(name = "SHAPE")  
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)  
public class Shape {
```

```
    @Id  
    @GeneratedValue(strategy = GenerationType.TABLE)  
    @Column(name = "Shape_Id")  
    intshapeld;
```

```
    public int getShapeld() {  
        returnshapeld;  
    }
```

```
    public void setShapeld(int shapeld) {  
        this.shapeld = shapeld;  
    }
```

```
    public String getShapeName() {  
        returnshapeName;  
    }
```

```
    public void setShapeName(String shapeName) {  
        this.shapeName = shapeName;  
    }
```

```
    @Column(name = "Shape_Name")
```



```
String shapeName;
```

```
public Shape() {
```

```
}
```

```
public Shape(String shapeName) {
```

```
    this.shapeName = shapeName;
```

```
}
```

```
//Client.java
```

```
package com.nareshit.hibernate.tableperconcreteclass;
```

```
import org.hibernate.Session;
```

```
import org.hibernate.SessionFactory;
```

```
import org.hibernate.cfg.Configuration;
```

```
import org.hibernate.service.ServiceRegistry;
```

```
import org.hibernate.service.ServiceRegistryBuilder;
```

```
public class Client {
```

```
publicstaticvoid main(String[] args) {
```

```
    Shape shape=new Shape("Sqaure");
```

```
    Rectangle rectangle=new Rectangle("Rectangle", 10, 20);
```

```
    Circle circle=new Circle("Circle", 4);
```

```
    Configuration configuration=new Configuration();
```

```
    configuration.configure();
```

```
    ServiceRegistry sr= new
```

```
ServiceRegistryBuilder().applySettings(configuration.getProperties()).buildServiceRegistry();
```

```
    SessionFactory sf=configuration.buildSessionFactory(sr);
```

```
    Session ss=sf.openSession();
```

```
    ss.beginTransaction();
```

```
    ss.save(shape);
```

```
    ss.save(rectangle);
```

```
    ss.save(circle);
```

```
    ss.getTransaction().commit();
```

```
    ss.close();
```

```
}
```

```
}
```

```
//hibernate.cfg.xml
```

```

<?xmlversion='1.0'encoding='utf-8'?>
<!DOCTYPEhibernate-configurationPUBLIC
"-//Hibernate/Hibernate Configuration DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">

<hibernate-configuration>

<session-factory>

<!-- Database connection settings -->
<propertyname="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
<propertyname="connection.url">jdbc:oracle:thin:@localhost:1521:ORA</property>
<propertyname="connection.username">scott</property>
<propertyname="connection.password">tiger</property>

<!-- JDBC connection pool (use the built-in) -->
<propertyname="connection.pool_size">1</property>

<!-- SQL dialect -->
<propertyname="dialect">org.hibernate.dialect.Oracle9Dialect</property>

<!-- Enable Hibernate's automatic session context management -->
<propertyname="current_session_context_class">thread</property>

<!-- Disable the second-level cache -->
<propertyname="cache.provider_class">org.hibernate.cache.NoCacheProvider</property>

<!-- Echo all executed SQL to stdout -->
<propertyname="show_sql">>true</property>

<!-- Drop and re-create the database schema on startup -->
<propertyname="hbm2ddl.auto">create</property>

<mappingclass="com.nareshit.hibernate.tableperconcreteclass.Shape"></mapping>
<mappingclass="com.nareshit.hibernate.tableperconcreteclass.Rectangle"></mapping>
<mappingclass="com.nareshit.hibernate.tableperconcreteclass.Circle"></mapping>

</session-factory>

</hibernate-configuration>

```

//Output of the program

```

Hibernate: create table CIRCLE (Shape_Id number(10,0) not null, Shape_Name varchar2(255 char),
Circle_Radius number(10,0), primary key (Shape_Id))
Hibernate: create table RECTANGLE (Shape_Id number(10,0) not null, Shape_Name varchar2(255 char),
Rectangle_Breadth number(10,0), Rectangle_Length number(10,0), primary key (Shape_Id))

```

Hibernate: create table SHAPE (Shape_Id number(10,0) not null, Shape_Name varchar2(255 char), primary key (Shape_Id))

Hibernate: create table hibernate_sequences (sequence_name varchar2(255 char), sequence_next_hi_value number(10,0))

Nov 22, 2014 7:14:46 PM org.hibernate.tool.hbm2ddl.SchemaExport execute

INFO: HHH000230: Schema export complete

Hibernate: select sequence_next_hi_value from hibernate_sequences where sequence_name = 'SHAPE' for update

Hibernate: insert into hibernate_sequences(sequence_name, sequence_next_hi_value) values('SHAPE', ?)

Hibernate: update hibernate_sequences set sequence_next_hi_value = ? where sequence_next_hi_value = ? and sequence_name = 'SHAPE'

Hibernate: insert into SHAPE (Shape_Name, Shape_Id) values (?, ?)

Hibernate: insert into RECTANGLE (Shape_Name, Rectangle_Breadth, Rectangle_Length, Shape_Id) values (?, ?, ?)

Hibernate: insert into CIRCLE (Shape_Name, Circle_Radius, Shape_Id) values (?, ?, ?)

Association Mapping :

Instead of taking single -large table with huge amount of columns for a project. It is recommended to design multiple data base tables & and keep them in a relationship/association. so that we can work with those table individually.

Primary key, foreign keys are used to form the relationships between the tables in Database.

Types of Relationships in DataBase :

1. one-to-one
2. one-to-many
3. many-to-one
4. many-to-many

p.k ---> p.k (one-to-one)

f.k(unique) ---> p.k (one-to-one)

p.k ----> f.k (one-to-many)

f.k-----> p.k (many-to-one)

table1----> linktable ----> table2(many-to-many)

To implement the association mapping in hibernate-mapping file we can use the following tags.

<one-to-one>

<one-to-many>

<many-to-one>

<many-to-many>

ONE-TO-ONE:-

In this association one entity object exactly mapping with/associated with One object of another entity.

An Example of this type of association is the Relationship between an Address & Person.

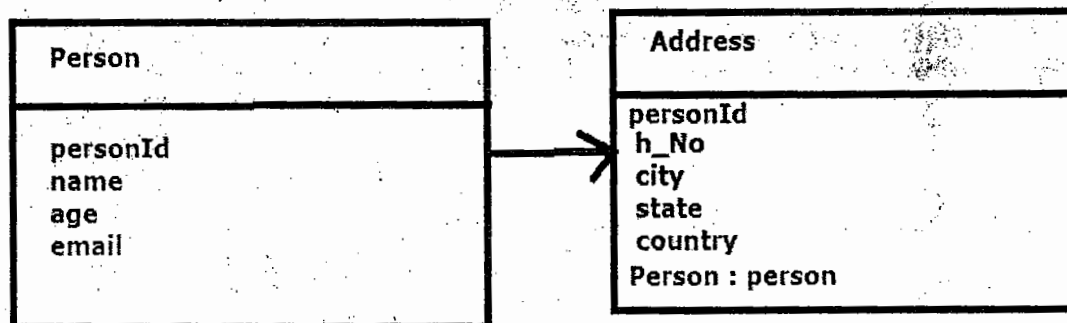
In one-to-One relationship each Person should have an unique address. Hence we will need two tables namely PERSON and ADDRESS to create this relation.

In Hibernate, one-to-one relationship between entities can be created by 2 different techniques.

These techniques are:

1. Using shared primary key:

In this technique, hibernate will ensure that it will use a common primary key value in both the tables. This way primary key of entity PERSON can safely be assumed the primary key of entity ADDRESS also. The example demonstrated shared primary key. The relational model is shown below:



one -to - one Relation class Diagram

Example Tables :

Person Table			
personId (pk)	name	age	email
1001	ramu	23	ramu@gmail.com
1002	raju	22	raju@gmail.com

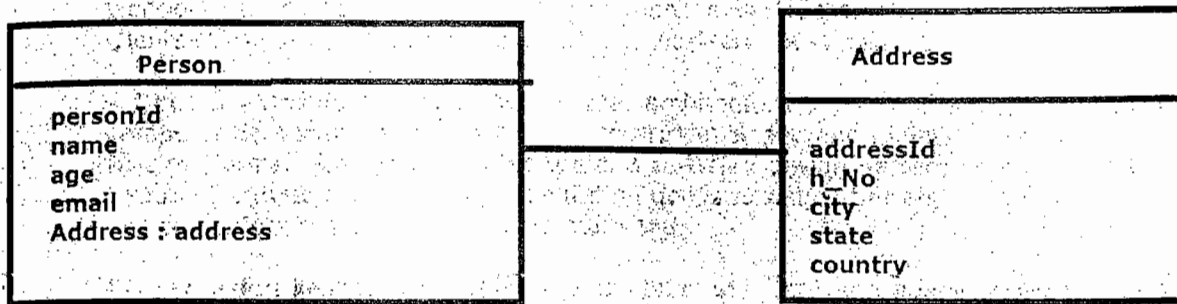
Address Table				
personId (pk)	h_No	city	state	country
1001	2-31/a	hyd	TS	INDIA
1002	1-13/b	hyd	TS	INDIA

2. Using Foreign Key Association:

In this association, a foreign key column is created in the owner entity.

For example, if we make PERSON as owner entity, then a extra column "ADDRESS_ID" will be

created in PERSON table. This column will store the foreign key for ADDRESS table. The relational model is shown below:



one -to- one relation class Diagram

Example Tables :

Person Table

personId (pk)	name	age	email	address_Id (fk)(unique)
1001	ramu	23	ramu@gmail.com	2001
1002	raju	22	raju@gmail.com	2002

Address Table

addressId (pk)	h_No	city	state	country
2001	2-31/a	hyd	TS	INDIA
2002	1-12/b	hyd	TS	INDIA

The following examples demonstrated shared primary key. The relational model is shown below:

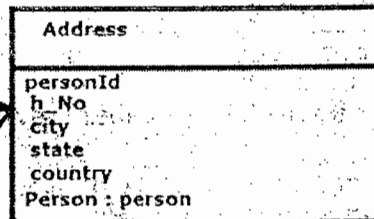
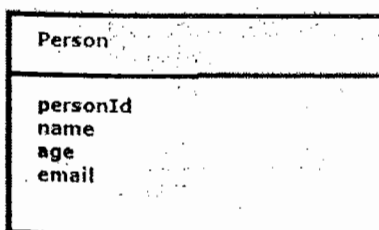
Example Table :

Person Table

personId (pk)	name	age	email
1001	ramu	23	ramu@gmail.com
1002	raju	22	raju@gmail.com

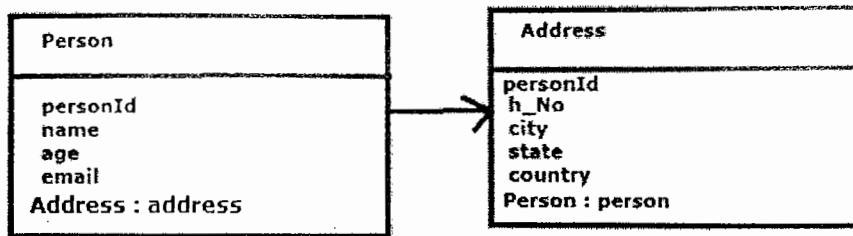
Address Table

personId (pk)	h_No	city	state	country
1001	2-31/a	hyd	TS	INDIA
1002	1-13/b	hyd	TS	INDIA



one -to- one Relation class Diagram

Uni Directional



one -to - one Releation class Diagram

Bi-Directional

One-to-One (pk-pk) UniDirectionalXmlMappingExample

SQL Script:

SQL>create table person (personId number(10,0) not null, name varchar2(12 char), age number(10,0), email varchar2(15 char), primary key (personId));

SQ>create table address (personId number(10,0) not null, h_No varchar2(20 char), city varchar2(20 char), state varchar2(20 char), country varchar2(20 char), primary key (personId));

one-to-one(pk-to-pk)UniDirectionalXMLMappingExample

src

- com.nareshit.client
 - Test.java
- com.nareshit.config
 - hibernate.cfg.xml
- com.nareshit.mapping
 - Address.hbm.xml
 - Person.hbm.xml
- com.nareshit.pojo
 - Address.java
 - Person.java

Person.java

```

package com.nareshit.pojo;

public class Person {
    private int personId,age;
    private String name,email;
    public int getPersonId() {
        return personId;
    }
    public void setPersonId(int personId) {
        this.personId = personId;
    }
    public int getAge() {
        return age;
    }
  
```

```
}  
public void setAge(int age) {  
    this.age = age;  
}  
public String getName() {  
    return name;  
}  
public void setName(String name) {  
    this.name = name;  
}  
public String getEmail() {  
    return email;  
}  
public void setEmail(String email) {  
    this.email = email;  
}  
}
```

Address.java

```
package com.nareshit.pojo;  
public class Address {  
    private int personId;  
    private String h_No,city,state,country;  
    private Person person;  
    public int getPersonId() {  
        return personId;  
    }  
    public void setPersonId(int personId) {  
        this.personId = personId;  
    }  
    public String getH_No() {  
        return h_No;  
    }  
    public void setH_No(String h_No) {  
        this.h_No = h_No;  
    }  
    public String getCity() {  
        return city;  
    }  
    public void setCity(String city) {  
        this.city = city;  
    }  
    public String getState() {  
        return state;  
    }  
}
```

```
public void setState(String state) {
    this.state = state;
}
public String getCountry() {
    return country;
}
public void setCountry(String country) {
    this.country = country;
}
public Person getPerson() {
    return person;
}
public void setPerson(Person person) {
    this.person = person;
}
}
```

Person.hbm.xml

```
<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
<class name="com.nareshit.pojo.Person" table="person">
<id name="personId">
<generator class="increment"/>
</id>
<property name="name" length="12"/>
<property name="age" length="3"/>
<property name="email" length="15"/>
</class>
</hibernate-mapping>
```

Address.hbm.xml

```
<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
<class name="com.nareshit.pojo.Address" table="address">
<id name="personId">
<generator class="foreign">
<param name="property">person</param>
</generator></id>
```



```

<property name="h_No" length="20"/>
<property name="city" length="20"/>
<property name="state" length="20"/>
<property name="country" length="20"/>
<one-to-one name="person" class="com.nareshit.pojo.Person"/>
</class>
</hibernate-mapping>

```

Note:- In the above example code Address object identity value is nothing but associated person class object identity value. For this we need to use foreign algorithm.

Foreign alg collects identity value from associated object and makes it has the identity value of current object. only foreign alg is capable of Collecting the identity value from associated object and assigns identity value of current object.

Test.java

```

package com.nareshit.client;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;
import com.nareshit.pojo.Address;
import com.nareshit.pojo.Person;
public class Test {
    public static void main(String[] args) {
        Configuration cfg=new Configuration();
        cfg.configure("/com/nareshit/config/hibernate.cfg.xml");
        SessionFactory factory=cfg.buildSessionFactory();
        Session session=factory.openSession();
        Transaction tx=session.beginTransaction();
        Person person=new Person();
        person.setName("ramu");
        person.setAge(23);
        person.setEmail("ramu@gmail.com");
        Address address=new Address();
        address.setH_No("2-31/a");
        address.setCity("hyd");
        address.setState("TS");
        address.setCountry("INDIA");
        address.setPerson(person);
        Integer id=(Integer)session.save(address);
        tx.commit();
        System.out.println("person created with Id :"+id);
    }
}

```

Example 2: One –to –One (pk-to-pk) BiDirectional XMLMappingExample

one-to-one(pk-to-pk)BiDirectionalXMLMappingExample

- src
 - com.nareshit.client
 - Test.java
 - com.nareshit.config
 - hibernate.cfg.xml
 - com.nareshit.mapping
 - Address.hbm.xml
 - Person.hbm.xml
 - com.nareshit.pojo
 - Address.java
 - Person.java

Person.java

```

package com.nareshit.pojo;
public class Person {
    private int personId,age;
    private String name,email;
    private Address address;

    public Address getAddress() {
        return address;
    }
    public void setAddress(Address address) {
        this.address = address;
    }
    public int getPersonId() {
        return personId;
    }
    public void setPersonId(int personId) {
        this.personId = personId;
    }
    public int getAge() {
        return age;
    }
    public void setAge(int age) {
        this.age = age;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {

```

```
        this.name = name;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
}
```

Address.java

```
package com.nareshit.pojo;
public class Address {
    private int personId;
    private String h_No,city,state,country;
    private Person person;
    public int getPersonId() {
        return personId;
    }
    public void setPersonId(int personId) {
        this.personId = personId;
    }
    public String getH_No() {
        return h_No;
    }
    public void setH_No(String h_No) {
        this.h_No = h_No;
    }
    public String getCity() {
        return city;
    }
    public void setCity(String city) {
        this.city = city;
    }
    public String getState() {
        return state;
    }
    public void setState(String state) {
        this.state = state;
    }
}
```

```
public String getCountry() {  
    return country;  
}  
public void setCountry(String country) {  
    this.country = country;  
}  
public Person getPerson() {  
    return person;  
}  
public void setPerson(Person person) {  
    this.person = person;  
}  
}
```

Person.hbm.xml

```
<!DOCTYPE hibernate-mapping PUBLIC  
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"  
    "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">  
<hibernate-mapping>  
  <class name="com.nareshit.pojo.Person" table="person">  
    <id name="personId">  
      <generator class="increment"/>  
    </id>  
    <property name="name" length="12"/>  
    <property name="age" length="3"/>  
    <property name="email" length="15"/>  
    <one-to-one name="address"  
      class="com.nareshit.pojo.Address"/>  
  </class>  
</hibernate-mapping>
```

Address.hbm.xml

```
<!DOCTYPE hibernate-mapping PUBLIC  
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"  
    "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">  
<hibernate-mapping>  
  <class name="com.nareshit.pojo.Address" table="address">  
    <id name="personId">  
      <generator class="foreign">  
        <param name="property">person</param>  
      </generator>  
    </id>  
  </class>  
</hibernate-mapping>
```

```

</id>
<property name="h_No" length="20"/>
<property name="city" length="20"/>
<property name="state" length="20"/>
<property name="country" length="20"/>
<one-to-one name="person" class="com.nareshit.pojo.Person" />
</class>
</hibernate-mapping>

```

hibernate.cfg.xml And Test.java files same as the

one-to-one(pk-pk)UniDirectionalXmlMappingExample

Example 3: One –to –One (pk - to-pk) Bidirectional Annotation Mapping Example

```

one-to-one(pk-to-pk)BiDirectionalAnnotationMappingExample
├── src
│   ├── com.nareshit.client
│   │   └── Test.java
│   ├── com.nareshit.config
│   │   └── hibernate.cfg.xml
│   ├── com.nareshit.pojo
│   │   ├── Address.java
│   │   └── Person.java
│   └── com.nareshit.utility
│       └── HibernateUtility.java

```

Person.java

```

package com.nareshit.pojo;
import javax.persistence.CascadeType;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.Id;
import javax.persistence.OneToOne;
import javax.persistence.PrimaryKeyJoinColumn;
import javax.persistence.Table;
import org.hibernate.annotations.GenericGenerator;
@Entity
@Table(name="person")
public class Person {
    @Id
    @GeneratedValue(generator="myGenerator")
    @GenericGenerator(name = "myGenerator", strategy = "increment")

```

```
@Column(length=12)
private int personId;
private int age;
@Column(length=20)
private String name,email;
@OneToOne(cascade=CascadeType.ALL)
@PrimaryKeyJoinColumn
private Address address;
public Address getAddress() {
    return address;
}
public void setAddress(Address address) {
    this.address = address;
}
public int getPersonId() {
    return personId;
}
public void setPersonId(int personId) {
    this.personId = personId;
}
public int getAge() {
    return age;
}
public void setAge(int age) {
    this.age = age;
}
public String getName() {
    return name;
}
public void setName(String name) {
    this.name = name;
}
public String getEmail() {
    return email;
}
public void setEmail(String email) {
    this.email = email;
}
}
```

Address.java

```
package com.nareshit.pojo;
import javax.persistence.*;
import org.hibernate.annotations.GenericGenerator;
import org.hibernate.annotations.Parameter;

@Entity
@Table(name="Address")
public class Address {
    @Id
    @GenericGenerator(name = "myGenerator",
        strategy = "foreign",
        parameters=@Parameter(name = "property",
            value = "person"))
    @GeneratedValue(generator="myGenerator")
    private int personId;
    @Column(length=20)
    private String h_No,city,state,country;
    @OneToOne(mappedBy = "address",cascade=CascadeType.ALL)
    private Person person;
    public int getPersonId() {
        return personId;
    }
    public void setPersonId(int personId) {
        this.personId = personId;
    }
    public String getH_No() {
        return h_No;
    }
    public void setH_No(String h_No) {
        this.h_No = h_No;
    }
    public String getCity() {
        return city;
    }
    public void setCity(String city) {
        this.city = city;
    }
    public String getState() {
        return state;
    }
}
```

```
public void setState(String state) {
    this.state = state;
}
public String getCountry() {
    return country;
}
public void setCountry(String country) {
    this.country = country;
}
public Person getPerson() {
    return person;
}
public void setPerson(Person person) {
    this.person = person;
}
}
```

hibernate.cfg.xml

```
<?xml version='1.0' encoding='UTF-8'?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
<session-factory>
<property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
<property name="connection.url">jdbc:oracle:thin:@localhost:1521:XE</property>
<property name="connection.username">system</property>
    <property name="connection.password">manager</property>
<property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
    <property name="hbm2ddl.auto">update</property>
    <property name="show_sql">>true</property>
    <mapping class="com.nareshit.pojo.Person"/>
<mapping class="com.nareshit.pojo.Address"/>
    </session-factory>
</hibernate-configuration>
```

HibernateUtility.java

```
package com.nareshit.utility;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
public class HibernateUtility {
private static SessionFactory factory;
static{
```



```
Configuration cfg=new Configuration();
cfg.configure("/com/nareshit/config/hibernate.cfg.xml");
factory=cfg.buildSessionFactory();
}
public static Session getSession(){
    Session session=null;
    if(factory!=null){
        session=factory.openSession();
    }
    return session;
}
}
```

Test.java

```
package com.nareshit.client;
import java.util.Iterator;
import java.util.List;

import org.hibernate.HibernateException;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;

import com.nareshit.pojo.Address;
import com.nareshit.pojo.Person;
import com.nareshit.utility.HibernateUtility;
public class Test {
    public static void main(String[] args) {
        Person person=new Person();
        person.setName("ramu");
        person.setAge(23);
        person.setEmail("ramu@gmail.com");
        Address address=new Address();
        address.setH_No("2-31/a");
        address.setCity("hyd");
        address.setState("TS");
        address.setCountry("INDIA");
        Test test=new Test();
        System.out.println("Testing of savePersonInfo(-,-)");
        Integer personId=test.savePersonInfo(person, address);
        System.out.println("Person saved : "+personId);
        System.out.println("Testing of listPersonInfo()");
        test.listPersonInfo();
    }
    public Integer savePersonInfo(Person person, Address address) {
```

```

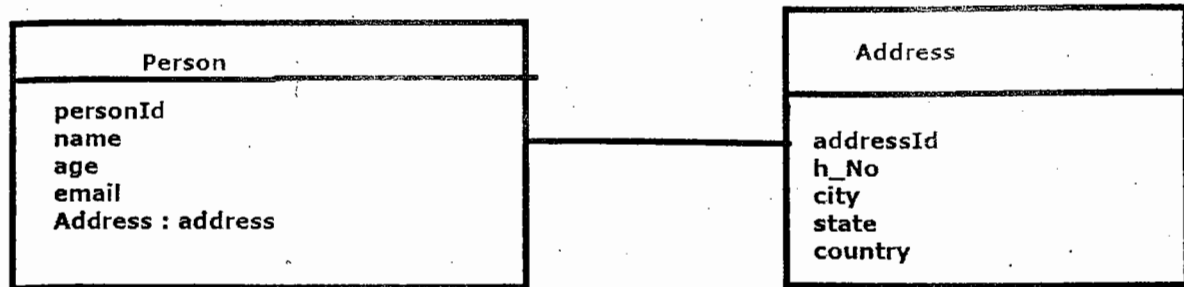
        Session session = HibernateUtility.getSession();
Integer personId = null;
Transaction transaction = null;
transaction = session.beginTransaction();
try {
    address.setPerson(person);
    personId=(Integer) session.save(address);
    transaction.commit();
} catch (HibernateException e) {
    transaction.rollback();
    e.printStackTrace();
} finally {
    session.close();
}
return personId;
}

public void listPersonInfo() {
    Session session = HibernateUtility.getSession();
    try {
        List<Person> personList = session.createQuery("from com.nareshit.pojo.Person").list();
        for (Iterator<Person> iterator = personList.iterator(); iterator
            .hasNext();) {
            Person person = (Person) iterator.next();
            System.out.println(person.getName());
            System.out.println(person.getEmail());
            System.out.println(person.getAddress().getH_No() + " "
                + person.getAddress().getCity() + " "
                + person.getAddress().getState());
        }
    } catch (HibernateException e) {
        e.printStackTrace();
    } finally {
        session.close();
    }
}
}

```

One-to-One Using foreign key association :-

1. In this association, a foreign key column is created in the owner entity. For example, if we make PERSON as owner entity, then a extra column "ADDRESS_ID" will be created in PERSON table. This column will store the foreign key for ADDRESS table. The relational model is shown below:



one -to- one relation class Diagram

Example Tables :

Person Table

personId (pk)	name	age	email	address_Id (fk)(unique)
1001	ramu	23	ramu@gmail.com	2001
1002	raju	22	raju@gmail.com	2002

Address Table

addressId (pk)	h_No	city	state	country
2001	2-31/a	hyd	TS	INDIA
2002	1-12/b	hyd	TS	INDIA

OnetoOne(fk(unique)-pk)AnnotationMappingExample1

- src
 - com.nareshit.client
 - Test.java
 - com.nareshit.config
 - hibernate.cfg.xml
 - com.nareshit.dao
 - PersonDao.java
 - com.nareshit.pojo
 - Address.java
 - Person.java
 - com.nareshit.utility
 - HibernateUtility.java

Person.java

```

package com.nareshit.pojo;
import javax.persistence.*;
import org.hibernate.annotations.GenericGenerator;

@Entity
@Table(name="person")
public class Person {
    @Id
    @GenericGenerator(name="myGenerator",strategy="increment")
    @GeneratedValue(generator="myGenerator")
    private int personId;
    @Column(length=12,name="name")
  
```

```
private String personName;
private byte age;
@Column(length=15,name="email")
private String email;
@OneToOne(cascade=CascadeType.ALL)
@JoinColumn(name="address_id",unique=true,nullable=false)
private Address address;
public Person(){
}
public Person(String personName, byte age, String email) {
    this.personName = personName;
    this.age = age;
    this.email = email;
}
public Address getAddress() {
    return address;
}
public void setAddress(Address address) {
    this.address = address;
}
public int getPersonId() {
    return personId;
}
public void setPersonId(int personId) {
    this.personId = personId;
}
public String getPersonName() {
    return personName;
}
public void setPersonName(String personName) {
    this.personName = personName;
}

public byte getAge() {
    return age;
}
public void setAge(byte age) {
    this.age = age;
}
public String getEmail() {
    return email;
}
public void setEmail(String email) {
    this.email = email;
}
}
```

Address.java

```
package com.nareshit.pojo;
import javax.persistence.*;
@Entity
@Table(name="Address")
public class Address {
    @Id
    private int addressId;
    @Column(length=15,name="h_No")
    private String h_No;
    @Column(length=20,name="city")
    private String city;
    @Column(length=20,name="state")
    private String state;
    @Column(length=20,name="country")
    private String country;
    public Address(){
        //zero-arg constructor
    }
    public Address(int addressId,String h_No,String city,String state,String country){
        this.addressId=addressId;
        this.h_No=h_No;
        this.city=city;
        this.state=state;
        this.country=country;
    }
    public int getAddressId() {
        return addressId;
    }
    public void setAddressId(int addressId) {
        this.addressId = addressId;
    }
    public String getH_No() {
        return h_No;
    }
    public void setH_No(String h_No) {
        this.h_No = h_No;
    }
    public String getCity() {
        return city;
    }
    public void setCity(String city) {
        this.city = city;
    }
    public String getState() {
        return state;
    }
```

```
}  
public void setState(String state) {  
    this.state = state;  
}  
public String getCountry() {  
    return country;  
}  
public void setCountry(String country) {  
    this.country = country;  
}}
```

PersonDao.java

```
/**  
 *  
 */  
package com.nareshit.dao;  
import org.hibernate.Session;  
import com.nareshit.pojo.Person;  
import com.nareshit.utility.HibernateUtility;  
  
/**  
 * @author Raja  
 */  
public class PersonDao {  
    public void createPerson(Person person){  
  
        Session session=HibernateUtility.getSession();  
        if(session!=null){  
            Integer personId=(Integer)session.save(person);  
            session.beginTransaction().commit();  
            System.out.println("person Id :"+personId);  
            session.close();  
        }  
    }  
  
    public void getPersonDetails(int personId){  
        Session session=HibernateUtility.getSession();  
        Person person=(Person) session.get(Person.class,personId);  
        System.out.println("Person Details");  
        System.out.println("Name :"+person.getPersonName());  
        System.out.println("Email :"+person.getEmail());  
    }  
}
```

```

System.out.println("Age :"+person.getAge());
System.out.println("City :"+person.getAddress().getCity());
System.out.println("State :"+person.getAddress().getState());
}
}

```

In one-to-one (fk- to -pk) association, refer the Address entity in Person class as follows:

```

1 @OneToOne
2 @JoinColumn(name="address_Id")
3 private Address address;

```

If no @JoinColumn is declared on the owner (Person) entity, the defaults apply. A join column(s) will be created in the owner entity table and its name will be the concatenation of the name of the relationship in the owner side, _ (underscore), and the name of the primary key column(s) in the owned side.

hibernate.cfg.xml

```

<!DOCTYPE hibernate-configuration PUBLIC
"-//Hibernate/Hibernate Configuration DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
<session-factory>
<property name="connection.driver_class">
oracle.jdbc.driver.OracleDriver</property>
<property name="connection.url">
jdbc:oracle:thin:@localhost:1521:XE</property>
<property name="connection.username">system</property>
<property name="connection.password">manager</property>
<property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
<property name="show_sql">true</property>
<property name="hbm2ddl.auto">update</property>
<mapping class="com.nareshit.pojo.Person"/>
<mapping class="com.nareshit.pojo.Address"/>
</session-factory>
</hibernate-configuration>

```

Test.java

```
/**
 *
 */

package com.nareshit.client;

import com.nareshit.dao.PersonDao;
import com.nareshit.pojo.Address;
import com.nareshit.pojo.Person;

/*
 * @author raja
 *
 */

public class Test {

    public static void main(String[] args){

        PersonDao personDao=new PersonDao();

        Person personDetails=
            new Person("ramu",new Byte((byte) 22),"ramu@gmail.com");
        Address address=new Address(2001,"2-31/a","hyd","TS","India");
        personDetails.setAddress(address);

        System.out.println("Testing of createPerson(-) ");

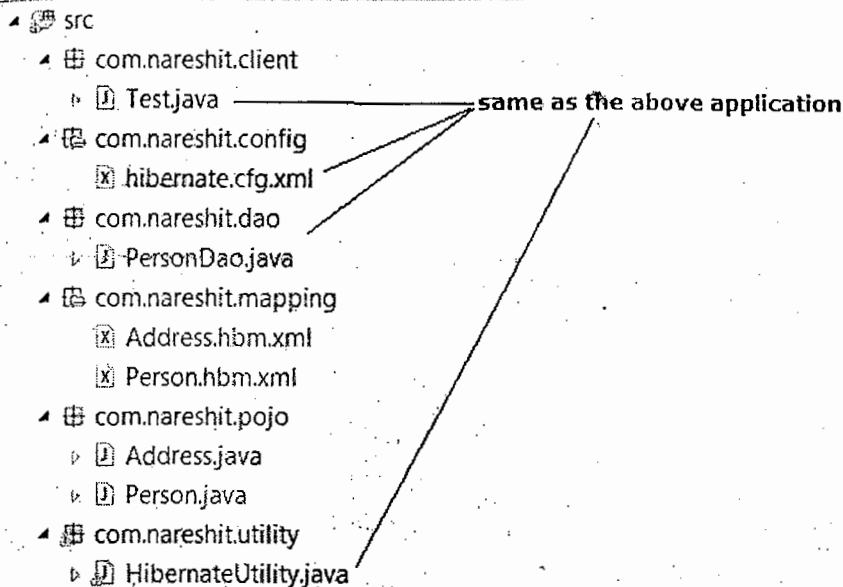
        personDao.createPerson(personDetails);

        System.out.println("Testing of getPersonDetails(1) : ");

        personDao.getPersonDetails(1);

    }

}
```


Example 2: one-to-one (fk-pk) Unidirectional XmlMappingExample**one-to-one (fk- to-pk) XmlMappingExample****Person.java**

```
package com.nareshit.pojo;
```

```
public class Person {
    private int personId;
    private String personName;
    private byte age;
    private String email;
    private Address address;
    public Person(){
    }
    public Person(String personName,byte age,String email){
        this.personName=personName;
        this.age=age;
        this.email=email;
    }
    public int getPersonId() {
        return personId;
    }
    public void setPersonId(int personId) {
        this.personId = personId;
    }
    public String getPersonName() {
        return personName;
    }
    public void setPersonName(String personName) {
        this.personName = personName;
    }
    public byte getAge() {
```

```
        return age;
    }
    public void setAge(byte age) {
        this.age = age;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
    public Address getAddress() {
        return address;
    }
    public void setAddress(Address address) {
        this.address = address;
    }
}
```

Address.java

```
package com.nareshit.pojo;
```

```
public class Address {
    private int addressId;
    private String h_No;
    private String city;
    private String state;
    private String country;
    public Address(){
        //zero-arg constructor
    }
    public Address(int addressId,String h_No,String city,String state,String country){
        this.addressId=addressId;
        this.h_No=h_No;
        this.city=city;
        this.state=state;
        this.country=country;
    }
    public int getAddressId() {
        return addressId;
    }
    public void setAddressId(int addressId) {
        this.addressId = addressId;
    }
    public String getH_No() {
        return h_No;
    }
}
```

```

public void setH_No(String h_No) {
    this.h_No = h_No;
}
public String getCity() {
    return city;
}
public void setCity(String city) {
    this.city = city;
}
public String getState() {
    return state;
}
public void setState(String state) {
    this.state = state;
}
public String getCountry() {
    return country;
}
public void setCountry(String country) {
    this.country = country;
}
}

```

Person.hbm.xml

```

<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
<class name="com.nareshit.pojo.Person"
    table="person">
<id name="personId">
<generator class="increment"></generator>
</id>
<property name="personName" column="name" length="12"/>
<property name="email" length="15"/>
<property name="age" length="3"/>
<many-to-one name="address" column="address_Id"
    unique="true" cascade="all" not-null="true"
    class="com.nareshit.pojo.Address"/>
</class>
</hibernate-mapping>

```

In the above mapping file

The **<many-to-one name="address" class="com.nareshit.pojo.Address" column="address_Id" unique="true" cascade="all" />** is the tag in the above hbm xml that configures the One To One

relation from Person to Address.

The column attribute describes the column name in the table mapping to this entity which is a reference key referring to the ID of the other entity. That is in simple the FK column name.

Address.hbm.xml

```
<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
<class name="com.nareshit.pojo.Address"
table="address">
<id name="addressId">
</id>
<property name="h_No" length="15"/>
<property name="city" length="15"/>
<property name="state" length="15"/>
<property name="country" length="15"/>
</class>
</hibernate-mapping>
```

Many-to-one:-

This type of Association relates many objects of an Entity to one Object of another Entity. An Example of this type of association is the relationship b/w Employee and a Department.

For Example Multiple Employee are associated with one Dept.

To map many to one relation we can use <many-to-one> element in hibernate mapping file of Employee persistence class.

Example in Employee mapping file:

```
<many-to-one name="dept"
class="com.nareshit.pojo.Department"
column="dept_no"/>
```

column attribute specifies the foreign key from the declaring to the referring table.

Employee_Details

empNo (pk)	name	salary	dept_No (fk)
101	sathish	15000.0	12
102	ramesh	16000.0	12
103	raju	14000.0	13
104	ramu	14500.0	13
105	pavan	15500.0	12

Department_Details

deptNo (pk)	deptName	location
12	Sales	hyd
13	Admin	hyd

Many-to-OneXMLMappingExample1

src

- com.nareshit.client
 - Test.java
- com.nareshit.config
 - hibernate.cfg.xml
- com.nareshit.dao
 - DepartmentDAO.java
 - EmployeeDAO.java
- com.nareshit.mapping
 - Department.hbm.xml
 - Employee.hbm.xml
- com.nareshit.pojo
 - Department.java
 - Employee.java
- com.nareshit.utility
 - HibernateUtility.java

Employee.java

```

package com.nareshit.pojo;
public class Employee {
    private int empNo;
    private String name;
    private double salary;
    private Department dept;
    public Employee(){
    }
    public Employee(String name,double salary){
        this.name=name;
        this.salary=salary;
    }
    public int getEmpNo() {
        return empNo;
    }
}

```

```
public void setEmpNo(int empNo) {
    this.empNo = empNo;
}
public String getName() {
    return name;
}
public void setName(String name) {
    this.name = name;
}
public double getSalary() {
    return salary;
}
public void setSalary(double salary) {
    this.salary = salary;
}
public Department getDept() {
    return dept;
}
public void setDept(Department dept) {
    this.dept = dept;
}
}
```

Department.java

```
package com.nareshit.pojo;
public class Department {
    private int deptNo;
    private String deptName, location;
    public Department(){
    }
    public Department(int deptNo, String deptName, String location){
        this.deptNo=deptNo;
        this.deptName=deptName;
        this.location=location;
    }
    public int getDeptNo() {
        return deptNo;
    }
    public void setDeptNo(int deptNo) {
        this.deptNo = deptNo;
    }
    public String getDeptName() {
        return deptName;
    }
}
```

```
public void setDeptName(String deptName) {  
    this.deptName = deptName;  
}  
public String getLocation() {  
    return location;  
}  
public void setLocation(String location) {  
    this.location = location;  
}  
}
```

Employee.hbm.xml

```
<!DOCTYPE hibernate-mapping PUBLIC  
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"  
    "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">  
<hibernate-mapping>  
    <class name="com.nareshit.pojo.Employee"  
        table="Employee_Details">  
        <id name="empNo" length="12">  
            <generator class="sequence">  
                <param name="sequence">EmpNo_Sequence</param>  
            </generator>  
        </id>  
        <property name="name" length="15"/>  
        <property name="salary" length="7"/>  
        <many-to-one name="dept"  
            class="com.nareshit.pojo.Department"  
            column="dept_no" not-null="true" cascade="all"/>  
        </class>  
    </hibernate-mapping>
```

Department.hbm.xml

```
<!DOCTYPE hibernate-mapping PUBLIC  
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"  
    "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">  
<hibernate-mapping>  
    <class name="com.nareshit.pojo.Department"  
        table="Department_Details">  
        <id name="deptNo">  
            </id>  
        <property name="deptName" length="15"/>
```

```
<property name="location" length="20"/>
</class>
</hibernate-mapping>
```

EmployeeDAO.java

```
package com.nareshit.dao;

import org.hibernate.Session;

import com.nareshit.pojo.Employee;
import com.nareshit.utility.HibernateUtility;

public class EmployeeDAO{

    public void createEmployee(Employee emp){
        Session session=HibernateUtility.getSession();
        session.save(emp);
        session.beginTransaction().commit();
    }

    public void getEmployeeDetails(int empNo){
        Session session=HibernateUtility.getSession();
        Employee employee=(Employee)session.get(Employee.class,empNo);
        System.out.println("Employee Details");
        System.out.println("=====");
        System.out.println("Emp No :"+employee.getEmpNo());
        System.out.println("Emp Name :"+employee.getName());
        System.out.println("Salary :"+employee.getSalary());
        System.out.println("DeptNo :"+employee.getDept().getDeptNo());
        System.out.println("DeptName :"+employee.getDept().getDeptName());
        System.out.println("DeptLocation :"+employee.getDept().getLocation());

    }

}
```

DepartmentDAO.java

```
package com.nareshit.dao;

import org.hibernate.Session;

import com.nareshit.pojo.Department;
import com.nareshit.utility.HibernateUtility;

public class DepartmentDAO {

    public void createDepartment(Department dept){
        Session session=HibernateUtility.getSession();
```



```
session.save(dept);
session.beginTransaction().commit();
}
}
```

Test.java

```
package com.nareshit.client;
import java.util.List;
import com.nareshit.dao.DepartmentDAO;
import com.nareshit.dao.EmployeeDAO;
import com.nareshit.pojo.Department;
import com.nareshit.pojo.Employee;
public class Test{
    public static void main(String[] args){
        EmployeeDAO empDao=new EmployeeDAO();
        DepartmentDAO deptDao=new DepartmentDAO();
        System.out.println("Testing of createDepartment(-)");
        Department salesDept=new Department(12,"Sales","hyd");
        deptDao.createDepartment(salesDept);
        Department adminDept=new Department(13,"Admin","hyd");
        deptDao.createDepartment(adminDept);
        System.out.println("Testing of createEmployee(-)");
        Employee emp1=new Employee("raja",15000.0);
        emp1.setDept(salesDept);
        empDao.createEmployee(emp1);
        Employee emp2=new Employee("ramesh",16000.0);
        emp2.setDept(salesDept);
        empDao.createEmployee(emp2);
        Employee emp3=new Employee("raju",14000.0);
        emp3.setDept(adminDept);
        empDao.createEmployee(emp3);
        Employee emp4=new Employee("ramu",14500.0);
        emp4.setDept(adminDept);
        empDao.createEmployee(emp4);
        Employee emp5=new Employee("pavan",15500.0);
        emp5.setDept(salesDept);
        empDao.createEmployee(emp5);
        System.out.println("Testing of getEmployeeDetails(-)");
        empDao.getEmployeeDetails(101);
    }
}
```

Example 2: Many-to-One Annotation Mapping Example

- ManytoOne Annotation Mapping Example

- src

-  com.nareshit.client
 -  Test.java
 -  com.nareshit.config
 -  hibernate.cfg.xml
 -  com.nareshit.dao
 -  DepartmentDAO.java
 -  EmployeeDAO.java
 -  com.nareshit.pojo
 -  Department.java
 -  Employee.java
 -  com.nareshit.utility
 -  HibernateUtility.java

Employee.java

```

package com.nareshit.pojo;
import javax.persistence.CascadeType;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.Id;
import javax.persistence.JoinColumn;
import javax.persistence.ManyToOne;
import javax.persistence.Table;

import org.hibernate.annotations.GenericGenerator;
@Entity
@Table(name="Employee_Details")
public class Employee {
    @Id
    @GeneratedValue(generator="myGenerator")
    @GenericGenerator(name="myGenerator",strategy="increment")
    private int empNo;
    @Column(length=12)
    private String name;
    private double salary;
    @ManyToOne(cascade=CascadeType.ALL)
    @JoinColumn(name="dept_no",nullable=false)
    private Department dept;
    public Employee(){
    }
    public Employee(String name,double salary){
        this.name=name;
        this.salary=salary;
    }
}

```

```
public int getEmpNo() {
    return empNo;
}
public void setEmpNo(int empNo) {
    this.empNo = empNo;
}
public String getName() {
    return name;
}
public void setName(String name) {
    this.name = name;
}
public double getSalary() {
    return salary;
}
public void setSalary(double salary) {
    this.salary = salary;
}
public Department getDept() {
    return dept;
}
public void setDept(Department dept) {
    this.dept = dept;
}
}
```

Note :

The @ManyToOne annotation is used to create the many-to-one relationship between the Employee and Department entities. The cascade option is used to cascade the required operations to the associated entity. If the cascade option is set to CascadeType.ALL then all the operations will be cascaded. For instance when you save a Employee object, the associated Department object will also be saved automatically.

Department.java

```
package com.nareshit.pojo;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;
@Entity
@Table(name="department")
public class Department {
    @Id
    private int deptNo;
    @Column(length=12)
    private String deptName;
```

```
@Column(length=12,name="location")
private String loc;
public Department(int deptNo,String deptName,String loc){
    this.deptName=deptName;
    this.loc=loc;
    this.deptNo=deptNo;
}
public Department(){
}
public int getDeptNo() {
    return deptNo;
}
public void setDeptNo(int deptNo) {
    this.deptNo = deptNo;
}
public String getDeptName() {
    return deptName;
}
public void setDeptName(String deptName) {
    this.deptName = deptName;
}
public String getLoc() {
    return loc;
}
public void setLoc(String loc) {
    this.loc = loc;
}
}
```

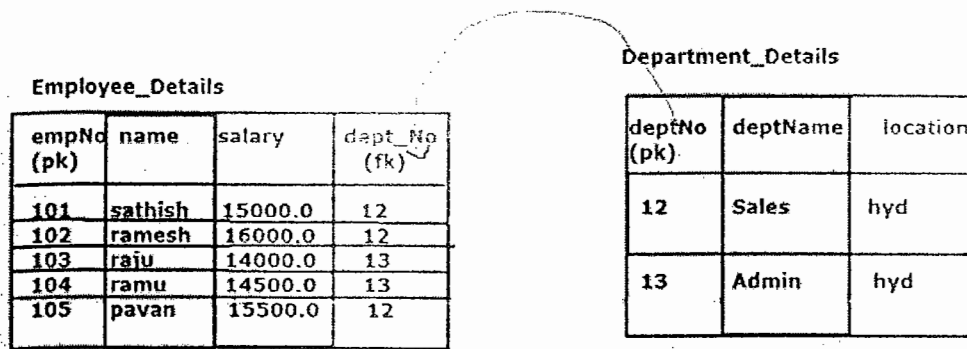
Note :- Remaining files are same as the above Many-to-one XmlMapping Example

One-To-Many:-

This type of association relates one entity object to many object's of another entity.

An Example of this type of association is the relationship

Between a department and the collection of Employees.



One-to-Many(pk-->fk)

Many-to-one (fk-->pk)

Employee_Details and Department_Details table contains One-to-many relationship. Each Department can be associated with multiple Employees and each Employee can have **only one** Department.

One-To-Many Relationship: A relationship in which each record in one table is **linked to multiple** records in another table

One-to-Many relation can be developed in four ways

1. Set
2. List
3. Bag
4. Array
5. Map

Bag

A <bag> is an unordered collection, which can contain duplicated elements. That means if you persist a bag with some order of elements, you cannot expect the same order retains when the collection is retrieved. There is not a "bag" concept in Java collections framework, so we just use a java.util.List to correspond to a <bag>.

A table BOOK_CHAPTER will be used to persist the following <bag> collection. It contains two columns, BOOK_ID and CHAPTER. Each element will be stored as a row in this table.

```
public class Book {  
    ...  
    private List chapters;  
}  
  
//hibernate mapping file  
<hibernate-mapping package="com.nareshit.hibernatemapping  
.bookshop">  
    <class name="Book" table="BOOK">  
        ...  
        <bag name="chapters" table="BOOK_CHAPTER">  
            <key column="BOOK_ID" />  
            <element column="CHAPTER" type="string" length="100" />  
        </bag>  
    </class>  
</hibernate-mapping>
```

Set

A `<set>` is very similar to a `<bag>`. The only difference is that set can only store unique objects. That means no duplicated elements can be contained in a set. When you add the same element to a set for second time, it will replace the old one. A set is unordered by default but we can ask it to be sorted, which we will discuss later. The corresponding type of a `<set>` in Java is `java.util.Set`. The table structure of a `<set>` is the same as a `<bag>`, i.e. contains two columns, `BOOK_ID` and `CHAPTER`.

```
public class Book {  
    ...  
    private Set chapters;  
  
    //hibernate mapping file  
    <hibernate-mapping package="com. nareshit.hibernatemapping.bookshop">  
        <class name="Book" table="BOOK">  
            ...  
            <set name="chapters" table="BOOK_CHAPTER">  
                <key column="BOOK_ID" />+  
  
                <element column="CHAPTER" type="string" length="100" />  
            </set>
```

List

A `<list>` is an indexed collection where the index will also be persisted. That means we can retain the order of the list when it is retrieved. It differs from a `<bag>` for it persists the element index while a `<bag>` does not. The corresponding type of a `<list>` in Java is `java.util.List`. The table structure for a `<list>` has one more column, `CHAPTER_INDEX`, than a `<bag>`. It is used for storing the element index. The type of this column is `int`.

```

public class Book {
...
private List chapters;
}
<hibernate-mapping package="com.
nareshit.hibernatemapping.bookshop">
<class name="Book" table="BOOK">
...
<list name="chapters" table="BOOK_CHAPTER">
<key column="BOOK_ID" />
<list-index column="CHAPTER_INDEX"/>
<element column="CHAPTER" type="string" length="100" />
</list>
</class>
</hibernate-mapping>

```

Array

An `<array>` has the same usage as a `<list>`. The only difference is that it corresponds to an array type in Java, not a `java.util.List`. It is seldom used unless we are mapping for legacy applications. In most cases, we should use `<list>` instead. That is because the size of an array cannot be increased or decreased dynamically, where a list can.

The table structure of an `<array>` is the same as a `<list>`, i.e. contains three columns, `BOOK_ID`, `CHAPTER` and `CHAPTER_INDEX`.

```

public class Book {
...
private String[] chapters;
}
<hibernate-mapping package="com. nareshit.hibernatemapping.bookshop">
<class name="Book" table="BOOK">
...
<array name="chapters" table="BOOK_CHAPTER">
<key column="BOOK_ID" />
<list-index column="CHAPTER_INDEX"/>
<element column="CHAPTER" type="string" length="100" />
</array>
</class>
</hibernate-mapping>

```

Map

A `<map>` is very similar to a `<list>`. The difference is that a map uses arbitrary keys to index the collection, not an integer index using in a list. A map stores its entries in key/value pairs. You can lookup the value by its key. The key of a map can be of any data types. The corresponding type of a `<map>` in Java is `java.util.Map`.

The table structure of the following `<map>` contains three columns, `BOOK_ID`, `CHAPTER` and

CHAPTER_KEY. Here we choose string as the data type of our key column.

```
public class Book {
...
private Map chapters;
}
<hibernate-mapping package="com. nareshit.hibernatemapping.bookshop">
<class name="Book" table="BOOK">
...
<map name="chapters" table="BOOK_CHAPTER">
<key column="BOOK_ID" />
<map-key column="CHAPTER_KEY" type="string" />
<element column="CHAPTER" type="string" length="100" />
</map>
</class>
</hibernate-mapping>
```

OneToManyAndManyToOneXMLMapping

- src
 - com.nareshit.client
 - Test.java
 - com.nareshit.config
 - hibernate.cfg.xml
 - com.nareshit.mapping
 - Department.hbm.xml
 - Employee.hbm.xml
 - com.nareshit.pojo
 - Department.java
 - Employee.java
 - com.nareshit.util.dao
 - DepartmentDAO.java
 - EmployeeDAO.java
 - com.nareshit.utility
 - HibernateUtility.java

Employee.java

```
package com.nareshit.pojo;
public class Employee {
private int empNo;
private String name;
private double salary;
private Department dept;
public Employee(){
}
public Employee(String name,double salary){
this.name=name;
this.salary=salary;
```



```
}  
public int getEmpNo() {  
    return empNo;  
}  
public void setEmpNo(int empNo) {  
    this.empNo = empNo;  
}  
public String getName() {  
    return name;  
}  
public void setName(String name) {  
    this.name = name;  
}  
public double getSalary() {  
    return salary;  
}  
public void setSalary(double salary) {  
    this.salary = salary;  
}  
public Department getDept() {  
    return dept;  
}  
public void setDept(Department dept) {  
    this.dept = dept;  
}  
}
```

Department.java

```
package com.nareshit.pojo;  
  
import java.util.List;  
import java.util.Set;  
  
public class Department {  
    private int deptNo;  
    private String deptName, loc;  
    private Set<Employee> employees;  
    public Department(int deptNo, String deptName, String loc) {  
        this.deptName = deptName;  
        this.loc = loc;  
        this.deptNo = deptNo;  
    }  
    public Department() {  
    }  
    public int getDeptNo() {  
        return deptNo;  
    }  
    public void setDeptNo(int deptNo) {
```

```

        this.deptNo = deptNo;
    }
    public String getDeptName() {
        return deptName;
    }
    public void setDeptName(String deptName) {
        this.deptName = deptName;
    }
    public String getLoc() {
        return loc;
    }
    public void setLoc(String loc) {
        this.loc = loc;
    }
    public Set<Employee> getEmployees() {
        return employees;
    }
    public void setEmployees(Set<Employee> employees) {
        this.employees = employees;
    }
}

```

Employee.hbm.xml

```

<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
<class name="com.nareshit.pojo.Employee" table="Employee_Details">
<id name="empNo" column="empNo">
<generator class="increment"/>
</id>
<property name="name" length="12"/>
<property name="salary" />
<many-to-one name="dept" class="com.nareshit.pojo.Department"
column="dept_no" not-null="true"/>
</class>
</hibernate-mapping>

```

Department.hbm.xml

```

<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
    <class name="com.nareshit.pojo.Department" table="Department_details">

```

```

        <id name="deptNo">
        </id>
        <property name="deptName" length="12" />
        <property name="loc" column="location" length="12" />
        <set name="employees" table="Employee_Details"
        inverse="true" lazy="true" fetch="select">
<key>
<column name="dept_no" not-null="true" />
</key>
        <one-to-many class="com.nareshit.pojo.Employee" />
    </set>
</class>
</hibernate-mapping>

```

EmployeeDAO.java

```

package com.nareshit.util.dao;
import java.util.List;
import org.hibernate.Query;
import org.hibernate.Session;
import com.nareshit.pojo.Employee;
import com.nareshit.utility.HibernateUtility;
public class EmployeeDAO{
    public void createEmployee(Employee emp){
        Session session=HibernateUtility.getSession();
        session.save(emp);
        session.beginTransaction().commit();
    }
    public Employee getEmployeeById(int empNo){
        Employee emp=null;
        Session session=HibernateUtility.getSession();
        emp=(Employee)session.get(Employee.class,empNo);
        return emp;
    }
    public List<Employee> getEmployeesByName(String name){
        List<Employee> list=null;
        Session session=HibernateUtility.getSession();
        String hql=
        "from com.nareshit.pojo.Employee as e where e.name like ?";
        Query query=session.createQuery(hql);
        query.setParameter(0,"%"+name+"%");
        list=query.list();
    }
}

```

```
return list;
```

DepartmentDAO.java

```
package com.nareshit.util.dao;
import java.util.List;
import org.hibernate.Query;
import org.hibernate.Session;
import com.nareshit.pojo.Department;
import com.nareshit.utility.HibernateUtility;
public class DepartmentDAO {
    public void createDepartment(Department dept){
        Session session=HibernateUtility.getSession();
        session.save(dept);
        session.beginTransaction().commit();
    }
    public Department getEmployeesByDeptName(String deptName){
        Department dept=null;
        Session session=HibernateUtility.getSession();
        String hql=
            "from com.nareshit.pojo.Department as d where d.deptName like ?";
        Query query=session.createQuery(hql);
        query.setParameter(0,deptName);
        List<Department> list=query.list();
        if(list.size()>0){
            dept= list.get(0);
        }
        return dept;
    }
}
```

hibernate.cfg.xml

```
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
<session-factory>
<property name="connection.driver_class">
oracle.jdbc.driver.OracleDriver</property>
<property name="connection.url">
jdbc:oracle:thin:@localhost:1521:XE</property>
<property name="connection.username">system</property>
```

```
<property name="connection.password">manager</property>
<property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
<property name="show_sql">true</property>
<property name="hbm2ddl.auto">create</property>
<mapping resource="com/nareshit/mapping/Employee.hbm.xml"/>
<mapping resource="com/nareshit/mapping/Department.hbm.xml"/>
</session-factory>
</hibernate-configuration>
```

Test.java

```
package com.nareshit.client;
import java.util.List;
import java.util.Set;
import com.nareshit.pojo.Department;
import com.nareshit.pojo.Employee;
import com.nareshit.util.dao.DepartmentDAO;
import com.nareshit.util.dao.EmployeeDAO;
/*
 * @author raja
 *
 */
public class Test {
    public static void main(String[] args){
        EmployeeDAO empDao=new EmployeeDAO();
        DepartmentDAO deptDao=new DepartmentDAO();
        System.out.println("create the departments ");

        Department salesDept=new Department(12,"Sales","hyd");
        deptDao.createDepartment(salesDept);

        Department adminDept=new Department(13,"Admin","hyd");
        deptDao.createDepartment(adminDept);

        System.out.println("create the employees");

        Employee emp1=new Employee("raja",15000.0);
        emp1.setDept(salesDept);
        empDao.createEmployee(emp1);

        Employee emp2=new Employee("ramesh",16000.0);
        emp2.setDept(salesDept);
        empDao.createEmployee(emp2);

        Employee emp3=new Employee("raju",14000.0);
        emp3.setDept(adminDept);
        empDao.createEmployee(emp3);
```

```
Employee emp4=new Employee("ramu",14500.0);
emp4.setDept(adminDept);
empDao.createEmployee(emp4);
```

```
Employee emp5=new Employee("pavan",15500.0);
emp5.setDept(salesDept);
empDao.createEmployee(emp5);
```

```
System.out.println("search Employee's By Id");
int empId=2;
Employee emp=empDao.getEmployeesById(empId);
//check whether the emp is null (OR) not
System.out.println("emp No : "+empId+" Details");
    System.out.println("Name :"+emp.getName());
    System.out.println("Salary :"+emp.getSalary());
    System.out.println("dept No :"+emp.getDept().getDeptNo());
    System.out.println("dept Name :"+emp.getDept().getDeptName());
System.out.println("Search Employee's By Name : ");
String name="aj";
List<Employee> list=empDao.getEmployeesByName(name);
    for(Employee empl:list){
        System.out.println("emp No : "+empl.getEmpNo()+" Details ");
        System.out.println("Name :"+empl.getName());
        System.out.println("Salary :"+empl.getSalary());

        System.out.println("dept No :"+empl.getDept().getDeptNo());
        System.out.println("dept Name :"+empl.getDept().getDeptName());
    }

System.out.println("Searching the Employee's By Dept Name :");
    String deptName="Sales";
Department dept1=deptDao.getEmployeesByDeptName(deptName);
System.out.println("Dept Name :"+dept1.getDeptName()+" : deptNo : "+dept1.getDeptNo());
Set<Employee> set= dept1.getEmployees();
for(Employee e:set){
    System.out.println("emp No : "+e.getEmpNo()+" Details ");
    System.out.println("Name :"+e.getName());
    System.out.println("Salary :"+e.getSalary());
}
}}
```

One-to-Many <bag> example

A <bag> is an unordered collection, which can contain duplicated elements. That means if you persist a bag with some order of elements, you cannot expect the same order retains when the collection is retrieved. There is not a "bag" concept in Java collections framework, so we just use a

java.util.List correspond to a <bag>.

To implement Bag in our one-to-many mapping example, we will do following changes:

Update Department Model Class

Department.java

Update Department.hbm.xml mapping file :-

```
Package com.nareshit.pojo;
import java.util.ArrayList;
import java.util.List;
public class Department {
    private int deptNo;
    private String deptName,loc;
    private List<Employee> employees;
    // Getter and Setter methods
}
```

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
<class name="com.nareshit.pojo.Department" table="Department_Details">
<id name="deptNo" column="deptNo" ></id>
<property name="deptName" column="DeptName"/>
<bag name="employees" table="employee_details" inverse="true" lazy="true" fetch="select">
<key><column name="dept_no" not-null="true" />
</key>
<one-to-many class="com.nareshit.pojo.Employee" />
</bag>
</class>
</hibernate-mapping>
Execute <bag> example
```

Execute the same Test.java file that we created in above example.

One-to-Many <list> example

A <list> is an indexed collection where the index will also be persisted. That means we can retain the order of the list when it is retrieved. It differs from <bag> for it persists the element Index while a <bag> does not. The corresponding type of a <list> in Java is java.util.List.

To implement List in our one-to-many mapping example, we will do following changes:

Update Department Model Class

Department.java

```
Package com.nareshit.pojo;
import java.util.ArrayList;
import java.util.List;
public class Department {
    private int deptNo;
    private String deptName, loc;
    private List<Employee> employees;
    // Getter and Setter methods
}
```

Update Department.hbm.xml mapping file :-

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
<hibernate-mapping >
<class name=" com.nareshit.pojo.Department" table="Department_Details">
<id name="deptNo" column="deptNo" ></id>
<property name="deptName" column="DeptName"/>
<list name="employees" table="employee_details"
    inverse="false" cascade="all">
<key column="dept_no" />
<list-index column="idx" />
<one-to-many class="com.nareshit.pojo.Employee" />
</list>
</class>

</hibernate-mapping>
```

In the above hibernate mapping xml file, note that we have added list tag to map a list of employees with department. Also a new index column "idx" is defined which will store the index of records. Note that inverse="false" is specified in the mapping which makes Department as the relationship owner. Thus when department object is saved, it automatically saves the Employees too. This is required so that Department can manage the index values for employees. The dept_no is given as key column.

Execute <List> example

Execute the same Test.java file that we created in above example.

One to Many relationship using Annotation mapping**Remove Hibernate Mapping (hbm) Files**

We are not going to use hibernate mapping files or hbm files as we will map the model using Java 5 Annotations. Delete the files employee.hbm.xml and department.hbm.xml in the above xml mapping example

Update Employee and Department Model Class as follows

Employee.java

```
package com.nareshit.pojo;

@Entity
@Table(name="Employee_Details")
public class Employee {
    @Id
    @GeneratedValue(generator="myGenerator")
    @GenericGenerator(name="myGenerator",strategy="increment")
    private int empNo;
    @Column(length=12)
    private String name;
    private double salary;
    @ManyToOne
    @JoinColumn(name="dept_no")
    private Department dept;

    //getters and setters
}
```

Department.java

```
package com.nareshit.pojo;

@Entity
@Table(name="department_details")
public class Department {
    @Id
    private int deptNo;
    @Column(length=12)
    private String deptName;
    @Column(length=12,name="location")
    private String loc;
    @OneToMany(mappedBy="dept")
    private Set<Employee> employees;

    //getters and setters
}
```

@OneToMany annotation defines a many-valued association with one-to-many multiplicity. If the collection is defined using generics to specify the element type, the associated target entity type need not be specified; otherwise the target entity class must be specified.

The association may be bidirectional. In a bidirectional relationship, one of the sides (and only one) has to be the owner: the owner is responsible for the association column(s) update. To declare a side as not responsible for the relationship, the attribute mappedBy is used. mappedBy refers to the property name of the association on the owner side.. As you can see, you don't have to (must not) declare the join column since it has already been declared on the owners side.

Inverse :

inverse keyword is responsible for managing the insert / update of the foreign key column.

An inverse keyword has the boolean value "true/false". The Default value of this keyword is 'false'

if the inverse keyword value is false, parent class is responsible for saving/updating the child and it's relationship.

If the inverse keyword value is true, an associated subclass is responsible for saving/updating itself.

Note :-

An inverse keyword is always used with the one-to-many and many-to-many.

It doesn't work with many-to-one relationship.

Many-to-Many:-

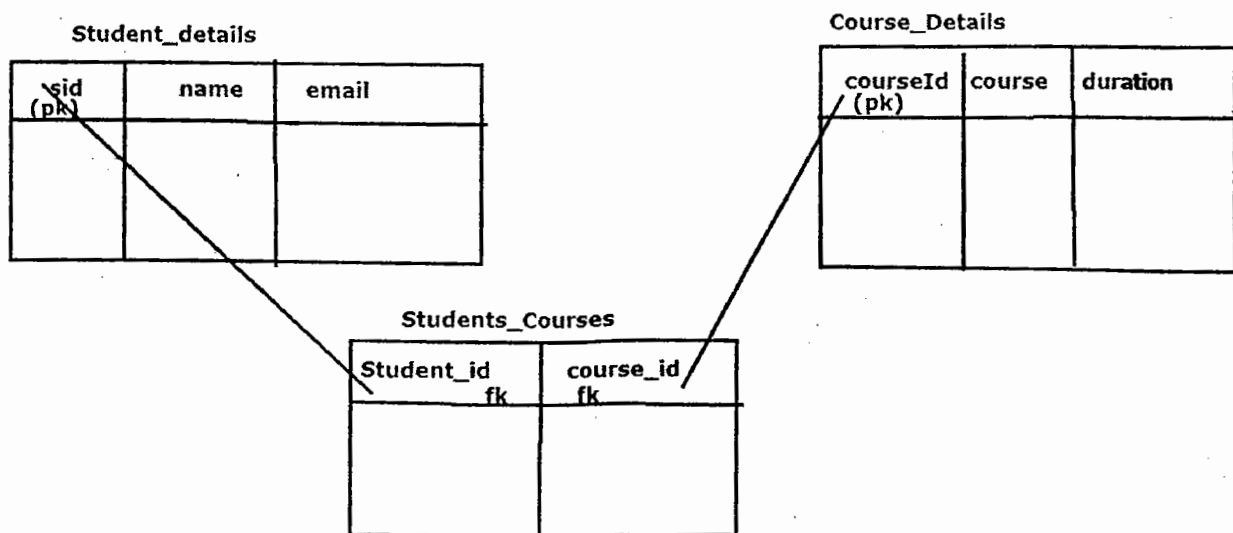
This type of relation relates many objects of an Entity to many object's of another Entity.

For Example:-

A relation ship between doctors and patients, one doctor may contains many no.of patients. one patient may consult(contains) many no.of doctors.

A relation ship between students and courses. there may be chance many student's registered for one course.(or) a student can register for multiple courses.

This Relation ship is possible only in Bidirectional. In this relation ship parent contains one (or) more No.of child obj's and child contains one (or) more No.of parent objects.



To define a many-to-many relation-ship in the database environment we can use link table(also known as collection table)

In the following example student entity maps to Student_Details table and course entity maps to Course_Details table. We do not have any entity class for the link table (in this case students_courses).

The students_courses table declares a foreign key column named as student_id referring the student_details table, and another fk column named as course_id referring the course_details table.

Many-To-Many Mapping Example (XML Mapping)

ManytoManyBidirectionalExample

src

- com.nareshit.client
 - Test.java
- com.nareshit.mapping
 - Course.hbm.xml
 - Student.hbm.xml
- com.nareshit.pojo
 - Course.java
 - Student.java
- hibernate.cfg.xml

Student.java

```
package com.nareshit.pojo;
```

```
import java.util.Set;
```

```
public class Student {
    private int sid;
    private String name,email;
    private Set<Course> courses;
    public Student(){
    }
    public Student(int sid,String name,String email){
        this.sid=sid;
        this.name=name;
        this.email=email;
    }

    public int getSid() {
        return sid;
    }

    public void setSid(int sid) {
        this.sid = sid;
    }
}
```

```
public String getName() {
    return name;
}
public void setName(String name) {
    this.name = name;
}
public String getEmail() {
    return email;
}
public void setEmail(String email) {
    this.email = email;
}
public Set<Course> getCourses() {
    return courses;
}
public void setCourses(Set<Course> courses) {
    this.courses = courses;
}
}
```

Course.java

```
package com.nareshit.pojo;
```

```
import java.util.Set;
```

```
public class Course {
    private int courseId;
    private String courseName,duration;
    private Set<Student> students;
    public Course(){
    }
    public Course(int courseId,String courseName,String duration){
        this.courseId=courseId;
        this.courseName=courseName;
        this.duration=duration;
    }

    public int getCourseId() {
        return courseId;
    }
    public void setCourseId(int courseId) {
        this.courseId = courseId;
    }
    public String getCourseName() {
        return courseName;
    }
}
```

```
public void setCourseName(String courseName) {
    this.courseName = courseName;
}
public String getDuration() {
    return duration;
}
public void setDuration(String duration) {
    this.duration = duration;
}
public Set<Student> getStudents() {
    return students;
}
public void setStudents(Set<Student> students) {
    this.students = students;
}}
```

Student.hbm.xml

```
<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
<class name="com.nareshit.pojo.Student"
    table="student_details">
<id name="sid">
<generator class="assigned"/>
</id>
<property name="name" length="12"/>
<property name="email" length="20"/>
<set name="courses" table="Students_Courses" cascade="all">
<key column="student_id"/>
<many-to-many class="com.nareshit.pojo.Course"
    column="course_id"/>
</set>
</class>
</hibernate-mapping>
```

Course.hbm.xml

```
<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
<class name="com.nareshit.pojo.Course"
    table="course_details">
<id name="courseId">
<generator class="assigned"/>
</id>
```

```

<property name="courseName" length="15"/>
<property name="duration" length="15"/>
<set name="students" table="Students_Courses" cascade="all">
<key column="course_id"/>
<many-to-many class="com.nareshit.pojo.Student"
column="student_id"/>
</set>
</class>
</hibernate-mapping>

```

hibernate.cfg.xml

```

<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
<session-factory>
<property name="connection.driver_class">
oracle.jdbc.driver.OracleDriver</property>
<property name="connection.url">
jdbc:oracle:thin:@localhost:1521:XE</property>
<property name="connection.username">system</property>
<property name="connection.password">manager</property>
<property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
<property name="show_sql">true</property>
<property name="hbm2ddl.auto">update</property>
<mapping resource="com/nareshit/mapping/Student.hbm.xml"/>
<mapping resource="com/nareshit/mapping/Course.hbm.xml"/>
</session-factory>
</hibernate-configuration>

```

Test.java

```

package com.nareshit.client;
public class Test {
public static void main(String[] args) {
SessionFactory factory=new Configuration().configure().buildSessionFactory();
    Session session=factory.openSession();
String hql="from com.nareshit.pojo.Course as c where c.courseName=?";
Query query=session.createQuery(hql);
query.setParameter(0,"java");
List<Course> list=query.list();
//check list is empty (OR) not
Course course=list.get(0);
System.out.println("Course Name :"+course.getCourseName());

```

```
Set<Student> setOfStudents=course.getStudents();
for(Student s:setOfStudents){
    System.out.println("Sid :"+s.getSid());
    System.out.println("Name :"+s.getName());
    System.out.println("Email :"+s.getEmail());
}
}
```

Bulk Operations In Hibernate :-

While working with Hibernate ,Operations on Object's can be done in two way's

- 1) Single row operations
- 2) Bulk Operations

In the case of Single row operations , the operations will be effected on a single row of a Database.

In the Case of Bulk operations, the operations will be effected on multiple rows at a time on Database.

Hibernate Framework Hhs Provided The Following 3 Techniques

To perform Bulk operations

- 1) HQL(Hibernate Query Language)
- 2) Criteria API
- 3) Native SQL

HQL(Hibernate Query Language):-

Till Now we have done the operations on single object(single row),but with the HQL we will performs updates,deletes,selects on multiple rows of data(multiple objects) at a time

To write complicated queries(or) multiple where conditions(or) to write where conditions on other than primary key columns (or) to work on multiple records we use the HQL features.

We can perform both select and non-select operations on a database by using HQL.

An object oriented form of SQL is called HQL.

HQL syntax is very much similar to SQL syntax.

HQL queries are formed by using entities and their properties

SQL queries are formed by using db tables and their columns.

SQL is a database dependent and HQL is a database Independent

Example:-

SQL Query

SQL:- select * from Employee ;

HQL Query

HQL:- select e from com.nareshit.pojo.Employee as e;

(or)

HQL :- from com.nareshit.pojo.Employee ;

(or)

HQL :- from com.nareshit.pojo.Employee as e ;

HQL and SQL keywords are not case sensitive.(Select,Delete,From)

IN SQL DB table columns and Dbtablename is not case-sensitive.

In HQL Entity name And Entity class variables are case-sensitive,

HQL queries database independent queries. (HQL queries internally converted into d/b specific SQL queries using Dialect class).

we can execute aggregate functions (MAX(),MIN(),AVG()...etc) using HQL.

To write very complicated queries (or) which are not supported by HQL , we have to use Native SQL. Native SQL queries are database dependent queries. Maximum try to avoid using Native SQL.

If we want to write the queries(HQL (or) Native SQL) declaratively(in mapping file(or) in annotations) we can use Named Queries.

Steps to work with HQL:-

Step 1: write the HQL query as per requirement.

Step2: create an org.hibernate.Query object by passing HQL query.

To do this we can call createQuery(String hql) method from Session interface.

public Query createQuery(String hql)

Example :

Query query = session.createQuery(hqlquery);

Note :-

session.getNamedQuery(String namedQueryName) method is used to create an object of Query for a named Hql query string defined in the mapping file.

(We will learn about the Named Queries later in this chapter) .

Step3 : if the Query contains parameters, set those values

Step4 : execute query object.

query.list(); (OR) query.iterate() -->[for executing select queries]

query.executeUpdate(); -->[for executing UPDATE/DELETE queries]

Query:-

Query is an interface, its implemented class as QueryImpl.

Query is an object oriented representation of Hibernate Query. The object of Query can be obtained by calling the session.createQuery(String hql) (OR)

session.getNamedQuery(String namedQueryName) method.

The Query interface provides many methods. Below are the commonly used methods:

1. `public int executeUpdate()` is used to execute the update or delete query.
2. `public List list()` returns the results of select query as a list.
3. `public Query setFirstResult(int rowno)` specifies the row number from where record will be retrieved.
4. `public Query setMaxResult(int rowno)` specifies the no. of records to be retrieved from the relation (table).
5. `public Query setParameter(int position, Object value)` it sets the value to the JDBC style query parameter.
6. `public Query setParameter(String name, Object value)` it sets the value to a named query parameter.

Differences between list() method iterate() method

- 1) `list()`-->directly returns the list data structure having selected records in the form of hb pojo class obj.
`iterate()`-->returns Iterator obj pointing into list data structure .
- 2) `list()`-->generates single select sql query to select the records from the table.
`iterate()`-->generates multiple select sql queries to select the records from db table.

Here first select query always returns identity values.

- 3) In the case of `list()` The returned HBpojo class obj will be initialized with the record irrespective of the obj utilization. It is good in standalone applications.

In the case of `iterate()` The returned HBpojo class obj will be initialized with records on demand basis that means lazy loading of records is having . good in web-app that contains multiple layers.

Note:

When HQL select query selects all the column values of dbtable records then the generated list data structure contains HB persistence class objects as element values by representing the records that are selected.

Example :

```

package com.nareshit.pojo;
public class Employee{
private int empNo;
private String name;
private double salary;
//required setters and getters
}

```

Employee Table

Empno	name	Salary
1001	ramu	5000.0
1002	raju	6000.0
1003	ajay	5500.0

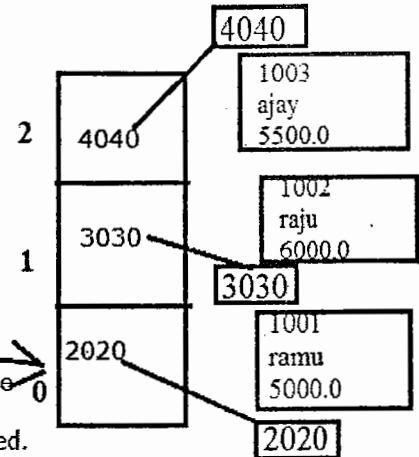
HQL Query :

```

String hql = "FROM com.nareshit.pojo.Employee";
Query query = session.createQuery(hql);
List<Employee> results = query.list();

```

list data structure contains HB persistence class objects as element values by representing the records that are selected.



If HQL select Query is selecting specific column values of a table then the generated list data structure contains java.lang.Object[] array as an element values by representing records that are selected.

```

package com.nareshit.pojo;
public class Employee{
private int empNo;
private String name;
private double salary;
//required setters and getters
}

```

Employee Table

Empno	name	Salary
1001	ramu	5000.0
1002	raju	6000.0
1003	ajay	5500.0

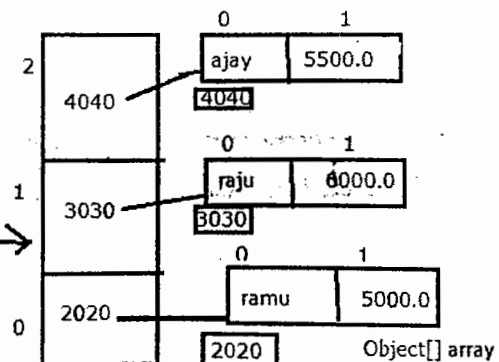
HQL Query :

```

String hql="select e.name,e.salary from
com.nareshit.pojo.Employee as e";
Query query=session.createQuery(hql);
List<Object[]> results=query.list();

```

list data structure contains java.lang.Object[] array as an element values by representing records that are selected



HqlProject1

src

- com.nareshit.client
 - Test.java
- com.nareshit.dao
 - EmployeeDao.java
 - EmployeeDaoImpl.java
- com.nareshit.pojo
 - Employee.java
- com.nareshit.utility
 - HibernateUtility.java
- com.nareshit.xml
 - hibernate.cfg.xml

Employee.java

```
package com.nareshit.pojo;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;
@Entity
@Table(name="employee")
public class Employee {
    @Id
    @Column(name="empno")
    private int empno;
    @Column(name="name")
    private String name;
    @Column(name="salary")
    private double salary;
    public int getEmpno() {
        return empno;
    }
    public void setEmpno(int empno) {
        this.empno = empno;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public double getSalary() {
        return salary;
    }
    public void setSalary(double salary) {
        this.salary = salary;
    }
}
```

```

    }
}

EmployeeDAO.java
package com.nareshit.dao;
import java.util.List;
import com.nareshit.pojo.Employee;
public interface EmployeeDAO{
    /**
     * this method is used to get the Employees By Name condition
     * @param name
     * @returns List<Employee>
     */
    List<Employee> getEmployeesByName(String name);
    /**
     * this method is used to get the Employees Name and salary
     * @param
     * @returns List<Object[]>
     */
    List<Object[]> getEmployees();
    /**
     * this method is used to update the EmployeeName
     * @param int ,String
     * @returns int
     */
    int updateEmployeeName(int empNo,String newName);
    /**
     * this method is used to delete the Employee
     * @param int
     * @returns int
     */
    int deleteEmployee(int empNo);
}

```

```

EmployeeDAOImpl.java
package com.nareshit.dao;
import java.util.List;
import org.hibernate.Query;
import org.hibernate.Session;
import com.nareshit.pojo.Employee;
import com.nareshit.utility.HibernateUtility;
public class EmployeeDAOImpl implements EmployeeDAO{
    public List<Employee> getEmployeesByName(String name) {
        List<Employee> results=null;
        Session session=HibernateUtility.getSession();
        if(session!=null){
            Query query=session.createQuery("from com.nareshit.pojo.Employee as e
where e.name like ?");
            query.setParameter(0,name);

```

```
        results=query.list();
        session.close();
    }
    return results;
} //end of getEmployeesByName
public List<Object[]> getEmployees() {
    List<Object[]> results=null;
    Session session=HibernateUtility.getSession();
    if(session!=null){
        Query query=session.createQuery("select e.name,e.salary from
com.nareshit.pojo.Employee as e");
        results=query.list();
        session.close();
    }
    return results;
} //end of getEmployees()

public int updateEmployeeName(int empNo, String newName) {
    int count=0;
    Session session=HibernateUtility.getSession();
    if(session!=null){
        Query query=session.createQuery("update com.nareshit.pojo.Employee as e
set e.name=? where e.empno=?");
        query.setParameter(0,newName);
        query.setParameter(1,empNo);
        count=query.executeUpdate();
        session.beginTransaction().commit();
        session.close();
    }
    return count;
} //end of updateEmployeeName(-,-)
public int deleteEmployee(int empNo) {
    int count=0;
    Session session=HibernateUtility.getSession();
    if(session!=null){
        Query query=session.createQuery("delete from com.nareshit.pojo.Employee
as e where e.empno=?");
        query.setParameter(0,empNo);
        count=query.executeUpdate();
        session.beginTransaction().commit();
        session.close();
    }
    return count;
} //end of deleteEmployee(-)
}
```

hibernate.cfg.xml

```
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
<session-factory>
<property name="connection.driver_class">
oracle.jdbc.driver.OracleDriver</property>
<property name="connection.url">
jdbc:oracle:thin:@localhost:1521:XE</property>
<property name="connection.username">system</property>
<property name="connection.password">manager</property>
<property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
<property name="show_sql">true</property>
<property name="hbm2ddl.auto">update</property>
<mapping class="com.nareshit.pojo.Employee"/>
</session-factory>
</hibernate-configuration>
```

HibernateUtility.java

```
package com.nareshit.utility;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
public class HibernateUtility {
    private static SessionFactory factory;
    static{
        Configuration cfg=new Configuration();
        cfg.configure("/com/nareshit/xml/hibernate.cfg.xml");
        factory=cfg.buildSessionFactory();
    }
    public static Session getSession(){
        Session session=null;
        if(factory!=null){
            session=factory.openSession();
        }
        return session;
    }
}
```

Test.java

```
package com.nareshit.client;
import java.util.List;
import com.nareshit.dao.EmployeeDao;
import com.nareshit.dao.EmployeeDaoImpl;
import com.nareshit.pojo.Employee;
public class Test {
```

```

public static void main(String[] args){
    EmployeeDao edao=new EmployeeDaoImpl();
    System.out.println("Testing of getEmployeeByName ");
    List<Employee> list1=edao.getEmployeesByName("ramu");
    for(Employee emp:list1){
        System.out.println("Emp No :"+emp.getEmpno());
        System.out.println("Name :"+emp.getName());
        System.out.println("Salary :"+emp.getSalary());
    }
    System.out.println("=====");
    System.out.println("Testing of getEmployees() ");
    List<Object[]> list2=edao.getEmployees();
    for(Object[] obj1:list2){
        for(Object obj2:obj1){
            System.out.println(obj2);
        }
    }

    System.out.println("=====");
    System.out.println("Testing of updateEmployeeName ");
    int updateCount=edao.updateEmployeeName(1001,"Ramurao");
    System.out.println("Update Count :"+updateCount);
    System.out.println("=====");
    System.out.println("Testing of deleteEmployee ");
    int deleteCount=edao.deleteEmployee(1002);
    System.out.println("Delete Count :"+deleteCount);
}
}

```

When we are preparing the HQL queries we can place 2 types of parameters

- Positional parameters(?)
- NamedParameters(:example)

These parameters helps the programmer to set input,conditional values to HQL queries as a java value. with out worrying about the syntax of HQL query.

To set the values to the above parametes we need to use setXxx(-,-) methods

(or) setParameter(-,-) methods

In Hibernate, the positional parameters index begins with '0' zero.

HQL Query with Positional Parameters:-

```

Query query=session.createQuery("select e from com.nareshit.pojo.Employee as e where
e.name like =? And e.salary >=?");

```

```
q.setParameter(0,"ramu");

q.setDouble(1,6000);

//execute the Query

List<Employee> list=query.list();
```

If the multiple positional parameter are there in a single HQL query ,programmer may be confused while identifying the index's of parameters.

To overcome this problem it is recommended to work with named parameters.And Named Parameters will increase the readability of the query.

HQL Query with Named Parameter:-

```
Query query=session.createQuery("select e.empNo, e.name from com.nareshit.pojo.Employee as e
where e.name like :name1 and e.salary >=:salary1");
```

```
//set values to Named parameters
```

```
query.setParameter("name1","ramu");
```

```
query.setDouble("salary1",7000.0);
```

```
List<Object[]> list=query.list();
```

Note:-

We can use both named parameters and positional parameters within the same query. But we should write positional parameters first, then named parameters.

```
String hql=" select e from com.nareshit.pojo.Employee as e where e.empNo=? and e.name
like :name1";
```

```
Query query=session.createQuery(hql);
query.setParameter(0,1001);
query.setParameter("name1","ramu");
List list=query.list();
```

uniqueResult():-

If we are expecting only one record as the result of query executing then we will call uniqueResult() method on query object . If query returns more than one record this method throws NonUniqueResultException.

Example 1 : Table

empno	name	salary
1001	ramu	6000.0
1002	ramu	5800.0
1003	ajay	5500.0

```
String hql="from com.nareshit.pojo.Employee as e e.name like :name1";
```

```
Query query=session.createQuery(hql);
```

```
query.setParameter("name1","ramu");
```

```
Employee emp=(Employee)query.uniqueResult();
```

In the above example code query returns more than one record so we get Exception
:org.hibernate.NonUniqueResultException :query did not return a unique result :2

If the query returns only one record uniqueResult() method executes the query and returns the resulted object.

Example 2:

```
String hql="from com.nareshit.pojo.Employee as e e.empNo =:empNo1";
```

```
Query query=session.createQuery(hql);
```

```
query.setParameter("empNo1",1001);
```

```
Employee emp=(Employee)query.uniqueResult();
```

```
System.out.println("Employee Details Are :");
```

```
System.out.println("EmpNo :"+emp.getEmpNo());
```

```
System.out.println("Name :"+emp.getName());
```

```
System.out.println("Salary :"+emp.getSalary());
```

Example 3:

```
String hql="select e.name from com.nareshit.pojo.Employee as e where e.empNo=:empNo1";
```

```
Query query=session.createQuery(hql);
```

```
query.setParameter("empNo1",1001);
```

```
String empName=(String) query.uniqueResult();
```

```
System.out.println("EmpName :"+empName);
```

Example 4:

```
String hql="select e.name,e.salary from com.nareshit.pojo.Employee as e where
```

```
e.empNo=:empNo1";
Query query=session.createQuery(hql);
query.setParameter("empNo1",1001);
Object[] obj=(Object[]) query.uniqueResult();
System.out.println("EmpName :"+obj[0]);
System.out.println("Emp Salary :"+obj[1]);
```

Note : Generally we will use this method where we should get only one record as a query result.

Aggregate Methods :

HQL supports a range of aggregate methods, similar to SQL. They work the same way in HQL as in SQL and following is the list of the available functions:

- | | | |
|---|---------------------------|--|
| 1 | avg(property name) | : The average of a property's value |
| 2 | count(property name or *) | : The number of times a property occurs in the results |
| 3 | max(property name) | : The maximum value of the property values |
| 4 | min(property name) | : The minimum value of the property values |
| 5 | sum(property name) | :The sum total of the property values |

Example 1 :

```
Session session = sessionFactory.openSession();
String hql ="Select min(e.salary) FROM Employee as e";
Query query = session.createQuery(hql);
System.out.println("Sum of Min salary : "+ query.uniqueResult());
```

Example 2 :

```
Session session = sessionFactory.openSession();
String hql ="Select max(e.salary) FROM Employee as e";
Query query = session.createQuery(hql);
System.out.println("Sum of Max salary : "+ query.uniqueResult());
```

Example 3 : Multiple Aggregate functions

```
Session session=factory.openSession();
String hql="select max(salary),min(salary) from com.nareshit.pojo.Employee";
List<Object[]> list=session.createQuery(hql).list();
for(Object[] obj:list){
System.out.println("Max Salary :"+obj[0]);
System.out.println("Min Salary:"+obj[1]);}
```

ORDER BY Clause :

To sort your HQL query's results, you will need to use the ORDER BY clause. You can order the

results by any property on the objects in the result set either ascending (ASC) or descending (DESC).

Note:-

By-default ORDER BY clause will take ascending order.

If we want to specify the order explicitly, we can write the queries as follows

HQL Query :- From com.nareshit.pojo.Employee e ORDER BY e.name ASC

HQL Query :- From com.nareshit.pojo.Employee e ORDER BY e.name DESC

Following is the simple example of using ORDER BY clause:

```
String hql = "FROM com.nareshit.pojo.Employee e WHERE e.empNo >=? ORDER BY e.salary DESC";
```

```
Query query = session.createQuery(hql);
```

```
query.setParameter(0,1001);
```

```
List<Employee> results = query.list();
```

If you want to sort by more than one property, you would just add the additional properties to the end of the order by clause, separated by commas as follows:

```
String hql = "FROM com.nareshit.pojo.Employee e WHERE e.empNo > = ? ORDER BY e.name DESC, e.salary DESC ";
```

```
Query query = session.createQuery(hql);
```

```
Query.setParameter(0,1001);
```

```
List<Employee> results = query.list();
```

Group by clause

The "group by" and "having" clauses are also supported by HQL. They will also be translated into SQL by Hibernate. Consider class Book having properties price, publishDate

```
select book.publishDate, avg(book.price)
```

```
from Book book
```

```
group by book.publishDate
```

```
select book.publishDate, avg(book.price)
```

```
from Book book
```

```
group by book.publishDate
```

```
having avg(book.price) > 10
```

Subqueries

We can use subqueries in HQL also. Be careful that subqueries may be not effective if writing improperly. For many cases, you can write equivalent queries with simple "select-from-where" statements.

```
from Book expensiveBook
```

```
where expensiveBook.price > (
```

```
select avg(book.price) from Book book
```

```
)
```

Named Queries

We can put our HQL statements in the mapping definitions and refer them by name in the code. They are called "Named Queries".

The named queries can be put in any mapping definitions. But for easier maintenance, we should centralize all the named queries in one mapping definitions, say NamedQuery.hbm.xml, or each mapping definition for a category. In addition, setting up a mechanism for the naming of queries is also beneficial.

For each named query, we need to assign a unique name to it. We should also put the query string in a `<![CDATA[...]]>` block to avoid conflicts with the special XML characters.

```
<hibernate-mapping>
<query name="Book.by.isbn">
<![CDATA[from Book where isbn = ?]]>
</query>
</hibernate-mapping>
```

To reference for a named query, we can use the `session.getNamedQuery()` method.

```
Query query = session.getNamedQuery("Book.by.isbn");
query.setString(0, "1932394419");
Book book = (Book) query.uniqueResult();
```

Fetching Strategies in Hibernate

Hibernate has few fetching strategies to optimize the Hibernate generated select statement, so that it can be as efficient as possible. The fetching strategy is declared in the mapping relationship to define how Hibernate fetch its related collections and entities.

Fetching Strategies

There are four fetching strategies

1. fetch-"join" = Disable the lazy loading, always load all the collections and entities.
2. fetch-"select" (default) = Lazy load all the collections and entities.
3. batch-size="N" = Fetching up to 'N' collections or entities, *Not record*.
4. fetch-"subselect" = Group its collection into a sub select statement.

Fetching strategies examples

Here's a "one-to-many relationship" example for the fetching strategies demonstration. A book is belong to many chapters.

Example to declare fetch strategies in XML file

```
...
<hibernate-mapping>
<class name="com.nareshit.hibernate.Book" table="book">
<set name="chapters" cascade="all" inverse="true"
    table="chapter" batch-size="10" fetch="select">
</key>
```

```

<column name="book_id" not-null="true" />
</key>
<one-to-many class=" com.nareshit.hibernate.Chapter " />
</set>
</class>
</hibernate-mapping>

```

Example to declare fetch strategies in annotation

```

...
@Entity
@Table(name = "book")
public class Book implements Serializable{
...
    @OneToMany(fetch = FetchType.LAZY, mappedBy = "book")
    @Cascade(CascadeType.ALL)
    @Fetch(FetchMode.SELECT)
    @BatchSize(size = 10)
    public Set<Chapter> getChapters() {
        return this.chapters;
    }
...
}

```

Let explore how fetch strategies affect the Hibernate generated SQL statement.

1. fetch="select" or @Fetch(FetchMode.SELECT)

This is the default fetching strategy. it enabled the lazy loading of all it's related collections. Let see the example...

```

//call select from book
Bookbook = (Book)session.get(Book.class, 114);
Set chapters = book.getChapters();
//call select from chapter
for ( Iterator iter = chapters.iterator(); iter.hasNext(); ) {
    Chapter sdr = (Chapter) iter.next();
    System.out.println(sdr.getChapterId());
}

```

Output

Hibernate:

```

select ...from book book0_
where book0_.book_id=?

```

Hibernate:

```

select ...from chapter chapter_
where chapter_.book_id=?

```

Hibernate generated two select statements

1. Select statement to retrieve the Book records -session.get(Book.class, 114)
2. Select its related collections – sets.iterator()

2. fetch="join" or @Fetch(FetchMode.JOIN)

The "join" fetching strategy will disabled the lazy loading of all it's related collections. Let see the Example...

```
//call select from book and chapter
Book book = (Book)session.get(Book.class, 114);
Set sets = book.getChapters();
//no extra select
for ( Iterator iter = sets.iterator();iter.hasNext(); ) {
    Chapter sdr = (Chapter) iter.next();
    System.out.println(sdr.getChapterId());
}
```

Output

Hibernate:

```
select ... from
  Book book_
left outer join
Chapter chapter_
  on book_.book_ID= chapter_.book_ID  where
  book_.book_ID=?
```

Hibernate generated only one select statement, it retrieve all its related collections when the Book is initialized. -session.get(Book.class, 114)

Select statement to retrieve the Book records and outer join its related collections.

3. batch-size="10" or @BatchSize(size = 10)

This 'batch size' fetching strategy is always misunderstanding by many Hibernate developers. Let see the *misunderstand* concept here...

```
Book book = (Book)session.get(Book.class, 114);
Set sets = book.getChapters();
```

```
for ( Iterator iter = sets.iterator();iter.hasNext(); ) {
    Chapter sdr = (Chapter) iter.next();
    System.out.println(sdr.getChapterId());
}
```

What is your expected result, is this per-fetch 10 records from collection? See the output

Output

Hibernate:

```
select ...from book book0_
where book0_.Book_ID=?
```

Hibernate:

```
select ...from chapter chapter0_  
where chapter0_.BOOK_ID=?
```

The batch-size did nothing here, it is not how batch-size work. See this statement.

The batch-size fetching strategy is not defined how many records inside in the collections are loaded. Instead, it defines how many collections should be loaded.

Another Example

Let see another example, you want to print out all the book records and its related chapter records (collections) one by one.

```
List<Book> list = session.createQuery("from Book").list();
```

```
for(Book book : list){
```

```
    Set sets = book.getChapters ();
```

```
    for ( Iterator iter = sets.iterator();iter.hasNext(); ) {  
        Chapter sdr = (Chapter) iter.next();  
        System.out.println(sdr.getchapterId());  
    }  
}
```

No batch-size fetching strategy

Output

Hibernate:

```
select ...  
from book book0_
```

Hibernate:

```
select ...  
from chapter chapter0_  
where chapter0_.BOOK_ID=?
```

Hibernate:

```
select ...  
from chapter chapter0_  
where chapter0_.book_id=?
```

Keep repeat the select statements....depend how many book records in your table.

If you have 20 book records in the database, the Hibernate's default fetching strategies will generate 20+1 select statements and hit the database.

1. Select statement to retrieve all the Book records.
2. Select its related collection
3. Select its related collection
4. Select its related collection

....

21. Select its related collection

The generated queries are not efficient and caused a serious performance issue.

Enabled the batch-size='10' fetching strategy

Let see another example with batch-size='10' is enabled.

Output

Hibernate:

```
select ...
from book book0_
```

Hibernate:

```
select ...
from chapter chapter0_
where
  chapter0_.book_id in (
    ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
  )
```

Now, Hibernate will per-fetch the collections, with a select *in* statement. If you have 20 bookrecords, it will generate 3 select statements.

1. Select statement to retrieve all the Book records.
2. Select In statement to per-fetch its related collections (10 collections a time)
3. Select In statement to per-fetch its related collections (next 10 collections a time)

With batch-size enabled, it simplify the select statements from 21 select statements to 3 select statements.

4. fetch="subselect" or @Fetch(FetchMode.SUBSELECT)

This fetching strategy is enable all its related collection in a sub select statement. Let see the same query again..

```
List<Book> list = session.createQuery("from Book").list();
for(Book book : list){
    Set sets = book.getChapters ();

    for ( Iterator iter = sets.iterator();iter.hasNext(); ) {
        Chapter sdr = (Chapter) iter.next();
        System.out.println(sdr.getChapterId());
    }
}
```


Output

Hibernate:

```
select ...  
from book book0_
```

Hibernate:

```
select ...  
from  
    chapter chapter0_  
where  
    chapter0_.book_id in (  
        select  
            chapter0_.book_id  
        from  
            book book0_  
    )
```

With "subselect" enabled, it will create two select statements.

1. Select statement to retrieve all the book records.
2. Select all its related collections in a sub select query.

Pagination in Hibernate :

Pagination is the very common problem for the most of the Enterprise applications. When we are retrieving thousands of records from the database, it is not good idea to retrieve all the records at the same time. So, we have to implement some sort of **pagination** concept in your application to restrict the number of rows to be fetched from the database.

Pagination can be implemented in Two ways

1. Front-End Pagination
2. Back-End Pagination

1. Front-End Pagination

In front-end pagination, we will hit the database only once, and get the results and store them in the memory(session/application scope), when user navigating from one page to another page, we retrieve the data from memory but not hitting the database again and again.

With this style of pagination we can achieve easy navigation, fast loading of the data but we have the below problems

- i) It takes more memory to hold results
- ii) If the data is updated in the database by other application, we cannot get the updated data because we are getting the data from memory instead of database.

Note : This type of pagination is suggestible if the number of resulted records are limited and the data is consistent data.

2. Back-End Pagination

In Back-end pagination ,we will hit the database again and again ,get's requested page details from database.

With this style of pagination we can achieve easy navigation, fast loading of data, takes memory to hold the result data And it will get always updated data.

But it gives less performance compared when compared with Front-end pagination.

Note : This type of pagination is suggestible ,if number of resulted records are unlimited and data is not consistent data.

To apply the pagination Hibernate provides two methods in org.hibernate.Query interface

public Query setFirstResult(int firstResult);→ It takes the argument which specifies from where record has to take.

public Query setMaxResults(int maxResults);→ It takes the argument which specifies how many records has to take.

Example :-

```
String hql="from com.nareshit.pojo.Employee as e";
Query query=session.createQuery(hql);
query.setFirstResult(2); //Means starts from 3 rd record
query.setMaxResults(4); //Means No.of records 4 to be displayed
List<Employee> list=query.list();
for(Employee emp:list){
    System.out.println("Emp No :"+emp.getEmpNo());
    System.out.println("Emp Name :"+emp.getName())
    System.out.println("Salary :"+emp.getSalary());
}
```

When we are using pagination ,In web page we are responsible to display page numbers ,When the user click on some page number ,we need to send the request to server,and get's corresponding page results .To do all these things we need to implement some logic.

The following Example showing Hibernate Pagination

Screen 1:

Welcome to Naresh it

EmployeeDetails

Screen 2:

Employee Details		
Employee Id	Employee Name	Employee Salary
1001	ajay	8000.0
1002	vijay	12000.0
1003	ramu	13000.0
1004	raju	32000.0

1 2 3

- HibernatePaginationExample
 - JAX-WS Web Services
 - Deployment Descriptor: HibernatePaginationExample
 - Java Resources
 - src
 - com.nareshit.config
 - hibernate.cfg.xml
 - com.nareshit.controller
 - EmployeeController.java
 - com.nareshit.dao
 - EmployeeDAO.java
 - EmployeeDAOImpl.java
 - com.nareshit.dao.factory
 - DAOFactory.java
 - com.nareshit.pojo
 - Employee.java
 - com.nareshit.service
 - EmployeeService.java
 - EmployeeServiceImpl.java
 - com.nareshit.util
 - HibernateUtility.java
 - WebContent
 - WEB-INF
 - lib
 - web.xml
 - employeeDetails.jsp
 - index.jsp

index.jsp

```
<html><head><center><h3>Welcome to Naresh it</h3></center>
</head>
<body><br/><hr/><br/>
<center>
<a href="employeesDetails.do">EmployeeDetails</a>
</center>
</body>
</html>
```

web.xml

```
<web-app>
  <servlet>
    <servlet-name>employeeController</servlet-name>
```

```
<servlet-class>com.nareshit.controller.EmployeeController
</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>employeeController</servlet-name>
  <url-pattern>*.do</url-pattern>
</servlet-mapping>
<welcome-file-list>
  <welcome-file>index.jsp</welcome-file>
</welcome-file-list>
</web-app>
```

EmployeeController.java

```
package com.nareshit.controller;
import java.io.IOException;
import java.util.List;
import javax.servlet.*;
import javax.servlet.http.*;
import com.nareshit.pojo.Employee;
import com.nareshit.service.*;
public class EmployeeController extends HttpServlet {
  private EmployeeService employeeService = null;
  public void init() {
    employeeService = new EmployeeServiceImpl();
  }
  protected void doGet(HttpServletRequest req, HttpServletResponse resp)
    throws ServletException, IOException {
    List<Employee> listOfEmployees = null;
    int pageIndex = 0;
    String startPageIndex = req.getParameter("pageIndex");
    if (startPageIndex == null) {
      pageIndex = 1;
    } else {
      pageIndex = Integer.parseInt(startPageIndex);
    }
    listOfEmployees = employeeService.getEmployees(pageIndex);
    long numberOfPages = employeeService.getNumberOfPages();
    req.setAttribute("employees", listOfEmployees);
    req.setAttribute("noOfPages", numberOfPages);
    RequestDispatcher rd =
    req.getRequestDispatcher("employeeDetails.jsp");
    rd.forward(req, resp);
  }
}
```

EmployeeService.java

```
package com.nareshit.service;
import java.util.List;
import com.nareshit.pojo.Employee;
public interface EmployeeService {
    public List<Employee> getEmployees(int pageIndex);
    public long getNumberOfPages();
    public int getStartPageIndex(int pageIndex);
    public long getTotalNumberOfEmployees();
}
```

EmployeeServiceImpl.java

```
package com.nareshit.service;
import java.util.List;
import com.nareshit.dao.factory.DAOFactory;
import com.nareshit.pojo.Employee;
public class EmployeeServiceImpl implements EmployeeService {
    private final int numberOfRecordsPerPage = 4;
    public int getStartPageIndex(int pageIndex){
        int startPageIndex = (pageIndex*numberOfRecordsPerPage)-
        numberOfRecordsPerPage;
        return startPageIndex;
    }
    public long getNumberOfPages(){
        long totalNumberOfRecords = getTotalNumberOfEmployees();
        long numberOfPages = totalNumberOfRecords/numberOfRecordsPerPage;
        if(totalNumberOfRecords > (numberOfPages*numberOfRecordsPerPage)){
            numberOfPages++;
        }
        return numberOfPages;
    }
    public List<Employee> getEmployees(int pageIndex) {
        List<Employee> listOfEmployees = null;

        listOfEmployees=DAOFactory.getEmployeeDAO().getEmployees(getStartPageIndex(pageIndex));

        return listOfEmployees;
    }
    public long getTotalNumberOfEmployees() {
        long count = DAOFactory.getEmployeeDAO().getTotalNumberRecords();
        return count;
    }
}
```

EmployeeDAO.java

```
package com.nareshit.dao;
import java.util.List;
import com.nareshit.pojo.Employee;
public interface EmployeeDAO {
    public List<Employee> getEmployees(int pageIndex);
    public long getTotalNumberRecords();
}
```

EmployeeDAOImpl.java

```
package com.nareshit.dao;
import java.util.List;
import org.hibernate.Query;
import org.hibernate.Session;
import com.nareshit.pojo.Employee;
import com.nareshit.util.HibernateUtility;
public class EmployeeDAOImpl implements EmployeeDAO {
    private static final int numberOfRecordsPerPage=4;
    public List<Employee> getEmployees(int pageIndex) {
        List<Employee> listOfEmployees = null;
        Session session = HibernateUtility.getSession();
        if(session!=null){
            String hql = "FROM com.nareshit.pojo.Employee";
            Query query = session.createQuery(hql);
            query.setFirstResult(pageIndex);
            query.setMaxResults(numberOfRecordsPerPage);
            listOfEmployees=query.list();
            HibernateUtility.closeSession(session);
        }
        return listOfEmployees;
    }

    public long getTotalNumberRecords() {
        long count=0L;
        Session session = HibernateUtility.getSession();
        String hql = "SELECT COUNT(*) FROM com.nareshit.pojo.Employee";
        Query query = session.createQuery(hql);
        count= (Long) query.uniqueResult();
        HibernateUtility.closeSession(session);
        return count;
    }
}
```

Employee.java

```
package com.nareshit.pojo;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity
@Table(name="employee")
public class Employee {
    @Id
    @Column(name="empNo")
    private int empNo;
    @Column(name="name",length=15)
    private String name;
    @Column(name="salary")
    private double salary;
    public int getEmpNo() {
        return empNo;
    }
    public void setEmpNo(int empNo) {
        this.empNo = empNo;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public double getSalary() {
        return salary;
    }
    public void setSalary(double salary) {
        this.salary = salary;
    }
}
```

DAOFactory.java

```
package com.nareshit.dao.factory;
import com.nareshit.dao.EmployeeDAO;
import com.nareshit.dao.EmployeeDAOImpl;
public class DAOFactory {
    private static EmployeeDAO employeeDao;
```

```

static{
    employeeDao=new EmployeeDAOImpl();
}
publicstatic EmployeeDAO getEmployeeDAO(){
    returnemployeeDao;
}
}

```

HibernateUtility.java

```

package com.nareshit.util;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
public class HibernateUtility {
    privatestatic Configuration cfg;
    privatestatic SessionFactory factory;
    static {
        cfg = new Configuration();
        cfg.configure("com/nareshit/config/hibernate.cfg.xml");
        factory = cfg.buildSessionFactory();
    }
    publicstatic Session getSession() {
        Session session = null;
        if (factory != null) {
            session = factory.openSession();
        }
        return session;
    }

    publicstaticvoid closeSession(Session session) {
        if (session != null) {
            session.close();
        }
    }
}

```

hibernate.cfg.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <property
name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
        <property name="connection.url">jdbc:oracle:thin:@localhost:1521:XE</property>
        <property name="connection.username">system</property>
        <property name="connection.password">manager</property>

```



```

        <property name="dialect">org.hibernate.dialect.Oracle10gDialect</property>
        <property name="show_sql">true</property>
        <property name="hbm2ddl.auto">update</property>
        <mapping class="com.nareshit.pojo.Employee" />
    </session-factory>
</hibernate-configuration>

```

employeeDetails.jsp

```

<%@page import="java.util.*"%>
<%@page import="com.nareshit.pojo.*"%>
<html>
<title>Employee Details</title>
<body>
    <%!Employee emp = null;
    List<Employee> list = null;
    Long noOfPages = null;%>
    <table align="center" style="margin-top: 150px;">
        <tr>
            <td colspan="3"><center
                style="font: bold; color: navy; color: white; background-color:
purple;">Employee
                Details</center></td>
        </tr>
        <tr>
            <th style="background: #03F; color: #FFF;">Employee Id</th>
            <th style="background: #03F; color: #FFF;">Employee Name</th>
            <th style="background: #03F; color: #FFF;">Employee Salary</th>
        </tr>
        <%
            list = (List) request.getAttribute("employees");
            if (list != null) {
                for (Employee employee : list) {
                    <td style="background: #CCC;"><%=employee.getEmpNo()%></td>
                    <td style="background: #CCC;"><%=employee.getName()%></td>
                    <td style="background: #CCC;"><%=employee.getSalary()%></td>
                </tr>
            }
            } else {
                <center>
                    <%
                        out.println("Employees Not Found");
                    %>
                </center>
            }
        <%

```

```

        <%
            }
        %>
    <tr>
        <td colspan="2">
            <%
                noOfPages = (Long) request.getAttribute("noOfPages");
                if (list != null) {
                    if (noOfPages != null) {
                        for (int i = 1; i <= noOfPages.longValue(); i++) {
                            String myurl =
                                "employeesDetails.do?pageIndex=" + i;
                            %><a href="<%=myurl%>"><%=i%></a><%
                        }
                    }
                }
            %>
        </td>
    </tr>
</table>
</body>
</html>

```

Criteria API

- The Criteria API provides an object-oriented way to retrieve persistent object's.
- This provides a simplified API to build queries dynamically at runtime.
- Criteria is an alternative way to write the queries without using HQL.
- The Criteria Api allows queries to build at runtime without direct string manipulations.
- Criteria is an API from hibernate and used to write queries in object oriented manner rather than sql(or) HQL.
- Criteria Api is more useful when we have variable no.of conditions in a query.
- We can execute only Select statements by using criteria, we can not execute Update, Delete, statements using crieteria.
- Criteria api also include aggregation methods

Steps to work with Criteria API:-

Step 1: Create org.hibernate.Criteria Object

```
Criteria criteria = session.createCriteria(EntityClassName.class);
```

Step 2: Create org.hibernate.criterion.Criterion Object per each condition of the query and add to Criteria object.

```
Criterion criterion = Restrictions.eq("propertyName", propertyValue);
```

```
criteria.add(criterion);
```

Step 3: Execute org.hibernate.Criteria object (by calling list() method on Criteria object)

```
List list = criteria.list();
```

- Criteria is an interface present in org.hibernate package used to represent the query in object oriented manner. It contains methods to add Criterion object's, Order Objects, Projection Object's, Pagination methods.....etc. CriteriaImpl is an implementation class of Criteria

To get Criteria Object we can use the following methods from session

```
public Criteria createCriteria(Class entityClass)
```

```
public Criteria createCriteria(String entityClassName)
```

- Restrictions is a utility class present in org.hibernate.criterion package which has methods to define conditions.
- Restrictions class methods return different XxxExpression classes which implement the Criterion interface. So Restrictions is a static factory for Criterion instances.
- Criterion is an interface present in org.hibernate.criterion package which is used to represent one condition in object oriented manner.
- By calling methods on Restrictions we will get Criterion Object.

RestrictionClass important methods

Expression class	Restrictions method	SQL operator/function
SimpleExpression	eq(prop, val)	=
SimpleExpression	ne(prop, val)	!=, <>
SimpleExpression	like(prop, val) [case-sensitive]	LIKE
SimpleExpression	gt(prop, val)	>
SimpleExpression	lt(prop, val)	<
SimpleExpression	ge(prop, val)	>=
SimpleExpression	le(prop, val)	<=
IdentifierExpression	idEq(val)	<id-column> =
ILikeExpression	ilike(prop, val) [case-in-sensitive]	LIKE
BetweenExpression	between(prop, val, val)	BETWEEN <lowerLimit> AND <upperLimit>
InExpression	in(prop, val[])	IN(val1, val2....)
NullExpression	isNull(prop)	<column> IS NULL
NullExpression	isNotNull(prop)	<column> IS NOT NULL
LogicalExpression	and (criterion1, criterion2)	<expr1> AND <expr2>
LogicalExpression	or(criterion1, criterion2)	<expr1> OR <expr2>
NotExpression	not(criterion)	!
EmptyExpression	isEmpty(prop)	EMPTY

Example code to read All the Employee records

```
Session session=sessionFactory.openSession();
```

```
Criteria criteria=session.createCriteria("com.nareshit.pojo.Employee");
```

```
List<Employee> employees=criteria.list();
```

```
for(Employee emp:employees){
```

```
System.out.println(emp.getName()+" : "+emp.getSalary());
```

```
}
```

Example code to read the Employee records whose name is "ramu"

```
Session session=sessionFactory.openSession();
```

```
Criteria crit = session.createCriteria(Employee.class);
```

```
Criterion nameCriterion=Restrictions.eq("name","ramu");
```

```
crit.add(nameCriterion);
```

```
List<Employee> results = crit.list();
```

Example code to read All the Employee records whose salary is > 8000.0

```
Session session=sessionFactory.openSession();
```

```
Criteria crit=session.createCriteria(Employee.class);
```

```
crit.add(Restrictions.gt("salary",new Double(8000.0)));
```

```
List<Employee> results = crit.list();
```

Example code to read All the Employee records whose name is starts with "ra" and salary is > 8000.0

```
Session session=sessionFactory.openSession();
```

```
Criteria crit=session.createCriteria(Employee.class);
```

```
Criterion nameCriterion= Restrictions.like("name","ra", MatchMode.START);
```

```
Criterion salaryCriterion= Restrictions.gt("salary",new Double(8000.0));
```

```
Criterion condition=Restrictions.and(nameCriterion,salaryCriterion);
```

```
crit.add(condition);
```

```
List<Employee> results = crit.list();
```

Associations

In HQL, we can reference an association property by its name to trigger an implicit join. But for criteria queries, will the same thing happen?

Consider example Book class with to have association with Publisher class

```
Criteria criteria = session.createCriteria(Book.class)
```

```
.add(Restrictions.like("name", "%Hibernate%"))
```

```
.add(Restrictions.eq("publisher.name", "Manning"));
```

```
List books = criteria.list();
```

If you execute the above code, an exception will be thrown for Hibernate could not resolve the property "publisher.name". That means implicit join is not supported for criteria queries. To join an association explicitly, you need to create a new criteria object for it.

```
Criteria criteria = session.createCriteria(Book.class)
```

```
.add(Restrictions.like("name", "%Hibernate%"))
```

```
.createCriteria("publisher")
```

```
.add(Restrictions.eq("name", "Manning"));
```

```
List books = criteria.list();
```

```
from Book book where book.name like '%Hibernate%' and book.publisher.name = 'Manning'
```

When using criteria query, you can specify the fetch mode of an association dynamically.

```
Criteria criteria = session.createCriteria(Book.class)
```

```
.add(Restrictions.like("name", "%Hibernate%"))
```

```
.setFetchMode("publisher", FetchMode.JOIN)
```

```
.setFetchMode("chapters", FetchMode.SELECT);
```

```
List books = criteria.list();
```

Projections :-

If we want to select particular properties (columns) (OR) to perform aggregate functions use Projections. We have done this in HQL also, but there we don't need any API support.

As part of HQL query it self we can specify required Properties(column) names.

But in criteria we have separate API to specify required properties.

Steps to work with Projections

Step1 :-

Create org.hibernate.criterion. Projection Object's

Example :

```
Projection nameProjection=Projections.property("name");
```

```
Projection salaryProjection =Projections.property("salary");
```

Projection is an interface ,which represents one selected property (OR)selected aggregate functions.

Projections is a utility class which contains methods to create

Projection Objects. These methods returns different xxxProjection Objects

Step 2:-

Create org.hibernate.criterion.ProjectionList Object

Example :

```
ProjectionList plist=Projections.projectionList();
```

ProjectionList is a class,which is used to hold multiple Projection Objects

Step 3:-

add Projection object's to ProjectionList

Example :

```
plist.add(nameProjection);
```

```
plist.add(salaryProjection);
```

Step 4:-

set projectionList object into criteria Object

Example :

```
criteria.setProjection(plist);
```

Examples on Projections :-

Example code to read the Employees names

```
Session session=sessionFactory.openSession();
```

```
Criteria crit = session.createCriteria(Employee.class);
```

```

Projection nameProjection=Projections.property("name");
crit.setProjection(nameProjection);
List<String> results=crit.list();

```

Example code to read the Employees names and salaries

```

Session session=sessionFactory.openSession();
Criteria crit = session.createCriteria(Employee.class);
Projection nameProjection=Projections.property("name");
Projection salaryProjection=Projections.property("salary");
ProjectionList plist=Projections.projectionList();
Plist.add(nameProjection);
Plist.add(salaryProjection);
crit.setProjection(plist);
List<Object[]> results=crit.list();

```

Note:-if only one selected Property is there then we can setProjection Object directly to Criteria,But if we havemultiple selected Properties,Then we should go forProjectionList.

Ordering and grouping

The results can be sorted in ascending or descending order also.

```

Criteria criteria = session.createCriteria(Book.class)
.addOrder(Order.asc("name"))
.addOrder(Order.desc("publishDate"));
List books = criteria.list();
from Book book

```

order by book.name asc, book.publishDate desc

Grouping is also supported for criteria queries. You can assign any properties as group properties and they will appear in the group by clause.

```

Criteria criteria = session.createCriteria(Book.class)
.setProjection(Projections.projectionList()
.add(Projections.groupProperty("publishDate"))
.add(Projections.avg("price"))
);
List books = criteria.list();
select book.publishDate, avg(book.price)
from Book book
group by book.publishDate

```

Querying by example

Criteria queries can be constructed through an "example" object. The object should be an instance of the persistent class you want to query. All null properties are ignored by default.

```

Book book = new Book();
book.setName("Hibernate");
book.setPrice(new Integer(100));
Example exampleBook = Example.create(book);

```

```
Criteria criteria = session.createCriteria(Book.class)
.add(exampleBook);
List books = criteria.list();
from Book book
```

where book.name = 'Hibernate' and book.price = 100

You can specify some creation rules for the construction of example object. For example, **exclude** some of the properties or enable "like" for string comparison.

```
Book book = new Book();
book.setName("%Hibernate%");
book.setPrice(new Integer(100));
Example exampleBook = Example.create(book)
.excludeProperty("price")
.enableLike();
```

```
Criteria criteria = session.createCriteria(Book.class)
.add(exampleBook);
List books = criteria.list();
from Book book
where book.name like '%Hibernate%'
```

Pagination methods of Criteria:-

While loading objects from the database instead of loading all objects at a time it is possible to load a particular no. of object's from a given point using methods criteria methods in criteria, we have the following two methods to get pagination behaviour.

public Criteria setFirstResult(int firstResult) : specifies the index of the first row to retrieve from. if not set, rows will be retrieved beginning from row 0 (zero).

public Criteria setMaxResult(int totalResult) : this method is used to specify the total number of records to be retrieved.

public Criteria addOrder(Order o) : used to specify ordering.

Example code to get the Employee records in ascending order on the basis of salary

```
Criteria crit=session.createCriteria(Employee.class);
crit.addOrder(Order.asc("salary"));
List<Employee> list=c.list();
```

Property class : org.hibernate.criterion.Property is a class available in hibernate API. Property is used to create Criterion objects (means creating conditions of the query). Property is just like Restrictions class, both are meant for creating Criterion objects.

You can create a Property by calling

```
Property propertyObj=Property.forName("property-Name")
```

Example Code:-

```
Criteria crit=session.createCriteria(Employee.class);
Property nameProperty=Property.forName("name");
Criterion nameCriterion=nameProperty.eq("raja");
```



```
Property salaryProperty=Property.forName("salary");
Criterion salaryCriterion=salaryProperty.gt(18000.0);
Criterion criterion=Restrictions.and(namecriterion,salarycriterion);
crit.add(criterion);
List<Employee> results=criteria.list();
```

Filters :

Filters are used to select some specific data from the database.

With Hibernate3 there is a new way to filtering the review of searches. Sometimes it is necessary to only process a subset of the record in the underlying database having tables. Hibernate filters are essential in those situations.

Other methodologies for these kind of problems is to use a database view or use a WHERE clause in the query or Hibernate Criteria API.

When to use Hibernate Filters

Suppose a web application that does the reporting for various flights. In future course there is some changes in requirement such that flights are to be shown as per their status (on time, delayed or cancelled).

This can also be done using a WHERE clause within the SQL SELECT query or Hibernate's HQL SELECT query. For a mini application it is fine to do this, but for a huge and complex application it might be a tiresome effort. Although it will be like searching each and every SQL query and making the changes in the existing code. But it has been thoroughly tested.

This can also be done using Hibernate's Criteria API but that also means changing the code at numerous places that is all working fine. Moreover in both the approaches, one needs to be very cautious so that they are not changing existing working SQL queries in inadvertent way.

Filters can be used like database views and data dictionary, but parameterized inside any application. This way they are essential when developers have very little control over DB operations. Here we are going to show you the usage of Hibernate filters to solve this problem. When the end users select the status, our application activates the flight's status for the end user's Hibernate session.

Any SQL query will only return the subset of flights with the user chooses status. **Flight status** is maintained at two places- Hibernate Session and flight status filter.

How to use Hibernate Filters

Hibernate filters are defined in Hibernate mapping documents (hbm.xml file)-which are easy to manage. One can programmatically turn active or sleep mode the filters in the application code. Though filters cannot be created dynamically, they can be parameterized which makes them quite

portable in nature. We suggest the filter on the column which is being used to enable/disable visibility rules. Please go through the demo application in which the filter is applied on flight status column and it must match a named parameter. After that we provide dynamic values at run time.

Demo Application

The example application is a very elementary flight reporting system.

A good amount of details in terms of working with Hibernate has been abstracted into HibernateUtil.java so that novice users can also use the sample application. The file 'hibernate.cfg.xml' can be used as it is. There is one hibernate configuration file. This application is ready to be executed provided you have MySQL DB previously installed. To make it run with other databases, we need to make the changes in 'hibernate.cfg.xml' file. DDL is also provided as part of the Zip file.

Getting started with the example

First define filters in the Hibernate mapping documents, using the <filter-def> XML element.

These filter declaration should contain the name of the filter and the names and types of any filter parameters. Demonstrate filter parameters with the <filter-param> XML element. Filter parameters are same to the named parameters for HQL queries. We need to specify a : (colon) before the parameter condition. Here is the mapping file from the code.

Listing 1: Mapping File

```
<?xml version="1.0"?>
<!DOCTYPE hibernate-mapping PUBLIC "-//Hibernate/Hibernate Mapping DTD"
"http://hibernate.sourceforge.net/hibernate-mapping">
<hibernate-mapping package="data">
<class name="Flight" table="hibernate_filter_demo.flightdb">
<id name="id" type="int" column="id">
<generator class="increment"/>
</id>
<property name="flightNo" type="string" length="10" column="flight_No"/>
<property name="src" type="string" length="15" column="source"/>
<property name="dest" type="string" length="15" column="destination"/>
<property name="info" type="string" length="20" column="info"/>
<strong><filter name="statusFilter" condition=":statusParam=status"/>
</class>
<filter-def name="statusFilter">
<filter-param name="statusParam" type="string"/>
</filter-def></strong>
</hibernate-mapping>
```

Now attach the filters to class or collection mapping elements. We can join a single filter to more than one class or collection. To do this, you add a <filter> XML element to each class or collection.

The <filter> XML element has two attributes viz. values and condition. The value references a filter definition while condition is analogous to a WHERE clause in HQL. Let's see the complete coding from the HibernateFilters.zip archive.

Note : Each <filter> XML element must correspond to a <filter-def> element. It may be possible to have more than one filter for each filter definition, and each class may have more than one filter.

Concept is to define all the filter parameters in one place and then refer them in the individual filter conditions.

In the java code, we can programmatically enable or hide the filter. By default the Hibernate Session doesn't have any filters visible on it.

Methods of Session Interface:

- public Filter enableFilter(String NameofFilter)
- public Filter getEnabledFilter(String NameofFilter)
- public void disableFilter(String NameofFilter)

Methods of Filter Interface:

- public Filter setParameter(String NameofFilter, Object value)
- public Filter setParameterList(String NameofFilter, Collection values)
- public Filter setParameterList(String NameofFilter, Object[] values)

setParameter() method is almost used. Be careful and define only the type of java object that we have mentioned in the parameter at the time of defining filter in the mapping file.

The two setParameterList() methods are useful for using IN clauses in your filters. If we want to use BETWEEN clauses, must use two different filter parameters with different names.

At the time of enabling the filter on session-use the name that you have provided in the mapping file for the filter name for the corresponding column in the table. Similarly string name should contain one of the possible values for that particular column.

Listing 2: This condition is set on the filter

```
public class HibernateFilterDemo{
    public static void main(String argsp[]){
        SessionFactory factory = HibernateUtil.getSessionFactory();
        Session session = factory.openSession();
        insertData("DL6159", "RFC", "JCK", "dealy 15 min", session);
        .....
        <strong>Filter filter = session.enableFilter("statusFilter");
        filter.setParameter("statusParam", "delayed");</strong>
        showData(session);
    }
}
```

```
.....
session.close();
}
public static void insertData(String fliteNo, String src,
String dest, String info, Session ses){
session.beginTransaction();
session.getTransaction().commit();
}
public static void showData(Session session){
session.beginTransaction();
Query query = session.createQuery("from Flight");
.....
session.getTransaction().commit();
}
}
```

Conclusion

Hibernate filters is one of the best option to DB views, Hibernate Criteria API and SQL where clause. It is an efficient way to separate database concerns from the remaining application code. They help in minimizing the complexity of HQL or SQL queries. Filters can be made visible/on as and when they are required.

Example Program on Filters

User.java

```
public class User{
    private int id;
    private String name;
    private boolean activated;

    public boolean isActivated(){
        return activated;
    }
    public void setActivated(boolean activated){
        this.activated = activated;
    }
    public int getId(){
        return id;
    }
    public void setId(int id){
        this.id = id;
    }
    public String getName(){
        return name;
    }
    public void setName(String name){
```

```
    this.name = name;
  }
}
```

User.hbm.xml

```
<?xml version='1.0' encoding='utf-8'?>
<!DOCTYPE hibernate-mapping
    PUBLIC "-//Hibernate/Hibernate Mapping DTD//EN"
    "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">

<hibernate-mapping>
<class name="User" table="user_data">
<id name="id" type="int">
<generator class="increment"/>
</id>
<property name="name" type="string" length="32"/>
<property name="activated" type="boolean"/>
<filter name="activatedFilter"
    condition=":activatedParam = activated"/>
</class>
<filter-def name="activatedFilter">
<filter-param name="activatedParam" type="boolean"/>
</filter-def>
</hibernate-mapping>
```

hibernate.cfg.xml

```
<?xml version='1.0' encoding='utf-8'?>
<!DOCTYPE hibernate-configuration
    PUBLIC "-//Hibernate/Hibernate Configuration DTD//EN"
    "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">

<hibernate-configuration>
<session-factory>
<property name="hibernate.connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
<property name="hibernate.connection.url">jdbc:oracle:thin:@localhost:1521:XE</property>
<property name="hibernate.connection.username">system</property>
<property name="connection.password">manager</property>
<property name="hibernate.dialect">org.hibernate.dialect.OracleDialect</property>
<property name="hibernate.autocommit">>true</property>
<property name="show_sql">>false</property>
```

```
<mapping resource="User.hbm.xml"/>
</session-factory>
</hibernate-configuration>
```

SimpleFilterExample.java

```
import java.util.Iterator;

import org.hibernate.cfg.Configuration;
import org.hibernate.Filter;
import org.hibernate.Query;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;

public class SimpleFilterExample{
    public static void main (String args[]){
        SessionFactory factory = new Configuration().configure().buildSessionFactory();
        Session session = factory.openSession();
        //insert the users
        insertUser("pavan",true,session);
        insertUser("chiru",false,session);
        insertUser("ramu",false,session);
        //Show all users
        System.out.println("===ALL USERS===");
        displayUsers(session);
        //Show activated users
        Filter filter = session.enableFilter("activatedFilter");
        filter.setParameter("activatedParam",new Boolean(true));
        System.out.println("===ACTIVATED USERS===");
        displayUsers(session);
        //Show non-activated users
        filter.setParameter("activatedParam",new Boolean(false));
        System.out.println("===NON-ACTIVATED USERS===");
        displayUsers(session);
        session.close();
    }

    public static void displayUsers(Session session){
        Query query = session.createQuery("from User");
        Iterator results = query.iterate();
        while (results.hasNext()){
            User user = (User) results.next();
            System.out.print(user.getName() + " is ");
            if (user.isActivated()){
                System.out.println("activated.");
            }
            else {

```

```

        System.out.println("not activated.");
    }
}

public static void insertUser(String name, boolean activated, Session session){
    Transaction trans = session.beginTransaction();
    User user = new User();
    user.setName(name);
    user.setActivated(activated);
    session.save(user);
    trans.commit();
}
}

```

Batch processing with HQL

Suppose that we are planning a promotion for the Hibernate books in our online bookshop. All the books having the word "Hibernate" in the name will have 10 dollars discount. We could do it by querying the matched books first and then update them one by one.

```

Session session = factory.openSession();
Transaction tx = null;
try {
    tx = session.beginTransaction();
    Query query = session.createQuery("from Book where name like ?");
    query.setString(0, "%Hibernate%");
    List books = query.list();
    for (Iterator iter = books.iterator(); iter.hasNext(); ) {
        Book book = (Book) iter.next();
        book.setPrice(new Integer(book.getPrice().intValue() - 10));
        session.saveOrUpdate(book);
    }
    tx.commit();
} catch (HibernateException e) {
    if (tx != null) tx.rollback();
    throw e;
} finally {
    session.close();
}

```

It can be easily noticed that this kind of update is very ineffective. If using SQL, we can write a single update statement for this task. Fortunately, HQL can also support this kind of batch update. Remember that it should be property names occurred in HQL statements, not column names.

```

Session session = factory.openSession();
Transaction tx = null;
try {
    tx = session.beginTransaction();

```

```
Query query = session.createQuery("update Book set price = price - ? where name like ?");
query.setInteger(0, 10);
query.setString(1, "%Hibernate%");
int count = query.executeUpdate();
tx.commit();
} catch (HibernateException e) {
    if (tx != null) tx.rollback();
    throw e;
} finally {
    session.close();
}
```

In addition to batch update, HQL can also support batch delete. Notice that the "from" keyword in the delete statement is optional.

```
Session session = factory.openSession();
Transaction tx = null;
try {
    tx = session.beginTransaction();
    Query query = session.createQuery("delete from Book where name like ?");
    query.setString(0, "%Hibernate%");
    int count = query.executeUpdate();
    tx.commit();
} catch (HibernateException e) {
    if (tx != null) tx.rollback();
    throw e;
} finally {
    session.close();
}
```

Querying with native SQL

In another business scenario, suppose we need to make some statistics on the top selling books for the users to refer. The process of statistics is very complicated and requires some native features provided by the database. HQL is not much helpful in this case. Hibernate supports using native SQL to query for objects.

For demo purpose, we won't implement that complicated statistics. We just create a view to emulate the result of query. The view TOP_SELLING_BOOK is created only by joining the BOOK and PUBLISHER table.

```
CREATE VIEW TOP_SELLING_BOOK (
    ID, ISBN, BOOK_NAME, PUBLISH_DATE, PRICE,
    PUBLISHER_ID, PUBLISHER_CODE, PUBLISHER_NAME, PUBLISHER_ADDRESS
) AS
SELECT book.ID, book.ISBN, book.BOOK_NAME, book.PUBLISH_DATE, book.PRICE,
book.PUBLISHER_ID, pub.CODE, pub.PUBLISHER_NAME, pub.ADDRESS
FROM BOOK book LEFT OUTER JOIN PUBLISHER pub ON book.PUBLISHER_ID = pub.ID
```


First, we retrieve the book objects from that view only and ignore the associated publishers. We use a native SQL to select all the columns related to a book. The `addEntity()` method is used to specify the type of resulting persistent objects.

```
String sql = "SELECT ID, ISBN, BOOK_NAME, PUBLISH_DATE, PRICE, PUBLISHER_ID  
FROM TOP_SELLING_BOOK  
WHERE BOOK_NAME LIKE ?";  
Query query = session.createSQLQuery(sql)  
  
.addEntity(Book.class)  
.setString(0, "%Hibernate%");  
List books = query.list();
```

Since all the columns in the result have the same names as in the mapping definition of `Book`, we can simply specify `{book.*}` in the select clause. Hibernate will replace the `{book.*}` with all the column names in our mapping definition.

```
String sql = "SELECT {book.*}  
FROM TOP_SELLING_BOOK book  
WHERE BOOK_NAME LIKE ?";  
Query query = session.createSQLQuery(sql)  
.addEntity("book", Book.class)  
.setString(0, "%Hibernate%");  
List books = query.list();
```

Next, let's consider the associated publishers. Since not all the column names are identical to those in the mapping definition of `Publisher`, we need to map them in the select clause of SQL manually.

The `addJoin()` method is used to specify the joined associations.

```
String sql = "SELECT {book.*},  
book.PUBLISHER_ID as {publisher.id},  
book.PUBLISHER_CODE as {publisher.code},  
book.PUBLISHER_NAME as {publisher.name},  
book.PUBLISHER_ADDRESS as {publisher.address}  
FROM TOP_SELLING_BOOK book  
WHERE BOOK_NAME LIKE ?";  
Query query = session.createSQLQuery(sql)  
.addEntity("book", Book.class)  
.addJoin("publisher", "book.publisher")  
.setString(0, "%Hibernate%");  
List books = query.list();
```

If we want to query for some simple values only, we can use the `addScalar()` method.

```
String sql = "SELECT max(book.PRICE) as maxPrice  
FROM TOP_SELLING_BOOK book  
WHERE BOOK_NAME LIKE ?";  
Query query = session.createSQLQuery(sql)  
.addScalar("maxPrice", Hibernate.INTEGER)
```

```
.setString(0, "%Hibernate%");
Integer maxPrice = (Integer) query.uniqueResult();
```

Named SQL queries

The native SQL statements can also be put in a mapping definition and referred by name in the Java code. We can use <return> and <return-join> to describe the resulting objects.

```
<hibernate-mapping package="com.nareshit.hibernate.bookshop">
<sql-query name="TopSellingBook.by.name">
<return alias="book" class="Book" />
<return-join alias="publisher" property="book.publisher"/>
<![CDATA[
SELECT {book.*},
book.PUBLISHER_ID as {publisher.id},
book.PUBLISHER_CODE as {publisher.code},
book.PUBLISHER_NAME as {publisher.name},
book.PUBLISHER_ADDRESS as {publisher.address}
FROM TOP_SELLING_BOOK book
WHERE BOOK_NAME LIKE ?
]]>
</sql-query>
</hibernate-mapping>
Query query = session.getNamedQuery("TopSellingBook.by.name")
.setString(0, "%Hibernate%");
List books = query.list();
```

The query for simple values can also be wrapped as a named query. In this case, <return-scalar> is used instead.

```
<hibernate-mapping package="com.nareshit.hibernate.bookshop">
...
<sql-query name="TopSellingBook.maxPrice.by.name">
<return-scalar column="maxPrice" type="int" />
<![CDATA[
SELECT max(book.PRICE) as maxPrice
FROM TOP_SELLING_BOOK book
WHERE BOOK_NAME LIKE ?
]]>
</sql-query>
</hibernate-mapping>
Query query = session.getNamedQuery("TopSellingBook.maxPrice.by.name")
.setString(0, "%Hibernate%");
Integer maxPrice = (Integer) query.uniqueResult();
```

For a named SQL query, the <return> and <return-join> can be grouped in a "result set mapping" and referenced in the <sql-query>. The advantage of using result set mapping is that it can be reused for multiple queries.

```

<hibernate-mapping package="com.nareshit.hibernate.bookshop">
<resultset name="bookPublisher">
<return alias="book" class="Book" />
<return-join alias="publisher" property="book.publisher" />
</resultset>
<sql-query name="TopSellingBook.by.name" resultset-ref="bookPublisher">
<![CDATA[
SELECT {book.*},
book.PUBLISHER_ID as {publisher.id},
book.PUBLISHER_CODE as {publisher.code},
book.PUBLISHER_NAME as {publisher.name},
book.PUBLISHER_ADDRESS as {publisher.address}
FROM TOP_SELLING_BOOK book
WHERE BOOK_NAME LIKE ?
]]>
</sql-query>
</hibernate-mapping>

```

In the result set mapping, we can further map each database column to an **object property**. It can simplify our SQL statement by removing the mapping from the select clause.

```

<hibernate-mapping package="com.nareshit.hibernate.bookshop">
<resultset name="bookPublisher">
<return alias="book" class="Book" />
<return-join alias="publisher" property="book.publisher">
<return-property name="id" column="PUBLISHER_ID" />
<return-property name="code" column="PUBLISHER_CODE" />
<return-property name="name" column="PUBLISHER_NAME" />
<return-property name="address" column="PUBLISHER_ADDRESS" />
</return-join>
</resultset>
<sql-query name="TopSellingBook.by.name" resultset-ref="bookPublisher">
<![CDATA[
SELECT {book.*},
book.PUBLISHER_ID as {publisher.id},
book.PUBLISHER_CODE as {publisher.code},
book.PUBLISHER_NAME as {publisher.name},
book.PUBLISHER_ADDRESS as {publisher.address}
FROM TOP_SELLING_BOOK book
WHERE BOOK_NAME LIKE ?
]]>
</sql-query>
</hibernate-mapping>

```

After moving the mapping of columns to the result set mapping, the select clause **can be simplified** to a "select all".

```

<hibernate-mapping package="com.nareshit.hibernate.bookshop">
...
<sql-query name="TopSellingBook.by.name" resultset-ref="bookPublisher">

```

```
<![CDATA[
SELECT *
FROM TOP_SELLING_BOOK book
WHERE BOOK_NAME LIKE ?
]]>
</sql-query>
</hibernate-mapping>
```

Example on native sql and batch processing

Suppose there is one situation in which you have to insert 1000000 records in to database in a time. So what to do in this situation...

In Native Solution in the Hibernate

```
Session session = sessionFactory.openSession();
Transaction tx = session.beginTransaction();
for ( int i=0; i<1000000; i++ )
{
    Student student = new Student(.....);
    session.save(student);
}
tx.commit();
session.close();
```

Because by default, Hibernate will cache all the persisted objects in the **session-level cache** and ultimately your application would fall over with an **OutOfMemoryException** somewhere around the 50,000th row. You can resolve this problem if you are using batch processing with Hibernate.

To use the batch processing feature, first set **hibernate.jdbc.batch_size** as batch size to a number either at 20 or 50 depending on object size. This will tell the hibernate container that every X rows to be inserted as batch. To implement this in your code we would need to do little modification as follows:

```
Session session = SessionFactory.openSession();
Transaction tx = session.beginTransaction();
for ( int i=0; i<1000000; i++ )
{
    Student student = new Student(.....);
    session.save(employee);
    if( i % 50 == 0 ) // Same as the JDBC batch size
    {
        //flush a batch of inserts and release memory:
        session.flush();
        session.clear();
    }
}
tx.commit();
session.close();
```

Above code will work fine for the INSERT operation, but if you want to make UPDATE operation then you can achieve using the following code:

```
Session session = sessionFactory.openSession();
Transaction tx = session.beginTransaction();
ScrollableResults studentCursor = session.createQuery("FROM STUDENT").scroll();
int count = 0;
while(studentCursor.next())
{
    Student student = (Student) studentCursor.get(0);
    student.setName("DEV");
    session.update(student);
    if ( ++count % 50 == 0 ) {
        session.flush();
        session.clear();
    }
}
tx.commit();
session.close();
```

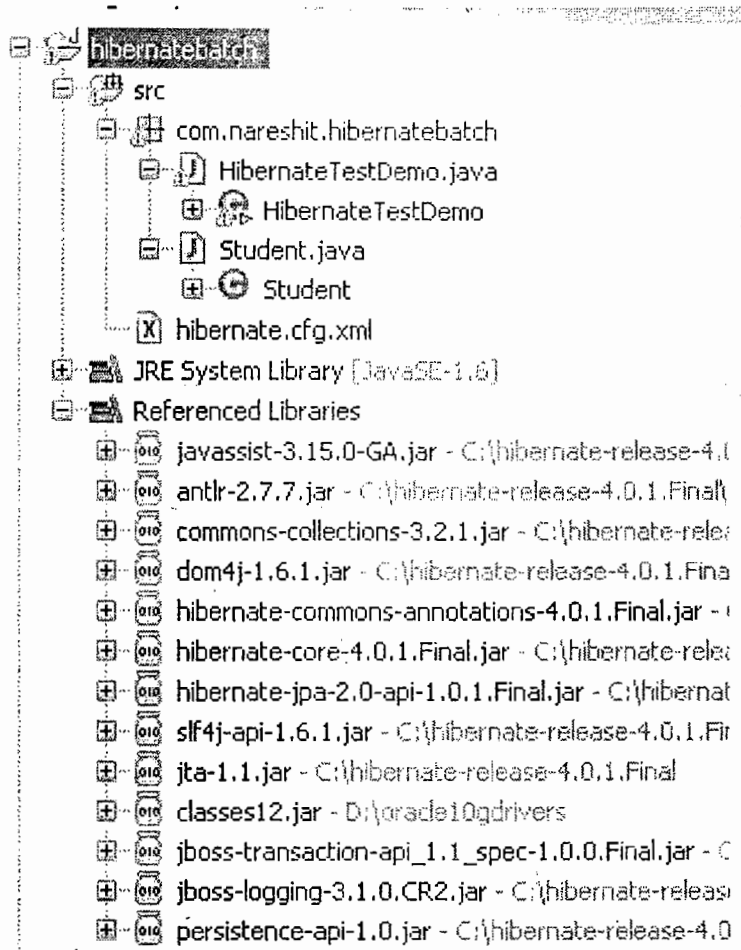
In Batch Processing Solution in the Hibernate

If you are undertaking batch processing you will need to enable the use of JDBC batching. This is absolutely essential if you want to achieve optimal performance. Set the JDBC batch size to a reasonable number (10-50).

hibernate.jdbc.batch_size 50

You can also do this kind of work in a process where interaction with the second-level cache is completely disabled:

hibernate.cache.use_second_level_cache false



```
//Student.java
```

```
package com.nareshit.hibernatebatch;
```

```
import java.io.Serializable;
```

```
import javax.persistence.Column;
```

```
import javax.persistence.Entity;
```

```
import javax.persistence.GeneratedValue;
```

```
import javax.persistence.GenerationType;
```

```
import javax.persistence.Id;
```

```
import javax.persistence.Table;
```

```
@Entity
```

```
@Table(name = "STUDENT")
```

```
public class Student implements Serializable {
```

```
/**
```

```
 * serialVersionUID
```

```
 */
```

```
private static final long serialVersionUID = 8633415090390966715L;
```

```
@Id
```

```
@Column(name = "ID")
```

```
@GeneratedValue(strategy = GenerationType.AUTO)
```

```
private int id;
```

```
@Column(name = "STUDENT_NAME")
```

```
private String studentName;
@Column(name = "ROLL_NUMBER")
private int rollNumber;
@Column(name = "COURSE")
private String course;

public int getId() {
    return id;
}

public void setId(int id) {
    this.id = id;
}

public String getStudentName() {
    return studentName;
}

public void setStudentName(String studentName) {
    this.studentName = studentName;
}

public int getRollNumber() {
    return rollNumber;
}

public void setRollNumber(int rollNumber) {
    this.rollNumber = rollNumber;
}

public String getCourse() {
    return course;
}

public void setCourse(String course) {
    this.course = course;
}

public String toString() {
    return "ROLL Number: " + rollNumber + "| Name: " + studentName
        + "| Course: " + course;
}
}
```

```
//HibernateTestDemo.java
package com.nareshit.hibernatebatch;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.AnnotationConfiguration;

import com.nareshit.hibernatebatch.Student;

public class HibernateTestDemo {

    /**
     * @param args
     */
    public static void main(String[] args) {
        SessionFactory sessionFactory = new AnnotationConfiguration()
            .configure().buildSessionFactory();
        Session session = sessionFactory.openSession();
        Transaction transaction = session.beginTransaction();

        for (int i = 0; i < 100000; i++) {
            String studentName = "Kumar " + i;
            int rollNumber = 9 + i;
            String course = "B.Tech " + i;
            Student student = new Student();
            student.setStudentName(studentName);
            student.setRollNumber(rollNumber);
            student.setCourse(course);
            session.save(student);
            if (i % 50 == 0) {
                session.flush();
                session.clear();
            }
        }
        transaction.commit();
        session.close();
    }
}
```

Execute HibernateTestDemo, you will see the output

Another example for Native-SQL

Department.java

```
package com.nareshit.pojo;
import java.io.Serializable;
import java.util.*;

public class Department implements Serializable{
```



```
private int deptno;
private String deptname,location;
Set<Employee> employees;
//required setters and getters
```

Employee.hbm.xml

```
package com.nareshit.pojo;
public class Employee {
    private int empno;
    private String name;
    private double salary;
    private Department department;
    public Employee() {
    }
    public Employee(String name,double salary) {
        this.name=name;
        this.salary=salary;
    }
    //required setters and getters
}
```

HibernateUtility.java

```
package com.nareshit.utility;
import org.hibernate.*;
import org.hibernate.cfg.Configuration;
public class HibernateUtility {
    private static SessionFactory factory;
    static{
        Configuration cfg=new Configuration();
        cfg.configure("hibernate.cfg.xml");
        factory=cfg.buildSessionFactory();
    }
    public static Session getSession(){
        Session session=null;
        if(factory!=null){
            session=factory.openSession();
        }
        return session;
    }
}
```

Department.hbm.xml

```
<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
<class name="com.nareshit.pojo.Department"
```

```

table="DepartmentDetails">
<id name="deptno" length="12">
</id>
<property name="deptname" length="15"/>
<property name="location" length="15"/>
<set name="employees" cascade="all"
table="employeedetails">
<key column="dept_no" />
<one-to-many class="com.nareshit.pojo.Employee" />
</set></class>
<sql-query name="getEmployeesByDeptNo"> select *from departmentdetails where deptno=?
</sql-query>
<hibernate-mapping>

```

Employee.hbm.xml

```

<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
<class name="com.nareshit.pojo.Employee" table="employeeDetails">
<id name="empno" length="12">
<generator class="increment"/></id>
<property name="name" length="15"/>
<property name="salary" length="7"/>
<many-to-one name="department" cascade="all"
class="com.nareshit.pojo.Department">
<column name="dept_no" />
</many-to-one>
</class>
</hibernate-mapping>

```

Hibernate.cfg.xml

```

<?xml version='1.0' encoding='UTF-8'?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
<session-factory>
    <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
    <property name="connection.url">jdbc:oracle:thin:@localhost:1521:XE</property>
    <property name="connection.username">system</property>
    <property name="connection.password">manager</property>
    <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
    <property name="hbm2ddl.auto">update</property>
    <property name="show_sql">true</property>
<mapping resource="Employee.hbm.xml"/>

```

```
<mapping resource="Department.hbm.xml"/>
</session-factory>
<hibernate-configuration>
```

Test.java

```
package com.nareshit.client;
import java.util.*;
import org.hibernate.*;
import org.hibernate.transform.Transformers;
import org.hibernate.type.StandardBasicTypes;
import com.nareshit.pojo.*;
import com.nareshit.utility.HibernateUtility;

public class Test {
    public static void main(String[] args){
        System.out.println("Testing of createEmployees(-)");
        createEmployees();
        System.out.println("=====");
        System.out.println("Testing of getEmployeesByName(-)");
        getEmployeesByName("ajay");
        System.out.println("=====");
        System.out.println("Testing of getEmployeesById(-)");
        getEmployeeById(2);
        System.out.println("=====");
        System.out.println("Testing of getEmployeesBySal(-)");
        getEmployeeDetailsBySal();
        System.out.println("=====");
        System.out.println("Testing of getEmployeesByDept(13)");
        getEmployeesByDept(13);
        System.out.println("=====");
    }

    public static void getEmployeesByName(String name){
        Session session=HibernateUtility.getSession();
        String sql="select * from EmployeeDetails where name=?";
        SQLQuery query=session.createSQLQuery(sql);
        query.setParameter(0,name);
        query.addEntity(Employee.class);
        //addEntity() is used to map the results with Entity Types
        List<Employee> list=query.list();
        for(Employee emp:list){
            System.out.println(emp.getEmpno());
            System.out.println(emp.getName());
        }
    }
}
```

```
System.out.println(emp.getSalary());
    Department d=emp.getDepartment();
    System.out.println(d.getDeptname()+" "+d.getDeptno());
}
} //end of getEmployeesByName(-)
public static void getEmployeeById(int empno){
    Session session=HibernateUtility.getSession();
    String sql="select e.empno as empno, e.name as empName ,e.salary as sal, e.dept_no as deptno,
    d.deptname as deptname, d.location as location from employeeDetails e join departmentdetails d
    on e.dept_no=d.deptno where empno="+empno;
    SQLQuery query=session.createSQLQuery(sql);
    query.addScalar("empName",StandardBasicTypes.STRING);
    query.addScalar("sal",StandardBasicTypes.DOUBLE);
    query.addScalar("deptno",StandardBasicTypes.INTEGER);
    query.addScalar("deptname",StandardBasicTypes.STRING);
    query.addScalar("location",StandardBasicTypes.STRING);
    List<Object[]> list=query.list();
    for(Object[] obj:list){
        for(Object obj1:obj){
            System.out.println(obj1);
        }
    }
}
public static void getEmployeeDetailsBySal(){
    Session session=HibernateUtility.getSession();
    String sql="select empno,name from employeeDetails where salary>=?";
    SQLQuery query=session.createSQLQuery(sql);
    query.setParameter(0,12000.0);
    query.addScalar("empno",StandardBasicTypes.INTEGER);
    query.addScalar("name",StandardBasicTypes.STRING);
    query.setResultTransformer(Transformers.aliasToBean(Employee.class));
    List<Employee> list=query.list();
    for(Employee emp:list){
        System.out.println(emp.getEmpno());
        System.out.println(emp.getName());
        System.out.println(emp.getSalary());
    }
}
public static void getEmployeesByDept(int deptno){
    Session session=HibernateUtility.getSession();
    SQLQuery query=(SQLQuery)session.getNamedQuery("getEmployeesByDeptNo");
```

```
query.setParameter(0,deptno);
query.addEntity(Department.class);
List<Department> list=query.list();
Department dept=list.get(0);
System.out.println("department Name : "+dept.getDeptname());
Set<Employee> set=dept.getEmployees();
System.out.println("Employees Details ");
for(Employee emp:set){
    System.out.println("Emp Num :"+emp.getEmpno());
    System.out.println("Emp Name :"+emp.getName());
    System.out.println("-----");
}
}
public static void createEmployees(){
Session session=HibernateUtility.getSession();
Department department = new Department();
department.setDeptno(13);
department.setDeptname("admin");
department.setLocation("hyd");
Employee emp1=new Employee("ramu",9000);
Employee emp2=new Employee("raju",12000);
Employee emp3=new Employee("ajay",13000);
Employee emp4=new Employee("ajay",11000);
Set<Employee> employees=new HashSet<Employee>();
employees.add(emp1);
employees.add(emp2);
employees.add(emp3);
employees.add(emp4);

//add the set to department
department.setEmployees(employees);
session.save(department);
session.beginTransaction().commit();
System.out.println("inserted");
}
}
```

Caching

Cache/buffer is a temprary memory that holds data for temporary period.The process of storing data in cache/buffer and using that data accross the muliple request/execution is called as

caching/buffering.

In client server environment the cache/buffer that resides at client side holds results given by server application and reduces n/w round trips b/w client and server across the multiple same request.

The buffer of browser window reduces n/w round trips b/w Browser window and website/web server. Caching in HB resides with HB based client App, holds HB pojo class obj's of selected records and reduces n/w round trips b/w client app and db s/w

To get maximum benefits from caching we need to delete content of cache/buffer that resides with client app at regular intervals.

Hibernate supports two levels of caching.

First level cache-->Resides at Session object specific to each Session object. it is built in cache.

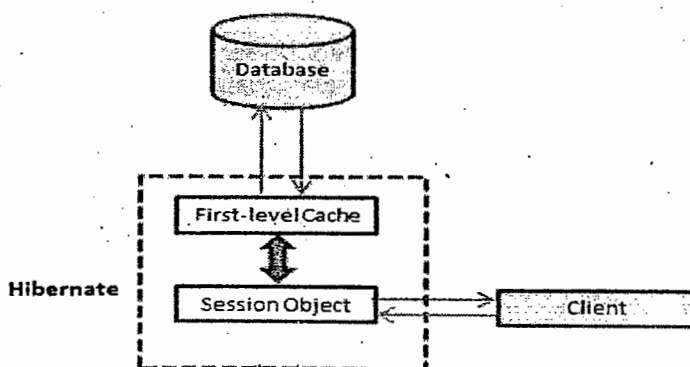
Second level cache-->Resides at SessionFactory obj it is not built in cache, configure explicitly.

First and Second level caches of HB s/w, maintains the results(records) given by db table in the form of HB pojo class objects.

First level cache is called local cache to session object.

First level cache in hibernate is enabled by default and you do not need to do anything to get this functionality working. In fact, you can not disable it even forcefully.

Its easy to understand the first level cache if we understand the fact that it is associated with Session object. As we know session object is created on demand from session factory and it is lost, once the session is closed. Similarly, first level cache associated with session object is available only till session object is live. It is available to session object only and is not accessible to any other session object in any other part of application.



1. First level cache is associated with "session" object and other session objects in application can not see it.
2. The scope of cache objects is of session. Once session is closed, cached objects are gone forever.
3. First level cache is enabled by default and you can not disable it.
4. When we query an entity first time, it is retrieved from database and stored in first level cache associated with hibernate session.
5. If we query same object again with same session object, it will be loaded from cache and no sql query will be executed.
6. The loaded entity can be removed from session using evict() method. The next loading of this entity will again make a database call if it has been removed using evict() method.
7. The whole session cache can be removed using clear() method. It will remove all the entities stored in cache.

Lets verify above facts using examples.

First level cache retrieval example

In this example, I am retrieving DepartmentEntity object from database using hibernate session. I will retrieve it multiple times, and will observe the sql logs to see the differences.

```
//Open the hibernate session
```

```
Session session = HibernateUtil.getSessionFactory().openSession();  
session.beginTransaction();
```

```
//fetch the department entity from database first time
```

```
DepartmentEntity department = (DepartmentEntity)  
session.load(DepartmentEntity.class, new Integer(1));  
System.out.println(department.getName());
```

```
//fetch the department entity again
```

```
department = (DepartmentEntity) session.load(DepartmentEntity.class, new Integer(1));  
System.out.println(department.getName());
```

```
session.getTransaction().commit();  
HibernateUtil.shutdown();
```

Output:

```
Hibernate: select department0_.ID as ID0_0_, department0_.NAME as NAME0_0_ from  
DEPARTMENT department0_ where department0_.ID=?
```

```
Human Resource
```

```
Human Resource
```

As you can see that second "session.load()" statement does not execute select query again and load the department entity directly.

First level cache retrieval example with new session

With new session, entity is fetched from database again irrespective of it is already present in any other session in application.

```
//Open the hibernate session
```

```
Session session = HibernateUtil.getSessionFactory().openSession();
session.beginTransaction();
```

```
Session sessionTemp = HibernateUtil.getSessionFactory().openSession();
sessionTemp.beginTransaction();
```

```
try
{
    //fetch the department entity from database first time
    DepartmentEntity department = (DepartmentEntity)
session.load(DepartmentEntity.class, new Integer(1));
    System.out.println(department.getName());
```

```
    //fetch the department entity again
    department = (DepartmentEntity) session.load(DepartmentEntity.class, new Integer(1));
    System.out.println(department.getName());
```

```
    department = (DepartmentEntity) sessionTemp.load(DepartmentEntity.class, new Integer(1));
    System.out.println(department.getName());
```

```
}
finally
{
    session.getTransaction().commit();
    HibernateUtil.shutdown();

    sessionTemp.getTransaction().commit();
    HibernateUtil.shutdown();
}
```

Output:

```
Hibernate: select department0_.ID as ID0_0_, department0_.NAME as NAME0_0_ from
DEPARTMENT department0_ where department0_.ID=?
```

Human Resource

Human Resource

```
Hibernate: select department0_.ID as ID0_0_, department0_.NAME as NAME0_0_ from
DEPARTMENT department0_ where department0_.ID=?
```

Human Resource

You can see that even if the department entity was stored in "session" object, still another database query was executed when we use another session object "sessionTemp".

Removing cache objects from first level cache example

Though we can not disable the first level cache in hibernate, but we can certainly remove some of objects from it when needed. This is done using two methods :

- `evict()`
- `clear()`

Here `evict()` is used to remove a particular object from cache associated with session, and `clear()` method is used to remove all cached objects associated with session. So they are essentially like remove one and remove all.

```
//Open the hibernate session
Session session = HibernateUtil.getSessionFactory().openSession();
session.beginTransaction();
try
{
    //fetch the department entity from database first time
    DepartmentEntity department = (DepartmentEntity)
    session.load(DepartmentEntity.class, new Integer(1));
    System.out.println(department.getName());

    //fetch the department entity again
    department = (DepartmentEntity) session.load(DepartmentEntity.class, new Integer(1));
    System.out.println(department.getName());

    session.evict(department);
    //session.clear();

    department = (DepartmentEntity) session.load(DepartmentEntity.class, new Integer(1));
    System.out.println(department.getName());
}
finally
{
    session.getTransaction().commit();
    HibernateUtil.shutdown();
}
```

Output:

```
Hibernate: select department0_.ID as ID0_0_, department0_.NAME as NAME0_0_ from
DEPARTMENT department0_ where department0_.ID=?
Human Resource
Human Resource
```

```
Hibernate: select department0_.ID as ID0_0_, department0_.NAME as NAME0_0_ from
DEPARTMENT department0_ where department0_.ID=?
Human Resource
```

Clearly, `evict()` method removed the department object from cache so that it was fetched again from database.

The general concept of caching persistent objects is that when an object is first read from external storage, a copy of it will be stored in the cache. For the subsequent readings of the same object, it can be retrieved from the cache directly. Since caches are typically stored in memory or local disk, it will be faster to read an object from cache than external storage. If using properly, caching can greatly improve the performance of our application.

As a high performance O/R mapping framework, Hibernate supports the caching of persistent objects at different levels. Suppose we get an object with same identifier for two times within a session, will Hibernate query the database for two times?

```
Session session = factory.openSession();
try {
    Book book1 = (Book) session.get(Book.class, id);
    Book book2 = (Book) session.get(Book.class, id);
} finally {
    session.close();
}
```

If we inspect the SQL statements executed by Hibernate, we will find that only one database query is made. That means Hibernate is caching our objects in the same session. This kind of caching is called "first level caching", whose caching scope is a session.

But how about getting an object with same identifier for two times in two different sessions?

```
Session session1 = factory.openSession();
try {
    Book book1 = (Book) session1.get(Book.class, id);
} finally {
    session1.close();
}
Session session2 = factory.openSession();
try {
    Book book2 = (Book) session2.get(Book.class, id);
} finally {
    session2.close();
}
```

The second level caching

Second level cache is called as global cache.

To turn on the second level caching, the first step is to choose a cache provider in the Hibernate configuration file "hibernate.cfg.xml". Hibernate supports several kinds of cache implementation, such as `EHCache`, `OSCache`, `SwarmCache` and `JBossCache`. In a non-distributed environment, we may simply choose `EHCache`, which is also the default cache provider for Hibernate.

```
<hibernate-configuration>
<session-factory>
...
<property name="cache.provider_class">
org.hibernate.cache.EhCacheProvider
</property>
...
</session-factory>
</hibernate-configuration>
```

We can configure EHCache through a configuration file "ehcache.xml" located in the source root folder. We can specify different settings for different "cache regions", which are used for storing different kinds of objects.

```
<ehcache>
<diskStore path="java.io.tmpdir" />
<defaultCache
maxElementsInMemory="10000"
eternal="false"
timeToIdleSeconds="120"
timeToLiveSeconds="120"
overflowToDisk="true"
/>
<cache name="com.nareshit.hibernate.bookshop.Book"
maxElementsInMemory="10000"
eternal="false"
timeToIdleSeconds="300"
timeToLiveSeconds="600"
overflowToDisk="true"
/>
</ehcache>
```

defaultCache will be used for all the persistent classes. We can also define persistent class explicitly by using the cache element.

eternal If we specify eternal="true", we don't need to define timeToIdleSeconds and timeToLiveSeconds attributes because it will be handled by hibernate internally. Specifying eternal="false" gives control to the programmer, but we need to define timeToIdleSeconds and timeToLiveSeconds attributes.

timeToIdleSeconds : It defines that how many seconds object can be idle in the second level cache.

timeToLiveSeconds : It defines that how many seconds object can be stored in the second level cache whether it is idle or not.

To monitor the caching activities of Hibernate at runtime, we can add the following line to the log4j configuration file "log4j.properties".

```
log4j.logger.org.hibernate.cache=debug
```

Now, we need to enable caching for a particular persistent class. The cached objects will be stored at a region having the same name as the persistent class, e.g.

"com.nareshit.hibernate.bookshop.Book".

There are several cache usages we can choose for a persistent class. If the persistent objects are read only and never modified, the most efficient usage is "read-only".

```
<hibernate-mapping package="com.nareshit.hibernate.bookshop">
<class name="Book" table="BOOK">
<cache usage="read-only" />
...
</class>
</hibernate-mapping>
```

After the caching of Book class being enabled, we can see that only one SQL query is made even the book object is loaded for two times in two different sessions.

```
Session session1 = factory.openSession();
try {
Book book1 = (Book) session1.get(Book.class, id);
} finally {
session1.close();
}
Session session2 = factory.openSession();
try {
Book book2 = (Book) session2.get(Book.class, id);
} finally {
session2.close();
}
```

However, if we modify the book object in one session and flush the changes, an exception will be thrown for updating a read-only object.

```
Session session1 = factory.openSession();
try {
Book book1 = (Book) session1.get(Book.class, id);
book1.setName("New Book");
session1.save(book1);
session1.flush();
} finally {
session1.close();
}
```

So for an updatable object, we should choose another cache usage "read-write". Hibernate will

invalidate the object from cache before it is updated.

```
<hibernate-mapping package="com.nareshit.hibernate.bookshop">
<class name="Book" table="BOOK">
<cache usage="read-write" />
...
</class>
</hibernate-mapping>
```

In some cases, e.g. the database updated by other applications, you may want to invalidate the cached objects manually. You can do it through the methods provided by the session factory. You can invalidate either one instance or all instances of a persistent class.

```
factory.evict(Book.class);
factory.evict(Book.class, id);
factory.evictEntity("com.nareshit.hibernate.bookshop.Book");
factory.evictEntity("com.nareshit.hibernate.bookshop.Book", id);
```

Caching associations

For the associated publisher object of a book, will it be also cached when its parent book object is cached?

```
<hibernate-mapping package="com.nareshit.hibernate.bookshop">
<class name="Book" table="BOOK">
<cache usage="read-write" />
...
<many-to-one name="publisher" class="Publisher" column="PUBLISHER_ID" />
</class>
</hibernate-mapping>
```

If you initialize the association in two different sessions, you will find that it is loaded for two times. It is because Hibernate caches only the identifier of publisher in the region of Book.

```
Session session1 = factory.openSession();
try {
Book book1 = (Book) session1.get(Book.class, id);
Hibernate.initialize(book1.getPublisher());
} finally {
session1.close();
}
Session session2 = factory.openSession();
try {
Book book2 = (Book) session2.get(Book.class, id);
Hibernate.initialize(book2.getPublisher());
} finally {
session2.close();
}
```

To cache the publisher objects in their own region, we need to enable the caching for the Publisher class also. We can do it the same way as for Book class. The cached publisher objects will be stored at a region with the name "com.nareshit.hibernate.bookshop.Publisher".

```
<hibernate-mapping package="com.nareshit.hibernate.bookshop">
```

```
<class name="Publisher" table="PUBLISHER">
<cache usage="read-write" />
```

```
...
```

```
</class>
</hibernate-mapping>
```

Caching collections

For the associated chapters of a book to be cached, we enable caching for the Chapter class. The cached chapter objects will be stored at a region with the name "com.nareshit.hibernate.bookshop.Chapter".

```
<hibernate-mapping package="com.nareshit.hibernate.bookshop">
<class name="Book" table="BOOK">
<cache usage="read-write" />
```

```
...
```

```
<set name="chapters" table="BOOK_CHAPTER">
<key column="BOOK_ID" />
<one-to-many class="Chapter" />
</set>
```

```
</class>
</hibernate-mapping>
<hibernate-mapping package="com.nareshit.hibernate.bookshop">
<class name="Chapter" table="CHAPTER">
<cache usage="read-write" />
```

```
...
```

```
<many-to-one name="book" class="Book" column="BOOK_ID" />
</class>
</hibernate-mapping>
```

Then we can try initializing the collection in two different sessions. We expect that no query should be made for the second session.

```
Session session1 = factory.openSession();
try {
Book book1 = (Book) session1.get(Book.class, id);
Hibernate.initialize(book1.getChapters());
} finally {
session1.close();
}
Session session2 = factory.openSession();
try {
Book book2 = (Book) session2.get(Book.class, id);
Hibernate.initialize(book2.getChapters());
} finally {
session2.close();
}
```

By inspecting the SQL statements, we can find that there is still one query made. The reason is that, unlike a many-to-one association, a collection will not be cached by default. We need to turn on it manually by specifying cache usage to the collection. For the chapter collection of a book, it will be stored at a region with the name "com.nareshit.hibernate.bookshop.Book.chapters".

How can a collection be cached by Hibernate? If the collection is storing simple values, the values

themselves will be cached. If the collection is storing persistent objects, the identifiers of the objects will be cached at the collection region and the persistent objects themselves will be cached at their own region.

```
<hibernate-mapping package="com.nareshit.hibernate.bookshop">
<class name="Book" table="BOOK">
<cache usage="read-write" />
...
<set name="chapters" table="BOOK_CHAPTER">
<cache usage="read-write" />
<key column="BOOK_ID" />
<one-to-many class="Chapter" />
</set>
</class>
</hibernate-mapping>
```

To invalidate a particular collection or all the collections in a region, we can use the following methods provided by the session factory.

```
factory.evictCollection("com.nareshit.hibernate.bookshop.Book.chapters");
factory.evictCollection("com.nareshit.hibernate.bookshop.Book.chapters", id);
```

For a bi-directional one-to-many/many-to-one association, you should call the `evictCollection()` method on the collection end after the single end is being updated.

```
Session session1 = factory.openSession();
try {
Book book1 = (Book) session1.get(Book.class, id);
Chapter chapter = (Chapter) book1.getChapters().iterator().next();
chapter.setBook(null);
session1.saveOrUpdate(chapter);
session1.flush();
factory.evictCollection("com.nareshit.hibernate.bookshop.Book.chapters", id);
} finally {
session1.close();
}
Session session2 = factory.openSession();
try {
Book book2 = (Book) session2.get(Book.class, id);
Hibernate.initialize(book2.getChapters());
} finally {
session2.close();
}
```

Caching queries

In addition to caching objects loaded by a session, a query with HQL can also be cached. Suppose that we are running the same query in two different sessions.

```
Session session1 = factory.openSession();
try {
Query query = session1.createQuery("from Book where name like ?");
query.setString(0, "%Hibernate%");
List books = query.list();
```

```

} finally {
session1.close();
}
Session session2 = factory.openSession();
try {
Query query = session2.createQuery("from Book where name like ?");
query.setString(0, "%Hibernate%");
List books = query.list();
} finally {
session2.close();
}

```

By default, the HQL queries are not cached. We must first enable the query cache in the Hibernate configuration file.

```

<hibernate-configuration>
<session-factory>
...
<property name="cache.use_query_cache">true</property>
...
</session-factory>
</hibernate-configuration>

```

Then we need to set the query to be cacheable before execution. The query result is cached at a region with the name "org.hibernate.cache.StandardQueryCache" by default.

How can a query be cached by Hibernate? If the query is returning simple values, the values themselves will be cached. If the query is returning persistent objects, the identifiers of the objects will be cached at the query region and the persistent objects themselves will be cached at their own region.

```

Session session1 = factory.openSession();
try {
Query query = session1.createQuery("from Book where name like ?");
query.setString(0, "%Hibernate%");
query.setCacheable(true);
List books = query.list();
} finally {
session1.close();
}
Session session2 = factory.openSession();
try {
Query query = session2.createQuery("from Book where name like ?");
query.setString(0, "%Hibernate%");
query.setCacheable(true);
List books = query.list();
} finally {
session2.close();
}

```

We can also specify the cache region for a query. This enables separating query caches at different regions and reducing the number of caches in one particular region.

...


```
query.setCacheable(true);  
query.setCacheRegion("com.nareshit.hibernate.bookshop.BookQuery");
```

Caching Strategies

1. **Read Only:** This caching strategy should be used for persistent objects that will always read but never updated. It's good for reading and caching application configuration and other static data that are never updated. This is the simplest strategy with best performance because there is no overload to check if the object is updated in database or not.
2. **Read Write:** It's good for persistent objects that can be updated by the hibernate application. However if the data is updated either through backend or other applications, then there is no way hibernate will know about it and data might be stale. So while using this strategy, make sure you are using Hibernate API for updating the data.
3. **Nonrestricted Read Write:** If the application only occasionally needs to update data and strict transaction isolation is not required, a nonstrict-read-write cache might be appropriate.
4. **Transactional:** The transactional cache strategy provides support for fully transactional cache providers such as JBoss TreeCache. Such a cache can only be used in a JTA environment and you must specify `hibernate.transaction.manager_lookup_class`.

There are so many second level caching implementations

EHcache:-

EH cache is lightweight and easy to use in process cache. it supports read only,readwrite, memory and disk-based caching . It does not supports clustering.

OS cache:-

it is another open source caching solution.

It is part of larger package,Which also provides caching for JSp pages (or) arbitrary objects.

it supports read only,readwrite,memory and disk-based caching. it also provides basic support for clustering via either java groups(or) JMS

SwarmCache:-

It is Simple clusted based caching solution based on java groups

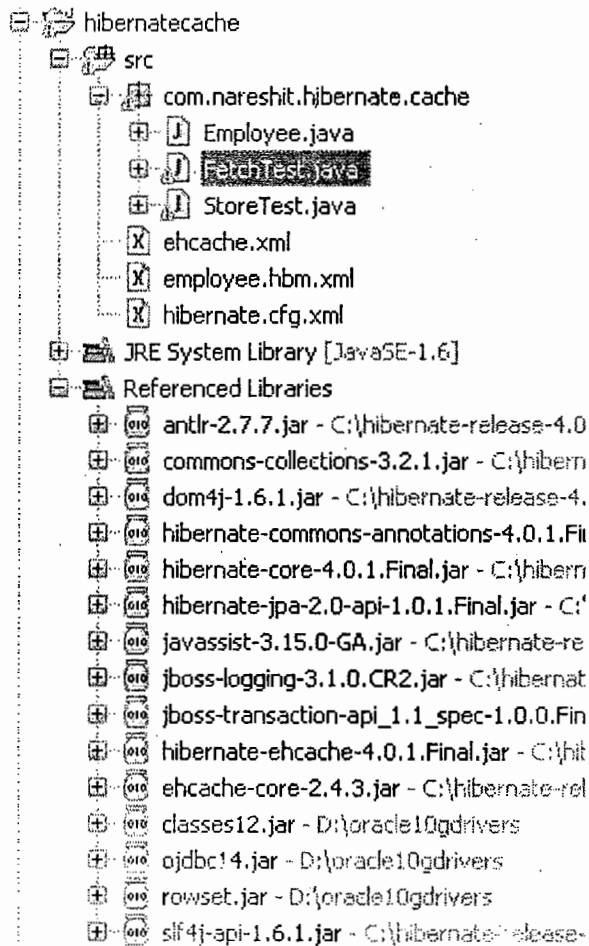
It supports read only,non-strict read and write i,e this type cache is suitable for applications that typically have many more read operations than write operations.

JBoss TreeCache:-

It is a power ful replicated(Synchronous(or) asynchronous) and transactional cache.It supported in JTA Environment

Let us illustrate second level cache with an example

Here is the eclipse directory structure



//Employee.java

```
package com.nareshit.hibernate.cache;
```

```
public class Employee {
    private int id;
    private String name;
    private float salary;
```

```
public Employee() {
    super();
}
```

```
public Employee(String name, float salary) {  
    super();  
    this.name = name;  
    this.salary = salary;  
}
```

```
public int getId() {  
    return id;  
}
```

```
public void setId(int id) {  
    this.id = id;  
}
```

```
public String getName() {  
    return name;  
}
```

```
public void setName(String name) {  
    this.name = name;  
}
```

```
public float getSalary() {  
    return salary;  
}
```

```
public void setSalary(float salary) {  
    this.salary = salary;  
}
```

```
}
```

```
//FetchTest.java
```

```
package com.nareshit.hibernate.cache;
```

```
import org.hibernate.Session;
```

```
import org.hibernate.SessionFactory;
```

```
import org.hibernate.cfg.Configuration;
```

```
public class FetchTest {
```

```
    public static void main(String[] args) {
```

```
        Configuration cfg=new Configuration().configure("hibernate.cfg.xml");
```

```
        SessionFactory factory=cfg.buildSessionFactory();
```

```
Session session1=factory.openSession();

Employee emp1=(Employee)session1.load(Employee.class,21);

System.out.println(emp1.getId()+" "+emp1.getName()+" "+emp1.getSalary());

session1.close();

Session session2=factory.openSession();

Employee emp2=(Employee)session2.load(Employee.class,21);

System.out.println(emp2.getId()+" "+emp2.getName()+" "+emp2.getSalary());

session2.close();

}
```

```
}
```

```
//StoreTest.java
```

```
package com.nareshit.hibernate.cache;
```

```
import org.hibernate.*;
```

```
import org.hibernate.cfg.*;
```

```
public class StoreTest {
```

```
public static void main(String[] args) {
```

```
    Session session=new  
Configuration().configure("hibernate.cfg.xml").buildSessionFactory().openSession();
```

```
    Transaction tx=session.beginTransaction();
```

```
    session.persist(new Employee("Ravi",50000));
```

```
    session.persist(new Employee("Kumar",70000));
```

```
        tx.commit();
```

```
    session.close();
```

```
}
```

```
}
```

```
//ehcache.xml
```

```
<?xml version="1.0"?>
```

```
<ehcache>
```

```
    <defaultCache maxElementsInMemory="100" eternal="false"
        timeToIdleSeconds="120" timeToLiveSeconds="200"/>
    <cache name="com.nareshit.hibernate.cache.Employee"
        maxElementsInMemory="100" eternal="false" timeToIdleSeconds="5"
        timeToLiveSeconds="200"/>
```

```
</ehcache>
```

```
//employee.hbm.xml
```

```
<?xml version='1.0' encoding='UTF-8'?>
<!DOCTYPE hibernate-mapping PUBLIC
"-//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
```

```
<hibernate-mapping>
<class name="com.nareshit.hibernate.cache.Employee" table="emp">
<cache usage="read-only"/>
<id name="id">
<generator class="native"></generator>
</id>
<property name="name"></property>
<property name="salary"></property>
</class>
```

```
</hibernate-mapping>
```

```
//hibernate.cfg.xml
```

```
<?xml version='1.0' encoding='UTF-8'?>
<!DOCTYPE hibernate-configuration PUBLIC
"-//Hibernate/Hibernate Configuration DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
```

```
<!-- Generated by MyEclipse Hibernate Tools. -->
```

```
<hibernate-configuration>
```

```
    <session-factory>
        <property name="show_sql">true</property>
        <property name="hbm2ddl.auto">update</property>
        <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
        <property name="connection.url">jdbc:oracle:thin:@localhost:1521:ora</property>
```

```
<propertyname="connection.username">scott</property>
<propertyname="connection.password">tiger</property>
```

```
<propertyname="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
<propertyname="hibernate.cache.use_second_level_cache">true</property>
<propertyname="hibernate.cache.use_query_cache">true</property>
```

```
<propertyname="hibernate.cache.region.factory_class">org.hibernate.cache.ehcache.EhCacheRegionFactory</property>
```

```
<mappingresource="employee.hbm.xml" />
</session-factory>
```

```
:/hibernate-configuration>
```

When StoreTest program, data will be inserted in emp table. Now when FetchTest is executed, the below is the output

```
hibernate: select employee0_.id as id0_0_, employee0_.name as name0_0_, employee0_.salary as salary0_0_ from emp1012 employee0_ where employee0_.id=?
?1 Ravi 50000.0
?1 Ravi 50000.0
```

If you can observe, sql query is executed only for first time, during second time load method call in FetchTest.java, the data is being retrieved from cache.

Example for QueryCache

To enable query cache, set <property name="hibernate.cache.use_query_cache">true</property> in hibernate.cfg.xml.

In order to cache query execution, use setCacheable(true) after creating Query object.

```
package com.nareshit.hibernate.cache;
```

```
import java.util.List;
import org.hibernate.Query;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
```

```
public class QueryTest {
```

```
    public static void main(String[] args) {
        SessionFactory sessionFactory = new Configuration().configure()
            .buildSessionFactory();
```

```
        /* creating first session object */
```

```

Session session = sessionFactory.openSession();
session.beginTransaction();

Query query1 = session
    .createQuery("from Employee emp where emp.id=21");
query1.setCacheable(true);
// setting query to be cacheable

List list1 = query1.list();
System.out.println("list data-->" + list1);
session.getTransaction().commit();
session.close(); // first session object closed

/* creating second session object */
Session session2 = sessionFactory.openSession();
session2.beginTransaction();
Query query2 = session2
    .createQuery("from Employee emp where emp.id=21");

query2.setCacheable(true);
// setting query to be cacheable
List list2 = query2.list();
System.out.println("list data-->" + list2);
session2.getTransaction().commit();
session2.close();
// second session object closed
    }
}

```

Output:

Hibernate: select employee0_.id as id0_, employee0_.name as name0_, employee0_.salary as salary0_ from emp1012 employee0_ where employee0_.id=21
list data-->[com.nareshit.hibernate.cache.Employee@1eb904d]
list data-->[com.nareshit.hibernate.cache.Employee@12fb0af]

When the query is executed 2 times, in the above program, as per the output, query to database is executed only for one time, second time, query result is retrieved from query cache.

Another example :**CachingExample :****Product.java**

```

package com.nit;
public class Product{
private int productId;

```

```
private String proName;  
private double price;  
//required setters and getters  
}
```

Ehcache.java

```
<ehcache>  
  
<defaultCache maxElementsInMemory="100" eternal="false"  
    timeToIdleSeconds="120" timeToLiveSeconds="200" />  
  
<cache name="com.nit.Product" maxElementsInMemory="5" eternal="false"  
    timeToIdleSeconds="5" timeToLiveSeconds="200" />  
  
</ehcache>
```

hibernate.cfg.xml

```
<!DOCTYPE hibernate-configuration PUBLIC  
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"  
    "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">  
  
<hibernate-configuration>  
  
    <session-factory>  
  
        <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>  
        <property name="connection.url">jdbc:oracle:thin:@localhost:1521:XE</property>  
        <property name="connection.username">system</property>  
        <property name="connection.password">manager</property>  
        <property name="cache.provider_class"> org.hibernate.cache.EhCacheProvider</property>  
        <property name="dialect">org.hibernate.dialect.OracleDialect</property>  
        <property name="show_sql">>true</property>  
        <mapping resource="Product.hbm.xml"></mapping>  
  
    </session-factory>
```


Product.hbm.xml

```

<!DOCTYPE hibernate-mapping PUBLIC
"-//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
<class name="com.nit.Product" table="products">
<cache usage="read-only" />
<id name="productId" />
<property name="proName" column="name" length="10"/>
<property name="price"/>
</class></hibernate-mapping>

package com.nit;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
public class Test{
public static void main(String[] args){
Configuration cfg = new Configuration();
cfg.configure("hibernate.cfg.xml");
SessionFactory factory = cfg.buildSessionFactory();
Session session1 = factory.openSession();
Object o=session1.load(Product.class,new Integer(1003));
Product s=(Product)o;
System.out.println("Loaded object product name is____"+s.getProName());
System.out.println("Object Loaded successfully.....!!");
session1.close();
System.out.println("-----");
System.out.println("Waiting.....");
try{
Thread.sleep(4900);
}
catch (Exception e) {
}
System.out.println("4.0 seconds compelted.....!!!!!!");
Session session2 = factory.openSession();
Object o2=session2.load(Product.class,new Integer(1003));
Product s2=(Product)o2;
System.out.println("Loaded object product name is____"+s2.getProName());
System.out.println("Object loaded successfully...!!");
session2.close();
}

```

```

try{
Thread.sleep(6000);
}
catch (Exception e) {
}
System.out.println("6 seconds compelted.....!!!!!!");
Session session3 = factory.openSession();
Object o3=session3.load(Product.class,new Integer(1003));
Product s3=(Product)o3;
System.out.println("Loaded object product name is____"+s3.getProName());
System.out.println("Object loaded successfully.....!!");
session3.close();
factory.close();
}}

```

Caching -Example 2:-

SearchStuedentController.java

```

package com.nareshit.controller;
import java.io.*;
import javax.servlet.*;
import com.nareshit.dao.StudentDAO;
import com.nareshit.pojo.Student;
public class SearchStudentController extends GenericServlet{
private StudentDAO studentDAO;
public void init(){
studentDAO =new StudentDAO();
}
public void service(ServletRequest request,
ServletResponse response)throws ServletException,IOException{
PrintWriter out = response.getWriter();
String error_msg=null;
try{
int sid =Integer.parseInt(request.getParameter("sid"));
Student s =studentDAO.getStudent(sid);
if(s==null){
error_msg="Student Not Found";
}
else {
request.setAttribute("student",s);
RequestDispatcher rd =
request.getRequestDispatcher("SearchStudentView.jsp");

```

```
rd.forward(request, response);
    }
} //end of try
catch(NumberFormatException ne){
    error_msg="Please Enter Valid Student Id ";
    System.out.println(ne);
} //end of catch
response.setContentType("text/html");
RequestDispatcher rd =
request.getRequestDispatcher("SearchStudentForm.html");
rd.include(request, response);
out.println("<hr/><i>");
out.println(error_msg);
out.println("</i>");
} //end of service
```

StudentDAO.java

```
package com.nareshit.dao;
import org.hibernate.Session;
import com.nareshit.pojo.Student;
import com.nareshit.utility.HibernateUtility;
public class StudentDAO {
    public Student getStudent(int sid){
        Student std=null;
        Session session=HibernateUtility.getSession();
        System.out.println("StudentDAO getStudent");
        if(session!=null){
            std=(Student)session.get(Student.class,sid);
        }
        return std;
    }
}
```

Student.java

```
package com.nareshit.pojo;
public class Student {
    private int sid,age;
    private String name,email;
    //required setters and getters
}
```

ehcache.java

```
<ehcache>
<defaultCache maxElementsInMemory="100" eternal="false"
    timeToldleSeconds="120" timeToLiveSeconds="200" />
<cache name="com.nareshit.pojo.Student" maxElementsInMemory="5" eternal="false"
    timeToldleSeconds="100" timeToLiveSeconds="200" /></ehcache>
```

hibernate.cfg.xml

```
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
<session-factory>
<property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
<property name="connection.url">jdbc:oracle:thin:@localhost:1521:XE</property>
<property name="connection.username">system</property>
<property name="connection.password">manager</property>
<property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
<property name="show_sql">>true</property>
<property name="cache.provider_class">
    org.hibernate.cache.EhCacheProvider
</property>
<mapping resource="Student.hbm.xml"/>
</session-factory>
```

Student.hbm.xml

```
<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
<class name="com.nareshit.pojo.Student" table="Student">
<cache usage="read-only" />
<id name="sid"></id>
<property name="name"></property>
<property name="age"></property>
<property name="email"></property>
</class>
```

web.xml

```
<web-app>
```

```

<servlet>
<servlet-name>searchStudentServlet</servlet-name>
<servlet-class>com.nareshit.controller.SearchStudentController
</servlet-class>
</servlet>
<servlet-mapping>
<servlet-name>searchStudentServlet</servlet-name>
<url-pattern>/searchStudent</url-pattern>
</servlet-mapping>
<welcome-file-list>
<welcome-file>SearchStudentForm.html</welcome-file>
</welcome-file-list>
</web-app>

```

```

<!-- SearchStudentForm.html -->
<b>Student Management System</b>
<br/>
<form action="searchStudent"><pre>
Student ID : <input type="text" name="sid"/>
<input type="submit" value="Search"/>

```

SearchStudentView.jsp

```

<%@ page import="com.nareshit.pojo.Student"%>
<%!
Student s=null;
%>
<html>
<head><center><h1>Student Management System</h1>
</center></head><br/><br/><hr/>
<body bgcolor="#FFFFFF">
<%
s=(Student)request.getAttribute("student");
%>
<center><TABLE BORDER=1>
<tr><th>SID</th><th>NAME</th>
<th>AGE</th><th>EMAIL</th></tr>
<tr>
<td><%= s.getSid()%></td>
<td><%= s.getName() %></td>
<td><%= s.getAge() %></td>
<td><%= s.getEmail() %></td>

```

```
</tr>
</TABLE></center>
<%@ include file="SearchStudentForm.html" %>
</body>
```

Transactions and Concurrency

Let's say a buyer logs in to the bookshop application and purchases a book. The following actions should take place in the event of purchase:

- Charge the buyer the cost of the book
- Reduce the stock of the book

If the charge on the credit card fails, the stock shouldn't be reduced. Also, when the book is out of stock, the buyer shouldn't be charged. That means either both actions should be successfully completed, or they should have no effect. These actions collectively are called a *transaction* or *unit of work*. In essence, transactions provide an all-or-nothing proposition.

The concept of transactions is inherited from database management systems. By definition, a transaction must be atomic, consistent, isolated, and durable (ACID):

- **Atomicity** means that if one step fails, then the whole unit of work fails.
- **Consistency** means that the transaction works on a set of data that is consistent before and after the transaction. The data must be clean after the transaction. From a database perspective, the clean and consistent state is maintained by integrity constraints. From an applications perspective, the consistent state is maintained by the business rules.
- **Isolation** means one transaction isn't visible to other transactions. Isolation makes sure the execution of a transaction doesn't affect other transactions.
- **Durability** means that when data has been persisted, it isn't lost.

In an application, you need to identify when a transaction begins and when it ends. The starting point and ending point of a transaction are called *transaction boundaries*, and the technique of identifying them in an application is called *transaction demarcation*. You can set the transaction boundaries either programmatically or declaratively.

There are two main approaches to concurrency control:

- **Optimistic**: Involves some kind of versioning to achieve control
- **Pessimistic**: Uses a locking mechanism to obtain control

Some of the methods that are used in concurrency control are as follows:

- **Two-phase locking**: This is a locking mechanism that uses two distinct phases to achieve concurrency. In the first phase of transaction execution, called the

expanding phase, locks are acquired, and no locks are released. In the second phase, called the *shrinking phase*, locks are released, and no new locks acquired. This guarantees serializability. The transactions are serialized in the order in which the locks are acquired. *Strict two-phase locking* is a subset of two-phase locking. All the write locks are released only at the end (after committing or aborting), and read locks are released regularly during phase 2. With both these locking mechanisms, a deadlock is possible but can be avoided if you maintain a canonical order for obtaining locks. So, if two processes need locks on A and B, then they both request first A and then B.

- **Serialization:** A *transaction schedule* is a sequential representation of two or more concurrent transactions. The transaction schedules have a property called *serializability*. Two transactions that are updating the same record are executed one after the other and don't overlap in time.

- **Time-stamp-based control:** This is a nonlocking mechanism for concurrency control. Every transaction is given a timestamp when it starts. Every object or record in the database also has a read timestamp and a write timestamp. These three timestamps are used to define the isolation rules that are used in this type of concurrency control.

- **Multiversion concurrency control:** When a transaction does a read on a database, a snapshot is created, and the data is read from that snapshot. This isolates the data from other concurrent transactions. When the transaction modifies a record, the database creates a new record version instead of overwriting the old record. This mechanism gives good performance because lock contention between concurrent transactions is minimized—in fact, lock contention is eliminated between read locks and write locks, which means read locks never block a write lock. Most current databases, such as Oracle, MySQL, SQL Server, and PostgreSQL, implement the multiversion concurrent control for concurrency.

Using Programmatic Transactions in a Standalone Java

If you're working on a standalone Java application, how do you achieve transaction demarcation? In a multiuser application, you need more than one connection to support multiple concurrent users.

It's also expensive to create new connections when you need to interact with the database. For this, you need to have a connection pool that creates and manages database connections. Every thread that needs to interact with the database requests a connection from the pool and executes its queries. After it's done, the connection is returned to the pool. Then, if another thread requests a connection, the connection pool may provide it with same connection.

Usually, an application server provides a connection pool. When you're working in a standalone Java application, you need to use a third-party solution that provides connection pools. Hibernate comes with an open source third-party connection pooling framework called C3P0. Apache also provides a connection-pooling framework called Commons DBCP. You need to configure Hibernate to use one of these frameworks. After the connection pool is configured, you can use the

Hibernate's Transaction API for transaction demarcation and use connections from the connection pool.

You need to add the Java **c3p0-0.9.1.jar** file that comes with Hibernate to the build path. In your Eclipse IDE, select your Java project, and right-click to edit build path.

In the **Hibernate.cfg.xml** file, add the following configuration:

```
<property
name="connection.provider_class">org.hibernate.connection.C3P0ConnectionProvider
</property>
<property name="hibernate.c3p0.min_size">5</property>
<property name="hibernate.c3p0.max_size">10</property>
<property name="hibernate.c3p0.timeout">300</property>
<property name="hibernate.c3p0.max_statements">50</property>
<property name="hibernate.c3p0.acquire_increment">1</property>
<property name="hibernate.c3p0.idle_test_period">3000</property>
```

Here's an explanation for each of the parameters you set:

- **connection.provider_class** specifies that **C3P0ConnectionProvider** is the class providing the connections.
- **min_size** is the minimum number of connections that are ready at all times.
- **max_size** is the maximum number of connections in the pool. This is the only property that is required for C3P0 to be enabled.
- **timeout** is the maximum idle time for a connection, after which the connection is removed from the pool.
- **max_statements** is the maximum number of prepared statements that can be cached.
- **idle_test_period** is the time in seconds before which a connection is automatically validated.
- **acquire_increment** is the number of connections acquired when the pool is exhausted.

These are all the settings required in the configuration file. Hibernate also provides a property called

hibernate.transaction.factory_class that you can set in the configuration file. It provides the factory

to use to instantiate transactions, and it defaults to **JDBCTransactionFactory**:

```
SessionFactory sessionFactory = new Configuration().configure().buildSessionFactory();
Session session = sessionFactory.openSession();
Transaction tx = null;
try {
tx = session.beginTransaction();
BookCh2 book = new BookCh2();
book.setName("Hibenate book");
book.setPrice(200);
book.setPublishDate(new Date());
session.saveOrUpdate(book);
```



```

tx.commit();
} catch (RuntimeException e) {
try{
if(tx != null)
{
tx.rollback();
}
} catch (RuntimeException ex)
{
log.error("Cannot rollback transaction");
}
throw e;
} finally {
session.close();
}

```

The session is provided by the `sessionFactory.openSession()` call. A database connection isn't opened when the session is created; this keeps the session creation a non-expensive task. The database connection is retrieved from the connection pool when the call `session.beginTransaction()` is made, and all queries are executed using this connection. The entities to be persisted are cached in the persistent context of the session. The `commit()` on the `Transaction` flushes the persistent context and completes the save of the entities. Another `Transaction` can be opened after the current `Transaction` is committed. Then, a new connection is provided by the connection pool. All resources and connections are released when `close()` is called on the `Session`.

The exceptions thrown by Hibernate are `RuntimeExceptions` and subtypes of `RuntimeExceptions`.

These exceptions are fatal, and hence you have to roll back your transactions. Because rolling back a transaction can also throw an exception, you have to call the `rollback` method within a `try/catch` block.

In JPA, you have to define the same configurations as for Hibernate, and the `EntityManager` API is used to manage transactions:

```

EntityManager manager = null;
EntityTransaction tx = null;
try {
EntityManagerFactory managerFactory =
Persistence.createEntityManagerFactory("book");
manager = managerFactory.createEntityManager();
tx = manager.getTransaction();
tx.begin();
BookCh2 newBook = new BookCh2();
newBook.setBookName("Hibernate book");
newBook.setPublishDate(new Date());
newBook.setPrice(new Long(50));
manager.persist(newBook);
tx.commit();
log.debug("Transaction committed");
}

```

```
}catch (RuntimeException e)
{
try
{
if(tx != null)
{
tx.rollback();
}
}catch(RuntimeException ex)
{
log.error("Cannot rollback transaction");
}
throw e;
} finally
{
manager.close();
}
```

getTransaction() is called on the EntityManager to get a transaction. The actual connection is requested from the pool with the transaction's **begin()** method. The **commit()** on the transaction flushes the persistent context and completes the save of the entities. The EntityManager's **close()** method is called in the **finally** block to make sure the session is closed even in the case of an exception.

Using Programmatic Transactions with JTA

Suppose you're using an application server that provides support to manage resources. Application servers like WebLogic, WebSphere, and JBoss can provide connections from a connection pool and manage transactions. How do you achieve programmatic transaction demarcation using the Java Transaction API (JTA)?

You have to configure a Java enterprise application with Hibernate as the persistent framework. You also need to configure Hibernate to use **JTATransactionFactory** to provide **JTATransaction** objects. You then need to use the **UserTransaction** interface to manage transactions programmatically.

You use WebLogic (version 9.2) as the application server to demonstrate using JTA for programmatic transaction demarcation. The assumption is that you're well versed with building and deploying your application.

To begin, you need to configure the data source on the application server. To do so, you need to start the WebLogic server, log in to the console, and configure the data source. You must also add the database driver to the server library. For Oracle, you add **classes12.jar** to the server library. Make sure you test connectivity from the console!

You need to provide the following information (with your values):

- **JNDI Name:** **local_oracle**. The application uses this to obtain database connections.
- **Database Name:** **BookShopDB**
- **Host Name:** **localhost**. If you're pointing to a remote database, then this value is

something like the IP of that machine.

- **Port: 1527.**
- **User: book.**
- **Password: book.**

After you successfully configure the data source on the application server, you need to configure Hibernate. The key properties you're required to provide are as follows:

- **Hibernate.connection.datasource:** The value of this property must be the same as the JNDI name of the data source you configured on the application server.
- **Hibernate.transaction.factory_class:** The default value for this is **JDBCTransactionFactory**. But because you want to use a JTA transaction for transaction demarcation, you need to define this as **JTATransactionFactory**.
- **Hibernate.transaction.manager_lookup_class:** The value of this transaction manager is dependent on the application server. Each application server has a different JTA implementation, so you have to tell Hibernate which JTA implementation to use. Hibernate supports most major application server implementation; you use

org.hibernate.transaction.WeblogicTransactionManagerLookup for WebLogic.

The following is the Hibernate configuration:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
"-//Hibernate/Hibernate Configuration DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
<session-factory name="book">
<property name="hibernate.connection.datasource">local_orcale</property>
<property
name="hibernate.transaction.factory_class">org.hibernate.transaction.JTATransactionFactory</
property>
<property
name="hibernate.transaction.manager_lookup_class">org.hibernate.transaction.WeblogicTransact
ionManagerLookup</property>
<property name="hibernate.dialect">org.hibernate.dialect.OracleDialect</property>
<property name="hibernate.show_sql">true</property>
<property name="hibernate.cache.use_second_level_cache">false</property>
<mapping resource="book.xml" />
</session-factory>
</hibernate-configuration>
```

You now need to write a class that provides **SessionFactory** for the complete application. To do so, you defined a utility class that has a **static** method to return the **SessionFactory**. Here's the code implementation:

```
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
public class HibernateUtil {
private HibernateUtil(){}
public static SessionFactory getSessionFactory()
{
SessionFactory factory = null;;
```

```
try {  
factory = new Configuration().configure().buildSessionFactory();  
} catch (Exception e) {  
e.printStackTrace();  
}  
return factory;  
}  
}
```

Now you're ready to use JTA for transaction demarcation. You get the `UserTransaction` by looking up the JNDI registry:

```
public void saveBook(Book book) throws NotSupportedException, SystemException,  
NamingException, Exception {  
System.out.println("Enter DAO Impl");  
Session session = null;  
UserTransaction tx = (UserTransaction)new InitialContext()  
.lookup("java:comp/UserTransaction");  
try {  
SessionFactory factory = HibernateUtil.getSessionFactory();  
tx.begin();  
session = factory.openSession();  
session.saveOrUpdate(book);  
session.flush();  
tx.commit();  
} catch (RuntimeException e) {  
try  
{  
tx.rollback();  
} catch (RuntimeException ex)  
{  
System.out.println("**** RuntimeException in BookDaoImpl ");  
}  
throw e;  
} finally  
{  
session.close();  
}  
}
```

Note that you explicitly call `session.flush()`. You need to do an explicit flush because Hibernate's default implementation doesn't flush the session. You can, however, override the default implementation by configuring the `hibernate.transaction.flush_before_completion` property. You also

configure the `hibernate.transaction.auto_close_session` property to avoid calling `session.close()` explicitly in every method. The Hibernate configuration file is as follows:

```
<session-factory name="book">  
<property name="hibernate.connection.datasource">local_oracle</property>  
</property>
```

```

name="hibernate.transaction.factory_class">org.hibernate.transaction.JTATransactionFactory</
property>
<property
name="hibernate.transaction.manager_lookup_class">org.hibernate.transaction.WeblogicTransa
ctionManagerLookup</property>
<property name="hibernate.transaction.flush_before_completion">true</property>
<property name="hibernate.transaction.auto_close_session">true</property>
<property name="hibernate.dialect">org.hibernate.dialect.OracleDialect</property>
<property name="hibernate.show_sql">true</property>
<property name="hibernate.cache.use_second_level_cache">false</property>
<!--<property name="hbm2ddl.auto">create</property-->
<mapping resource="book.xml" />
</session-factory>

```

And the code looks a little simpler, as shown here:

```

Session session = null;
UserTransaction tx = (UserTransaction)new
InitialContext().lookup("java:comp/UserTransaction");
try {
SessionFactory factory = HibernateUtil.getSessionFactory();
tx.begin();
session = factory.openSession();
session.saveOrUpdate(book);
tx.commit();
}catch (RuntimeException e) {
try
{
tx.rollback();
}catch(RuntimeException ex)
{
System.out.println("**** RuntimeException in BookDaoImpl ");
}
throw e;
}

```

In JPA, the implementation is very similar to Hibernate:

```

EntityManager manager = null;
UserTransaction tx = (UserTransaction)new InitialContext()
.lookup("java:comp/UserTransaction");
try {
EntityManagerFactory managerFactory = HibernateUtil.getFactory();
manager = managerFactory.createEntityManager();
tx.begin();
manager.persist(book);
tx.commit();
}catch (RuntimeException e)
{
try
{

```

```

if(tx != null)
{
tx.rollback();
}
} catch(RuntimeException ex)
{
System.out.println ("Cannot rollback transaction");
}
throw e;
} finally {
manager.close();
}

```

Enabling Optimistic Concurrency Control

Suppose two transactions are trying to update a record in the database. The first transaction updates and commits successfully. The second transaction tries to update and fails. So, the transaction is rolled back. The problem is that the first update is lost. Or, what if the second transaction successfully updates, as illustrated in below table. The changes made by the first transaction are overwritten.

Lost Updates: Transaction 1 Updates Are Lost

Time	Transaction Account
T1	Transaction 1 begins.
T2	Transaction 2 begins.
T3	Transaction 1 updates record R1.
T4	Transaction 2 updates record R1.
T5	Transaction 1 commits.
T6	Transaction 2 commits.

How do you handle such cases of lost updates? And how do you enable versioning in Hibernate?

You need to understand isolation levels in order to choose a concurrency control mechanism.

Access to database records is classified as reads and writes. The concurrency control mechanisms define the rules that dictate when to allow reads and writes.

A *dirty read* occurs when one transaction reads changes made by another transaction that haven't yet been committed (refer below table). Basically, a dirty read means reading uncommitted data.

Dirty Read: A Transaction Reading Uncommitted Data

Time	Transaction Account
T1	Transaction 1 begins.
T2	Transaction 2 begins.
T3	Transaction 1 updates record R1.
T4	Transaction 2 reads uncommitted record R1.
T5	Transaction 1 rolls back its update.
T6	Transaction 2 commits.

An *unrepeatable read* occurs when a transaction reads a record twice and the record state is different between the first and the second read. This happens when another transaction updates the state of the record between the two reads

Unrepeatable Read: A Transaction Reading a Record Twice

Time	Transaction Account
T1	Transaction 1 begins.
T2	Transaction 1 reads record R1.
T3	Transaction 2 begins.
T4	Transaction 2 updates record R1.
T5	Transaction 2 commits.
T6	Transaction 1 reads record R1 (the record R1 read at time T2 is in a different state than at time T6).
T7	Transaction 1 commits.

A *phantom read* occurs when a transaction executes two identical queries, and the collection of rows returned by the second query is different from the first. This also happens when another transaction inserts records into or deletes records from the table between the two reads.

Phantom Read: Reading a Range of Data That Changes in Size During a Transaction

Time	Transaction Account
T1	Transaction 1 begins.
T2	Transaction 1 reads a range of records RG1.
T3	Transaction 2 begins.
T4	Transaction 2 inserts records.
T5	Transaction 2 commits.
T6	Transaction 1 reads the range of records RG1 (RG1's size has changed from time T2 to T6).
T7	Transaction 1 commits.

Isolation defines how and when changes made by one transaction are made visible to other transactions. Isolation is one of the ACID properties. For better performance and concurrency control, isolation is divided by the ANSI SQL standard into levels that define the degree of locking when you select data. The four isolation levels are as follows.

- **Serializable:** Transactions are executed serially, one after the other. This isolation level allows a transaction to acquire read locks or write locks for the entire range of data that it affects. The Serializable isolation level prevents dirty reads, unrepeatable reads, and phantom reads, but it can cause scalability issues for an application.
- **Repeatable Read:** Read locks and write locks are acquired. This isolation level doesn't permit dirty reads or unrepeatable reads. It also doesn't acquire a range lock, which means it permits phantom reads. A read lock prevents any write locks from being acquired by other concurrent transactions. This level can still have some scalability issues.
- **Read Committed:** Read locks are acquired and released immediately, and write locks are acquired and released at the end of the transaction. Dirty reads aren't allowed in this isolation level, but unrepeatable reads and phantom reads are permitted. By using the combination of persistent context and versioning, you can

achieve the Repeatable Read isolation level.

- *Read Uncommitted*: Changes made by one transaction are made visible to other transactions before they're committed. All types of reads, including dirty reads, are permitted. This isolation level isn't recommended for use. If a transaction's uncommitted changes are rolled back, other concurrent transactions may be seriously affected.

Summarizing the Reads That Are Permitted for Various Isolation Levels

Isolation level	Dirty Read	Unrepeated Read	Phantom Read
Serializable - - -			
Repeatable Read - - Permitted			
Read Committed - Permitted	Permitted		
Read Uncommitted Permitted		Permitted	Permitted

Every database management system has a default setting for the isolation level. You can change the default isolation level in the DBMS configuration. On JDBC, you can set the isolation level by using a property called `hibernate.connection.isolation`. Hibernate uses the following values to set a particular isolation level:

- 8: Serializable isolation
- 4: Repeatable Read isolation
- 2: Read Committed isolation
- 1: Read Uncommitted isolation

This setting is applicable only when the connection isn't obtained from an application server. In this scenario, you need to change the isolation level in the application server configuration. Now, let's come back to the case of lost updates described at the beginning section.

You've seen a case where an update made by transaction 1 is lost when transaction 2 commits. Most applications are database connections with Read Committed isolation and use the optimistic concurrency control. One way of implementing optimistic control is to use versioning. Hibernate provides automatic versioning. Each entity can have a version, which can be a number or a timestamp. Hibernate increments the number when the entity is modified. If you're saving with an old version (which is the case for transaction 2 in the lost-update problem), Hibernate compares the versions automatically and throws an exception.

To add a version number to an entity, you add a property called **version** of type **int** or **Integer**:

```
public class BookCh2 {
    private long isbn;
    private String name;
    private Date publishDate;
    private int price;
    private int version;
}
```

In the `book.xml` configuration file, you add the **version** element. Note that the **version** element

must be placed immediately after the `id` element:

```
<hibernate-mapping package="com.nareshit.hibernate.chapter2" auto-import="false" >
<import class="BookCh2" rename="bkch2"/>
<class name="BookCh2" table="BOOK" dynamic-insert="true" dynamic-update="true"
schema="BOOK">
<id name="isbn" column="isbn" type="long">
<generator class="hilo">
</generator>
</id>
<version name="version" access="field" column="version"></version>
<property name="name" type="string" column="BOOK_NAME" />
<property name="publishDate" type="date" column="PUBLISH_DATE" />
<property name="price" type="int" column="PRICE" />
</class>
</hibernate-mapping>
```

A new column called `version` is created in the `BOOK` table. Using JPA, you add the `version` variable to

the `Book` class and annotate it with the `Version` element:

```
@Entity (name="bkch2")
@org.hibernate.annotations.Entity(dynamicInsert = true, dynamicUpdate = true)
@Table (name="BOOK")
public class BookCh2 {
@Id
@GeneratedValue (strategy=GenerationType.TABLE)
@Column (name="ISBN")
private long isbn;
@Version
@Column (name="version")
private Integer version;
@Column (name="book_Name")
private String bookName;
/*@Column (name="publisher_code")
String publisherCode;*/
@Column (name="publish_date")
private Date publishDate;
@Column (name="price")
private Long price;
// getters and setters
}
```

You can also use timestamps to version by adding a variable of type `Date`:

```
public class BookCh2 implements Serializable{
private long isbn;
private String name;
private Date publishDate;
private int price;
```

```
private Date timestamp;
// getters and setters
}
```

The XML mapping file has a timestamp element as shown here:

```
<hibernate-mapping package="com.nareshit.hibernate.chapter2" auto-import="false" >
<import class="BookCh2" rename="bkch2"/>
<class name="BookCh2" table="BOOK" dynamic-insert="true" dynamic-update="true"
schema="BOOK">
<id name="isbn" column="isbn" type="long">
<generator class="hilo">
</generator>
</id>
<timestamp name="timestamp" access="field" column="timestamp"></timestamp>
<property name="name" type="string" column="BOOK_NAME" />
<property name="publishDate" type="date" column="PUBLISH_DATE" />
<property name="price" type="int" column="PRICE" />
</class>
</hibernate-mapping>
```

You can also implement versioning without a version or timestamp by using the attribute **optimistic-lock** on the class mapping. It works when the entity is retrieved and modified in the same session. It doesn't work with detached objects. If you need to use optimistic concurrency control with

detached objects, you must use a version or timestamp:

```
<hibernate-mapping package="com.nareshit.hibernate.chapter2" auto-import="false" >
<import class="BookCh2" rename="bkch2"/>
<class name="BookCh2" table="BOOK" dynamic-insert="true" dynamic-update="true"
schema="BOOK" optimistic-lock="all">
<id name="isbn" column="isbn" type="long">
<generator class="hilo">
</generator>
</id>
<property name="name" type="string" column="BOOK_NAME" />
<property name="publishDate" type="date" column="PUBLISH_DATE" />
<property name="price" type="int" column="PRICE" />
</class>
</hibernate-mapping>
```

This isn't a popular option because it's slower and is complex to implement. In addition, JPA does not standardize this technique. So, if you need to use optimistic locking in JPA, you must use Hibernate's

annotations, as shown here:

```
@Entity (name="bkch2")
@org.hibernate.annotations.Entity
(dynamicInsert = true, dynamicUpdate = true,
optimisticLock=org.hibernate.annotations.OptimisticLockType.ALL)
@Table (name="BOOK")
public class BookCh2 {
```

```

@Id
@GeneratedValue (strategy=GenerationType.TABLE)
@Column (name="ISBN")
private long isbn;
@Version
@Column (name="version")
private Integer version;
@Column (name="book_Name")
private String bookName;
/*@Column (name="publisher_code")
String publisherCode;*/
@Column (name="publish_date")
private Date publishDate;
@Column (name="price")
private Long price;
// getters and setters
}

```

Using Pessimistic Concurrency Control

Most applications have Read Committed as the isolation level. This isolation level permits unrepeatable reads, which isn't desirable. One way to avoid unrepeatable reads is to implement versioning in the application. This upgrading of the isolation level from Read Committed to Repeatable Read comes with scalability issues. So, as an application developer, you may not want to make an application-wide upgrade to versioning—you may just want to upgrade the isolation level on a per-unit basis. To do so, Hibernate provides the `lock()` method in the Hibernate session object. The following code demonstrates how an unrepeatable read can happen. You first get the book using `session.get()`, and then you use a query to read the name of the book. If an update happens between these two calls to the book's record in the database by a concurrent transaction, you have a case of unrepeatable reads:

```

Session session = getSession();
Transaction tx = null;
tx = session.beginTransaction();
BookCh2 book = (BookCh2)session.get(BookCh2.class, new Long(32769));
String name = (String) session.createQuery("select b.name from bkch2 b where b.isbn = :isbn")
.setParameter("isbn", book.getIsbn()).uniqueResult();
System.out.println("BOOK's Name- "+name);
tx.commit();
session.close();

```

You can use `session.lock` to upgrade the isolation level. The previous code is updated as follows:

```

Session session = getSession();
Transaction tx = null;
tx = session.beginTransaction();
BookCh2 book = (BookCh2)session.get(BookCh2.class, new Long(32769));
session.lock(book, LockMode.UPGRADE);

```

```
String name = (String) session.createQuery("select b.name from bkch2 b where b.isbn = :isbn")
```

```
.setParameter("isbn", book.getIsbn()).uniqueResult();
```

```
System.out.println("BOOK's Name- "+name);
```

```
tx.commit();
```

```
session.close();
```

Or you can directly call

```
BookCh2 book = (BookCh2)session.get(BookCh2.class, new Long(32769),LockMode.UPGRADE);
```

LockMode.UPGRADE creates a lock on the specific book record in the database. Now, other concurrent

transactions can't modify the book record.

Hibernate supports the following lock modes:

- **LockMode.NONE**: This is the default lock mode. If an object is requested with this lock mode, a **READ** lock is obtained if it's necessary to read the state from the database, rather than pull it from a cache.
- **LockMode.READ**: In this lock mode, an object is read from the database. The object's version is checked, just as in memory.
- **LockMode.UPGRADE**: Objects loaded in this lock mode are materialized using an SQL **select ... for update**. It's equivalent to **LockModeType.READ** in Java Persistence.
- **LockMode.UPGRADE_NOWAIT**: This lock mode attempts to obtain an upgrade lock using an Oracle-style **select for update nowait**. The semantics of this lock mode, once obtained, are the same as **UPGRADE**.
- **LockMode.FORCE**: This lock mode results in a forced version increment. It's equivalent to **LockModeType.Write** in Java Persistence
- **LockMode.WRITE**: A **WRITE** lock is obtained when an object is updated or inserted. This lock mode is for internal use only and isn't a valid mode for **load()** or **lock()**.

```
=====
1 App1) Basic Application to Save obj/record
2 =====
3 -----hibernate.cfg.xml-----
4 <!DOCTYPE hibernate-configuration PUBLIC
5 "-//Hibernate/Hibernate Configuration DTD//EN"
6 "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
7
8 <hibernate-configuration>
9   <session-factory>
10     <!--connection properties -->
11     <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
12     <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
13     <property name="connection.username">scott</property>
14     <property name="connection.password">tiger</property>
15     <!-- hibernate properties -->
16     <property name="show_sql">>true</property>
17     <property name="hibernate.dialect">org.hibernate.dialect.Oracle10gDialect</property>
18     <!-- mapping file -->
19     <mapping resource="com/nt/cfgs/Employee.hbm.xml"/>
20   </session-factory>
21
22 </hibernate-configuration>
23 -----Employee.hbm.xml-----
24 <!DOCTYPE hibernate-mapping PUBLIC
25 "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
26 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
27
28 <hibernate-mapping>
29   <class name="com.nt.domain.EmpDetails" table="Employee">
30     <id name="no" column="EID"/> <!-- singular ID Field cfg -->
31     <property name="fname" column="FIRSTNAME" />
32     <property name="lname" column="LASTNAME" />
33     <property name="mail" column="EMAIL" />
34   </class>
35 </hibernate-mapping>
36 -----EmpDetails.java-----
37 package com.nt.domain;
38
39 public class EmpDetails {
40   //Properties
41   private int no;
42   private String fname,lname,mail;
43   //0-param constructor
44   public EmpDetails(){
45     System.out.println("EmpDetails:0-param constructor");
46   }
47   // full constructor
48   public EmpDetails(int no, String fname, String lname, String mail) {
49     System.out.println("EmpDetails: 4-param constructor");
50     this.no = no;
51     this.fname = fname;
52     this.lname = lname;
53     this.mail = mail;
54   }
55   //write setter and getters methods
56   public int getNo() {
57     return no;
58   }
59   public void setNo(int no) {
60     this.no = no;
61   }
62   public String getFname() {
63     return fname;
64   }
65 }
```

```

66     public void setFname(String fname) {
67         this.fname = fname;
68     }
69     public String getLname() {
70         return lname;
71     }
72     public void setLname(String lname) {
73         this.lname = lname;
74     }
75     public String getMail() {
76         return mail;
77     }
78     public void setMail(String mail) {
79         this.mail = mail;
80     }
81     // add toString()
82     @Override
83     public String toString() {
84         return "EmpDetails [no=" + no + ", fname=" + fname + ", lname=" + lname
85             + ", mail=" + mail + "];"
86     }
87 }
88 }
89 -----ClientApp.java-----
90 package com.nt.test;
91
92 import org.hibernate.Session;
93 import org.hibernate.SessionFactory;
94 import org.hibernate.Transaction;
95 import org.hibernate.cfg.Configuration;
96 import com.nt.domain.EmpDetails;
97
98 public class ClientApp {
99     public static void main(String[] args) throws Exception {
100         //Activate Hibernate F.w
101         Configuration cfg=new Configuration();
102         //read both xml files data to store InMemory MetaData
103         cfg=cfg.configure("/com/nt/cfgs/hibernate.cfg.xml");
104         // Get HB SessionFactory
105         SessionFactory factory=cfg.buildSessionFactory();
106         // Open Session
107         Session ses=factory.openSession();
108         package com.nt.test;
109
110 import org.hibernate.Session;
111 import org.hibernate.SessionFactory;
112 import org.hibernate.Transaction;
113 import org.hibernate.cfg.Configuration;
114 import com.nt.domain.EmpDetails;
115
116 public class ClientApp {
117     public static void main(String[] args) throws Exception {
118         //Activate Hibernate F.w
119         Configuration cfg=new Configuration();
120         //read both xml files data to store InMemory MetaData
121         cfg=cfg.configure("/com/nt/cfgs/hibernate.cfg.xml");
122         // Get HB SessionFactory
123         SessionFactory factory=cfg.buildSessionFactory();
124         // Open Session
125         Session ses=factory.openSession();
126
127         /*Transaction tx=null;
128         try{
129             //Begin Transaction
130             tx=ses.beginTransaction();

```

```

131         //model class obj with data
132         EmpDetails details=new EmpDetails();
133         details.setNo(1011);
134         //save object (insert record)
135         ses.save(details);
136         tx.commit();
137         System.out.println("Object Deleted/Record deleted");
138     }//try
139     catch(Exception e){
140         // tx.rollback();
141     }*/
142
143     Thread.sleep(30000);
144
145     //close Objects
146     ses.close();
147     factory.close();
148 }//main
149 }//class
150
151     //close Objects
152     ses.close();
153     factory.close();
154 }//main
155 }//class
156 -----ClientApp1.java-----
157 package com.nt.test;
158
159 import org.hibernate.Session;
160 import org.hibernate.SessionFactory;
161 import org.hibernate.Transaction;
162 import org.hibernate.cfg.Configuration;
163 import com.nt.domain.EmpDetails;
164
165 public class ClientApp1 {
166     public static void main(String[] args) {
167         //Activate Hibernate F.w
168         Configuration cfg=new Configuration();
169         //read both xml files data to store InMemory MetaData
170         cfg=cfg.configure("/com/nt/cfgs/hibernate.cfg.xml");
171         // Get HB SessionFactory
172         SessionFactory factory=cfg.buildSessionFactory();
173         // Open Session
174         Session ses=factory.openSession();
175
176         /* //Load obj /select a record
177         EmpDetails ed=(EmpDetails)ses.get(EmpDetails.class,1001);
178         //Display obj data
179         if(ed!=null){
180             System.out.println(ed.getNo()+" "+ed.getFname()+" "+ed.getLname()+" "+ed.getMail());
181         }
182         else{
183             System.out.println("Object/Record not found");
184         } */
185
186         //Load obj using ses.load(-,-) method
187         /* EmpDetails ed=(EmpDetails)ses.get(EmpDetails.class,1234);
188         System.out.println(ed); */
189
190         EmpDetails ed=(EmpDetails)ses.get(EmpDetails.class,1234);
191         Transaction tx=null;
192         try{
193             tx=ses.beginTransaction();
194             ed.setMail("x1@y2.com");
195             ed.setMail("x4@y5.com");

```

```

196         ed.setLname("rao3");
197         tx.commit();
198     }
199     catch(Exception e){
200         tx.rollback();
201     }
202
203     //close Objects
204     ses.close();
205     factory.close();
206 } //main
207 } //class
208 -----ClientApp2.java-----
209 package com.nt.test;
210
211 import org.hibernate.Session;
212 import org.hibernate.SessionFactory;
213 import org.hibernate.Transaction;
214 import org.hibernate.cfg.Configuration;
215 import com.nt.domain.EmpDetails;
216
217 public class ClientApp2 {
218     public static void main(String[] args) {
219
220         Session ses=new Configuration().configure("/com/nt/cfgs/hibernate.cfg.xml")
221         .buildSessionFactory().openSession();
222
223
224         //Approach1 to update record/object
225
226         /* EmpDetails ed=new EmpDetails(1003,"rajesh","rao","x@gmai.com");
227         Transaction tx=null;
228         try{
229             tx=ses.beginTransaction();
230             ses.update(ed);
231             tx.commit();
232         }
233         catch(Exception e){
234             tx.rollback();
235         } */
236
237         /* //Approach2 to update record/object
238         EmpDetails ed=(EmpDetails)ses.get(EmpDetails.class,103);
239         if(ed==null){
240             System.out.println("Record/obj not found ");
241         }
242         else{
243             Transaction tx=null;
244             try{
245                 tx=ses.beginTransaction();
246                 ed.setMail("x@yahoo.com");
247                 tx.commit();
248                 System.out.println("Record modified");
249             } //try
250             catch(Exception e){
251                 tx.rollback();
252             }
253             } //else */
254
255         /* //Approach3 to update record/object
256         EmpDetails ed=(EmpDetails)ses.get(EmpDetails.class,1003);
257         if(ed==null){
258             System.out.println("Record/obj not found ");
259         }
260         else{
261             Transaction tx=null;

```



```
261         try{
262             tx=ses.beginTransaction();
263             ed.setMail("x1@yahoo.com");
264             ses.update(ed);
265             tx.commit();
266             System.out.println("Record modified");
267         }//try
268         catch(Exception e){
269             tx.rollback();
270         }
271     }//else */
272
273     /* //Saving or updating object/record using ses.saveOrUpdate(-)
274     EmpDetails ed=new EmpDetails(1021,"ramesh2","rao1","x@y.com");
275     Transaction tx=null;
276     try{
277         tx=ses.beginTransaction();
278         ses.saveOrUpdate(ed);
279         tx.commit();
280     }
281     catch(Exception e){
282         tx.rollback();
283     } */
284
285     //Saving or updating obj/record using ses.merge(-)
286     EmpDetails ed=new EmpDetails(1001,"rakesh1","chari2","chari@x.com");
287     Transaction tx=null;
288     try{
289         tx=ses.beginTransaction();
290         EmpDetails ed1=(EmpDetails)s ses.merge(ed);
291         tx.commit();
292         System.out.println(ed1);
293     }//try
294     catch(Exception e){
295         tx.rollback();
296     }
297
298     //close Objects
299     ses.close();
300
301 }//main
302 }//class
303 -----ClientApp3.java-----
304 package com.nt.test;
305
306 import org.hibernate.Session;
307 import org.hibernate.SessionFactory;
308 import org.hibernate.Transaction;
309 import org.hibernate.cfg.Configuration;
310 import com.nt.domain.EmpDetails;
311
312 public class ClientApp3 {
313     public static void main(String[] args) {
314         //Activate Hibernate F.w
315         Configuration cfg=new Configuration();
316         //read both xml files data to store InMemory MetaData
317         cfg=cfg.configure("/com/nt/cfgs/hibernate.cfg.xml");
318         // Get HB SessionFactory
319         SessionFactory factory=cfg.buildSessionFactory();
320         // Open Session
321         Session ses=factory.openSession();
322
323         /* //Approach1 to delete obj/record
324         EmpDetails ed=new EmpDetails();
325         ed.setNo(1011);
```

```
326 Transaction tx=null;
327 try{
328     tx=ses.beginTransaction();
329     ses.delete(ed);
330     tx.commit();
331     System.out.println("Object/record deleted");
332 }
333 catch(Exception e){
334     tx.rollback();
335 } */
336
337 /* //Approach2 to delete obj/record
338 EmpDetails ed=(EmpDetails)sess.get(EmpDetails.class,8901);
339 if(ed==null)
340     System.out.println("Record/obj not found");
341 else{
342     Transaction tx=null;
343     try{
344         tx=ses.beginTransaction();
345         ses.delete(ed);
346         tx.commit();
347         System.out.println("Object/record deleted");
348     }
349     catch(Exception e){
350         tx.rollback();
351     }
352 } //else */
353 //Performing non-select persistence operations on Db table
354 // with out using Transaction env..
355
356
357
358 //close Objects
359 sess.close();
360 factory.close();
361 } //main
362 } //class
```

```
363 -----ClientApp4.java-----
364 package com.nt.test;
365
366 import org.hibernate.Session;
367 import org.hibernate.SessionFactory;
368 import org.hibernate.Transaction;
369 import org.hibernate.cfg.Configuration;
370 import com.nt.domain.EmpDetails;
371
372 public class ClientApp4 {
373     public static void main(String[] args) {
374         //Activate Hibernate F.w
375         Configuration cfg=new Configuration();
376
377         cfg.setProperty("hibernate.connection.driver_class","oracle.jdbc.driver.OracleDriver");
378         cfg.setProperty("hibernate.connection.url","jdbc:oracle:thin:@localhost:1521:xe");
379         cfg.setProperty("hibernate.connection.username","scott");
380         cfg.setProperty("hibernate.connection.password","tiger1");
381         cfg.setProperty("show_sql","true");
382
383         cfg.addFile("com/nt/cfgs/Employee.hbm.xml");
384
385
386         // Get HB SessionFactory
387         SessionFactory factory=cfg.buildSessionFactory();
388         // Open Session
389         Session ses=factory.openSession();
390     }
391 }
```

```
391         //load obj/record
392         EmpDetails ed=(EmpDetails)ses.get(EmpDetails.class,1005);
393         System.out.println(ed);
394
395     }
396 }
397
398 -----ClientApp5.java-----
399 package com.nt.test;
400
401 import org.hibernate.Session;
402 import org.hibernate.SessionFactory;
403 import org.hibernate.Transaction;
404 import org.hibernate.cfg.Configuration;
405 import com.nt.domain.EmpDetails;
406
407 public class ClientApp5 {
408     public static void main(String[] args)throws Exception {
409         //Activate Hibernate F.w
410         Configuration cfg=new Configuration();
411         //read both xml files data to store InMemory MetaData
412         cfg=cfg.configure("/com/nt/cfgs/hibernate.cfg.xml");
413         // Get HB SessionFactory
414         SessionFactory factory=cfg.buildSessionFactory();
415         // Open Session
416         Session ses=factory.openSession();
417
418         EmpDetails ed=null; // Transient state
419
420         ed=(EmpDetails)ses.get(EmpDetails.class,1001); //Persistent
421         Transaction tx=ses.beginTransaction();
422         ed.setMail("xx1@yy.com"); // persistent (Modifications will reflect to Db table)
423         tx.commit();
424         ses.clear();
425         Thread.sleep(20000);
426         ed.setMail("zz@yy.com"); // Detached state
427
428         tx=ses.beginTransaction();
429         ses.merge(ed); //Persistent state
430         tx.commit();
431
432
433     } //main
434 } //class
435
436 -----ClientApp6.java-----
437 package com.nt.test;
438
439 import org.hibernate.Session;
440 import org.hibernate.SessionFactory;
441 import org.hibernate.Transaction;
442 import org.hibernate.cfg.Configuration;
443
444 import com.nt.domain.EmpDetails;
445
446 public class ClientApp6 {
447     public static void main(String[] args){
448         //Activate Hibernate F.w
449         Configuration cfg=new Configuration();
450         //read both xml files data to store InMemory MetaData
451         cfg=cfg.configure("/com/nt/cfgs/hibernate.cfg.xml");
452         // Get HB SessionFactory
453         SessionFactory factory=cfg.buildSessionFactory();
454         // Open Session
```

```

36     Session ses1=factory.openSession();
37
38     EmpDetails ed1=null; //transient state
39     ed1=(EmpDetails)ses1.get(EmpDetails.class,1001); //persistent state
40     ses1.close();
41     ed1.setMail("x62@y1.com"); //Detached state
42     // (Modifications will not be reflected)
43
44     Session ses2=factory.openSession();
45     EmpDetails ed2=null; //transient state
46     ed2=(EmpDetails)ses2.get(EmpDetails.class,1001); //persistent state
47     Transaction tx=null;
48     try{
49         tx=ses2.beginTransaction();
50         ses2.saveOrUpdate(ed1);
51         ed2.setLname("rakesh3");
52         tx.commit();
53     }//try
54     catch(Exception e){
55         tx.rollback();
56         e.printStackTrace();
57     }
58 }//main
59 }//class
60
61 =====
62 App2) CURD Persistence operations using DAO pattern
63 =====
64 -----hibernate.cfg.xml-----
65 <!DOCTYPE hibernate-configuration PUBLIC
66 "-//Hibernate/Hibernate Configuration DTD//EN"
67 "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
68 <hibernate-configuration>
69     <session-factory>
70         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
71         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
72         <property name="connection.username">scott</property>
73         <property name="connection.password">tiger</property>
74
75         <property name="hibernate.hbm2ddl.auto">update</property>
76         <property name="show_sql">>true</property>
77         <property name="hibernate.dialect">org.hibernate.dialect.Oracle10gDialect</property>
78
79         <mapping resource="com/nt/cfgs/Employee.hbm.xml"/>
80     </session-factory>
81
82 </hibernate-configuration>
83 -----Employee.hbm.xml-----
84 <!DOCTYPE hibernate-mapping PUBLIC
85 "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
86 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
87 <hibernate-mapping>
88     <class name="com.nt.domain.EmpDetails" table="Employee">
89         <id name="no" column="EID" type="int"/> <!-- singular ID Field cfg -->
90         <property name="fname" column="FIRSTNAME" length="20" type="string"/>
91         <property name="lname" column="LASTNAME" length="20" type="string"/>
92         <property name="mail" column="EMAIL" length="15" type="string" />
93     </class>
94 </hibernate-mapping>
95 -----EmpDetails.java-----
96 package com.nt.domain;
97
98
99
100

```

```
521 public class EmpDetails {
522     //Properties
523     private int no;
524     private String fname,lname,mail;
525     //0-param constructor
526     public EmpDetails(){
527         System.out.println("EmpDetails:0-param constructor");
528     }
529     // full constructor
530     public EmpDetails(int no, String fname, String lname, String mail) {
531         System.out.println("EmpDetails: 4-param constructor");
532         this.no = no;
533         this.fname = fname;
534         this.lname = lname;
535         this.mail = mail;
536     }
537     //write setter and getters methods
538     public int getNo() {
539         return no;
540     }
541     public void setNo(int no) {
542         this.no = no;
543     }
544     public String getFname() {
545         return fname;
546     }
547     public void setFname(String fname) {
548         this.fname = fname;
549     }
550     public String getLname() {
551         return lname;
552     }
553     public void setLname(String lname) {
554         this.lname = lname;
555     }
556     public String getMail() {
557         return mail;
558     }
559     public void setMail(String mail) {
560         this.mail = mail;
561     }
562     // add toString()
563     @Override
564     public String toString() {
565         return "EmpDetails [no=" + no + ", fname=" + fname + ", lname=" + lname
566             + ", mail=" + mail + "];"
567     }
568
569     String xyz;
570     public String getXyz() {
571         return xyz;
572     }
573     public void setXyz(String xyz) {
574         this.xyz = xyz;
575     }
576
577 }
578 -----EmpDAO.java-----
```

```
579 package com.nt.dao;
580
581 import com.nt.domain.EmpDetails;
582
583 public interface EmpDAO {
584     public boolean registerEmployee(int no,String fname,String lname,String mail);
585 }
```

```
586     public EmpDetails search(int no);
587     public boolean  modifyEmail(int no,String newmaild);
588     public boolean  removeEmp(int no);
589
590 }
591 -----EmpDAOImpl.java-----
592 package com.nt.dao;
593
594 import org.hibernate.Session;
595 import org.hibernate.Transaction;
596
597 import com.nt.domain.EmpDetails;
598 import com.nt.utility.HibernateUtil;
599
600 public class EmpDAOImpl implements EmpDAO {
601
602     @Override
603     public boolean registerEmployee(int no, String fname, String lname,
604         String mail) {
605         // Get Session
606         Session ses=HibernateUtil.getSession();
607         Transaction tx=null;
608         try{
609             tx=ses.beginTransaction();
610             //create Domain class obj
611             EmpDetails details=new EmpDetails(no,fname,lname,mail);
612             //save obj
613             ses.save(details);
614             tx.commit();
615             return true;
616         }
617         catch(Exception e){
618             tx.rollback();
619             return false;
620         }
621     } //method
622
623     @Override
624     public EmpDetails search(int no) {
625         // get Session obj
626         Session ses=HibernateUtil.getSession();
627         System.out.println(ses.hashCode());
628         //Load obj/record
629         EmpDetails details=(EmpDetails)ses.get(EmpDetails.class,no);
630         if(details!=null)
631             return details;
632         else
633             return null;
634     }
635
636     @Override
637     public boolean modifyEmail(int no, String newmaild) {
638         // Get Hb Session obj
639         Session ses=HibernateUtil.getSession();
640         System.out.println(ses.hashCode());
641         //Load obj/record
642         EmpDetails details=(EmpDetails)ses.get(EmpDetails.class,no);
643         if(details!=null){
644             details.setMail(newmaild);
645         }
646         else{
647             return false;
648         }
649     }
650 }
```

```
651         Transaction tx=null;
652         try{
653             tx=ses.beginTransaction();
654             //update obj;
655             ses.update(details);
656             tx.commit();
657             return true;
658         }
659         catch(Exception e){
660             tx.rollback();
661             return false;
662         }
663     }//method
664
665
666     @Override
667     public boolean removeEmp(int no) {
668         //Get Session obj
669         Session ses=HibernateUtil.getSession();
670
671
672         //Load obj
673         EmpDetails details=(EmpDetails)ses.get(EmpDetails.class,no);
674         Transaction tx=null;
675         if(details!=null){
676             try{
677                 tx=ses.beginTransaction();
678                 //delete obj
679                 ses.delete(details);
680                 tx.commit();
681                 return true;
682             }
683             catch(Exception e){
684                 tx.rollback();
685                 return false;
686             }
687         }//if
688         else{
689             return false;
690         }
691     }//method
692
693 }//class
```

-----EmpDAOFactory.java-----

```
695 package com.nt.dao;
696
697 public class EmpDAOFactory {
698
699     public static EmpDAO getDAOInstance(){
700         return new EmpDAOImpl();
701     }//method
702 }//class
```

-----HibernateUtil.java-----

```
704 package com.nt.utility;
705
706 import org.hibernate.Session;
707 import org.hibernate.SessionFactory;
708 import org.hibernate.cfg.Configuration;
709
710 public class HibernateUtil {
711     private static SessionFactory factory;
712     private static ThreadLocal<Session> tl=new ThreadLocal<Session>();
713
714     static{
715         try{
```

```

716     factory=new Configuration().configure("/com/nt/cfgs/hibernate.cfg.xml").buildSessionFactory();
717 }
718 catch(Exception e){
719     e.printStackTrace();
720 }
721
722 }//static block
723
724 static Session ses=null;
725 public static Session getSession(){
726     try{
727         System.out.println("Thread name"+Thread.currentThread().getName());
728         System.out.println("factory hashCode"+factory.hashCode());
729         if(tl.get()==null){
730             ses=factory.openSession();
731             tl.set(ses);
732             return ses;
733         }
734         else{
735             ses=tl.get();
736             return ses;
737         }
738     }//try
739     catch(Exception e){
740         return null;
741     }//catch
742 }//method
743
744 public static void closeSession(){
745     try{
746         if(tl.get()!=null){
747             tl.get().close();
748         }
749     }
750     catch(Exception e){
751         e.printStackTrace();
752     }
753 }//closeSession
754 public static void closeSessionFactory(){
755     factory.close();
756 }
757 }//class
758 -----ClientApp.java-----
759 package com.nt.test;
760
761 import com.nt.dao.EmpDAO;
762 import com.nt.dao.EmpDAOFactory;
763
764 public class ClientApp {
765     public static void main(String[] args)throws Exception {
766         //get DAO Object
767         EmpDAO dao=EmpDAOFactory.getDAOInstance();
768         // Insert record
769         //System.out.println("Inseted?"+"dao.registerEmployee(1003,"ramesh","rao" ,"x@y.com"));
770         System.out.println("1003 emp details"+dao.search(1001));
771     }
772 }
773
774
775 =====
776 App3) App using Controller,Service,DAO clases with VO class,BO class
777 =====
778 -----hibernate.cfg.xml-----
779 <!DOCTYPE hibernate-configuration PUBLIC
780 "-//Hibernate/Hibernate Configuration DTD//EN"

```



```

781 "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
782
783 <hibernate-configuration>
784   <session-factory>
785     <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
786     <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
787     <property name="connection.username">scott</property>
788     <property name="connection.password">tiger</property>
789
790     <property name="hibernate.hbm2ddl.auto">update</property>
791     <property name="show_sql">>true</property>
792     <property name="hibernate.dialect">org.hibernate.dialect.Oracle10gDialect</property>
793
794     <mapping resource="com/nt/cfgs/Student.hbm.xml"/>
795   </session-factory>
796
797 </hibernate-configuration>
798 -----Student.hbm.xml-----
799 <!DOCTYPE hibernate-mapping PUBLIC
800   "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
801   "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
802
803
804 <hibernate-mapping>
805   <class name="com.nt.bo.StudentBO" table="Student_Tab">
806     <id name="stno"/> <!-- singular ID Field cfg -->
807     <property name="stname" />
808     <property name="total"/>
809     <property name="avg" />
810
811   </class>
812 </hibernate-mapping>
813 /* Student_tab
814   |---->stno n(5) pk
815   |---->stname (vc2)
816   |---->total n(5)
817   |---->avg (n) (7,2) */
818 -----HibernateUtil.java-----
819 package com.nt.utility;
820
821 import org.hibernate.Session;
822 import org.hibernate.SessionFactory;
823 import org.hibernate.cfg.Configuration;
824
825 public class HibernateUtil {
826   private static SessionFactory factory;
827   private static ThreadLocal<Session> tl=new ThreadLocal<Session>();
828
829   static{
830     try{
831       Configuration cfg=new Configuration().configure("/com/nt/cfgs/hibernate.cfg.xml");
832       ServiceRegistryBuilder builder=new ServiceRegistryBuilder();
833       ServiceRegistry registry=builder.applySettings(cfg.getProperties()).buildServiceRegistry();
834       //BuildSessionFactory
835       factory=cfg.buildSessionFactory(registry);
836     }
837     catch(Exception e){
838       e.printStackTrace();
839     }
840
841   }//static block
842
843   static Session ses=null;
844   public static Session getSession(){
845     try{

```

```
846         if(tl.get()==null){
847             ses=factory.openSession();
848             tl.set(ses);
849             return ses;
850         }
851         else{
852             ses=tl.get();
853             return ses;
854         }
855     }//try
856     catch(Exception e){
857         return null;
858     }//catch
859 }//method
860
861 public static void closeSession(){
862     try{
863         ses.close();
864     }
865     catch(Exception e){
866         e.printStackTrace();
867     }
868 }//closeSession
869 }//class
```

```
870 -----StudentVO.java-----
871 package com.nt.vo;
872
873 public class StudentVO {
874     private int sno;
875     private String sname;
876     private int m1,m2,m3;
877
878     public StudentVO(int sno, String sname, int m1, int m2, int m3) {
879         System.out.println("StudentVO:5-param constructor");
880         this.sno = sno;
881         this.sname = sname;
882         this.m1 = m1;
883         this.m2 = m2;
884         this.m3 = m3;
885     }
886     public StudentVO(){
887         System.out.println("StudentVO:0-param constructor");
888     }
889     public int getSno() {
890         return sno;
891     }
892     public void setSno(int sno) {
893         this.sno = sno;
894     }
895     public String getSname() {
896         return sname;
897     }
898     public void setSname(String sname) {
899         this.sname = sname;
900     }
901     public int getM1() {
902         return m1;
903     }
904     public void setM1(int m1) {
905         this.m1 = m1;
906     }
907     public int getM2() {
908         return m2;
909     }
910     public void setM2(int m2) {
```

```
911         this.m2 = m2;
912     }
913     public int getM3() {
914         return m3;
915     }
916     public void setM3(int m3) {
917         this.m3 = m3;
918     }
919     @Override
920     public String toString() {
921         return "StudentVO [sno=" + sno + ", sname=" + sname + ", m1=" + m1
922             + ", m2=" + m2 + ", m3=" + m3 + "]";
923     }
924 }
925 -----StudentBO.java-----
926 package com.nt.bo;
927
928 public class StudentBO {
929     private int stno;
930     private String stname;
931     private int total;
932     private float avg;
933
934     public StudentBO(){
935         System.out.println("StudentBO:0-param constructor");
936     }
937
938
939
940     public StudentBO(int stno, String stname, int total, float avg) {
941         System.out.println("StudentBO:4-param constructor");
942         this.stno = stno;
943         this.stname = stname;
944         this.total = total;
945         this.avg = avg;
946     }
947
948     public int getStno() {
949         return stno;
950     }
951     public void setStno(int stno) {
952         this.stno = stno;
953     }
954     public String getStname() {
955         return stname;
956     }
957     public void setStname(String stname) {
958         this.stname = stname;
959     }
960     public int getTotal() {
961         return total;
962     }
963     public void setTotal(int total) {
964         this.total = total;
965     }
966     public float getAvg() {
967         return avg;
968     }
969     public void setAvg(float avg) {
970         this.avg = avg;
971     }
972
973     @Override
974     public String toString() {
975         return "StudentBO [stno=" + stno + ", stname=" + stname + ", total="
```

```
76         + total + ", avg=" + avg + "];"
77     }
78 }
79 -----StudentController.java-----
80 package com.nt.controller;
81
82 import com.nt.Service.StudentService;
83 import com.nt.vo.StudentVO;
84
85 public class StudentController {
86
87     public int registerStudent(int no,String name,int m1,int m2,int m3){
88         // Convert inputs VO class obj
89         StudentVO stvo=new StudentVO(no,name,m1,m2,m3);
90
91         //Create Service class obj
92         StudentService service=new StudentService();
93         int id=service.process(stvo);
94
95         return id;
96     }
97 }
98 }
99 -----StudentService.java-----
00 package com.nt.Service;
01
02 import com.nt.bo.StudentBO;
03 import com.nt.dao.StudentDAO;
04 import com.nt.dao.StudentDAOFactory;
05 import com.nt.vo.StudentVO;
06
07 public class StudentService {
08
09
10     public int process(StudentVO stvo){
11         // write b.logic /Service logic
12         int total=stvo.getM1()+stvo.getM2()+stvo.getM3();
13         float avg=(float)total/3.0f;
14
15         //create BO /Domain class obj
16         StudentBO stbo=new StudentBO();
17         stbo.setStno(stvo.getSno());
18         stbo.setSname(stvo.getSname());
19         stbo.setTotal(total);
20         stbo.setAvg(avg);
21
22         // Use DAO class for persitence operations
23         StudentDAO stdao=StudentDAOFactory.getInstance();
24         int id=stdao.insert(stbo);
25         return id;
26     } //method
27 } //class
28
29 -----StudentDAO.java-----
30 package com.nt.dao;
31
32 import com.nt.bo.StudentBO;
33
34 public interface StudentDAO {
35
36     public int insert(StudentBO stbo);
37
38 }
39 -----StudentDAOImpl.java-----
40 package com.nt.dao;
```

```

1041
1042 import org.hibernate.Session;
1043 import org.hibernate.Transaction;
1044
1045 import com.nt.bo.StudentBO;
1046 import com.nt.utility.HibernateUtil;
1047
1048 public class StudentDAOImpl implements StudentDAO {
1049
1050     @Override
1051     public int insert(StudentBO stbo) {
1052         //get HB Session obj
1053         Session ses=HibernateUtil.getSession();
1054         Transaction tx=null;
1055         try{
1056             tx=ses.beginTransaction();
1057             int id=(Integer)ses.save(stbo);
1058             tx.commit();
1059             //close Session
1060             HibernateUtil.closeSession();
1061             return id;
1062         }
1063         catch(Exception e){
1064             tx.rollback();
1065             //close Session
1066             HibernateUtil.closeSession();
1067             return -1;
1068         }
1069
1070     } //method
1071
1072 } //class
1073 -----StudentDAOFactory.java-----
1074 package com.nt.dao;
1075
1076 public class StudentDAOFactory {
1077
1078     public static StudentDAO getInstance(){
1079         return new StudentDAOImpl();
1080     }
1081 }
1082 -----ClientApp.java-----
1083 package com.nt.test;
1084
1085 import com.nt.controller.StudentController;
1086
1087 public class ClientApp {
1088
1089     public static void main(String[] args) {
1090         // Get Controller class obj
1091         StudentController controller=new StudentController();
1092         // call method
1093         int id=controller.registerStudent(101,"Raja",90,80,60);
1094         if(id!=-1)
1095             System.out.println("Student Registered with id"+id);
1096         else
1097             System.out.println("Registration Failed");
1098     } //main
1099 } //class
1100
1101 =====
1102 App3) Talking with two DB s/ws
1103 =====
1104 -----hibernate-ora.cfg.xml-----
1105 <!DOCTYPE hibernate-configuration

```

```

1106 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
1107 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
1108
1109 <hibernate-configuration>
1110     <session-factory>
1111         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
1112         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
1113         <property name="connection.username">scott</property>
1114         <property name="connection.password">tiger</property>
1115         <property name="show_sql">>true</property>
1116         <property name="hibernate.hbm2ddl.auto">update</property>
1117         <mapping resource="com/nt/cfgs/Employee.hbm.xml"/>
1118     </session-factory>
1119 </hibernate-configuration>
1120
1121 -----hibernate-mysql.cfg.xml-----
1122 <!DOCTYPE hibernate-configuration
1123 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
1124 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
1125
1126 <hibernate-configuration>
1127     <session-factory>
1128         <property name="connection.driver_class">com.mysql.jdbc.Driver</property>
1129         <property name="connection.url">jdbc:mysql:///NtDb1</property>
1130         <property name="connection.username">root</property>
1131         <property name="connection.password">root</property>
1132         <property name="dialect">org.hibernate.dialect.MySQL5Dialect</property>
1133         <property name="show_sql">>true</property>
1134         <mapping resource="com/nt/cfgs/Employee.hbm.xml"/>
1135     </session-factory>
1136 </hibernate-configuration>
1137 -----Employee.hbm.xml-----
1138 <!DOCTYPE hibernate-mapping SYSTEM
1139 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
1140
1141 <hibernate-mapping>
1142     <class name="com.nt.domain.EmpDetails" table="Employee">
1143         <id name="no" column="EID"/> <!--Singular Id field cfg -->
1144         <property name="fname" column="FIRSTNAME" length="20"/>
1145         <property name="lname" column="LASTNAME" length="30"/>
1146         <property name="mail" column="EMAIL" length="20" unique="true" not-null="true"/>
1147     </class>
1148 </hibernate-mapping>
1149
1150 -----MySqlHibernateUtil.java-----
1151 package com.nt.utility;
1152 import org.hibernate.Session;
1153 import org.hibernate.SessionFactory;
1154 import org.hibernate.cfg.Configuration;
1155
1156 public class MySqlHibernateUtil {
1157     private static SessionFactory factory=null;
1158     static{
1159         try{
1160             Configuration cfg=new Configuration().configure("/com/nt/cfgs/hibernate-mysql.cfg.xml");
1161             ServiceRegistryBuilder builder=new ServiceRegistryBuilder();
1162             ServiceRegistry registry=builder.applySettings(cfg.getProperties()).buildServiceRegistry();
1163             //BuildSessionFactory
1164             factory=cfg.buildSessionFactory(registry);
1165             catch(Exception e){
1166                 e.printStackTrace();
1167             }
1168         }
1169     }
1170     static ThreadLocal <Session> tl=new ThreadLocal<Session>();

```

```
1171     static Session ses=null;
1172     public static Session getSession(){
1173     try{
1174         if(tl.get()==null){ // get from ThreadLocal
1175             ses=factory.openSession();
1176             tl.set(ses); // set into ThreadLocal
1177             return ses;
1178         }
1179         else{
1180             return tl.get(); //get from ThreadLocal
1181         }
1182     }//try
1183     catch(Exception e){
1184         return null;
1185     }
1186 }//method
1187
1188     public static void closeSession(){
1189     try{
1190         ses.close();
1191     }
1192     catch(Exception e){
1193         e.printStackTrace();
1194     }
1195     } //closeSession()
1196 }//class
1197 -----OracleHibenrateUtility.java-----
1198 package com.nt.utility;
1199 import org.hibernate.Session;
1200 import org.hibernate.SessionFactory;
1201 import org.hibernate.cfg.Configuration;
1202
1203 public class OracleHibernateUtil {
1204     private static SessionFactory factory=null;
1205     static{
1206     try{
1207         Configuration cfg=new Configuration().configure("/com/nt/cfgs/hibernate-ora.cfg.xml");
1208         ServiceRegistryBuilder builder=new ServiceRegistryBuilder();
1209         ServiceRegistry registry=builder.applySettings(cfg.getProperties()).buildServiceRegistry();
1210         //BuildSessionFactory
1211         factory=cfg.buildSessionFactory(registry);
1212     }
1213     catch(Exception e){
1214         e.printStackTrace();
1215     }
1216 }//static
1217
1218     static ThreadLocal <Session> tl=new ThreadLocal<Session>();
1219     static Session ses=null;
1220     public static Session getSession(){
1221     try{
1222         if(tl.get()==null){ // get from ThreadLocal
1223             ses=factory.openSession();
1224             tl.set(ses); // set into ThreadLocal
1225             return ses;
1226         }
1227         else{
1228             return tl.get(); //get from ThreadLocal
1229         }
1230     }//try
1231     catch(Exception e){
1232         return null;
1233     }
1234 }//method
1235
```

```
1236     public static void closeSession(){
1237         try{
1238             ses.close();
1239         }
1240         catch(Exception e){
1241             e.printStackTrace();
1242         }
1243     } //closeSession()
1244 }//class
1245
1246 -----EmpDetails.java-----
1247 package com.nt.domain;
1248
1249 public class EmpDetails {
1250     //properties
1251     private int no;
1252     private String fname,lname,mail;
1253
1254     public EmpDetails(){
1255         System.out.println("EmpDetails:0-param constructor");
1256     }
1257     public EmpDetails(int no, String fname, String lname,String mail) {
1258         System.out.println("EmpDetails:4-param constructor");
1259         this.no = no;
1260         this.fname = fname;
1261         this.lname = lname;
1262         this.mail=mail;
1263     }
1264     public int getNo() {
1265         return no;
1266     }
1267     public void setNo(int no) {
1268         this.no = no;
1269     }
1270     public String getFname() {
1271         return fname;
1272     }
1273     public void setFname(String fname) {
1274         this.fname = fname;
1275     }
1276     public String getLname() {
1277         return lname;
1278     }
1279     public void setLname(String lname) {
1280         this.lname = lname;
1281     }
1282     public String getMail() {
1283         return mail;
1284     }
1285     public void setMail(String mail) {
1286         this.mail = mail;
1287     }
1288     @Override
1289     public String toString() {
1290         return "EmpDetails [no=" + no + ", fname=" + fname + ", lname=" + lname
1291             + ", mail=" + mail + "]\n";
1292     }
1293 }
1294 -----EmployeeDAO.java-----
1295 package com.nt.dao;
1296
1297 public interface EmployeeDAO {
1298     public String trasferEmployee(int no);
1299 }
1300 }
```



```
1301 -----EmployeeDAOImpl.java-----package com.nt.dao;
1302
1303 import org.hibernate.Transaction;
1304
1305 import org.hibernate.Session;
1306
1307 import com.nt.domain.EmpDetails;
1308 import com.nt.utility.MySqlHibernateUtil;
1309 import com.nt.utility.OracleHibernateUtil;
1310
1311 public class EmployeeDAOImpl implements EmployeeDAO {
1312
1313     @Override
1314     public String trasferEmployee(int no) {
1315         //HB Session of oracle
1316         Session orases=OracleHibernateUtil.getSession();
1317         //HB Session if mysql
1318         Session mysqlses=MySqlHibernateUtil.getSession();
1319         //get object from oracle
1320         EmpDetails eb=(EmpDetails)orases.get(EmpDetails.class,no);
1321         if(eb==null)
1322             return "employee not found";
1323
1324         //insert obj into mysql
1325         Transaction tx=null;
1326         try{
1327             tx=mysqlses.beginTransaction();
1328             if(eb!=null){
1329                 mysqlses.save(eb);
1330             }
1331             tx.commit();
1332         }//try
1333         catch(Exception e){
1334             tx.rollback();
1335             return "Employee not trasferred";
1336         }
1337
1338         //delete obj from Oralce
1339         tx=null;
1340         try{
1341             tx=orases.beginTransaction();
1342             if(eb!=null){
1343                 orases.delete(eb);
1344             }
1345             tx.commit();
1346         }//try
1347         catch(Exception e){
1348             tx.rollback();
1349             return "Employee transfered but not deleted from source";
1350         }
1351         OracleHibernateUtil.closeSession();
1352         MySqlHibernateUtil.closeSession();
1353         return "Emp Transfered";
1354     }//method
1355 }//class
1356
1357 -----EmployeeDAOFactory.java-----
1358 package com.nt.dao;
1359
1360 public class EmployeeDAOFactory {
1361     public static EmployeeDAO getEmployeeDAO(){
1362         return new EmployeeDAOImpl();
1363     }
1364 }
1365
```

```

1366 -----ClientApp.java-----
1367 package com.nt.test;
1368
1369 import com.nt.dao.EmployeeDAO;
1370 import com.nt.dao.EmployeeDAOFactory;
1371
1372 public class ClientApp {
1373     public static void main(String[] args) {
1374         EmployeeDAO dao=EmployeeDAOFactory.getEmployeeDAO();
1375         System.out.println(dao.trasferEmployee(101));
1376     }
1377 }
1378
1379 =====
1380 App4) Object Versioning
1381 =====
1382 <!DOCTYPE hibernate-configuration
1383 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
1384 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
1385
1386 <hibernate-configuration>
1387     <session-factory>
1388         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
1389         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
1390         <property name="connection.username">scott</property>
1391         <property name="connection.password">tiger</property>
1392         <property name="show_sql">>true</property>
1393         <property name="hibernate.hbm2ddl.auto">update</property>
1394         <mapping resource="com/nt/cfgs/Employee.hbm.xml"/>
1395     </session-factory>
1396 </hibernate-configuration>
1397
1398 -----Employee.hbm.xml-----
1399 <!DOCTYPE hibernate-mapping SYSTEM
1400 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
1401
1402 <hibernate-mapping>
1403     <class name="com.nt.domain.EmpDetails" table="Employee">
1404         <id name="no" column="EID"/> <!--Singular Id field cfg -->
1405         <version name="ver" column="ver_col"/>
1406         <timestamp name="tstamp"/>
1407         <property name="fname" column="FIRSTNAME" />
1408         <property name="lname" column="LASTNAME" />
1409         <property name="mail" column="EMAIL" />
1410     </class>
1411 </hibernate-mapping>
1412
1413 -----EmpDetails.java-----
1414 // EmpDetails.java (Domain class/model class/Persistence class)
1415 package com.nt.domain;
1416 public class EmpDetails {
1417     //Properties
1418     private int no;
1419     private String fname, lname, mail;
1420     private int ver;
1421
1422     public int getVer() {
1423         return ver;
1424     }
1425
1426     public void setVer(int ver) {
1427         this.ver = ver;
1428     }
1429
1430     public EmpDetails(){

```

```
1431         System.out.println("EmpDetails:0-param constructor");
1432     }
1433
1434     //write setXxx(-) and getXxx() methods
1435     public int getNo() {
1436         return no;
1437     }
1438     public void setNo(int no) {
1439         this.no = no;
1440     }
1441     public String getFname() {
1442         return fname;
1443     }
1444     public void setFname(String fname) {
1445         this.fname = fname;
1446     }
1447     public String getLname() {
1448         return lname;
1449     }
1450     public void setLname(String lname) {
1451         this.lname = lname;
1452     }
1453     public String getMail() {
1454         return mail;
1455     }
1456     public void setMail(String mail) {
1457         this.mail = mail;
1458     }
1459
1460     @Override
1461     public String toString() {
1462         return "EmpDetails [no=" + no + ", fname=" + fname + ", lname=" + lname
1463             + ", mail=" + mail + "]\n";
1464     }
1465 }
```

-----HibernateUtility.java-----

Same as previous App

-----Testlient.java-----

```
1469 package com.nt.test;
1470
1471 import org.hibernate.Session;
1472 import org.hibernate.SessionFactory;
1473 import org.hibernate.Transaction;
1474 import org.hibernate.cfg.Configuration;
1475
1476 import com.nt.domain.EmpDetails;
1477
1478 public class TestClient {
1479
1480     public static void main(String[] args) throws Exception{
1481
1482         //Get Session
1483         Session ses=HibernateUtility.getSession();
1484
1485         /* Insert Record
1486         EmpDetails eb=new EmpDetails();
1487         eb.setNo(1010); eb.setFname("rajesh"); eb.setLname("rao");
1488         eb.setMail("rao@x.com");
1489
1490         Transaction tx=null;
1491         try{
1492             tx=ses.beginTransaction();
1493             ses.save(eb);
1494             tx.commit();
1495         }//try
```

```
1496     catch(Exception e){
1497         tx.rollback();
1498     } */
1499
1500     /**/Update object
1501     EmpDetails eb=(EmpDetails)ses.get(EmpDetails.class,1010);
1502     Transaction tx=null;
1503     try{
1504         tx=ses.beginTransaction();
1505         eb.setMail("x1@z2.com");
1506         tx.commit();
1507     }
1508     catch(Exception e){
1509         tx.rollback();
1510     } */
1511
1512     EmpDetails eb=new EmpDetails();
1513     eb.setNo(1010);
1514     eb.setFname("rajesh");
1515     eb.setLname("rao");
1516     eb.setMail("rao@x.com");
1517     Transaction tx=ses.beginTransaction();
1518     ses.update(eb);
1519     tx.commit();
1520
1521
1522     //Close objs
1523     ses.close();
1524     factory.close();
1525 }//main
1526 }//class
```

App5) Object Time Stamping Feature

```
1531 -----hibernate.cfg.xml-----
1532 <!DOCTYPE hibernate-configuration
1533 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
1534 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
1535
1536 <hibernate-configuration>
1537     <session-factory>
1538         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
1539         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
1540         <property name="connection.username">scott</property>
1541         <property name="connection.password">tiger</property>
1542         <property name="show_sql">>true</property>
1543         <property name="hibernate.hbm2ddl.auto">update</property>
1544         <mapping resource="com/nt/cfgs/Employee.hbm.xml"/>
1545     </session-factory>
1546 </hibernate-configuration>
1547
1548 -----Employee.hbm.xml-----
1549 <!DOCTYPE hibernate-mapping SYSTEM
1550 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
1551
1552 <hibernate-mapping>
1553     <class name="com.nt.domain.EmpDetails" table="Employee">
1554         <id name="no" column="EID"/> <!-- Singular Id field cfg -->
1555         <property name="fname" column="FIRSTNAME" />
1556         <property name="lname" column="LASTNAME" />
1557         <property name="mail" column="EMAIL" />
1558
1559     </class>
1560 </hibernate-mapping>
```

```
1561 -----EmpDetails.java-----
1562 // EmpDetails.java (Domain class/model class/Persistence class)
1563 package com.nt.domain;
1564 public class EmpDetails {
1565     //Properties
1566     private int no;
1567     private String fname, lname, mail;
1568     private java.sql.Timestamp ts;
1569
1570
1571     public java.sql.Timestamp getTs() {
1572         return ts;
1573     }
1574
1575     public void setTs(java.sql.Timestamp ts) {
1576         this.ts = ts;
1577     }
1578
1579     public EmpDetails(){
1580         System.out.println("EmpDetails:0-param constructor");
1581     }
1582
1583     //write setXxx(-) and getXxx() methods
1584     public int getNo() {
1585         return no;
1586     }
1587     public void setNo(int no) {
1588         this.no = no;
1589     }
1590     public String getFname() {
1591         return fname;
1592     }
1593     public void setFname(String fname) {
1594         this.fname = fname;
1595     }
1596     public String getLname() {
1597         return lname;
1598     }
1599     public void setLname(String lname) {
1600         this.lname = lname;
1601     }
1602     public String getMail() {
1603         return mail;
1604     }
1605     public void setMail(String mail) {
1606         this.mail = mail;
1607     }
1608
1609     @Override
1610     public String toString() {
1611         return "EmpDetails [no=" + no + ", fname=" + fname + ", lname=" + lname
1612             + ", mail=" + mail + "];"
1613     }
1614 }
1615
1616 -----TestClient.java-----
1617 package com.nt.test;
1618
1619 import org.hibernate.Session;
1620 import org.hibernate.SessionFactory;
1621 import org.hibernate.Transaction;
1622 import org.hibernate.cfg.Configuration;
1623
1624 import com.nt.domain.EmpDetails;
1625
```

```

1626 public class TestClient {
1627
1628     public static void main(String[] args) throws Exception{
1629
1630         //Get Session
1631         Session ses=HibernateUtil.openSession();
1632
1633         /*      //Save object
1634             EmpDetails eb=new EmpDetails();
1635             eb.setNo(1101); eb.setFname("Ravi"); eb.setLname("rao"); eb.setMail("rao@x.com");
1636
1637             Transaction tx=null;
1638             try{
1639                 tx=ses.beginTransaction();
1640                 ses.save(eb);
1641                 tx.commit();
1642             }//try
1643             catch(Exception e){
1644                 tx.rollback();
1645             }
1646             System.out.println("Obj is saved at"+eb.getTs()); */
1647         //Update the Object
1648         EmpDetails eb=(EmpDetails)s ses.load(EmpDetails.class,1101);
1649         Transaction tx=ses.beginTransaction();
1650         eb.setMail("raja@x.com");
1651         tx.commit();
1652         System.out.println("object is modified at"+eb.getTs());
1653         //note: tstamp col holds the record modification date,time.
1654
1655         //close objs
1656         ses.close();
1657         factory.close();
1658     }//main
1659 }//class
1660

```

App6) Example App Composite Primarykey

```

1664 -----hibernate.cfg.xml-----
1665 <!DOCTYPE hibernate-configuration
1666 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
1667 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
1668
1669 <hibernate-configuration>
1670     <session-factory>
1671         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
1672         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
1673         <property name="connection.username">scott</property>
1674         <property name="connection.password">tiger</property>
1675         <property name="show_sql">>true</property>
1676         <property name="hibernate.hbm2ddl.auto">update</property>
1677         <mapping resource="com/nt/cfgs/Employee.hbm.xml"/>
1678     </session-factory>
1679 </hibernate-configuration>
1680
1681 -----Employee.hbm.xml-----
1682 <!DOCTYPE hibernate-mapping SYSTEM
1683 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
1684
1685 <hibernate-mapping>
1686     <class name="com.nt.domain.EmpDetails" table="Employee">
1687         <composite-id name="id" class="com.nt.domain.EmployeeId">
1688             <key-property name="no" column="EID"/>
1689             <key-property name="fname" column="FIRSTNAME"/>
1690         </composite-id>

```

```
1691         <property name="lname" column="LASTNAME" />
1692         <property name="mail" column="EMAIL"/>
1693     </class>
1694 </hibernate-mapping>
```

```
1696 -----EmpDetails.java-----
```

```
1697 // EmpDetails.java (Domain class)
1698 package com.nt.domain;
1699 public class EmpDetails {
1700     //Properties
1701     private EmployeeId id;
1702     private String lname, mail;
1703
1704     //write setXxx(-) and getXxx() methods with toString()
1705     public EmployeeId getId() {
1706         return id;
1707     }
1708     public void setId(EmployeeId id) {
1709         this.id = id;
1710     }
1711     public String getLname() {
1712         return lname;
1713     }
1714     public void setLname(String lname) {
1715         this.lname = lname;
1716     }
1717     public String getMail() {
1718         return mail;
1719     }
1720     public void setMail(String mail) {
1721         this.mail = mail;
1722     }
1723     @Override
1724     public String toString() {
1725         return "EmpDetails [id=" + id + ", lname=" + lname + ", mail=" + mail + "];";
1726     }
1727
1728
1729 }
```

```
1730 -----EmployeeId.java-----
```

```
1731 package com.nt.domain;
1732
1733 import java.io.Serializable;
1734
1735 public class EmployeeId implements Serializable {
1736     private int no;
1737     private String fname;
1738     //write setXxx(-) , getXxx() methods and toString()
1739     public int getNo() {
1740         return no;
1741     }
1742     public void setNo(int no) {
1743         this.no = no;
1744     }
1745     public String getFname() {
1746         return fname;
1747     }
1748     public void setFname(String fname) {
1749         this.fname = fname;
1750     }
1751
1752     @Override
1753     public String toString() {
1754         return "EmployeeId [no=" + no + ", fname=" + fname + "];";
1755     }
1756 }
```

```

1756 }
1757 -----TestClient.java-----
1758 package com.nt.test;
1759
1760 import org.hibernate.Session;
1761 import org.hibernate.SessionFactory;
1762 import org.hibernate.Transaction;
1763 import org.hibernate.cfg.Configuration;
1764
1765 import com.nt.domain.EmpDetails;
1766 import com.nt.domain.EmployeeId;
1767
1768 public class TestClient {
1769
1770     public static void main(String[] args) throws Exception {
1771
1772         //Get Session
1773         Session ses=HibernateUtil.getSession();
1774
1775         /* Transaction tx=null;
1776            try{
1777                tx=ses.beginTransaction();
1778                // Id obj
1779                EmployeeId id=new EmployeeId();
1780                id.setNo(345); id.setFname("rajesh");
1781
1782                //Domain obj
1783                EmpDetails eb=new EmpDetails();
1784                eb.setId(id);
1785                eb.setLname("rao");
1786                eb.setMail("rao@x.com");
1787
1788                EmployeeId idval=(EmployeeId)ses.save(eb);
1789
1790                System.out.println("Generated Id value"+idval);
1791                tx.commit();
1792            }
1793            catch(Exception e){
1794                tx.rollback();
1795            }*/
1796
1797         //To Load obj using Composite Id value
1798         //prepare id value
1799         EmployeeId idval=new EmployeeId();
1800         idval.setNo(345); idval.setFname("rajesh");
1801
1802         EmpDetails eb=(EmpDetails)ses.get(EmpDetails.class, idval);
1803         System.out.println(eb);
1804
1805
1806         //close objs
1807         ses.close();
1808         factory.close();
1809     } //main
1810 } //class
1811 =====
1812 App8 ) Generators
1813 =====
1814 -----hibernate.cfg.xml-----
1815 <!DOCTYPE hibernate-configuration
1816 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
1817 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
1818
1819 <hibernate-configuration>
1820     <session-factory>

```



```

1821 <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
1822 <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
1823 <property name="connection.username">scott</property>
1824 <property name="connection.password">tiger</property>
1825 <property name="show_sql">true</property>
1826 <property name="dialect">org.hibernate.dialect.Oracle10gDialect</property>
1827
1828 <property name="connection.pool_size">15</property>
1829 <property name="hibernate.hbm2ddl.auto">update</property>
1830 <mapping resource="com/nt/cfgs/Employee.hbm.xml"/>
1831 </session-factory>
1832 </hibernate-configuration>
1833
1834 -----EmpDetails.java-----
1835 // EmpDetails.java (Domain class/model class/Persistence class)
1836 package com.nt.domain;
1837 public class EmpDetails {
1838     //Properties
1839     private String no;
1840     private String fname, lname, mail;
1841
1842     public EmpDetails(){
1843         System.out.println("EmpDetails:0-param constructor");
1844     }
1845
1846     //write setXxx(-) and getXxx() methods
1847     public String getNo() {
1848         return no;
1849     }
1850     public void setNo(String no) {
1851         this.no = no;
1852     }
1853     public String getFname() {
1854         return fname;
1855     }
1856     public void setFname(String fname) {
1857         this.fname = fname;
1858     }
1859     public String getLname() {
1860         return lname;
1861     }
1862     public void setLname(String lname) {
1863         this.lname = lname;
1864     }
1865     public String getMail() {
1866         return mail;
1867     }
1868     public void setMail(String mail) {
1869         this.mail = mail;
1870     }
1871
1872     @Override
1873     public String toString() {
1874         return "EmpDetails [no=" + no + ", fname=" + fname + ", lname=" + lname
1875             + ", mail=" + mail + "]";
1876     }
1877 }
1878 -----Employee.hbm.xml-----
1879 <!DOCTYPE hibernate-mapping SYSTEM
1880 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
1881
1882 <hibernate-mapping>
1883     <class name="com.nt.domain.EmpDetails" table="Employee">
1884         <id name="no" column="EID">
1885             <generator class="increment"/>

```

```

1886     </id>
1887     <property name="fname" column="FIRSTNAME" />
1888     <property name="lname" column="LASTNAME" />
1889     <property name="mail" column="EMAIL" />
1890
1891     </class>
1892 </hibernate-mapping>
1893 -----TestClient.java-----
1894 package com.nt.test;
1895
1896 import org.hibernate.Session;
1897 import org.hibernate.SessionFactory;
1898 import org.hibernate.Transaction;
1899 import org.hibernate.cfg.Configuration;
1900
1901 import com.nt.domain.EmpDetails;
1902
1903 public class TestClient {
1904     public static void main(String[] args) throws Exception{
1905         //Get HB Session
1906         Session ses=HibernateUtil.getSession();
1907         Transaction tx=null;
1908         try{
1909             tx=ses.beginTransaction();
1910             EmpDetails eb=new EmpDetails();
1911             eb.setFname("raja");
1912             eb.setLname("rao"); eb.setMail("rao@x.com");
1913             int idval=(String)ses.save(eb);
1914             System.out.println("The generated id value"+idval);
1915             tx.commit();
1916         }//try
1917         catch(Exception e){
1918             tx.rollback();
1919         }
1920
1921         //close objs
1922         ses.close();
1923         factory.close();
1924     }//main
1925 }//class
1926
1927 =====
1928 App9 (Custom Generator)
1929 =====
1930 -----hibernate.cfg.xml-----
1931 <!DOCTYPE hibernate-configuration
1932 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
1933 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
1934
1935 <hibernate-configuration>
1936     <session-factory>
1937         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
1938         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
1939         <property name="connection.username">scott</property>
1940         <property name="connection.password">tiger</property>
1941         <property name="show_sql">>true</property>
1942         <property name="hibernate.hbm2ddl.auto">update</property>
1943         <mapping resource="com/nt/cfgs/Employee.hbm.xml"/>
1944     </session-factory>
1945 </hibernate-configuration>
1946 -----Employee.hbm.xml-----
1947 <!DOCTYPE hibernate-mapping SYSTEM
1948 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
1949
1950 <hibernate-mapping>

```

```
1951     <class name="com.nt.domain.EmpDetails" table="Employee">
1952         <id name="no" column="EID">
1953             <generator class="com.nt.generator.CustRandomGenerator"/>
1954         </id>
1955         <property name="fname" column="FIRSTNAME" length="20"/>
1956         <property name="lname" column="LASTNAME" length="30"/>
1957         <property name="mail" column="EMAIL" length="20"/>
1958
1959     </class>
1960 </hibernate-mapping>
1961 -----CustRandomGenerator.java-----
1962 package com.nt.generator;
1963
1964 import java.io.Serializable;
1965 import java.util.Random;
1966 import org.hibernate.HibernateException;
1967 import org.hibernate.engine.spi.SessionImplementor;
1968 import org.hibernate.id.IdentifierGenerator;
1969
1970 public class CustRandomGenerator implements IdentifierGenerator {
1971
1972     @Override
1973     public Serializable generate(SessionImplementor session, Object object)
1974         throws HibernateException {
1975         Random rad=new Random();
1976         int val=rad.nextInt(10000);
1977         return val;
1978     }
1979 }
1980
1981 -----EmpDetails.java-----
1982 // EmpDetails.java (Domain class/model class/Persistence class)
1983 package com.nt.domain;
1984 public class EmpDetails {
1985     //Properties
1986     private int no;
1987     private String fname, lname, mail;
1988
1989     public EmpDetails(){
1990         System.out.println("EmpDetails:0-param constructor");
1991     }
1992
1993     //write setXxx(-) and getXxx() methods
1994     public int getNo() {
1995         return no;
1996     }
1997     public void setNo(int no) {
1998         this.no = no;
1999     }
2000     public String getFname() {
2001         return fname;
2002     }
2003     public void setFname(String fname) {
2004         this.fname = fname;
2005     }
2006     public String getLname() {
2007         return lname;
2008     }
2009     public void setLname(String lname) {
2010         this.lname = lname;
2011     }
2012     public String getMail() {
2013         return mail;
2014     }
2015     public void setMail(String mail) {
```

```

2016         this.mail = mail;
2017     }
2018
2019     @Override
2020     public String toString() {
2021         return "EmpDetails [no=" + no + ", fname=" + fname + ", lname=" + lname
2022             + ", mail=" + mail + "];";
2023     }
2024 }
2025 -----HibernateUtil.java-----
2026 --->same<----
2027 -----TestClient.java-----
2028 package com.nt.test;
2029
2030 import org.hibernate.Session;
2031 import org.hibernate.SessionFactory;
2032 import org.hibernate.Transaction;
2033 import org.hibernate.cfg.Configuration;
2034
2035 import com.nt.domain.EmpDetails;
2036
2037 public class TestClient {
2038
2039     public static void main(String[] args) throws Exception{
2040
2041         //Get HB Session
2042         Session ses=HibernateUtil.getSession();
2043         Transaction tx=null;
2044         try{
2045             tx=ses.beginTransaction();
2046             EmpDetails eb=new EmpDetails();
2047             eb.setFname("Raja"); eb.setLname("Rao");
2048             eb.setMail("rao@x.com");
2049             int no=(Integer)ses.save(eb);
2050             System.out.println("id value"+no);
2051             tx.commit();
2052         }
2053         catch(Exception e){
2054             tx.rollback();
2055         }
2056
2057         ses.close();
2058         factory.close();
2059     } //main
2060 } //class
2061
2062 =====
2063 App10) Custom Generator2
2064 =====
2065 <!DOCTYPE hibernate-configuration
2066 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
2067 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
2068
2069 <hibernate-configuration>
2070     <session-factory>
2071         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
2072         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
2073         <property name="connection.username">scott</property>
2074         <property name="connection.password">tiger</property>
2075         <property name="show_sql">>true</property>
2076         <property name="dialect">org.hibernate.dialect.Oracle10gDialect</property>
2077
2078         <property name="connection.pool_size">15</property>
2079         <property name="hibernate.hbm2ddl.auto">update</property>
2080         <mapping resource="com/nt/cfgs/Employee.hbm.xml"/>

```

```
2081     </session-factory>
2082 </hibernate-configuration>
2083
2084 -----Employee.hbm.xml-----
2085 <!DOCTYPE hibernate-mapping SYSTEM
2086 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
2087
2088 <hibernate-mapping>
2089     <class name="com.nt.domain.EmpDetails" table="Employee">
2090         <id name="no" column="EID">
2091             <generator class="com.nt.generator.CustIdGenerator"/>
2092         </id>
2093         <property name="fname" column="FIRSTNAME" />
2094         <property name="lname" column="LASTNAME" />
2095         <property name="mail" column="EMAIL" />
2096
2097     </class>
2098 </hibernate-mapping>
2099 -----EmpDetails.java-----
2100 // EmpDetails.java (Domain class/model class/Persistence class)
2101 package com.nt.domain;
2102 public class EmpDetails {
2103     //Properties
2104     private String no;
2105     private String fname, lname, mail;
2106
2107     public EmpDetails(String no, String fname, String lname, String mail) {
2108         this.no = no;
2109         this.fname = fname;
2110         this.lname = lname;
2111         this.mail = mail;
2112     }
2113
2114
2115
2116     public EmpDetails(){
2117         System.out.println("EmpDetails:0-param constructor");
2118     }
2119
2120
2121
2122     //write setXxx(-) and getXxx() methods
2123     public String getNo() {
2124         return no;
2125     }
2126     public void setNo(String no) {
2127         this.no = no;
2128     }
2129     public String getFname() {
2130         return fname;
2131     }
2132     public void setFname(String fname) {
2133         this.fname = fname;
2134     }
2135     public String getLname() {
2136         return lname;
2137     }
2138     public void setLname(String lname) {
2139         this.lname = lname;
2140     }
2141     public String getMail() {
2142         return mail;
2143     }
2144     public void setMail(String mail) {
2145         this.mail = mail;
```

```
2146     }
2147
2148     @Override
2149     public String toString() {
2150         return "EmpDetails [no=" + no + ", fname=" + fname + ", lname=" + lname
2151             + ", mail=" + mail + "]\n";
2152     }
2153 }
2154
2155 -----CustomIdGenerator.java-----
2156 package com.nt.generator;
2157
2158 import java.io.Serializable;
2159 import java.sql.Connection;
2160 import java.sql.PreparedStatement;
2161 import java.sql.ResultSet;
2162
2163 import org.hibernate.HibernateException;
2164 import org.hibernate.engine.SessionImplementor;
2165 import org.hibernate.id.IdentifierGenerator;
2166
2167 public class CustIdGenerator implements IdentifierGenerator {
2168
2169     @Override
2170     public Serializable generate(SessionImplementor session, Object object)
2171         throws HibernateException {
2172         try{
2173             Connection con=session.connection();
2174             PreparedStatement ps=con.prepareStatement("select seq1.nextVal from dual");
2175             ResultSet rs=ps.executeQuery();
2176             int val=0;
2177             if(rs.next())
2178             {
2179                 val=rs.getInt(1);
2180             }
2181
2182             if(val<=9)
2183                 return "E00"+val;
2184             else if(val<=99)
2185                 return "E0"+val;
2186             else
2187                 return "E"+val;
2188         }//try
2189         catch(Exception e){
2190             throw new IllegalStateException("Can not generate id value");
2191         }
2192     }//generate(-,-)
2193 }//class
2194
2195 -----EmpDAO.java-----
2196 package com.nt.dao;
2197
2198 import com.nt.domain.EmpDetails;
2199
2200 public interface EmpDAO {
2201     public String registerEmp(EmpDetails obj);
2202 }
2203
2204 -----EmpDAOImpl.java-----
2205 package com.nt.dao;
2206
2207 import org.hibernate.Session;
2208 import org.hibernate.Transaction;
2209
2210 import com.nt.domain.EmpDetails;
```

```

2211 import com.nt.utility.HibernateUtil;
2212
2213 public class EmpDAOImpl implements EmpDAO{
2214
2215     @Override
2216     public String registerEmp(EmpDetails eb) {
2217         Transaction tx=null;
2218         try{
2219             Session ses=HibernateUtil.getSession();
2220             tx=ses.beginTransaction();
2221             String id=(String)ses.save(eb);
2222             tx.commit();
2223             HibernateUtil.closeSession();
2224             return "Emp is registered with"+id;
2225         }//try
2226         catch(Exception e){
2227             tx.rollback();
2228             HibernateUtil.closeSession();
2229             return "Emp registration failed";
2230         }//catch
2231     }//method
2232 }//class

```

-----EmpDAOFactory.java-----

```

2235 package com.nt.dao;
2236
2237 public class EmpDAOFactory {
2238     public static EmpDAO getInstance(){
2239         return new EmpDAOImpl();
2240     }
2241 }

```

-----HibernateUtil.java-----

same previous Apps

-----ClientApp.java-----

```

2246 package com.nt.test;
2247
2248 import com.nt.dao.EmpDAO;
2249 import com.nt.dao.EmpDAOFactory;
2250 import com.nt.domain.EmpDetails;
2251
2252 public class ClientApp {
2253
2254     public static void main(String[] args) {
2255         // Get DAO obj
2256         EmpDAO dao=EmpDAOFactory.getInstance();
2257         //create EmpDetails obj
2258         EmpDetails eb=new EmpDetails("", "raja", "chari", "x@y.com");
2259         String result=dao.registerEmp(eb);
2260         System.out.println(result);
2261     }//method
2262 }//class

```

App 10) Using Hibernate in web application

-----hibernate.cfg.xml-----

```

2267 <!DOCTYPE hibernate-configuration
2268 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
2269 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
2270
2271 <hibernate-configuration>
2272     <session-factory>
2273         <property name="connection.provider_class">
2274             org.hibernate.service.jdbc.connections.internal.DatasourceConnectionProviderImpl
2275         </property>

```

```
2276     <property name="connection.datasource">DsJndi</property>
2277     <property name="show_sql">true</property>
2278     <mapping resource="com/nt/cfgs/Employee.hbm.xml"/>
2279 </session-factory>
2280 </hibernate-configuration>
2281
2282 -----web.xml-----
2283
2284 <web-app >
2285
2286     <servlet>
2287         <servlet-name>abc</servlet-name>
2288         <servlet-class>com.nt.web.DBServlet</servlet-class>
2289     </servlet>
2290     <servlet-mapping>
2291         <servlet-name>abc</servlet-name>
2292         <url-pattern>/dburl</url-pattern>
2293     </servlet-mapping>
2294
2295     <welcome-file-list>
2296         <welcome-file>Form.html</welcome-file>
2297     </welcome-file-list>
2298 </web-app>
2299 -----Employee.hbm.xml-----
2300 <!DOCTYPE hibernate-mapping SYSTEM
2301     "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
2302
2303 <hibernate-mapping>
2304     <class name="com.nt.model.EmpDetails" table="Employee">
2305         <id name="no" column="EID"/> <!--Singular Id field cfg -->
2306         <property name="fname" column="FIRSTNAME" length="20"/>
2307         <property name="lname" column="LASTNAME" length="30"/>
2308         <property name="mail" column="EMAIL" length="20" />
2309     </class>
2310 </hibernate-mapping>
2311
2312 -----EmpDao.java-----
2313 package com.nt.dao;
2314 import com.nt.model.EmpDetails;
2315 public interface EmpDao {
2316     public EmpDetails search(int no);
2317 }
2318 -----EmpDaoImpl.java-----
2319 package com.nt.dao;
2320
2321 import org.hibernate.Session;
2322
2323 import com.nt.model.EmpDetails;
2324 import com.nt.utility.HibernateUtil;
2325
2326 public class EmpDaoImpl implements EmpDao {
2327
2328     @Override
2329     public EmpDetails search(int no) {
2330         //get Session
2331         Session ses=HibernateUtil.getSession();
2332         //load object
2333         EmpDetails eb=(EmpDetails)ses.get(EmpDetails.class,no);
2334
2335         if(eb==null)
2336             return null;
2337         else
2338             return eb;
2339     } //method
2340 } //class
```



```

2341
2342 -----EmpDaoFactory.java-----
2343 package com.nt.dao;
2344
2345 public class EmpDAOFactory {
2346
2347     public static EmpDao getDAOInstance()
2348     {
2349         return new EmpDaoImpl();
2350     }
2351 }
2352
2353 -----DBServlet.java-----
2354 package com.nt.web;
2355
2356 import java.io.IOException;
2357 import java.io.PrintWriter;
2358
2359 import javax.servlet.ServletException;
2360 import javax.servlet.http.HttpServlet;
2361 import javax.servlet.http.HttpServletRequest;
2362 import javax.servlet.http.HttpServletResponse;
2363
2364 import com.nt.dao.EmpDAOFactory;
2365 import com.nt.dao.EmpDao;
2366 import com.nt.model.EmpDetails;
2367 import com.nt.utility.HibernateUtil;
2368
2369 public class DBServlet extends HttpServlet {
2370
2371     public void doGet(HttpServletRequest req,HttpServletResponse res)throws ServletException,IOException
2372     { //get PrintWriter
2373         PrintWriter pw=res.getWriter();
2374         res.setContentType("text/html");
2375
2376         //read form data
2377         int no=Integer.parseInt(req.getParameter("teno"));
2378         // get DAO obj
2379         EmpDao dao=EmpDAOFactory.getDAOInstance();
2380         EmpDetails eb=dao.search(no);
2381         if(eb!=null) {
2382             pw.println("Emp Details:"+eb);
2383         }
2384         else
2385         {
2386             pw.println("Emp not found");
2387         }
2388         //close stream
2389         pw.close();
2390     }//doGet(-,-)
2391
2392     public void doPost(HttpServletRequest req,HttpServletResponse res)throws ServletException,IOExcep
2393     {
2394         doGet(req,res);
2395     }
2396
2397 }//class
2398
2399 =====
2400 App11) HQL
2401 =====
2402 -----hibernate.cfg.xml-----
2403 <!DOCTYPE hibernate-configuration
2404     PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
2405     "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">

```

```

2406
2407 <hibernate-configuration>
2408   <session-factory>
2409     <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
2410     <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
2411     <property name="connection.username">scott</property>
2412     <property name="connection.password">tiger</property>
2413     <property name="show_sql">>true</property>
2414     <property name="hibernate.hbm2ddl.auto">update</property>
2415     <mapping resource="com/nt/cfgs/Employee.hbm.xml"/>
2416   </session-factory>
2417 </hibernate-configuration>
2418 -----Employee.hbm.xml-----
2419 <!DOCTYPE hibernate-mapping SYSTEM
2420   "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
2421
2422 <hibernate-mapping>
2423   <class name="com.nt.domain.EmpDetails" table="Employee">
2424     <id name="no" column="EID"/> <!--Singular Id field cfg -->
2425     <property name="fname" column="FIRSTNAME" />
2426     <property name="lname" column="LASTNAME" />
2427     <property name="mail" column="EMAIL" />
2428   </class>
2429   <query name="ntest1" >
2430     from EmpDetails where mail like :domain
2431   </query>
2432
2433   <query name="ntest2" >
2434     <![CDATA[ from EmpDetails where no>=:min and no <= :max ]]>
2435   </query>
2436
2437 </hibernate-mapping>
2438 -----EmpDetails.java-----
2439 // EmpDetails.java (Domain class/model class/Persistence class)
2440 package com.nt.domain;
2441 public class EmpDetails {
2442   //Properties
2443   private int no;
2444   private String fname, lname, mail;
2445
2446   public EmpDetails(){
2447     System.out.println("EmpDetails:0-param constructor");
2448   }
2449
2450   //write setXxx(-) and getXxx() methods
2451   public int getNo() {
2452     return no;
2453   }
2454   public void setNo(int no) {
2455     this.no = no;
2456   }
2457   public String getFname() {
2458     return fname;
2459   }
2460   public void setFname(String fname) {
2461     this.fname = fname;
2462   }
2463   public String getLname() {
2464     return lname;
2465   }
2466   public void setLname(String lname) {
2467     this.lname = lname;
2468   }
2469   public String getMail() {
2470     return mail;

```

```

2471     }
2472     public void setMail(String mail) {
2473         this.mail = mail;
2474     }
2475
2476     @Override
2477     public String toString() {
2478         return "EmpDetails [no=" + no + ", fname=" + fname + ", lname=" + lname
2479             + ", mail=" + mail + "];"
2480     }
2481 }
2482 }
2483 -----HibernateUtil.java-----
2484 --->same<-----
2485 -----HQLTest.java-----
2486 package com.nt.test;
2487
2488 import java.util.List;
2489
2490 import org.hibernate.Query;
2491 import org.hibernate.Session;
2492 import org.hibernate.cfg.Configuration;
2493
2494 import com.nt.domain.EmpDetails;
2495
2496 public class HQLTest {
2497     public static void main(String[] args) {
2498         //Get Session obj
2499         Session ses=HibernateUtil.getSession();
2500
2501         /* //prepare Query obj
2502            Query q1=ses.createQuery("from EmpDetails ");
2503            //execute HQL Query
2504            List <EmpDetails>l=q1.list();
2505            //process results
2506            for(EmpDetails eb:l){
2507                System.out.println(eb);
2508            }//for */
2509
2510            /*Query q1=ses.createQuery("from EmpDetails");
2511            //execute HQL Query
2512            Iterator<EmpDetails> it=q1.iterate();
2513            //process the results
2514            while(it.hasNext()){
2515                EmpDetails eb=it.next();
2516                System.out.println(eb);
2517            }*/
2518
2519
2520            /* Query q2=ses.createQuery("from EmpDetails where no>=? and no<=?");
2521            //set param value
2522            q2.setInteger(0,100);
2523            q2.setInteger(1,500);
2524            //execute HQL Query
2525            List <EmpDetails>l=q2.list();
2526            //process the Result
2527            for(EmpDetails eb:l){
2528                System.out.println(eb);
2529            }//for */
2530
2531
2532            /*Query q3=ses.createQuery("from EmpDetails where no>=? and no<=?");
2533
2534            //set param values
2535            q3.setInteger(0,100);

```

```

2536         q3.setInteger(1,500);
2537         //execute HQL Query
2538         List <EmpDetails> l=q3.list();
2539         //process the results
2540         for(EmpDetails eb:l){
2541             System.out.println(eb);
2542         } //for */
2543
2544     /* //retrieving partial data from objs (specific multiple col values from Db table)
2545        Query q4=ses.createQuery("select fname,mail from EmpDetails where no>=:min");
2546        //set-param values
2547        q4.setInteger("min",300);
2548        //execute HQL Query
2549        List <Object[]> l=q4.list();
2550        //process the result
2551        for(Object[] row:l){ // to process List
2552            for(Object p:row){ // to process row[]
2553                System.out.print(p+" ");
2554            }
2555            System.out.println();
2556        } */
2557
2558     /* //Using iterate
2559        Query q4=ses.createQuery("select fname,mail from EmpDetails where no>=:min");
2560        //set param values
2561        q4.setInteger("min",300);
2562        //execute HQL Query
2563        Iterator <Object[]> it=q4.iterate();
2564        //process the result
2565        while(it.hasNext()){
2566            Object row[]=it.next();
2567            for(Object p:row){
2568                System.out.print(p+" ");
2569            } //for
2570            System.out.println();
2571        } //while */
2572
2573     /* // getting single property/single col value
2574        Query q5=ses.createQuery("select no from EmpDetails ");
2575        // execute HQL Query
2576        List<Integer> l=q5.list();
2577        //process the results
2578        for(Integer n:l)
2579            System.out.println(n+" "); */
2580
2581     //working with count(*) aggregate Function
2582     /* Query q6=ses.createQuery("select count(*) from EmpDetails");
2583        List<Long> l=q6.list();
2584        long cnt=l.get(0);
2585        System.out.println("Records count:"+cnt); */
2586
2587     /* //working with multiple aggregate functions
2588        Query q7=ses.createQuery("select count(*),max(no),min(no),avg(no),sum(no) from EmpDetails");
2589        List <Object[]> l=q7.list();
2590        Object res[]=l.get(0);
2591        System.out.println("count :"+res[0]);
2592        System.out.println("max value of no :"+res[1]);
2593        System.out.println("min value of no :"+res[2]);
2594        System.out.println("avg of no :"+res[3]);
2595        System.out.println("sum of no col value :"+res[4]); */
2596
2597     /* // HQL Sub queries
2598        Query q8=ses.createQuery("from EmpDetails where no=(select max(no) from EmpDetails)");
2599        List<EmpDetails> l=q8.list();
2600        EmpDetails eb=l.get(0);

```

```

2601         System.out.println(eb); */
2602
2603     /* Transaction tx=null;
2604        try{
2605            tx=ses.beginTransaction();
2606            Query q9=ses.createQuery("delete from EmpDetails where mail like :domain");
2607            q9.setString("domain","%y.com");
2608            int res=q9.executeUpdate();
2609            System.out.println("no.of records that are effected"+res);
2610            tx.commit();
2611        }
2612        catch(Exception e){
2613            tx.rollback();
2614            e.printStackTrace();
2615        } */
2616
2617     /* //Working with Named Query
2618        Query q2=ses.getNamedQuery("ntest2"); //accessing the named query
2619        // set param values
2620        q2.setInteger("min",100);
2621        q2.setInteger("max",400);
2622        // execute the HQL Query
2623        List <EmpDetails> l=q2.list();
2624        //process the Result
2625        for(EmpDetails eb: l){
2626            System.out.println(eb);
2627        } */
2628
2629        //Pagination in Hibernate using HQL
2630        //prepare HQL Query
2631        Query q9=ses.createQuery("from EmpDetails ");
2632        // set pagination params
2633        q9.setFirstResult(1);
2634        q9.setMaxResults(2);
2635        //execute HQL Query
2636        List<EmpDetails> list=q9.list();
2637        //process the results
2638        for(EmpDetails eb:list)
2639        {
2640            System.out.println(eb);
2641        }
2642
2643        //close Session
2644        ses.close();
2645    } //main
2646 } //class

```

```

2648 =====
2649 App12 ) HQL Insert
2650 =====

```

```

2651 -----hibernate.cfg.xml-----
2652 <!DOCTYPE hibernate-configuration
2653 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
2654 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
2655
2656 <hibernate-configuration>
2657     <session-factory>
2658         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
2659         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
2660         <property name="connection.username">scott</property>
2661         <property name="connection.password">tiger</property>
2662         <property name="show_sql">true</property>
2663         <property name="hibernate.hbm2ddl.auto">update</property>
2664         <mapping resource="com/nt/cfgs/Member.hbm.xml"/>
2665     </session-factory>

```

```
2666 </hibernate-configuration>
2667
2668 -----Member.hbm.xml-----
2669 <!DOCTYPE hibernate-mapping SYSTEM
2670 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
2671
2672 <hibernate-mapping>
2673     <class name="com.nt.domain.EmpDetails" table="Employee">
2674         <id name="no" column="EID"/>
2675         <property name="fname" column="FIRSTNAME" />
2676         <property name="lname" column="LASTNAME" />
2677         <property name="mail" column="EMAIL" />
2678     </class>
2679
2680     <class name="com.nt.domain.NGOMember" table="NGOMember">
2681         <id name="no" column="MID"/>
2682         <property name="fname" column="FIRSTNAME" />
2683         <property name="lname" column="LASTNAME" />
2684         <property name="mail" column="EMAIL" />
2685     </class>
2686
2687 </hibernate-mapping>
2688
2689 -----EmpDetails.java-----
2690 // EmpDetails.java (Domain class/model class/Persistence class)
2691 package com.nt.domain;
2692 public class EmpDetails {
2693     //Properties
2694     private int no;
2695     private String fname, lname, mail;
2696
2697     public EmpDetails(){
2698         System.out.println("EmpDetails:0-param constructor");
2699     }
2700
2701     //write setXxx(-) and getXxx() methods
2702     public int getNo() {
2703         return no;
2704     }
2705     public void setNo(int no) {
2706         this.no = no;
2707     }
2708     public String getFname() {
2709         return fname;
2710     }
2711     public void setFname(String fname) {
2712         this.fname = fname;
2713     }
2714     public String getLname() {
2715         return lname;
2716     }
2717     public void setLname(String lname) {
2718         this.lname = lname;
2719     }
2720     public String getMail() {
2721         return mail;
2722     }
2723     public void setMail(String mail) {
2724         this.mail = mail;
2725     }
2726
2727     @Override
2728     public String toString() {
2729         return "EmpDetails [no=" + no + ", fname=" + fname + ", lname=" + lname
2730             + ", mail=" + mail + "];
```

```
2731     }
2732 }
2733 -----NGOMember.java-----
2734 package com.nt.domain;
2735
2736 public class NGOMember {
2737     private int no;
2738     private String fname,lname,mail;
2739     public int getNo() {
2740         return no;
2741     }
2742     public void setNo(int no) {
2743         this.no = no;
2744     }
2745     public String getFname() {
2746         return fname;
2747     }
2748     public void setFname(String fname) {
2749         this.fname = fname;
2750     }
2751     public String getLname() {
2752         return lname;
2753     }
2754     public void setLname(String lname) {
2755         this.lname = lname;
2756     }
2757     public String getMail() {
2758         return mail;
2759     }
2760     public void setMail(String mail) {
2761         this.mail = mail;
2762     }
2763     @Override
2764     public String toString() {
2765         return "NGOMember [no=" + no + ", fname=" + fname + ", lname=" + lname
2766             + ", mail=" + mail + "];"
2767     }
2768 }
2769 -----HibernateUtil.java-----
2770 ----->same<-----
2771 -----HQLInsertTest.java-----
2772 package com.nt.test;
2773
2774 import org.hibernate.*;
2775
2776 import org.hibernate.Query;
2777 import org.hibernate.Session;
2778
2779 import com.nt.utility.HibernateUtil;
2780
2781 public class HQLInsertTest {
2782
2783     public static void main(String[] args) {
2784         // get HB session
2785         Session ses=HibernateUtil.getSession();
2786
2787         /* //prepare Query
2788         Query q1=ses.createQuery("insert into NGOMember(no,fname,lname,mail)
2789             select no,fname,lname,mail from EmpDetails where mail like :domain");
2790         //set param value
2791         q1.setString("domain","%x.com");
2792         Transaction tx=null;
2793         try{
2794             tx=ses.beginTransaction();
2795             int result=q1.executeUpdate();
```

```

2796         tx.commit();
2797         System.out.println("no.of that are effected"+result);
2798     }
2799     catch(Exception e){
2800         tx.rollback();
2801     } */
2802
2803     Query q2=ses.createQuery("insert into EmpDetails values(:no,:fn,:ln,:mail)");
2804     q2.setInteger("no",1001);
2805     q2.setString("fn","raja");
2806     q2.setString("ln","rao");
2807     q2.setString("mail","x@z.com");
2808     int res=q2.executeUpdate();
2809 }
2810 }
2811 =====
2812 App13) HQL with DAO
2813 =====
2814 -----hibernate.cfg.xml-----
2815 <!DOCTYPE hibernate-configuration
2816 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
2817 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
2818
2819 <hibernate-configuration>
2820     <session-factory>
2821         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
2822         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
2823         <property name="connection.username">scott</property>
2824         <property name="connection.password">tiger</property>
2825         <property name="show_sql">>true</property>
2826         <property name="hibernate.hbm2ddl.auto">update</property>
2827         <mapping resource="com/nt/cfgs/Employee.hbm.xml"/>
2828     </session-factory>
2829 </hibernate-configuration>
2830 -----Employee.hbm.xml-----
2831 <!DOCTYPE hibernate-mapping SYSTEM
2832 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
2833
2834 <hibernate-mapping>
2835     <class name="com.nt.domain.EmpDetails" table="Employee">
2836         <id name="no" column="EID"/> <!--Singular Id field cfg -->
2837         <property name="fname" column="FIRSTNAME" />
2838         <property name="lname" column="LASTNAME" />
2839         <property name="mail" column="EMAIL" />
2840     </class>
2841 </hibernate-mapping>
2842 -----EmpDetails.java-----
2843 // EmpDetails.java (Domain class/model class/Persistence class)
2844 package com.nt.domain;
2845 public class EmpDetails {
2846     //Properties
2847     private int no;
2848     private String fname, lname, mail;
2849
2850     public EmpDetails(){
2851         System.out.println("EmpDetails:0-param constructor");
2852     }
2853
2854     //write setXxx(-) and getXxx() methods
2855     public int getNo() {
2856         return no;
2857     }
2858     public void setNo(int no) {
2859         this.no = no;
2860     }

```



```
2861     public String getFname() {
2862         return fname;
2863     }
2864     public void setFname(String fname) {
2865         this.fname = fname;
2866     }
2867     public String getLname() {
2868         return lname;
2869     }
2870     public void setLname(String lname) {
2871         this.lname = lname;
2872     }
2873     public String getMail() {
2874         return mail;
2875     }
2876     public void setMail(String mail) {
2877         this.mail = mail;
2878     }
2879
2880     @Override
2881     public String toString() {
2882         return "EmpDetails [no=" + no + ", fname=" + fname + ", lname=" + lname
2883             + ", mail=" + mail + "]\n";
2884     }
2885 }
2886 -----EmpDao.java-----
2887 package com.nt.dao;
2888
2889 import java.util.List;
2890
2891 import com.nt.domain.EmpDetails;
2892
2893 public interface EmpDao {
2894     public List<EmpDetails> findEmployees(String domain);
2895     public List<String> findNames(int start, int end);
2896     public boolean modifyEmail(int eno, String newmail);
2897     public boolean deleteEmployee(int eno);
2898 }
2899 -----EmpDaoImpl.java-----
2900 package com.nt.dao;
2901
2902 import java.util.List;
2903
2904 import org.hibernate.Query;
2905 import org.hibernate.Session;
2906 import org.hibernate.Transaction;
2907
2908 import com.nt.domain.EmpDetails;
2909 import com.nt.utility.HibernateUtil;
2910
2911 public class EmpDaoImpl implements EmpDao {
2912
2913     private static final String FIND_EMPLOYEES="from EmpDetails where mail like :domain";
2914     private static final String FIND_NAMES="select fname from EmpDetails where no>=:start and no<=:end";
2915     private static final String UPDATE_EMAIL="update EmpDetails set mail=:newmail where no=:eid";
2916     private static final String DELETE_EMP="delete from EmpDetails where no=:eid";
2917
2918
2919
2920     @Override
2921     public List<EmpDetails> findEmployees(String domain) {
2922         // get Session obj
2923         Session ses=HibernateUtil.getSession();
2924         //execute HQL
2925         Query query=ses.createQuery(FIND_EMPLOYEES);
```

```
2926         query.setString("domain","%" +domain);
2927
2928         List<EmpDetails> list=query.list();
2929         return list;
2930     }
2931
2932     @Override
2933     public List<String> findNames(int start, int end) {
2934         //get HB Session
2935         Session session=HibernateUtil.getSession();
2936         //execute HQL
2937         Query query=session.createQuery(FIND_NAMES);
2938         query.setInteger("start",start);
2939         query.setInteger("end",end);
2940         List<String> list=query.list();
2941         return list;
2942     }
2943
2944     @Override
2945     public boolean modifyEmail(int eno,String newmail) {
2946         // get HB Session
2947         Session session=HibernateUtil.getSession();
2948         //execute HQL
2949         Query query=session.createQuery(UPDATE_EMAIL);
2950         query.setString("newmail",newmail);
2951         query.setInteger("eid",eno);
2952         Transaction tx=null;
2953         int result=0;
2954         try{
2955             tx=session.beginTransaction();
2956             result=query.executeUpdate();
2957             tx.commit();
2958         }
2959         catch(Exception e){
2960             tx.rollback();
2961         }
2962
2963         if(result==0)
2964             return false;
2965         else
2966             return true;
2967     } //method
2968
2969     @Override
2970     public boolean deleteEmployee(int eno) {
2971         // get HB Session obj
2972         Session session=HibernateUtil.getSession();
2973         // execute HQL Query
2974         Query query=session.createQuery(DELETE_EMP);
2975         query.setInteger("eid",eno);
2976         Transaction tx=null;
2977         int result=0;
2978         try{
2979             tx=session.beginTransaction();
2980             result=query.executeUpdate();
2981             tx.commit();
2982         }
2983         catch(Exception e){
2984             tx.rollback();
2985         }
2986         if(result==0)
2987             return false;
2988         else
2989             return true;
2990     }
```

```

2991 }
2992 -----EmpDaoFactory.java-----
2993 package com.nt.dao;
2994
2995 public class EmpDaoFactory {
2996
2997     public static EmpDao getInstance()
2998     {
2999         return new EmpDaoImpl();
3000     }
3001 }
3002 }
3003 -----HQLTest.java-----
3004 package com.nt.test;
3005
3006 import java.util.List;
3007
3008 import com.nt.dao.EmpDao;
3009 import com.nt.dao.EmpDaoFactory;
3010 import com.nt.domain.EmpDetails;
3011
3012 public class HQLTest {
3013     public static void main(String[] args) {
3014         //get EmpDao
3015         EmpDao dao=EmpDaoFactory.getInstance();
3016         // find Emp Details
3017         List <EmpDetails> list=dao.findEmployees("x.com");
3018         System.out.println("Employee belogining x.com domain");
3019         for(EmpDetails eb: list){
3020             System.out.println(eb);
3021         }
3022
3023         //find names
3024         List <String> names=dao.findNames(200,500);
3025         System.out.println("Employees in the 200 through 500"+names);
3026
3027         System.out.println("Modifying Email ID.....");
3028         System.out.println("Email Id modified"+dao.modifyEmail(256,"raja@xyz.com"));
3029
3030         System.out.println("Deleting Record");
3031         System.out.println("Record Deleted?" +dao.deleteEmployee(3456));
3032
3033     } //main
3034 } //class
3035 =====
3036 App14) Basic App with Annotations
3037 =====
3038 -----hibernate.cfg.xml-----
3039 <!DOCTYPE hibernate-configuration
3040 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
3041 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
3042
3043 <hibernate-configuration>
3044     <session-factory>
3045         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
3046         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
3047         <property name="connection.username">scott</property>
3048         <property name="connection.password">tiger</property>
3049         <property name="show_sql">>true</property>
3050         <mapping class="com.nt.model.Student"/>
3051     </session-factory>
3052 </hibernate-configuration>
3053 -----Student.java-----
3054 package com.nt.model;
3055

```

```
3056 import javax.persistence.Column;
3057 import javax.persistence.Entity;
3058 import javax.persistence.Id;
3059 import javax.persistence.Table;
3060 import javax.persistence.Transient;
3061
3062 @Entity
3063 @Table(name="Student_Tab")
3064 public class Student {
3065     private int sno;
3066     private String sname;
3067     //@Transient
3068     private String address;
3069
3070     public Student(){
3071         System.out.println("Student:0-param constructor");
3072     }
3073
3074     public Student(int sno, String sname, String address) {
3075         this.sno = sno;
3076         this.sname = sname;
3077         this.address = address;
3078     }
3079     //write full constructor
3080
3081     //write setXxx(-) and getXxx() methods
3082     @Column(name="stno")
3083     @Id
3084     public int getSno() {
3085         return sno;
3086     }
3087     public void setSno(int sno) {
3088         this.sno = sno;
3089     }
3090     @Column(name="stname")
3091     public String getSname() {
3092         return sname;
3093     }
3094     public void setSname(String sname) {
3095         this.sname = sname;
3096     }
3097
3098     @Column(name="stadd")
3099     public String getAddress() {
3100         return address;
3101     }
3102
3103
3104     public void setAddress(String address) {
3105         this.address = address;
3106     }
3107     @Override
3108     public String toString() {
3109         return "Student [sno=" + sno + ", sname=" + sname + ", address="
3110             + address + "]\n";
3111     }
3112 }
3113 -----TestClient.java-----
3114 package com.nt.test;
3115
3116 import java.util.List;
3117
3118 import org.hibernate.Query;
3119 import org.hibernate.Session;
3120 import org.hibernate.SessionFactory;
```

```

3121 import org.hibernate.cfg.Configuration;
3122 import org.hibernate.service.ServiceRegistry;
3123 import org.hibernate.service.ServiceRegistryBuilder;
3124
3125 import com.nt.model.Student;
3126
3127 public class TestClient {
3128
3129     public static void main(String[] args) {
3130         //Activate HB f/w
3131         Configuration cfg=new Configuration();
3132         //Load hibernate cfg file
3133         cfg=cfg.configure("/com/nt/cfgs/hibernate.cfg.xml");
3134         // Create Service Registry
3135         ServiceRegistryBuilder builder=new ServiceRegistryBuilder();
3136         ServiceRegistry registry=builder.applySettings(cfg.getProperties()).buildServiceRegistry();
3137         //BuildSessionFactory
3138         SessionFactory factory=cfg.buildSessionFactory(registry);
3139         //Get HB Session
3140         Session ses=factory.openSession();
3141
3142         /* // save the obj
3143         Transaction tx=null;
3144         try{
3145             tx=ses.beginTransaction();
3146             Student st=new Student(102,"raja1","hyd1");
3147             int id=(Integer)ses.save(st);
3148             System.out.println("Generated Id value"+id);
3149             tx.commit();
3150         }//try
3151         catch(Exception e){
3152             tx.rollback();
3153         } */
3154
3155         /* //Perform select operation
3156         Student st=(Student)ses.get(Student.class,101);
3157         System.out.println(st); */
3158
3159         //HQL Query
3160         Query q=ses.createQuery("from Student where sno>=:min and sno<=:max");
3161         q.setInteger("min",100);
3162         q.setInteger("max",200);
3163         List<Student> l=q.list();
3164         for(Student st:l)
3165         {
3166             System.out.println(st);
3167         }
3168
3169         //close objs
3170         ses.close();
3171         factory.close();
3172
3173     }
3174 }

```

=====
 App15) Object Versioning using Annotations
 =====

```

3179 -----hibernate.cfg.xml-----
3180 <!DOCTYPE hibernate-configuration
3181 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
3182 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
3183
3184 <hibernate-configuration>
3185     <session-factory>

```

```
3186     <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
3187     <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
3188     <property name="connection.username">scott</property>
3189     <property name="connection.password">tiger</property>
3190     <property name="show_sql">true</property>
3191     <mapping class="com.nt.model.Student"/>
3192 </session-factory>
3193 </hibernate-configuration>
3194 -----Student.java-----
3195 package com.nt.model;
3196
3197 import javax.persistence.Column;
3198 import javax.persistence.Entity;
3199 import javax.persistence.Id;
3200 import javax.persistence.Table;
3201 import javax.persistence.Version;
3202
3203 @Entity
3204 @Table(name="Student_Tab")
3205 public class Student {
3206     private int sno;
3207     private String sname;
3208     // @Transient
3209     private String address;
3210     private int ver;
3211
3212     public Student(){
3213         System.out.println("Student:0-param constructor");
3214     }
3215
3216     public Student(int sno, String sname, String address) {
3217         this.sno = sno;
3218         this.sname = sname;
3219         this.address = address;
3220     }
3221     // write full constructor
3222
3223     // write setXxx(-) and getXxx() methods
3224     @Column(name="stno")
3225     @Id
3226     public int getSno() {
3227         return sno;
3228     }
3229     public void setSno(int sno) {
3230         this.sno = sno;
3231     }
3232     @Column(name="stname")
3233     public String getSname() {
3234         return sname;
3235     }
3236     public void setSname(String sname) {
3237         this.sname = sname;
3238     }
3239
3240     @Column(name="stadd")
3241     public String getAddress() {
3242         return address;
3243     }
3244
3245     public void setAddress(String address) {
3246         this.address = address;
3247     }
3248 }
3249
3250 @Version
```

```
3251     @Column(name="ver_col")
3252     public int getVer(){
3253         return ver;
3254     }
3255     public void setVer(int ver){
3256         this.ver=ver;
3257     }
3258     @Override
3259     public String toString() {
3260         return "Student [sno=" + sno + ", sname=" + sname + ", address="
3261             + address + "];"
3262     }
3263 }
3264 }
3265 -----TestClient.java-----
3266 package com.nt.test;
3267
3268 import org.hibernate.Session;
3269 import org.hibernate.SessionFactory;
3270 import org.hibernate.Transaction;
3271 import org.hibernate.cfg.Configuration;
3272 import org.hibernate.service.ServiceRegistry;
3273 import org.hibernate.service.ServiceRegistryBuilder;
3274
3275 import com.nt.model.Student;
3276
3277 public class TestClient {
3278
3279     public static void main(String[] args) {
3280         //Get Session obj
3281         Configuration cfg=new Configuration();
3282         cfg=cfg.configure("/com/nt/cfgs/hibernate.cfg.xml");
3283         ServiceRegistryBuilder builder=new ServiceRegistryBuilder();
3284         ServiceRegistry registry=builder.applySettings(cfg.getProperties()).buildServiceRegistry();
3285         SessionFactory factory=cfg.buildSessionFactory(registry);
3286         Session ses=factory.openSession();
3287
3288         /* // save the obj
3289         Transaction tx=null;
3290         try{
3291             tx=ses.beginTransaction();
3292             Student st=new Student(102,"raja1","hyd1");
3293             int id=(Integer)ses.save(st);
3294             System.out.println("Generated Id value"+id);
3295             tx.commit();
3296         }//try
3297         catch(Exception e){
3298             tx.rollback();
3299         } */
3300
3301         /* //Perform select operation
3302         Student st=(Student)ses.get(Student.class,101);
3303         System.out.println(st); */
3304
3305         /* //HQL Query
3306         Query q=ses.createQuery("from Student where sno>=:min and sno<=:max");
3307         q.setInteger("min",100);
3308         q.setInteger("max",200);
3309         List<Student> l=q.list();
3310         for(Student st:l)
3311         {
3312             System.out.println(st);
3313         } */
3314
3315         //Load obj
```

```

3316         Student st=(Student)ses.get(Student.class,101);
3317         //modify the obj
3318         Transaction tx=ses.beginTransaction();
3319         st.setAddress("delhi2");
3320         tx.commit();
3321         System.out.println("Record is modified for"+st.getVer()+" times");
3322
3323         //close objs
3324         ses.close();
3325         factory.close();
3326
3327     }
3328 }
3329 =====
3330 App16 >>>>>>>>TimeStamp operation using Annotations
3331 =====
3332 -----Student.java-----
3333 <!DOCTYPE hibernate-configuration
3334 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
3335 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
3336
3337 <hibernate-configuration>
3338     <session-factory>
3339         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
3340         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
3341         <property name="connection.username">scott</property>
3342         <property name="connection.password">tiger</property>
3343         <property name="show_sql">>true</property>
3344         <mapping class="com.nt.model.Student"/>
3345     </session-factory>
3346 </hibernate-configuration>
3347 -----Student.java-----
3348 package com.nt.model;
3349
3350 import java.util.Date;
3351
3352 import javax.persistence.Column;
3353 import javax.persistence.Entity;
3354 import javax.persistence.Id;
3355 import javax.persistence.Table;
3356 import javax.persistence.Temporal;
3357 import javax.persistence.TemporalType;
3358
3359 @Entity
3360 @Table(name="Student_Tab")
3361 public class Student {
3362     private int sno;
3363     private String sname;
3364     // @Transient
3365     private String address;
3366     private int ver;
3367
3368     public Student(){
3369         System.out.println("Student:0-param constructor");
3370     }
3371
3372     public Student(int sno, String sname, String address) {
3373         this.sno = sno;
3374         this.sname = sname;
3375         this.address = address;
3376     }
3377     //write full constructor
3378
3379     //write setXxx(-) and getXxx() methods
3380     @Column(name="stno")

```



```
3381 @Id
3382 public int getSno() {
3383     return sno;
3384 }
3385 public void setSno(int sno) {
3386     this.sno = sno;
3387 }
3388 @Column(name="sname")
3389 public String getSname() {
3390     return sname;
3391 }
3392 public void setSname(String sname) {
3393     this.sname = sname;
3394 }
3395
3396 @Column(name="stadd")
3397 public String getAddress() {
3398     return address;
3399 }
3400
3401
3402 public void setAddress(String address) {
3403     this.address = address;
3404 }
3405
3406 Date txtime=new Date();
3407
3408 @Column(name="ts")
3409 @Temporal(TemporalType.DATE)
3410 public Date getTxtime() {
3411     return txtime;
3412 }
3413
3414 public void setTxtime(Date txtime) {
3415     this.txtime = txtime;
3416 }
3417
3418
3419 @Override
3420 public String toString() {
3421     return "Student [sno=" + sno + ", sname=" + sname + ", address="
3422         + address + "]";
3423 }
3424
3425 }
3426
3427 -----TestClient.java-----
3428 package com.nt.test;
3429
3430 import org.hibernate.Session;
3431 import org.hibernate.SessionFactory;
3432 import org.hibernate.Transaction;
3433 import org.hibernate.cfg.Configuration;
3434 import org.hibernate.service.ServiceRegistry;
3435 import org.hibernate.service.ServiceRegistryBuilder;
3436
3437 import com.nt.model.Student;
3438
3439 public class TestClient {
3440
3441     public static void main(String[] args) {
3442         //Get Session obj
3443         Configuration cfg=new Configuration();
3444         cfg=cfg.configure("/com/nt/cfgs/hibernate.cfg.xml");
3445         ServiceRegistryBuilder builder=new ServiceRegistryBuilder();
```

```

3446 ServiceRegistry registry=builder.applySettings(cfg.getProperties()).buildServiceRegistry();
3447 SessionFactory factory=cfg.buildSessionFactory(registry);
3448 Session ses=factory.openSession();
3449
3450 // save the obj
3451 Transaction tx=null;
3452 try{
3453     tx=ses.beginTransaction();
3454     Student st=new Student(101,"raja","hyd");
3455     int id=(Integer)ses.save(st);
3456     System.out.println("Generated Id value"+id);
3457     tx.commit();
3458 }//try
3459 catch(Exception e){
3460     tx.rollback();
3461 }
3462
3463 /* //Perform select operation
3464 Student st=(Student)ses.get(Student.class,101);
3465 System.out.println(st); */
3466
3467 /* //HQL Query
3468 Query q=ses.createQuery("from Student where sno>=:min and sno<=:max");
3469 q.setInteger("min",100);
3470 q.setInteger("max",200);
3471 List<Student> l=q.list();
3472 for(Student st:l)
3473 {
3474     System.out.println(st);
3475 } */
3476
3477 /* //Load obj
3478 Student st=(Student)ses.get(Student.class,101);
3479 //modify the obj
3480 Transaction tx=ses.beginTransaction();
3481 st.setAddress("delhi2");
3482 tx.commit();
3483 System.out.println("Record is modified for"+st.getVer()+" times"); */
3484
3485 //close objs
3486 ses.close();
3487 factory.close();
3488
3489 }
3490 }

```

App17 (Annotations Based Named HQL Queries & Generators)

```

3494 <!DOCTYPE hibernate-configuration
3495 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
3496 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
3497
3498 <hibernate-configuration>
3499     <session-factory>
3500         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
3501         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
3502         <property name="connection.username">scott</property>
3503         <property name="connection.password">tiger</property>
3504         <property name="show_sql">>true</property>
3505         <mapping class="com.nt.model.Student"/>
3506     </session-factory>
3507 </hibernate-configuration>
3508 -----Student.java-----
3509 package com.nt.model;
3510

```

```
3511 import javax.persistence.Column;
3512 import javax.persistence.Entity;
3513 import javax.persistence.GeneratedValue;
3514 import javax.persistence.Id;
3515 import javax.persistence.NamedNativeQueries;
3516 import javax.persistence.NamedNativeQuery;
3517 import javax.persistence.Table;
3518
3519 import org.hibernate.annotations.GenericGenerator;
3520 import org.hibernate.annotations.Parameter;
3521
3522 /*@Entity
3523 @Table(name="Student_Tab")
3524 @NamedQueries({@NamedQuery(name="ntest1",query="from Student where sno<=:min"),
3525                 @NamedQuery(name="ntest2",query="delete from Student where address=:city")
3526                })*/
3527 @Entity
3528 @Table(name="Student_Tab")
3529 @NamedNativeQueries({@NamedNativeQuery(name="ntest1",query="select * from student_Tab where sta
3530                                resultClass=Student.class),
3531                      @NamedNativeQuery(name="ntest2",query="select stno,stadd from Student_Tab")
3532                     })
3533 public class Student {
3534     private int sno;
3535     private String sname;
3536     //@Transient
3537     private String address;
3538
3539     public Student(){
3540         System.out.println("Student:0-param constructor");
3541     }
3542
3543     public Student(int sno, String sname, String address) {
3544         this.sno = sno;
3545         this.sname = sname;
3546         this.address = address;
3547     }
3548     //write full constructor
3549
3550     //write setXxx(-) and getXxx() methods
3551     /* @Column(name="stno")
3552        @Id
3553        @GenericGenerator(name="gen1",strategy="increment")
3554        @GeneratedValue(generator="gen1") */
3555
3556     @Column(name="stno")
3557     @Id
3558     @GenericGenerator(name="gen2",strategy="sequence",
3559                       parameters={@Parameter(name="sequence",value="myseq1")})
3560     )
3561     @GeneratedValue(generator="gen2")
3562     public int getSno() {
3563         return sno;
3564     }
3565
3566     public void setSno(int sno) {
3567         this.sno = sno;
3568     }
3569     @Column(name="stname")
3570     public String getSname() {
3571         return sname;
3572     }
3573     public void setSname(String sname) {
3574         this.sname = sname;
3575     }
```

```
3576
3577     @Column(name="stadd")
3578     public String getAddress() {
3579         return address;
3580     }
3581
3582
3583     public void setAddress(String address) {
3584         this.address = address;
3585     }
3586     @Override
3587     public String toString() {
3588         return "Student {sno="+ sno + ", sname="+ sname + ", address="
3589             + address + "}";
3590     }
3591
3592
3593 }
3594 -----TestClient.java-----
3595 package com.nt.test;
3596
3597 import java.util.List;
3598
3599 import org.hibernate.Query;
3600 import org.hibernate.Session;
3601 import org.hibernate.SessionFactory;
3602 import org.hibernate.Transaction;
3603 import org.hibernate.cfg.Configuration;
3604 import org.hibernate.service.ServiceRegistry;
3605 import org.hibernate.service.ServiceRegistryBuilder;
3606
3607 import com.nt.model.Student;
3608
3609 public class TestClient {
3610
3611     public static void main(String[] args) {
3612         //Get Session obj
3613         Configuration cfg=new Configuration();
3614         cfg=cfg.configure("/com/nt/cfgs/hibernate.cfg.xml");
3615         ServiceRegistryBuilder builder=new ServiceRegistryBuilder();
3616         ServiceRegistry registry=builder.applySettings(cfg.getProperties()).buildServiceRegistry();
3617         SessionFactory factory=cfg.buildSessionFactory(registry);
3618         Session ses=factory.openSession();
3619
3620         /*// save the obj
3621         Transaction tx=null;
3622         try{
3623             tx=ses.beginTransaction();
3624             Student st=new Student(102,"raja1","hyd1");
3625             int id=(Integer)ses.save(st);
3626             System.out.println("Generated Id value"+id);
3627             tx.commit();
3628         }//try
3629         catch(Exception e){
3630             tx.rollback();
3631         } */
3632
3633         /* //Perform select operation
3634         Student st=(Student)ses.get(Student.class,101);
3635         System.out.println(st); */
3636
3637         /* //HQL Query
3638         Query q=ses.createQuery("from Student where sno>=:min and sno<=:max");
3639         q.setInteger("min",100);
3640         q.setInteger("max",200);
```

```

3641     List<Student> l=q.list();
3642     for(Student st:l)
3643     {
3644         System.out.println(st);
3645     } */
3646
3647     /* // Access Named Query1 (ntest1)
3648     Query q1=ses.getNamedQuery("ntest1");
3649     q1.setInteger("min",200);
3650     List<Student> list=q1.list();
3651     for(Student st:list){
3652         System.out.println(st);
3653     }
3654
3655     //Access Named Query2(ntest2)
3656     Transaction tx=null;
3657     try{
3658         tx=ses.beginTransaction();
3659         Query q2=ses.getNamedQuery("ntest2");
3660         q2.setString("city","hyd1");
3661         int res=q2.executeUpdate();
3662         tx.commit();
3663         System.out.println(res+" no.of records are deleted");
3664     }//try
3665     catch(Exception e){
3666         tx.rollback();
3667     } */
3668
3669     /* // Access NamedNativeSQL Query (ntest2)
3670     Query q1=ses.getNamedQuery("ntest1");
3671     q1.setString("city","hyd");
3672     List <Student>list=q1.list();
3673     for(Student st:list){
3674         System.out.println(st);
3675     }
3676
3677     //Accesss NamedNativeSQL Quey (ntest2)
3678     Query q2=ses.getNamedQuery("ntest2");
3679     List <Object[]>list1=q2.list();
3680     for(Object[] row:list1){
3681         System.out.println(row[0]+" "+row[1]);
3682     } */
3683
3684     //close objs
3685     ses.close();
3686     factory.close();
3687
3688 }
3689 }
3690
3691 =====
3692 App18 (Annotations based Composite ID)
3693 =====
3694 -----hibernate.cfg.xml-----
3695
3696 <!DOCTYPE hibernate-configuration
3697 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
3698 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
3699
3700 <hibernate-configuration>
3701     <session-factory>
3702         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
3703         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
3704         <property name="connection.username">scott</property>
3705         <property name="connection.password">tiger</property>

```

```
3706     <property name="show_sql">true</property>
3707     <mapping class="com.nt.model.Student"/>
3708     </session-factory>
3709 </hibernate-configuration>
3710 -----Student.java-----
3711 package com.nt.model;
3712
3713 import javax.persistence.Column;
3714 import javax.persistence.EmbeddedId;
3715 import javax.persistence.Entity;
3716 import javax.persistence.Id;
3717 import javax.persistence.Table;
3718
3719 @Entity
3720 @Table(name="Student_Tab")
3721 public class Student {
3722
3723     @EmbeddedId
3724     private StudentId stid;
3725     // @Transient
3726     @Column(name="stadd")
3727     private String address;
3728
3729     public Student(){
3730         System.out.println("Student:0-param constructor");
3731     }
3732
3733     public Student(StudentId stid, String address) {
3734         this.stid = stid;
3735         this.address = address;
3736     }
3737     // write setXxx(-) and getXxx() methods
3738
3739     public StudentId getStid() {
3740         return stid;
3741     }
3742
3743     public void setStid(StudentId stid) {
3744         this.stid = stid;
3745     }
3746
3747     public String getAddress() {
3748         return address;
3749     }
3750
3751     public void setAddress(String address) {
3752         this.address = address;
3753     }
3754
3755     @Override
3756     public String toString() {
3757         return "Student [stid=" + stid + ", address=" + address + "];";
3758     }
3759 }
3760 -----StudentId.java-----
3761 package com.nt.model;
3762
3763 import java.io.Serializable;
3764
3765 import javax.persistence.Column;
3766 import javax.persistence.Embeddable;
3767
3768 @Embeddable
3769 public class StudentId implements Serializable{
3770     @Column(name="stno")
```

```
3771 private int sno;
3772 @Column(name="sname")
3773 private String sname;
3774
3775 public StudentId(int sno, String sname) {
3776     this.sno = sno;
3777     this.sname = sname;
3778 }
3779 public StudentId(){
3780
3781 }
3782 public int getSno() {
3783     return sno;
3784 }
3785 public void setSno(int sno) {
3786     this.sno = sno;
3787 }
3788 public String getSname() {
3789     return sname;
3790 }
3791 public void setSname(String sname) {
3792     this.sname = sname;
3793 }
3794 @Override
3795 public String toString() {
3796     return "StudentId [sno=" + sno + ", sname=" + sname + "];"
3797 }
3798 }
```

3799 -----TestClient.java-----

```
3800 package com.nt.test;
3801
3802 import org.hibernate.Session;
3803 import org.hibernate.SessionFactory;
3804 import org.hibernate.Transaction;
3805 import org.hibernate.cfg.Configuration;
3806 import org.hibernate.service.ServiceRegistry;
3807 import org.hibernate.service.ServiceRegistryBuilder;
3808
3809 import com.nt.model.Student;
3810 import com.nt.model.StudentId;
3811
3812 public class TestClient {
3813
3814     public static void main(String[] args) {
3815         //Get Session obj
3816         Configuration cfg=new Configuration();
3817         cfg=cfg.configure("/com/nt/cfgs/hibernate.cfg.xml");
3818         ServiceRegistryBuilder builder=new ServiceRegistryBuilder();
3819         ServiceRegistry registry=builder.applySettings(cfg.getProperties()).buildServiceRegistry();
3820         SessionFactory factory=cfg.buildSessionFactory(registry);
3821         Session ses=factory.openSession();
3822
3823         /* // save the obj
3824         Transaction tx=null;
3825         try{
3826             tx=ses.beginTransaction();
3827             StudentId id=new StudentId(103,"Raja2");
3828             Student st=new Student(id,"hyd2");
3829             StudentId id1=(StudentId)ses.save(st);
3830             System.out.println("Generated Id value"+id1);
3831             tx.commit();
3832         }//try
3833         catch(Exception e){
3834             tx.rollback();
3835         }*/
3836     }
```

```

3836
3837 //Load obj
3838 StudentId sid=new StudentId(101,"Raja");
3839 Student st=(Student)ses.get(Student.class, sid);
3840 System.out.println(st);
3841
3842
3843 //close objs
3844 ses.close();
3845 factory.close();
3846
3847 }
3848 }
3849 =====
3850 App19 (Native SQL)
3851 =====
3852 -----hibernate.cfg.xml-----
3853 <!DOCTYPE hibernate-configuration
3854 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
3855 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
3856
3857 <hibernate-configuration>
3858     <session-factory>
3859         <!-- connection properties -->
3860         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
3861         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
3862         <property name="connection.username">scott</property>
3863         <property name="connection.password">tiger</property>
3864         <!-- hibernate properties -->
3865         <property name="show_sql">true</property>
3866         <!-- mapping file names -->
3867         <mapping resource="com/nt/cfgs/Employee.hbm.xml"/>
3868     </session-factory>
3869 </hibernate-configuration>
3870 -----Employee.hbm.xml-----
3871 <!DOCTYPE hibernate-mapping SYSTEM
3872 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
3873
3874 <hibernate-mapping>
3875     <class name="com.nt.domain.EmpDetails" table="Employee">
3876         <id name="no" column="EID"/> <!-- Singular Id field cfg -->
3877         <property name="fname" column="FIRSTNAME"/>
3878         <property name="lname" column="LASTNAME"/>
3879         <property name="mail" column="EMAIL"/>
3880     </class>
3881     <sql-query name="ntest1">
3882         <return class="com.nt.domain.EmpDetails"/>
3883         select * from Employee where email like :domain
3884     </sql-query>
3885 </hibernate-mapping>
3886
3887 -----EmpDetails.java-----
3888 package com.nt.domain;
3889
3890 public class EmpDetails {
3891     //properties
3892     private int no;
3893     private String fname,lname,mail;
3894
3895     public EmpDetails(){
3896         System.out.println("EmpDetails:0-param constructor");
3897     }
3898     public EmpDetails(int no, String fname, String lname,String mail) {
3899         System.out.println("EmpDetails:4-param constructor");
3900         this.no = no;

```



```

3901         this.fname = fname;
3902         this.lname = lname;
3903         this.mail=mail;
3904     }
3905     public int getNo() {
3906         return no;
3907     }
3908     public void setNo(int no) {
3909         this.no = no;
3910     }
3911     public String getFname() {
3912         return fname;
3913     }
3914     public void setFname(String fname) {
3915         this.fname = fname;
3916     }
3917     public String getLname() {
3918         return lname;
3919     }
3920     public void setLname(String lname) {
3921         this.lname = lname;
3922     }
3923     public String getMail() {
3924         return mail;
3925     }
3926     public void setMail(String mail) {
3927         this.mail = mail;
3928     }
3929     @Override
3930     public String toString() {
3931         return "EmpDetails [no=" + no + ", fname=" + fname + ", lname=" + lname
3932             + ", mail=" + mail + "];"
3933     }
3934 }

```

```

3935 -----HibernateUtil.java-----
3936 package com.nt.utility;
3937 import org.hibernate.Session;
3938 import org.hibernate.SessionFactory;
3939 import org.hibernate.cfg.Configuration;
3940 import org.hibernate.service.ServiceRegistry;
3941 import org.hibernate.service.ServiceRegistryBuilder;
3942
3943 public class HibernateUtil {
3944     private static SessionFactory factory=null;
3945     static{
3946         try{
3947             Configuration cfg=new Configuration().configure("/com/nt/cfgs/hibernate.cfg.xml");
3948             ServiceRegistryBuilder builder=new ServiceRegistryBuilder();
3949             ServiceRegistry registry=builder.applySettings(cfg.getProperties()).buildServiceRegistry()
3950             factory=cfg.buildSessionFactory(registry);
3951         }
3952         catch(Exception e){
3953             e.printStackTrace();
3954         }
3955     }//static
3956
3957     static ThreadLocal <Session> tl=new ThreadLocal<Session>();
3958     static Session ses=null;
3959     public static Session getSession(){
3960         try{
3961             if(tl.get()==null){ // get from ThreadLocal
3962                 ses=factory.openSession();
3963                 tl.set(ses); // set into ThreadLocal
3964                 return ses;
3965             }

```

```
3966         else{
3967             return tl.get(); //get from ThreadLocal
3968         }
3969     } //try
3970     catch(Exception e){
3971         return null;
3972     }
3973 } //method
3974
3975 public static void closeSession(){
3976     try{
3977         ses.close();
3978     }
3979     catch(Exception e){
3980         e.printStackTrace();
3981     }
3982 } //closeSession()
3983 } //class
3984
3985 -----NativeSQLTest.java-----
3986 package com.nt.test;
3987
3988 import java.util.List;
3989
3990 import org.hibernate.Query;
3991 import org.hibernate.Session;
3992
3993 import com.nt.domain.EmpDetails;
3994 import com.nt.utility.HibernateUtil;
3995
3996 public class NativeSQLTest {
3997     public static void main(String[] args) {
3998
3999         //Get HB Session obj
4000         Session ses=HibernateUtil.getSession();
4001
4002         /*
4003         //On Entity Query
4004         SQLQuery q1=ses.createSQLQuery("select * from employe where email like :domain");
4005         q1.setString("domain","%x.com");
4006
4007         //add Domain class
4008         q1.addEntity(EmpDetails.class);
4009
4010         //Iterator<EmpDetails> it=q1.iterate();
4011         List <EmpDetails> list=q1.list();
4012         for(EmpDetails eb: list){
4013             System.out.println(eb);
4014         } //for */
4015
4016         /* // On Scalar Query
4017         SQLQuery q2=ses.createSQLQuery("select EID,FIRSTNAME from Employee");
4018         //map col results with Hibernate data types (This optional to do but recomanded)
4019         q2.addScalar("EID",StandardBasicTypes.INTEGER);
4020         q2.addScalar("FIRSTNAME",StandardBasicTypes.STRING);
4021         //execute Query
4022         List <Object[]> list =q2.list();
4023         //process the result
4024         for(Object[] row:list){
4025             for(Object o:row){
4026                 System.out.print(o+" ");
4027             } //for
4028             System.out.println();
4029         } //for */
4030     }
```

```

4031
4032  /* //Executing non-select Native SQL query to insert record
4033      Transaction tx=null;
4034      try{
4035          tx=ses.beginTransaction();
4036          SQLQuery q3=ses.createSQLQuery("insert into Employee values(:no,:fname,:lname,:mai
4037          q3.setInteger("no",1001);
4038          q3.setString("fname","Raja");
4039          q3.setString("lname","Rao");
4040          q3.setString("mail","x@y.com");
4041          int res=q3.executeUpdate();
4042          if(res==0)
4043              System.out.println("record not inserted");
4044          else
4045              System.out.println("record inserted");
4046          tx.commit();
4047      }//try
4048      catch(Exception e){
4049          tx.rollback();
4050      } */
4051
4052      // get Access to Named Native SQL Query
4053      Query q=ses.getNamedQuery("ntest1");
4054      q.setString("domain","%x.com");
4055      List <EmpDetails> list=q.list();
4056      for(EmpDetails eb: list){
4057          System.out.println(eb);
4058      }
4059
4060
4061      //close Session obj
4062      ses.close();
4063  }//main
4064 }//class
4065 =====
4066 Calling PL/sql Function from Hibernate App
4067 =====
4068 -----hibernate.cfg.xml-----
4069 <!DOCTYPE hibernate-configuration
4070 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4071 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
4072
4073 <hibernate-configuration>
4074     <session-factory>
4075         <!-- connection properties -->
4076         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
4077         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
4078         <property name="connection.username">scott</property>
4079         <property name="connection.password">tiger</property>
4080         <!-- hibernate properties -->
4081         <property name="show_sql">>true</property>
4082         <!-- mapping file names -->
4083         <mapping resource="com/nt/cfgs/Employee.hbm.xml"/>
4084     </session-factory>
4085 </hibernate-configuration>
4086 -----Employee.hbm.xml-----
4087 <!DOCTYPE hibernate-configuration
4088 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4089 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
4090
4091 <hibernate-configuration>
4092     <session-factory>
4093         <!-- connection properties -->
4094         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
4095         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>

```

```
4096     <property name="connection.username">scott</property>
4097     <property name="connection.password">tiger</property>
4098     <!-- hibernate properties -->
4099     <property name="show_sql">true</property>
4100     <!-- mapping file names -->
4101     <mapping resource="com/nt/cfgs/Employee.hbm.xml"/>
4102 </session-factory>
4103 </hibernate-configuration>
4104 -----Employee.hbm.xml-----
4105 <!DOCTYPE hibernate-mapping SYSTEM
4106 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
4107
4108 <hibernate-mapping>
4109     <class name="com.nt.domain.EmpDetails" table="Employee">
4110         <id name="no" column="EID"/> <!--Singular Id field cfg -->
4111         <property name="fname" column="FIRSTNAME"/>
4112         <property name="lname" column="LASTNAME"/>
4113         <property name="mail" column="EMAIL"/>
4114     </class>
4115     <sql-query name="fxtest" callable="true">
4116         <return class="com.nt.domain.EmpDetails"/>
4117         {?=call GET_EMP_DETAILS(:min,:max)}
4118     </sql-query>
4119
4120 </hibernate-mapping>
4121
4122 -----EmpDetails.java-----
4123 package com.nt.domain;
4124
4125 public class EmpDetails {
4126     //properties
4127     private int no;
4128     private String fname,lname,mail;
4129
4130     public EmpDetails(){
4131         System.out.println("EmpDetails:0-param constructor");
4132     }
4133     public EmpDetails(int no, String fname, String lname,String mail){
4134         System.out.println("EmpDetails:4-param constructor");
4135         this.no = no;
4136         this.fname = fname;
4137         this.lname = lname;
4138         this.mail=mail;
4139     }
4140     public int getNo() {
4141         return no;
4142     }
4143     public void setNo(int no) {
4144         this.no = no;
4145     }
4146     public String getFname() {
4147         return fname;
4148     }
4149     public void setFname(String fname) {
4150         this.fname = fname;
4151     }
4152     public String getLname() {
4153         return lname;
4154     }
4155     public void setLname(String lname) {
4156         this.lname = lname;
4157     }
4158     public String getMail() {
4159         return mail;
4160     }
```

```

4161     public void setMail(String mail) {
4162         this.mail = mail;
4163     }
4164     @Override
4165     public String toString() {
4166         return "EmpDetails [no=" + no + ", fname=" + fname + ", lname=" + lname
4167             + ", mail=" + mail + "];"
4168     }
4169 }
4170 -----HibernateUtil.java-----
4171 <-----Same ----->
4172 -----FunctionTest.java-----
4173 package com.nt.test;
4174
4175 import java.util.List;
4176
4177 import org.hibernate.Query;
4178 import org.hibernate.Session;
4179
4180 import com.nt.domain.EmpDetails;
4181 import com.nt.utility.HibernateUtil;
4182
4183 public class FunctionTest {
4184     public static void main(String[] args) {
4185
4186         //Get HB Session obj
4187         Session ses=HibernateUtil.getSession();
4188
4189         /*
4190         //On Entity Query
4191         SQLQuery q1=ses.createSQLQuery("select * from employee where email like 'domain'");
4192         q1.setString("domain","%x.com");
4193
4194         //add Domain class
4195         q1.addEntity(EmpDetails.class);
4196
4197         //Iterator<EmpDetails> it=q1.iterate();
4198         List <EmpDetails>list=q1.list();
4199         for(EmpDetails eb: list){
4200             System.out.println(eb);
4201         }//for */
4202
4203         /* // On Scalar Query
4204         SQLQuery q2=ses.createSQLQuery("select EID,FIRSTNAME from Employee");
4205         //map col results with Hibernate data types (This optional to do but recommended)
4206         q2.addScalar("EID",StandardBasicTypes.INTEGER);
4207         q2.addScalar("FIRSTNAME",StandardBasicTypes.STRING);
4208         //execute Query
4209         List <Object[]> list =q2.list();
4210         //process the result
4211         for(Object[] row:list){
4212             for(Object o:row){
4213                 System.out.print(o+" ");
4214             }//for
4215             System.out.println();
4216         }//for */
4217
4218         /* //Executing non-select Native SQL query to insert record
4219         Transaction tx=null;
4220         try{
4221             tx=ses.beginTransaction();
4222             SQLQuery q3=ses.createSQLQuery("insert into Employee values(:no,:fname,:lname,
4223             q3.setInteger("no",1001);
4224             q3.setString("fname","Raja");
4225             tx.commit();

```

```

4226         q3.setString("lname", "Rao");
4227         q3.setString("mail", "x@y.com");
4228         int res=q3.executeUpdate();
4229         if(res==0)
4230             System.out.println("record not inserted");
4231         else
4232             System.out.println("record inserted");
4233         tx.commit();
4234     } //try
4235     catch(Exception e){
4236         tx.rollback();
4237     } /*
4238
4239     /* // get Access to Named Native SQL Query
4240     Query q=ses.getNamedQuery("ntest1");
4241     q.setString("domain", "%x.com");
4242     List <EmpDetails> list=q.list();
4243     for(EmpDetails eb: list){
4244         System.out.println(eb);
4245     } /*
4246
4247     //get Access to Named NativeSQL Query
4248     Query q1=ses.getNamedQuery("fxtest");
4249     q1.setInteger("min",100);
4250     q1.setInteger("max",300);
4251     q1.setFirstResult(0);
4252     q1.setMaxResults(3);
4253     List <EmpDetails> list=q1.list();
4254     for(EmpDetails eb: list){
4255         System.out.println(eb);
4256     }
4257
4258     //close Session obj
4259     ses.close();
4260 } //main
4261 } //class
4262 =====
4263 App21) Calling PL/sql procedure from Hibernate App
4264 =====
4265 -----hibernate.cfg.xml-----
4266 <!DOCTYPE hibernate-configuration
4267 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4268 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
4269
4270 <hibernate-configuration>
4271     <session-factory>
4272         <!-- connection properties -->
4273         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
4274         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
4275         <property name="connection.username">scott</property>
4276         <property name="connection.password">tiger</property>
4277         <!-- hibernate properties -->
4278         <property name="show_sql">true</property>
4279         <!-- mapping file names -->
4280         <mapping resource="com/nt/cfgs/Employee.hbm.xml"/>
4281     </session-factory>
4282 </hibernate-configuration>
4283 -----Employee.hbm.xml-----
4284 <!DOCTYPE hibernate-mapping SYSTEM
4285 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
4286
4287 <hibernate-mapping>
4288     <class name="com.nt.domain.EmpDetails" table="Employee">
4289         <id name="no" column="EID"/> <!-- Singular Id field cfg -->
4290         <property name="fname" column="FIRSTNAME"/>

```



```

4291         <property name="lname" column="LASTNAME"/>
4292         <property name="mail" column="EMAIL"/>
4293     </class>
4294     <sql-query name="fxtest" callable="true">
4295         <return class="com.nt.domain.EmpDetails"/>
4296         {?=call GET_EMP_DETAILS(:min,:max)}
4297     </sql-query>
4298
4299     <sql-query name="Proc_test" callable="true">
4300         <return-scalar column="firstname" type="string"/>
4301         <return-scalar column="lastname" type="string"/>
4302         { call GET_EMP_DETAILS_FOR_MAIL(?, :domain)}
4303     </sql-query>
4304
4305 </hibernate-mapping>
4306
4307 -----EmpDetails.java-----
4308 package com.nt.domain;
4309
4310 public class EmpDetails {
4311     //properties
4312     private int no;
4313     private String fname,lname,mail;
4314
4315     public EmpDetails(){
4316         System.out.println("EmpDetails:0-param constructor");
4317     }
4318     public EmpDetails(int no, String fname, String lname,String mail) {
4319         System.out.println("EmpDetails:4-param constructor");
4320         this.no = no;
4321         this.fname = fname;
4322         this.lname = lname;
4323         this.mail=mail;
4324     }
4325     public int getNo() {
4326         return no;
4327     }
4328     public void setNo(int no) {
4329         this.no = no;
4330     }
4331     public String getFname() {
4332         return fname;
4333     }
4334     public void setFname(String fname) {
4335         this.fname = fname;
4336     }
4337     public String getLname() {
4338         return lname;
4339     }
4340     public void setLname(String lname) {
4341         this.lname = lname;
4342     }
4343     public String getMail() {
4344         return mail;
4345     }
4346     public void setMail(String mail) {
4347         this.mail = mail;
4348     }
4349     @Override
4350     public String toString() {
4351         return "EmpDetails [no=" + no + ", fname=" + fname + ", lname=" + lname
4352             + ", mail=" + mail + "];"
4353     }
4354 }
4355

```

```
4356 -----ProcedureTest.java-----
4357 package com.nt.test;
4358
4359 import java.util.List;
4360
4361 import org.hibernate.Query;
4362 import org.hibernate.Session;
4363
4364 import com.nt.domain.EmpDetails;
4365 import com.nt.utility.HibernateUtil;
4366
4367 public class ProcedureTest {
4368     public static void main(String[] args) {
4369
4370         //Get HB Session obj
4371         Session ses=HibernateUtil.getSession();
4372         // get Named Query
4373         Query q1=ses.getNamedQuery("Proc_test");
4374         //set param values
4375         q1.setString("domain","%x.com");
4376         //execute query
4377         List <Object[]>list=q1.list();
4378         //process the result
4379         for(Object row[]:list){
4380             System.out.println(row[0]+" "+row[1]);
4381         }//for
4382
4383
4384
4385
4386
4387
4388     /*
4389         //On Entity Query
4390         SQLQuery q1=ses.createSQLQuery("select * from employe where email like :domain");
4391         q1.setString("domain","%x.com");
4392
4393         //add Domain class
4394         q1.addEntity(EmpDetails.class);
4395
4396         //Iterator<EmpDetails> it=q1.iterate();
4397         List <EmpDetails>list=q1.list();
4398         for(EmpDetails eb: list){
4399             System.out.println(eb);
4400         }//for */
4401
4402     /* // On Scalar Query
4403         SQLQuery q2=ses.createSQLQuery("select EID,FIRSTNAME from Employee");
4404         //map col results with Hibernate data types (This optional to do but recomanded)
4405         q2.addScalar("EID",StandardBasicTypes.INTEGER);
4406         q2.addScalar("FIRSTNAME",StandardBasicTypes.STRING);
4407         //execute Query
4408         List <Object[]> list =q2.list();
4409         //process the result
4410         for(Object[] row:list){
4411             for(Object o:row){
4412                 System.out.print(o+" ");
4413             }//for
4414             System.out.println();
4415         }//for */
4416
4417     /* //Executing non-select Native SQL query to insert record
4418         Transaction tx=null;
4419         try{
```



```

4421         tx=ses.beginTransaction();
4422         SQLQuery q3=ses.createSQLQuery("insert into Employee values(:no,:fname,:lname,:mail)");
4423         q3.setInteger("no",1001);
4424         q3.setString("fname","Raja");
4425         q3.setString("lname","Rao");
4426         q3.setString("mail","x@y.com");
4427         int res=q3.executeUpdate();
4428         if(res==0)
4429             System.out.println("record not inserted");
4430         else
4431             System.out.println("record inserted");
4432         tx.commit();
4433     }//try
4434     catch(Exception e){
4435         tx.rollback();
4436     } */
4437
4438 /* // get Access to Named Native SQL Query
4439 Query q=ses.getNamedQuery("ntest1");
4440 q.setString("domain","%x.com");
4441 List <EmpDetails> list=q.list();
4442 for(EmpDetails eb: list){
4443     System.out.println(eb);
4444 } */
4445
4446 /* //get Access to Named NativeSQL Query
4447 Query q1=ses.getNamedQuery("fxtest");
4448 q1.setInteger("min",100);
4449 q1.setInteger("max",300);
4450 q1.setFirstResult(0);
4451 q1.setMaxResults(3);
4452 List <EmpDetails> list=q1.list();
4453 for(EmpDetails eb: list){
4454     System.out.println(eb);
4455 }*/
4456
4457 //close Session obj
4458 ses.close();
4459 }//main
4460 }//class
4461 =====
4462 App22 (Calling pl/sql procedure that is not following hibernate rules
4463 =====
4464 -----hibernate.cfg.xml-----
4465 <!DOCTYPE hibernate-configuration
4466 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4467 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
4468
4469 <hibernate-configuration>
4470     <session-factory>
4471         <!-- connection properties -->
4472         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
4473         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
4474         <property name="connection.username">scott</property>
4475         <property name="connection.password">tiger</property>
4476         <!-- hibernate properties -->
4477         <property name="show_sql">true</property>
4478     </session-factory>
4479 </hibernate-configuration>
4480 -----HibernateUtil.java-----
4481 <-----Same <-----
4482 -----ProcedureTest.java-----
4483 package com.nt.test;
4484
4485

```

```

4486 import java.sql.CallableStatement;
4487 import java.sql.Connection;
4488 import java.sql.SQLException;
4489 import java.sql.Types;
4490
4491 import org.hibernate.Session;
4492 import org.hibernate.jdbc.ReturningWork;
4493
4494 import com.nt.utility.HibernateUtil;
4495
4496 public class ProcedureTest {
4497     public static void main(String[] args) {
4498
4499         // Get HB Session obj
4500         Session ses=HibernateUtil.getSession();
4501         // call pl/sql procedure
4502         String result=ses.doReturningWork(new ReturningWork<String>(){
4503             @Override
4504             public String execute(Connection con) throws SQLException {
4505                 //write plain jdbc/java jdbc code
4506                 try{
4507                     //create CallableStatement obj
4508                     CallableStatement cs=con.prepareCall("{ call Auth_Pro(?,?,?) }");
4509                     //register OUT param with jdbc types
4510                     cs.registerOutParameter(3,Types.VARCHAR);
4511                     //set values to IN Params
4512                     cs.setString(1, "raja");
4513                     cs.setString(2,"khan");
4514                     //call pl/sql procedure
4515                     cs.execute();
4516                     //get Result from OUT param
4517                     String result=cs.getString(3);
4518                     return result;
4519                 }
4520                 catch(Exception e){
4521                     e.printStackTrace();
4522                     return "Internal problem";
4523                 }
4524             } //method
4525         } //inner class
4526         ); //end of method call
4527         System.out.println("Result is:::"+result);
4528         //close Session obj
4529         ses.close();
4530     } //main
4531 } //class
4532
4533 =====
4534 App23 ) App on QBC (Query By Criteria )
4535 =====
4536 -----hibernate.cfg.xml-----
4537 <!DOCTYPE hibernate-configuration
4538 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4539 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
4540
4541 <hibernate-configuration>
4542     <session-factory>
4543         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
4544         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
4545         <property name="connection.username">scott</property>
4546         <property name="connection.password">tiger</property>
4547         <property name="show_sql">>true</property>
4548         <property name="hibernate.hbm2ddl.auto">update</property>
4549         <mapping resource="com/nt/cfgs/Employee.hbm.xml"/>
4550     </session-factory>

```

```

4551 </hibernate-configuration>
4552
4553 -----Employee.hbm.xml-----
4554 <!DOCTYPE hibernate-mapping SYSTEM
4555 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
4556
4557 <hibernate-mapping>
4558     <class name="com.nt.domain.EmpDetails" table="Employee">
4559         <id name="no" column="EID"/> <!--Singular Id field cfg -->
4560         <property name="fname" column="FIRSTNAME" length="20"/>
4561         <property name="lname" column="LASTNAME" length="30"/>
4562         <property name="mail" column="EMAIL" length="20" unique="true" not-null="true"/>
4563     </class>
4564 </hibernate-mapping>
4565 -----EmpDetails.java-----
4566 // EmpDetails.java (Domain class/model class/Persistence class)
4567 package com.nt.domain;
4568 public class EmpDetails {
4569     //Properties
4570     private int no;
4571     private String fname, lname, mail;
4572
4573     public EmpDetails(){
4574         System.out.println("EmpDetails:0-param constructor");
4575     }
4576
4577     //write setXxx(-) and getXxx() methods
4578     public int getNo() {
4579         return no;
4580     }
4581
4582     public void setNo(int no) {
4583         this.no = no;
4584     }
4585
4586     public String getFname() {
4587         return fname;
4588     }
4589
4590     public void setFname(String fname) {
4591         this.fname = fname;
4592     }
4593
4594     public String getLname() {
4595         return lname;
4596     }
4597
4598     public void setLname(String lname) {
4599         this.lname = lname;
4600     }
4601
4602     public String getMail() {
4603         return mail;
4604     }
4605
4606     public void setMail(String mail) {
4607         this.mail = mail;
4608     }
4609
4610     @Override
4611     public String toString() {
4612         return "EmpDetails [no=" + no + ", fname=" + fname + ", lname=" + lname
4613             + ", mail=" + mail + "];"
4614     }
4615 }
4616 -----HibernateUtil.java-----
4617 <----- Same ----->
4618 -----CriteriaTest.java-----
4619 package com.nt.test;
4620
4621 import java.util.List;

```

```
4616
4617 import org.hibernate.Criteria;
4618 import org.hibernate.Session;
4619 import org.hibernate.criterion.Projection;
4620 import org.hibernate.criterion.ProjectionList;
4621 import org.hibernate.criterion.Projections;
4622
4623 import com.nt.domain.EmpDetails;
4624 import com.nt.utility.HibernateUtil;
4625
4626 public class CriteriaTest {
4627
4628     public static void main(String[] args) {
4629
4630         //Get HSession obj
4631         Session ses=HibernateUtil.getSession();
4632
4633         /* //get Criteria Obj
4634         Criteria ct=sess.createCriteria(com.nt.domain.EmpDetails.class);
4635         // execute logic
4636         List<EmpDetails> list=ct.list();
4637         //process the result
4638         for(EmpDetails eb:list){
4639             System.out.println(eb);
4640         } */
4641
4642         /* Criteria ct1=sess.createCriteria(EmpDetails.class);
4643         //prepare conditions
4644         Criterion cond1=Restrictions.between("no",100,900);
4645         Criterion cond2=Restrictions.like("mail","%x.com");
4646         //add conditions
4647         ct1.add(cond1); ct1.add(cond2);
4648         //execute QBC logic
4649         List <EmpDetails>list=ct1.list();
4650         //process the result
4651         for(EmpDetails eb:list)
4652             System.out.println(eb); */
4653
4654         /* Criteria ct2=sess.createCriteria(EmpDetails.class);
4655         //prepared conditions
4656         Criterion cond1=Restrictions.ge("no", 200);
4657         Criterion cond2=Restrictions.le("no",400);
4658         Criterion andcond=Restrictions.and(cond1,cond2);
4659         Criterion cond3=Restrictions.ilike("mail","%x.com");
4660         Criterion orcond=Restrictions.or(andcond,cond3);
4661         //add conditions
4662         ct2.add(orcond);
4663         //prepare Order obj
4664         Order o1=Order.desc("no");
4665         //add Order obj to Criteria obj
4666         ct2.addOrder(o1);
4667
4668         //execute the logic
4669         List <EmpDetails>list =ct2.list();
4670         //process the Results
4671         for(EmpDetails eb: list){
4672             System.out.println(eb);
4673         } */
4674
4675         /* Criteria ct3=sess.createCriteria(EmpDetails.class);
4676         //frame condition through sql
4677         Criterion cond1=Restrictions.sqlRestriction(" firstname like 'r%'");
4678         //add condition
4679         ct3.add(cond1);
4680
```

```

4681      //execute logic
4682      List <EmpDetails>list =ct3.list();
4683      //process the results
4684      for(EmpDetails eb:list){
4685          System.out.println(eb);
4686      } */
4687
4688      /* //working with Projections to get specific multiple property values(col values)
4689      Criteria ct=ses.createCriteria(EmpDetails.class);
4690      //prepare projection objs
4691      Projection p1=Projections.property("no");
4692      Projection p2=Projections.property("mail");
4693      //add Projection objs to ProjectionList
4694      ProjectionList pl=Projections.projectionList();
4695      pl.add(p1); pl.add(p2);
4696      //set ProjectList to Criteria obj
4697      ct.setProjection(pl);
4698      //execute Logic
4699      List <Object[]>list=ct.list();
4700      //process the results
4701      for(Object row[]:list){
4702          System.out.println(row[0]+" "+row[1]);
4703      }//for */
4704
4705      //Using Projections with Criteria API to work with aggregate operations
4706      Criteria ct4=ses.createCriteria(EmpDetails.class);
4707      // Prepare multiple Projection objs for aggregate functions
4708      Projection p1=Projections.count("no");
4709      Projection p2=Projections.avg("no");
4710      Projection p3=Projections.max("no");
4711      //add Multiple Projection objs to ProjectionList
4712      ProjectionList pl=Projections.projectionList();
4713      pl.add(p1); pl.add(p2); pl.add(p3);
4714      //set ProjectionList to Criteria
4715      ct4.setProjection(pl);
4716      //execute logic
4717      List <Object[]>list=ct4.list();
4718      //get Result
4719      Object res[]=(Object[])list.get(0);
4720      System.out.println("records count"+res[0]);
4721      System.out.println("avg value of EID col"+res[1]);
4722      System.out.println("max value of EID col"+res[2]);
4723
4724      //close session
4725      ses.close();
4726
4727
4728  }
4729
4730 }
4731 =====
4732 App24) Application on Filters
4733 =====
4734 -----hibernate.cfg.xml-----
4735 <!DOCTYPE hibernate-configuration
4736 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4737 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
4738
4739 <hibernate-configuration>
4740     <session-factory>
4741         <!-- connection properties -->
4742         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
4743         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
4744         <property name="connection.username">scott</property>
4745         <property name="connection.password">tiger</property>

```

```

4746      <!-- hibernate properties -->
4747      <property name="show_sql">true</property>
4748      <!-- mapping file names -->
4749      <mapping resource="com/nt/cfgs/Employee.hbm.xml"/>
4750  </session-factory>
4751 </hibernate-configuration>
4752 -----Employee.hbm.xml-----
4753 <!DOCTYPE hibernate-mapping SYSTEM
4754 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
4755
4756 <hibernate-mapping>
4757     <class name="com.nt.domain.EmpDetails" table="Employee">
4758         <id name="no" column="EID"/> <!-- Singular Id field cfg -->
4759         <property name="fname" column="FIRSTNAME"/>
4760         <property name="lname" column="LASTNAME"/>
4761         <property name="mail" column="EMAIL"/>
4762         <filter name="myfilter" condition="eid>=:p1 and eid &lt;=:p2"/>
4763     </class>
4764
4765     <filter-def name="myfilter" >
4766         <filter-param name="p1" type="int" />
4767         <filter-param name="p2" type="int" />
4768     </filter-def>
4769
4770 </hibernate-mapping>
4771 -----EmpDetails.java-----
4772 package com.nt.domain;
4773
4774 public class EmpDetails {
4775     //properties
4776     private int no;
4777     private String fname,lname,mail;
4778
4779     public EmpDetails(){
4780         System.out.println("EmpDetails:0-param constructor");
4781     }
4782     public EmpDetails(int no, String fname, String lname,String mail) {
4783         System.out.println("EmpDetails:4-param constructor");
4784         this.no = no;
4785         this.fname = fname;
4786         this.lname = lname;
4787         this.mail=mail;
4788     }
4789     public int getNo() {
4790         return no;
4791     }
4792     public void setNo(int no) {
4793         this.no = no;
4794     }
4795     public String getFname() {
4796         return fname;
4797     }
4798     public void setFname(String fname) {
4799         this.fname = fname;
4800     }
4801     public String getLname() {
4802         return lname;
4803     }
4804     public void setLname(String lname) {
4805         this.lname = lname;
4806     }
4807     public String getMail() {
4808         return mail;
4809     }
4810     public void setMail(String mail) {

```

```

4811         this.mail = mail;
4812     }
4813     @Override
4814     public String toString() {
4815         return "EmpDetails [no=" + no + ", fname=" + fname + ", lname=" + lname
4816             + ", mail=" + mail + "];"
4817     }
4818 }

```

```

4819 -----HibernateUtil.java-----

```

```

4820 <-----same----->

```

```

4821 -----FilterTest.java-----

```

```

4822 package com.nt.test;
4823
4824 import java.util.List;
4825
4826 import org.hibernate.Criteria;
4827 import org.hibernate.Filter;
4828 import org.hibernate.Session;
4829
4830 import com.nt.domain.EmpDetails;
4831 import com.nt.utility.HibernateUtil;
4832
4833 public class FilterTest {
4834
4835     public static void main(String[] args) throws Exception {
4836
4837         //Get Session obj
4838         Session ses=HibernateUtil.getSession();
4839
4840         //enable filter
4841         Filter f1=s ses.enableFilter("myfilter");
4842         //set filter param values
4843         f1.setParameter("p1",100);
4844         f1.setParameter("p2",300);
4845         /* //testing1
4846             Query q1=s ses.createQuery("select count(*) from EmpDetails");
4847             List <Long>list=q1.list(); //query executes with filter condition
4848             System.out.println("Count:"+list.get(0));
4849             //dsiable filter
4850             ses.disableFilter("myfilter");
4851             //testing2
4852             q1=s ses.createQuery("select count(*) from EmpDetails");
4853             list=q1.list(); //query executes with out filter condition
4854             System.out.println("Count:"+list.get(0)); */
4855
4856         /* //On Native SQL
4857             SQLQuery q1=s ses.createSQLQuery("select * from Employee");
4858             q1.addEntity(EmpDetails.class);
4859             List <EmpDetails> list=q1.list();
4860             for(EmpDetails eb:list){
4861                 System.out.println(eb);
4862             } */
4863         //On Criteria API
4864         Criteria ct=s ses.createCriteria(EmpDetails.class);
4865         List <EmpDetails> list=ct.list();
4866         for(EmpDetails eb:list){
4867             System.out.println(eb);
4868         }
4869
4870         // close session
4871         ses.close();
4872     } //main
4873 } //class
4874
4875 App25) App on Annotations based Hibernate Filters

```

```

4876 =====
4877 -----hibernate.cfg.xml-----
4878 <!DOCTYPE hibernate-configuration
4879 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4880 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
4881
4882 <hibernate-configuration>
4883     <session-factory>
4884         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
4885         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
4886         <property name="connection.username">scott</property>
4887         <property name="connection.password">tiger</property>
4888         <property name="show_sql">>true</property>
4889         <mapping class="com.nt.model.Student"/>
4890     </session-factory>
4891 </hibernate-configuration>
4892 -----Student.java-----
4893 package com.nt.model;
4894
4895 import javax.persistence.Column;
4896 import javax.persistence.Entity;
4897 import javax.persistence.Id;
4898 import javax.persistence.Table;
4899
4900 import org.hibernate.annotations.Filter;
4901 import org.hibernate.annotations.FilterDef;
4902 import org.hibernate.annotations.ParamDef;
4903
4904 @Entity
4905 @Table(name="Student_Tab")
4906 @FilterDef(name="myfilter",
4907 parameters={@ParamDef(name="p1",type="integer"),
4908             @ParamDef(name="p2",type="integer")
4909             })
4910 @Filter(name="myfilter", condition="stno>=:p1 and stno<=:p2")
4911 public class Student {
4912     private int sno;
4913     private String sname;
4914     // @Transient
4915     private String address;
4916
4917     public Student(){
4918         System.out.println("Student:0-param constructor");
4919     }
4920
4921     public Student(int sno, String sname, String address) {
4922         this.sno = sno;
4923         this.sname = sname;
4924         this.address = address;
4925     }
4926     //write full constructor
4927
4928     //write setXxx(-) and getXxx() methods
4929     @Column(name="stno")
4930     @Id
4931     public int getSno() {
4932         return sno;
4933     }
4934     public void setSno(int sno) {
4935         this.sno = sno;
4936     }
4937     @Column(name="stname")
4938     public String getSname() {
4939         return sname;
4940     }

```



```

4941     public void setName(String sname) {
4942         this.sname = sname;
4943     }
4944
4945     @Column(name="stadd")
4946     public String getAddress() {
4947         return address;
4948     }
4949
4950
4951     public void setAddress(String address) {
4952         this.address = address;
4953     }
4954     @Override
4955     public String toString() {
4956         return "Student [sno=" + sno + ", sname=" + sname + ", address="
4957             + address + "]";
4958     }
4959 }

```

4960 -----HibernateUtil.java-----

4961 <--- Same----->

4962 -----FilterTest.java-----

```

4963 package com.nt.test;
4964
4965 import java.util.List;
4966
4967 import org.hibernate.Filter;
4968 import org.hibernate.Query;
4969 import org.hibernate.Session;
4970
4971 import com.nt.model.Student;
4972 import com.nt.utility.HibernateUtil;
4973
4974 public class FilterTest {
4975
4976     public static void main(String[] args) {
4977         //Get Session obj
4978         Session ses=HibernateUtil.getSession();
4979         //enable filter
4980         Filter f1=ses.enableFilter("myfilter");
4981         //set filter param values
4982         f1.setParameter("p1",100);
4983         f1.setParameter("p2",300);
4984         Query q1=ses.createQuery("from Student");
4985         List<Student>list=q1.list();
4986         for(Student st:list){
4987             System.out.println(st);
4988         }
4989         //Disable Filter
4990         ses.disableFilter("myfilter");
4991         Query q2=ses.createQuery("from Student");
4992         List<Student>list1=q2.list();
4993         for(Student st:list1){
4994             System.out.println(st);
4995         }
4996         //close objs
4997         ses.close();
4998
4999     }
5000 }

```

5001 =====

5002 App) Application on Component Mapping

5003 =====

5004

5005 -----hibernate.cfg.xml-----

```
5006 <!DOCTYPE hibernate-configuration
5007 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
5008 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
5009
5010 <hibernate-configuration>
5011     <session-factory>
5012         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
5013         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
5014         <property name="connection.username">scott</property>
5015         <property name="connection.password">tiger</property>
5016         <property name="show_sql">true</property>
5017         <mapping resource="com/nt/cfgs/Person.hbm.xml"/>
5018     </session-factory>
5019 </hibernate-configuration>
5020
5021 -----Person.hbm.xml-----
5022 <!DOCTYPE hibernate-mapping SYSTEM
5023 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
5024
5025 <hibernate-mapping>
5026     <class name="com.nt.model.Person" table="Person">
5027         <id name="pid" />
5028         <property name="pname"/>
5029         <component name="pjob" class="com.nt.model.JobType">
5030             <property name="job"/>
5031             <property name="salary"/>
5032             <property name="department"/>
5033         </component>
5034     </class>
5035 </hibernate-mapping>
5036
5037 -----Person.java-----
5038 package com.nt.model;
5039
5040 public class Person {
5041     private int pid;
5042
5043     private String pname;
5044     private JobType pjob;
5045
5046     public Person(){
5047         System.out.println("Person:0-param constructor");
5048     }
5049
5050     public Person(int pid, String pname, JobType pjob) {
5051         System.out.println("Person:3-param cosntructor");
5052         this.pid = pid;
5053         this.pname = pname;
5054         this.pjob = pjob;
5055     }
5056
5057     public int getPid() {
5058         return pid;
5059     }
5060     public void setPid(int pid) {
5061         this.pid = pid;
5062     }
5063     public String getPname() {
5064         return pname;
5065     }
5066     public void setPname(String pname) {
5067         this.pname = pname;
5068     }
5069     public JobType getPjob() {
5070         return pjob;
```

```
5071     }
5072     public void setPjob(JobType pjob) {
5073         this.pjob = pjob;
5074     }
5075     @Override
5076     public String toString() {
5077         return "Person [pid=" + pid + ", pname=" + pname + ", pjob=" + pjob
5078             + "]";
5079     }
5080
5081
5082 }
5083
5084 /*
5085 create table person
5086 (pid number(5) primary key,
5087  pname varchar2(20),
5088  job varchar2(20),
5089  salary number(8,4),
5090  department number(5)
5091 );
5092 */
5093 -----JobType.java-----
5094 package com.nt.model;
5095
5096 public class JobType {
5097     private String job;
5098     private double salary;
5099     private int department;
5100
5101     public JobType(String job, double salary, int department) {
5102         System.out.println("JobType: 3-param constructor");
5103         this.job = job;
5104         this.salary = salary;
5105         this.department = department;
5106     }
5107
5108
5109
5110     public JobType(){
5111         System.out.println("JobType:0-param constructor");
5112     }
5113
5114
5115
5116     public void setJob(String job) {
5117         this.job = job;
5118     }
5119     public void setSalary(double salary) {
5120         this.salary = salary;
5121     }
5122     public void setDepartment(int department) {
5123         this.department = department;
5124     }
5125
5126
5127
5128     public String getJob() {
5129         return job;
5130     }
5131
5132
5133
5134     public double getSalary() {
5135         return salary;
```

```
5136     }
5137
5138
5139
5140     public int getDepartment() {
5141         return department;
5142     }
5143
5144
5145
5146     @Override
5147     public String toString() {
5148         return "JobType [job=" + job + ", salary=" + salary + ", department="
5149             + department + "]";
5150     }
5151 }
5152 -----HibernateUtil.java-----
5153 >>>same>>>
5154 -----TestClient.java-----
5155 package com.nt.test;
5156
5157 import org.hibernate.Session;
5158 import org.hibernate.Transaction;
5159 import com.nt.model.JobType;
5160 import com.nt.model.Person;
5161 import com.nt.utility.HibernateUtil;
5162
5163 public class TestClient {
5164
5165     public static void main(String[] args) {
5166         //Get Session obj
5167         Session ses=HibernateUtil.getSession();
5168         //Save objs
5169         Transaction tx=null;
5170         try{
5171             tx=ses.beginTransaction();
5172
5173             JobType jt1=new JobType("clerk",9000,1001);
5174             Person p1=new Person();
5175             p1.setPid(103); p1.setPname("Raja"); p1.setPjob(jt1);
5176
5177             JobType jt2=new JobType("manager",18000,1002);
5178             Person p2=new Person();
5179             p2.setPid(104); p2.setPname("Rajesh"); p2.setPjob(jt2);
5180             //save objs
5181             ses.save(p1);
5182             ses.save(p2);
5183
5184             tx.commit();
5185         }//try
5186         catch(Exception e){
5187
5188             tx.rollback();
5189             e.printStackTrace();
5190         }
5191
5192         //close session
5193         ses.close();
5194
5195     }
5196 }
5197
5198 }
5199 -----SelectTest.java-----
5200 package com.nt.test;
```

```
5201
5202 import java.util.List;
5203
5204 import org.hibernate.Query;
5205 import org.hibernate.Session;
5206
5207 import com.nt.model.JobType;
5208 import com.nt.model.Person;
5209 import com.nt.utility.HibernateUtil;
5210
5211 public class SelectTest {
5212
5213     public static void main(String[] args) {
5214
5215         //Get HB Session obj
5216         Session ses=HibernateUtil.getSession();
5217         //load objs
5218         Query q1=ses.createQuery("from Person where pjob.job=:desg");
5219         q1.setString("desg","clerk");
5220         List <Person>list=q1.list();
5221         for(Person p:list){
5222             JobType jt=p.getPjob();
5223             System.out.println(p.getPid()+" "+p.getPname()+" "+jt.getJob()
5224                 +" "+jt.getSalary()+" "+jt.getDepartment());
5225         }//for
5226     }
5227 }
```

5228 =====
5229 App27) Application on Annotations based Component mapping
5230 =====

5231 -----hibernate.cfg.xml-----

```
5232 <!DOCTYPE hibernate-configuration
5233 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
5234 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
5235
5236 <hibernate-configuration>
5237     <session-factory>
5238         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
5239         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
5240         <property name="connection.username">scott</property>
5241         <property name="connection.password">tiger</property>
5242         <property name="show_sql">>true</property>
5243         <mapping class="com.nt.model.Person"/>
5244     </session-factory>
5245 </hibernate-configuration>
```

5246 -----Person.java-----

```
5247
5248 package com.nt.model;
5249
5250 import javax.persistence.Embedded;
5251 import javax.persistence.Entity;
5252 import javax.persistence.GeneratedValue;
5253 import javax.persistence.Id;
5254 import javax.persistence.Table;
5255
5256 import org.hibernate.annotations.GenericGenerator;
5257
5258 @Table(name="person")
5259 @Entity
5260 public class Person {
5261     @Id
5262     @GenericGenerator(name="gen1",strategy="increment")
5263     @GeneratedValue(generator="gen1")
5264     private int pid;
```

```
5266     private String pname;
5267     @Embedded
5268     private JobType pjob;
5269
5270     public Person(){
5271         System.out.println("Person:0-param constructor");
5272     }
5273
5274     public Person(int pid, String pname, JobType pjob) {
5275         System.out.println("Person:3-param constructor");
5276         this.pid = pid;
5277         this.pname = pname;
5278         this.pjob = pjob;
5279     }
5280
5281     public int getPid() {
5282         return pid;
5283     }
5284     public void setPid(int pid) {
5285         this.pid = pid;
5286     }
5287     public String getPname() {
5288         return pname;
5289     }
5290     public void setPname(String pname) {
5291         this.pname = pname;
5292     }
5293     public JobType getPjob() {
5294         return pjob;
5295     }
5296     public void setPjob(JobType pjob) {
5297         this.pjob = pjob;
5298     }
5299     @Override
5300     public String toString() {
5301         return "Person [pid=" + pid + ", pname=" + pname + ", pjob=" + pjob
5302             + "]\n";
5303     }
5304
5305 }
5306
5307 /*
5308 create table person
5309 (pid number(5) primary key,
5310  pname varchar2(20),
5311  job varchar2(20),
5312  salary number(8,4),
5313  department number(5)
5314 );
5315 */
5316
5317 -----JobType.java-----
5318 package com.nt.model;
5319
5320 import javax.persistence.Embeddable;
5321
5322 @Embeddable
5323 public class JobType {
5324     private String job;
5325     private double salary;
5326     private int department;
5327
5328     public JobType(String job, double salary, int department) {
5329         System.out.println("JobType: 3-param constructor");
5330         this.job = job;
5331     }
5332 }
```

```
5331         this.salary = salary;
5332         this.department = department;
5333     }
5334
5335
5336
5337     public JobType(){
5338         System.out.println("JobType:0-param constructor");
5339     }
5340
5341
5342
5343     public void setJob(String job) {
5344         this.job = job;
5345     }
5346     public void setSalary(double salary) {
5347         this.salary = salary;
5348     }
5349     public void setDepartment(int department) {
5350         this.department = department;
5351     }
5352
5353
5354
5355     public String getJob() {
5356         return job;
5357     }
5358
5359
5360
5361     public double getSalary() {
5362         return salary;
5363     }
5364
5365
5366
5367     public int getDepartment() {
5368         return department;
5369     }
5370
5371
5372
5373     @Override
5374     public String toString() {
5375         return "JobType [job=" + job + ", salary=" + salary + ", department="
5376             + department + "]";
5377     }
5378
5379
5380
5381 }
5382 -----InsertClient.java-----
5383 package com.nt.test;
5384
5385 import org.hibernate.Session;
5386 import org.hibernate.Transaction;
5387
5388 import com.nt.model.JobType;
5389 import com.nt.model.Person;
5390 import com.nt.utility.HibernateUtil;
5391
5392 public class InsertClient {
5393
5394     public static void main(String[] args) {
5395         //Get Session obj
```

```
5396 Session ses=HibernateUtil.getSession();
5397 //Save objs
5398 Transaction tx=null;
5399 try{
5400     tx=ses.beginTransaction();
5401
5402     JobType jt1=new JobType("clerk",9000,1001);
5403     Person p1=new Person();
5404     p1.setPname("Raja1"); p1.setPjob(jt1);
5405
5406     JobType jt2=new JobType("manager",18000,1002);
5407     Person p2=new Person();
5408     p2.setPname("Rajesh1"); p2.setPjob(jt2);
5409     //save objs
5410     ses.save(p1);
5411     ses.save(p2);
5412
5413     tx.commit();
5414 }//try
5415 catch(Exception e){
5416
5417     tx.rollback();
5418     e.printStackTrace();
5419 }
5420 //close session
5421 ses.close();
5422
5423 }
5424
5425 }
5426
5427 }
5428 -----SelectTest.java-----
5429 package com.nt.test;
5430
5431 import java.util.List;
5432
5433 import org.hibernate.Query;
5434 import org.hibernate.Session;
5435
5436 import com.nt.model.JobType;
5437 import com.nt.model.Person;
5438 import com.nt.utility.HibernateUtil;
5439
5440 public class SelectTest {
5441
5442     public static void main(String[] args) {
5443
5444         //Get HB Session obj
5445         Session ses=HibernateUtil.getSession();
5446         //load objs
5447         Query q1=ses.createQuery("from Person where pjob.job=:desg");
5448         q1.setString("desg","clerk");
5449         List <Person>list=q1.list();
5450         for(Person p:list){
5451             JobType jt=p.getPjob();
5452             System.out.println(p.getPid()+" "+p.getPname()+" "+jt.getJob()
5453                 +" "+jt.getSalary()+" "+jt.getDepartment());
5454         }//for
5455     }
5456 }
5457
5458 =====
5459 App28) Inheritance Mapping (Table Class hierarchy)
5460 =====
```



```

5461 -----hibernate.cfg.xml-----
5462 <!DOCTYPE hibernate-configuration
5463 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
5464 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
5465
5466 <hibernate-configuration>
5467     <session-factory>
5468         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
5469         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
5470         <property name="connection.username">scott</property>
5471         <property name="connection.password">tiger</property>
5472         <property name="show_sql">>true</property>
5473         <mapping resource="com/nt/cfgs/Person.hbm.xml"/>
5474     </session-factory>
5475 </hibernate-configuration>
5476
5477 -----Person.hbm.xml-----
5478 <!DOCTYPE hibernate-mapping SYSTEM
5479 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
5480
5481 <hibernate-mapping>
5482     <class name="com.nt.model.Person" table="in_persons" discriminator-value="per" >
5483         <id name="id" />
5484         <discriminator column="person_type" type="string" length="5" />
5485         <property name="name" />
5486         <property name="company" />
5487         <subclass name="com.nt.model.Employee" discriminator-value="emp">
5488             <property name="salary" />
5489             <property name="department" />
5490         </subclass>
5491         <subclass name="com.nt.model.Customer" discriminator-value="cust">
5492             <property name="address" />
5493         </subclass>
5494     </class>
5495 </hibernate-mapping>
5496 /*1) table for hierarchy
5497
5498     create table in_persons
5499     (
5500         id number(5) primary key,
5501         name varchar2(20),
5502         company varchar2(20),
5503         salary number(8,2),
5504         department number(4),
5505         address varchar2(30),
5506         person_type varchar2(5)
5507     );
5508 */
5509 -----Person.java-----
5510 package com.nt.model;
5511
5512 public class Person {
5513     private int id;
5514     private String name;
5515     private String company;
5516
5517     public Person(){
5518         System.out.println("Person:0-param constructor");
5519     }
5520
5521     public Person(int id, String name, String company) {
5522         System.out.println("Person:3-param constructor");
5523         this.id = id;
5524         this.name = name;
5525         this.company = company;

```

```
5526     }
5527
5528     public int getId() {
5529         return id;
5530     }
5531
5532     public void setId(int id) {
5533         this.id = id;
5534     }
5535
5536     public String getName() {
5537         return name;
5538     }
5539
5540     public void setName(String name) {
5541         this.name = name;
5542     }
5543
5544     public String getCompany() {
5545         return company;
5546     }
5547
5548     public void setCompany(String company) {
5549         this.company = company;
5550     }
5551
5552     @Override
5553     public String toString() {
5554         return "Person [id=" + id + ", name=" + name + ", company=" + company
5555             + "]";
5556     }
5557
5558 }
5559
5560
5561 /*
5562 1) table for hierarchy
5563
5564     create table in_persons
5565     (
5566         id number(5) primary key,
5567         name varchar2(20),
5568         company varchar2(20),
5569         salary number(8,2),
5570         department number(4),
5571         address varchar2(30),
5572         person_type varchar2(5)
5573     );
5574 */
5575
5576 -----Customer.java-----
5577 package com.nt.model;
5578
5579 public class Customer extends Person {
5580
5581
5582     private String address;
5583
5584     public Customer(){
5585         System.out.println("Customer:0-param constructor");
5586     }
5587
5588     public Customer(int id,String name,String company,String address) {
5589         super(id,name,company);
5590         System.out.println("Customer:4-param constructor");
5591     }
5592 }
```

```
5591         this.address = address;
5592     }
5593
5594     public String getAddress() {
5595         return address;
5596     }
5597
5598     public void setAddress(String address) {
5599         this.address = address;
5600     }
5601
5602     @Override
5603     public String toString() {
5604         return "Customer Id=" + getId() + ", Name=" + getName()
5605             + ", Company=" + getCompany()+",address="+getAddress();
5606     }
5607 }
5608
5609
5610
5611 }
5612 -----Employee.java-----
5613 package com.nt.model;
5614
5615 public class Employee extends Person {
5616
5617     private double salary;
5618     private int department;
5619
5620     public Employee(){
5621         System.out.println("Employee:0-param constructor");
5622     }
5623
5624
5625
5626     public Employee(int id, String name, String company,double salary,int department) {
5627         super(id, name, company);
5628         this.salary=salary;
5629         this.department=department;
5630         System.out.println("Employee:5 -param constructor");
5631     }
5632
5633
5634
5635     public double getSalary() {
5636         return salary;
5637     }
5638
5639     public void setSalary(double salary) {
5640         this.salary = salary;
5641     }
5642
5643     public int getDepartment() {
5644         return department;
5645     }
5646
5647     public void setDepartment(int department) {
5648         this.department = department;
5649     }
5650
5651
5652
5653     @Override
5654     public String toString() {
5655         return "Employee [ Id=" + getId() + ", Name=" + getName()
```

```
5656         + ",Company=" + getCompany()
5657         +"salary=" + salary + ", department=" + department;
5658     }
5659
5660 }//class
5661
5662 -----InhInsertTest.java-----
5663 package com.nt.test;
5664
5665 import org.hibernate.Session;
5666 import org.hibernate.Transaction;
5667
5668 import com.nt.model.Customer;
5669 import com.nt.model.Employee;
5670 import com.nt.model.Person;
5671 import com.nt.utility.HibernateUtil;
5672
5673 public class InhInsertTest {
5674
5675     public static void main(String[] args) {
5676         // Get HB Session obj
5677         Session ses=HibernateUtil.getSession();
5678         // Save objs
5679         Transaction tx=null;
5680         try{
5681             tx=ses.beginTransaction();
5682             Person p=new Person(101,"raja","HCL");
5683             Employee emp=new Employee(102,"ravi","Wipro",89000,1111);
5684             Customer cust=new Customer(103,"ramesh","HCL","hyd");
5685             ses.save(p); ses.save(emp); ses.save(cust);
5686             tx.commit();
5687         }//try
5688         catch(Exception e){
5689             e.printStackTrace();
5690             tx.rollback();
5691         }
5692
5693         //close Session
5694         HibernateUtil.closeSession();
5695     }//main
5696 }//class
5697
5698 -----InhSelectTest.java-----
5699 package com.nt.test;
5700
5701 import java.util.List;
5702
5703 import org.hibernate.Query;
5704 import org.hibernate.Session;
5705
5706 import com.nt.model.Customer;
5707 import com.nt.model.Employee;
5708 import com.nt.model.Person;
5709 import com.nt.utility.HibernateUtil;
5710
5711 public class InhSelectTest {
5712
5713     public static void main(String[] args) {
5714         //Get HB Session obj
5715         Session ses=HibernateUtil.getSession();
5716         //load objs
5717         // Using Employee Domain class
5718         Query q1=ses.createQuery("from Employee"); // gives 1 record/obj
5719         List <Employee> list=q1.list();
5720         for(Employee emp: list){
```

```

5721         System.out.println(emp);
5722     }
5723     System.out.println("-----");
5724     //Using Customer Domain class
5725     Query q2=ses.createQuery("from Customer"); // gives 1 record/obj
5726     List <Customer>list2=q2.list();
5727     for(Customer cust: list2){
5728         System.out.println(cust);
5729     }
5730     System.out.println("-----");
5731     //Using Person Domain class
5732     Query q3=ses.createQuery("from Person"); // gives 3 records/objs
5733     List <Person>list3=q3.list();
5734     for(Person per: list3){
5735         System.out.println(per);
5736     }
5737
5738 }//main
5739 }//class
5740
5741 =====
5742 App29) App on Table per sub class
5743 =====
5744 <!DOCTYPE hibernate-configuration
5745 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
5746 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
5747
5748 <hibernate-configuration>
5749     <session-factory>
5750         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
5751         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
5752         <property name="connection.username">scott</property>
5753         <property name="connection.password">tiger</property>
5754         <property name="show_sql">>true</property>
5755         <mapping resource="com/nt/cfgs/Person.hbm.xml"/>
5756     </session-factory>
5757 </hibernate-configuration>
5758 -----Person.hbm.xml-----
5759 <!DOCTYPE hibernate-mapping SYSTEM
5760 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
5761
5762 <hibernate-mapping>
5763     <class name="com.nt.model.Person" table="in_persons2" >
5764         <id name="id" />
5765         <property name="name" />
5766         <property name="company" />
5767         <joined-subclass name="com.nt.model.Employee" table="in_employees2">
5768             <key column="person_id"/>
5769             <property name="salary" />
5770             <property name="department" />
5771         </joined-subclass>
5772         <joined-subclass name="com.nt.model.Customer" table="in_customers2">
5773             <key column="person_id"/>
5774             <property name="address" />
5775         </joined-subclass>
5776     </class>
5777 </hibernate-mapping>
5778 <!--
5779
5780 1) table per subclass
5781
5782     create table in_persons2
5783     (
5784         id number(5) primary key,
5785         name varchar2(20),

```

```

5786     company varchar2(20)
5787 );
5788 create table in_employees2
5789 (
5790     salary number(8,2),
5791     department number(4),
5792     person_id number(5) references in_persons2(id)
5793 );
5794 create table in_customers2
5795 (
5796     address varchar2(30),
5797     person_id number(5) references in_persons2(id)
5798 );
5799 -->
5800 note: Remaining all resources are same as previous App
5801 -----
5802 =====
5803 App30 Inheritance mapping (Table per concrete class)
5804 =====
5805 -----hibernate.cfg.xml-----
5806 <!DOCTYPE hibernate-configuration
5807 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
5808 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
5809
5810 <hibernate-configuration>
5811     <session-factory>
5812         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
5813         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
5814         <property name="connection.username">scott</property>
5815         <property name="connection.password">tiger</property>
5816         <property name="show_sql">>true</property>
5817         <mapping resource="com/nt/cfgs/Person.hbm.xml"/>
5818     </session-factory>
5819 </hibernate-configuration>
5820 -----Person.hbm.xml-----
5821 <!DOCTYPE hibernate-mapping SYSTEM
5822 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
5823
5824 <hibernate-mapping>
5825 <!-- <class name="com.nt.model.Person" table="in_persons3" >
5826     <id name="id" />
5827     <property name="name" />
5828     <property name="company" />
5829     <union-subclass name="com.nt.model.Employee" table="in_employees3">
5830         <property name="salary" />
5831         <property name="department" />
5832     </union-subclass>
5833     <union-subclass name="com.nt.model.Customer" table="in_customers3">
5834         <property name="address" />
5835     </union-subclass>
5836 </class> -->
5837 <class name="com.nt.model.Person" table="in_persons3">
5838     <id name="id" />
5839     <property name="name" />
5840     <property name="company" />
5841 </class>
5842 <class name="com.nt.model.Employee" table="in_employees3">
5843     <id name="id" />
5844     <property name="name" />
5845     <property name="company" />
5846     <property name="salary" />
5847     <property name="department" />
5848 </class>
5849 <class name="com.nt.model.Customer" table="in_customers3">
5850     <id name="id" />

```

```

5851         <property name="name" />
5852         <property name="company" />
5853         <property name="address"/>
5854     </class>
5855 </hibernate-mapping>
5856 -----
5857 note: Remaining all resources are same as previous App
5858
5859 =====
5860 App31 ) Annotations based Inheritance mapping (Table per class hierarchy)
5861 =====
5862 -----hibernate.cfg.xml-----
5863 <!DOCTYPE hibernate-configuration
5864 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
5865 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
5866
5867 <hibernate-configuration>
5868     <session-factory>
5869         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
5870         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
5871         <property name="connection.username">scott</property>
5872         <property name="connection.password">tiger</property>
5873         <property name="show_sql">>true</property>
5874
5875         <mapping class="com.nt.model.Payment"/>
5876         <mapping class="com.nt.model.CreditCardPayment"/>
5877         <mapping class="com.nt.model.ChequePayment"/>
5878
5879     </session-factory>
5880 </hibernate-configuration>
5881 -----Payment.java-----
5882 package com.nt.model;
5883
5884 import javax.persistence.Column;
5885 import javax.persistence.DiscriminatorColumn;
5886 import javax.persistence.DiscriminatorType;
5887 import javax.persistence.Entity;
5888 import javax.persistence.Id;
5889 import javax.persistence.Inheritance;
5890 import javax.persistence.InheritanceType;
5891 import javax.persistence.Table;
5892
5893 @Table(name="payment1")
5894 @Entity
5895 @Inheritance(strategy=InheritanceType.SINGLE_TABLE)
5896 @DiscriminatorColumn(name="paytype",discriminatorType=DiscriminatorType.STRING,length=10)
5897 public class Payment {
5898     @Id
5899     @Column(name="payid")
5900     private int payId;
5901
5902     @Column(name="amount")
5903     private int amount;
5904
5905     public Payment(){
5906         System.out.println("Payment:0-param constructor");
5907     }
5908     public Payment(int payId,int amount){
5909         System.out.println("Payment:2-param constructor");
5910         this.payId=payId;
5911         this.amount=amount;
5912     }
5913
5914     public int getPayId() {
5915

```

```
5916         return payId;
5917     }
5918
5919     public void setPayId(int payId) {
5920         this.payId = payId;
5921     }
5922
5923     public int getAmount() {
5924         return amount;
5925     }
5926
5927     public void setAmount(int amount) {
5928         this.amount = amount;
5929     }
5930     @Override
5931     public String toString() {
5932         return "Payment [payId=" + payId + ", amount=" + amount + "]";
5933     }
5934 }
5935
5936 /* create table payment1(payid number(5) primary key,
5937         amount number(7),
5938         cctype varchar2(15),
5939         chtype varchar2(15),
5940         paytype varchar2(10)
5941     );
5942
5943 */
5944 -----ChequePayment.java-----
5945 package com.nt.model;
5946
5947 import javax.persistence.Column;
5948 import javax.persistence.DiscriminatorValue;
5949 import javax.persistence.Entity;
5950
5951 @Entity
5952 @DiscriminatorValue("CHEQUE")
5953 public class ChequePayment extends Payment {
5954     @Column(name="chtype")
5955     private String chequeType;
5956
5957
5958     public ChequePayment(){
5959         System.out.println("ChequePayment:0-param constructor");
5960     }
5961     public ChequePayment(int payid,int amount,String chequeType){
5962         super(payid,amount);
5963         System.out.println("ChequePayment:3-param constructor");
5964         this.chequeType=chequeType;
5965     }
5966     public String getChequeType() {
5967         return chequeType;
5968     }
5969     public void setChequeType(String chequeType) {
5970         this.chequeType = chequeType;
5971     }
5972     @Override
5973     public String toString() {
5974         return "ChequePayment [chequeType=" + chequeType + ", PayId="
5975             + getPayId() + ", Amount=" + getAmount() ;
5976     }
5977
5978 }
5979
5980 -----CreditPayment.java-----
```



```

5981 package com.nt.model;
5982
5983 import javax.persistence.Column;
5984 import javax.persistence.DiscriminatorValue;
5985 import javax.persistence.Entity;
5986
5987 @Entity
5988 @DiscriminatorValue("CREDIT")
5989 public class CreditCardPayment extends Payment {
5990     @Column(name = "cctype")
5991     private String cardType;
5992
5993     public CreditCardPayment(){
5994         System.out.println("CreditCard:0-param constructor");
5995     }
5996     public CreditCardPayment(int payid,int amount,String cardType){
5997         super(payid,amount);
5998         System.out.println("CreditCard:3-param constructor");
5999         this.cardType=cardType;
6000     }
6001     public String getCardType() {
6002         return cardType;
6003     }
6004     public void setCardType(String cardType) {
6005         this.cardType = cardType;
6006     }
6007     @Override
6008     public String toString() {
6009         return "CreditCardPayment [cardType=" + cardType + ",PayId="
6010             + getPayId() + ",Amount=" + getAmount() ;
6011     }
6012 }
6013 }
6014 -----HibernateUtil.java-----
6015 <----same---->
6016 -----InhInsertTest.java-----
6017 package com.nt.test;
6018
6019 import org.hibernate.Session;
6020 import org.hibernate.Transaction;
6021
6022 import com.nt.model.ChequePayment;
6023 import com.nt.model.CreditCardPayment;
6024 import com.nt.utility.HibernateUtil;
6025
6026 public class InhInsertTest {
6027
6028     public static void main(String[] args) {
6029         // Get Session obj
6030         Session ses=HibernateUtil.getSession();
6031         //save objs
6032         Transaction tx=null;
6033         try{
6034             tx=ses.beginTransaction();
6035             CreditCardPayment cd=new CreditCardPayment(1,2000,"Visa");
6036             ChequePayment cp=new ChequePayment(2,3000,"self");
6037             ses.save(cd); ses.save(cp);
6038             tx.commit();
6039         }//try
6040         catch(Exception e){
6041             tx.rollback();
6042         }
6043         //Close Session
6044         HibernateUtil.closeSession();
6045     }

```

```

6046
6047 }
6048
6049 -----InhSelectTest.java-----
6050 package com.nt.test;
6051
6052 import java.util.List;
6053
6054 import org.hibernate.Query;
6055 import org.hibernate.Session;
6056
6057 import com.nt.model.ChequePayment;
6058 import com.nt.model.CreditCardPayment;
6059 import com.nt.utility.HibernateUtil;
6060
6061 public class InhSelectTest {
6062
6063     public static void main(String[] args) {
6064         // Get Session
6065         Session ses=HibernateUtil.getSession();
6066         //Using CreditCardPayment
6067         Query q1=ses.createQuery("from CreditCardPayment");
6068         List<CreditCardPayment> list=q1.list();
6069         for(CreditCardPayment cd:list){
6070             System.out.println(cd);
6071         }
6072         System.out.println("-----");
6073         Query q2=ses.createQuery("from ChequePayment");
6074         List<ChequePayment> list1=q2.list();
6075         for(ChequePayment cp:list1){
6076             System.out.println(cp);
6077         }
6078         //close Session
6079         HibernateUtil.closeSession();
6080     }
6081 }
6082 =====
6083 App32 (Annotations based Inheritance mapping ) (Table per sub class)
6084 =====
6085 <!DOCTYPE hibernate-configuration
6086 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
6087 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
6088
6089 <hibernate-configuration>
6090     <session-factory>
6091         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
6092         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
6093         <property name="connection.username">scott</property>
6094         <property name="connection.password">tiger</property>
6095         <property name="show_sql">>true</property>
6096
6097         <mapping class="com.nt.model.Payment"/>
6098         <mapping class="com.nt.model.CreditCardPayment"/>
6099         <mapping class="com.nt.model.ChequePayment"/>
6100
6101     </session-factory>
6102 </hibernate-configuration>
6103 -----Payment.java-----
6104 package com.nt.model;
6105
6106 import javax.persistence.Column;
6107 import javax.persistence.DiscriminatorColumn;
6108 import javax.persistence.DiscriminatorType;
6109 import javax.persistence.Entity;

```

```
6111 import javax.persistence.Id;
6112 import javax.persistence.Inheritance;
6113 import javax.persistence.InheritanceType;
6114 import javax.persistence.Table;
6115
6116 @Table(name="payment2")
6117 @Entity
6118 @Inheritance(strategy=InheritanceType.JOINED)
6119 public class Payment {
6120     @Id
6121     @Column(name="payid")
6122     private int payId;
6123
6124     @Column(name="amount")
6125     private int amount;
6126
6127     public Payment(){
6128         System.out.println("Payment:0-param constructor");
6129     }
6130     public Payment(int payId,int amount){
6131         System.out.println("Payment:2-param constructor");
6132         this.payId=payId;
6133         this.amount=amount;
6134     }
6135
6136     public int getPayId() {
6137         return payId;
6138     }
6139
6140     public void setPayId(int payId) {
6141         this.payId = payId;
6142     }
6143
6144     public int getAmount() {
6145         return amount;
6146     }
6147
6148     public void setAmount(int amount) {
6149         this.amount = amount;
6150     }
6151     @Override
6152     public String toString() {
6153         return "Payment [payId=" + payId + ", amount=" + amount + "]";
6154     }
6155 }
6156
6157 /* create table payment2(payid number(5) primary key,amount number(7));
6158
6159     create table credit_payment2(cctype varchar2(20),payment_id references payment2(payid));
6160
6161     create table cheque_payment2(chtype varchar2(20),payment_id references payment2(payid));
6162
6163 */
6164
6165 -----CreditCardPayment.java-----
6166 package com.nt.model;
6167
6168 import javax.persistence.Column;
6169 import javax.persistence.Entity;
6170 import javax.persistence.PrimaryKeyJoinColumn;
6171 import javax.persistence.Table;
6172
6173 @Entity
6174 @Table(name="credit_payment2")
```

```
6176 @PrimaryKeyJoinColumn(name="payment_id")
6177 public class CreditCardPayment extends Payment {
6178     @Column(name = "cctype")
6179     private String cardType;
6180
6181     public CreditCardPayment(){
6182         System.out.println("CreditCard:0-param constructor");
6183     }
6184     public CreditCardPayment(int payid,int amount,String cardType){
6185         super(payid,amount);
6186         System.out.println("CreditCard:3-param constructor");
6187         this.cardType=cardType;
6188     }
6189     public String getCardType() {
6190         return cardType;
6191     }
6192     public void setCardType(String cardType) {
6193         this.cardType = cardType;
6194     }
6195     @Override
6196     public String toString() {
6197         return "CreditCardPayment [cardType=" + cardType + ",PayId="
6198             + getPayId() + ",Amount=" + getAmount() ;
6199     }
6200
6201
6202
6203
6204 }
6205 -----ChequePayment.java-----
6206 package com.nt.model;
6207
6208 import javax.persistence.Column;
6209 import javax.persistence.Entity;
6210 import javax.persistence.PrimaryKeyJoinColumn;
6211 import javax.persistence.Table;
6212
6213 @Entity
6214 @Table(name="cheque_payment2")
6215 @PrimaryKeyJoinColumn(name="payment_id")
6216 public class ChequePayment extends Payment {
6217     @Column(name="chtype")
6218     private String chequeType;
6219
6220
6221     public ChequePayment(){
6222         System.out.println("ChequePayment:0-param constructor");
6223     }
6224     public ChequePayment(int payid,int amount,String chequeType){
6225         super(payid,amount);
6226         System.out.println("ChequePayment:3-param constructor");
6227         this.chequeType=chequeType;
6228     }
6229     public String getChequeType() {
6230         return chequeType;
6231     }
6232     public void setChequeType(String chequeType) {
6233         this.chequeType = chequeType;
6234     }
6235     @Override
6236     public String toString() {
6237         return "ChequePayment [chequeType=" + chequeType + ", PayId="
6238             + getPayId() + ", Amount=" + getAmount() ;
6239     }
6240
```

```

6241
6242 }
6243 -----
6244 note : InhInsertTest.java, InhSelectTest.java, HibernateUtil.java are same as previous App
6245 =====
6246 App 33 (Annotations based Inheritance mapping) (Table per concrete class)
6247 =====
6248 -----hibernate.cfg.xml-----
6249 <!DOCTYPE hibernate-configuration
6250 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
6251 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
6252
6253 <hibernate-configuration>
6254     <session-factory>
6255         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
6256         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
6257         <property name="connection.username">scott</property>
6258         <property name="connection.password">tiger</property>
6259         <property name="show_sql">>true</property>
6260         <mapping class="com.nt.model.Payment"/>
6261         <mapping class="com.nt.model.CreditCardPayment"/>
6262         <mapping class="com.nt.model.ChequePayment"/>
6263
6264     </session-factory>
6265 </hibernate-configuration>
6266 -----Payment.java-----
6267 package com.nt.model;
6268
6269 import javax.persistence.Column;
6270 import javax.persistence.DiscriminatorColumn;
6271 import javax.persistence.DiscriminatorType;
6272 import javax.persistence.Entity;
6273 import javax.persistence.Id;
6274 import javax.persistence.Inheritance;
6275 import javax.persistence.InheritanceType;
6276 import javax.persistence.Table;
6277
6278 @Table(name="payment3")
6279 @Entity
6280 @Inheritance(strategy=InheritanceType.TABLE_PER_CLASS)
6281 public class Payment {
6282     @Id
6283     @Column(name="payid")
6284     private int payId;
6285
6286     @Column(name="amount")
6287     private int amount;
6288
6289     public Payment(){
6290         System.out.println("Payment:0-param constructor");
6291     }
6292     public Payment(int payId,int amount){
6293         System.out.println("Payment:2-param constructor");
6294         this.payId=payId;
6295         this.amount=amount;
6296     }
6297
6298     public int getPayId() {
6299         return payId;
6300     }
6301
6302     public void setPayId(int payId) {
6303         this.payId = payId;
6304     }
6305 }

```

```
6306
6307     public int getAmount() {
6308         return amount;
6309     }
6310
6311     public void setAmount(int amount) {
6312         this.amount = amount;
6313     }
6314     @Override
6315     public String toString() {
6316         return "Payment [payId=" + payId + ", amount=" + amount + "]";
6317     }
6318 }
6319
6320 /* create table payment3(payid number(5) primary key, amount number(7));
6321
6322     create table credit_payment3(payid number(5) primary key,amount number(7),cctype varchar2(20))
6323
6324     create table cheque_payment3(payid number(5) primary key,amount number(7),chtype varchar2(20))
6325
6326
6327
6328
6329 */
6330 -----ChequePayment.java-----
6331 package com.nt.model;
6332
6333 import javax.persistence.Column;
6334 import javax.persistence.Entity;
6335 import javax.persistence.PrimaryKeyJoinColumn;
6336 import javax.persistence.Table;
6337
6338 @Entity
6339 @Table(name="cheque_payment3")
6340 public class ChequePayment extends Payment {
6341     @Column(name="chtype")
6342     private String chequeType;
6343
6344
6345     public ChequePayment(){
6346         System.out.println("ChequePayment:0-param constructor");
6347     }
6348     public ChequePayment(int payid,int amount,String chequeType){
6349         super(payid,amount);
6350         System.out.println("ChequePayment:3-param constructor");
6351         this.chequeType=chequeType;
6352     }
6353     public String getChequeType() {
6354         return chequeType;
6355     }
6356     public void setChequeType(String chequeType) {
6357         this.chequeType = chequeType;
6358     }
6359     @Override
6360     public String toString() {
6361         return "ChequePayment [chequeType=" + chequeType + ", PayId="
6362             + getPayId() + ", Amount=" + getAmount() ;
6363     }
6364
6365 }
6366
6367 -----CreditCardPayment.java-----
6368 package com.nt.model;
6369
6370 import javax.persistence.Column;
```

```

6371 import javax.persistence.Entity;
6372 import javax.persistence.PrimaryKeyJoinColumn;
6373 import javax.persistence.Table;
6374
6375 @Entity
6376 @Table(name="credit_payment3")
6377 public class CreditCardPayment extends Payment {
6378     @Column(name = "cctype")
6379     private String cardType;
6380
6381     public CreditCardPayment(){
6382         System.out.println("CreditCard:0-param constructor");
6383     }
6384     public CreditCardPayment(int payid,int amount,String cardType){
6385         super(payid,amount);
6386         System.out.println("CreditCard:3-param constructor");
6387         this.cardType=cardType;
6388     }
6389     public String getCardType() {
6390         return cardType;
6391     }
6392     public void setCardType(String cardType) {
6393         this.cardType = cardType;
6394     }
6395     @Override
6396     public String toString() {
6397         return "CreditCardPayment [cardType=" + cardType + ",PayId="
6398             + getPayId() + ",Amount=" + getAmount() ;
6399     }
6400 }
6401 }

```

6402 -----
6403 note : InhInsertTest.java, InhSelectTest.java, HibernateUtil.java are same as previous App

6404 =====
6405 App35 Working Large objects

```

6406 =====
6407 <?xml version='1.0' encoding='UTF-8'?>
6408 <!DOCTYPE hibernate-configuration PUBLIC
6409     "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
6410     "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
6411
6412 <hibernate-configuration>
6413     <session-factory>
6414         <property name="hibernate.connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
6415         <property name="hibernate.connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
6416         <property name="hibernate.connection.username">scott</property>
6417         <property name="hibernate.connection.password">tiger</property>
6418         <property name="dialect">org.hibernate.dialect.Oracle10gDialect</property>
6419     </session-factory>
6420 </hibernate-configuration>
6421
6422 -----Employee.hbm.xml-----
6423 <?xml version="1.0"?>
6424 <!DOCTYPE hibernate-mapping PUBLIC "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
6425     "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
6426
6427
6428
6429
6430 <hibernate-mapping>
6431     <class name="com.nt.model.EmpDetails" table="EmployeeProfile">
6432         <id name="eid" column="emp_id"/>
6433         <property name="ename" column="emp_name"/>
6434         <property name="eresume" column="emp_resume" type="clob"/>
6435         <property name="ephoto" column="emp_photo" type="blob"/>

```

```
6436     </class>
6437 </hibernate-mapping>
6438
6439
6440 <!--
6441 create table EmployeeProfile
6442 (
6443     emp_id number(5) primary key,
6444     emp_name varchar2(20),
6445     emp_resume clob,
6446     emp_photo blob
6447 );
6448 -->
6449 -----EmpBean.java-----
6450 package com.nt.model;
6451
6452 import java.sql.Blob;
6453 import java.sql.Clob;
6454
6455 public class EmpDetails
6456 {
6457
6458     private int eid;
6459     private String ename;
6460     private Clob eresume;
6461     private Blob ephoto;
6462
6463     public int getEid() {
6464         return eid;
6465     }
6466     public void setEid(int eid) {
6467         this.eid = eid;
6468     }
6469     public String getEname() {
6470         return ename;
6471     }
6472     public void setEname(String ename) {
6473         this.ename = ename;
6474     }
6475     public Clob getEresume() {
6476         return eresume;
6477     }
6478     public void setEresume(Clob eresume) {
6479         this.eresume = eresume;
6480     }
6481     public Blob getEphoto() {
6482         return ephoto;
6483     }
6484     public void setEphoto(Blob ephoto) {
6485         this.ephoto = ephoto;
6486     }
6487
6488 }
6489 -----FileUtility.java-----
6490 package com.nt.utility;
6491
6492 import java.io.BufferedReader;
6493 import java.io.File;
6494 import java.io.FileInputStream;
6495 import java.io.FileOutputStream;
6496 import java.io.FileReader;
6497 import java.io.FileWriter;
6498 import java.io.IOException;
6499 import java.io.InputStream;
6500 import java.io.Reader;
```



```
6501 import java.sql.Blob;
6502 import java.sql.Clob;
6503
6504
6505 public class FileUtility
6506 {
6507
6508
6509 public static String readFileAsString(String filePath)throws Exception
6510 {
6511     String fileData="";
6512     BufferedReader reader = new BufferedReader(new FileReader(filePath));
6513     char[] buf = new char[1024];
6514     int numRead=0;
6515     while((numRead=reader.read(buf)) != -1){
6516         String readData = String.valueOf(buf, 0, numRead);
6517         fileData+=readData;
6518     }
6519     reader.close();
6520     return fileData.toString();
6521 }//readFileAsString
6522
6523
6524 //Returns the contents of the file in a byte array.
6525 public static byte[] getByteArrayFromFile(String fpath)
6526 {
6527     try{
6528         File file=new File(fpath);
6529         InputStream is = new FileInputStream(file);
6530         long length = file.length();
6531
6532         // Create the byte array to hold the data
6533         byte[] bytes = new byte[(int)length];
6534
6535         // Read in the bytes
6536         int offset = 0;
6537         int numRead = 0;
6538         while (offset < bytes.length && (numRead=is.read(bytes, offset, bytes.length-offset)) >= 0) {
6539             offset += numRead;
6540         }
6541
6542         // Ensure all the bytes have been read in
6543         if (offset < bytes.length) {
6544             throw new IOException("Could not completely read file "+file.getName());
6545         }
6546
6547         // Close the input stream and return bytes
6548         is.close();
6549         return bytes;
6550     }catch(Exception e){return null;}
6551
6552
6553 }
6554
6555 public static String createFileFromClob(Clob clb,String fpath)
6556 {
6557     try
6558     {
6559         Reader reader=clb.getCharacterStream();
6560         File data = new File(fpath);
6561         FileWriter writer = new FileWriter(data);
6562         char[] buffer = new char[1];
6563         while (reader.read(buffer) > 0) {
6564             writer.write(buffer);
6565         }
6566     }
6567 }
```

```
6566     writer.close();
6567     }catch(Exception e) {}
6568     return "Data Stored Successfully in "+fpath;
6569 }
6570
6571 public static String createFileFromBlob(Blob blb,String fpath)
6572 {
6573     try
6574     {
6575         InputStream in=blb.getBinaryStream();
6576         FileOutputStream fos=new FileOutputStream(fpath);
6577         int bytesRead=0;
6578         byte buffer[]=new byte[4096];
6579
6580         while((bytesRead=in.read(buffer))!=-1)
6581         {
6582             fos.write(buffer,0,bytesRead);
6583         }
6584
6585         fos.close();
6586
6587     }catch(Exception e) {}
6588     return "photo stored Successfully in"+fpath;
6589 }
6590
6591 }
6592 -----HibernateUtility.java-----
6593 ----->Same<-----
6594 -----InsertClient.java-----
6595 package com.nt.test;
6596
6597 import java.sql.Blob;
6598 import java.sql.Clob;
6599
6600 import org.hibernate.Hibernate;
6601 import org.hibernate.Session;
6602 import org.hibernate.SessionFactory;
6603 import org.hibernate.Transaction;
6604 import org.hibernate.cfg.Configuration;
6605
6606 import com.nt.model.EmpDetails;
6607 import com.nt.utility.FileUtility;
6608 import com.nt.utility.HibernateUtil;
6609
6610 public class InsertClient
6611 {
6612     public static void main(String s[])throws Exception
6613     {
6614         // Get Session obj
6615         Session ses=HibernateUtil.getSession();
6616
6617         String fileContent=FileUtility.readFileAsString("c:\\store\\resume.txt");
6618         Clob resume=Hibernate.createClob(fileContent);
6619
6620
6621         byte fileByte[]=FileUtility.getBytesArrayFromFile("c:/store/koala.jpg");
6622         Blob photo=Hibernate.createBlob(fileByte);
6623
6624         EmpDetails eb=new EmpDetails();
6625         eb.setEid(1001);
6626         eb.setEname("raja");
6627         eb.setEresume(resume);
6628         eb.setEphoto(photo);
6629
6630         Transaction tx=ses.beginTransaction();
```

```

6631         ses.save(eb);
6632         tx.commit();
6633         System.out.println("Successfully Inserted");
6634
6635         HibernateUtil.closeSession();
6636
6637     }
6638 }
6639 -----RetrieveClient.java-----
6640 package com.nt.test;
6641
6642 import java.sql.Blob;
6643 import java.sql.Clob;
6644
6645 import org.hibernate.Session;
6646 import org.hibernate.SessionFactory;
6647 import org.hibernate.cfg.Configuration;
6648
6649 import com.nt.model.EmpDetails;
6650 import com.nt.utility.FileUtility;
6651 import com.nt.utility.HibernateUtil;
6652
6653
6654 public class RetrieveClient
6655 {
6656     public static void main(String s[])throws Exception
6657     {
6658         Session ses=HibernateUtil.getSession();
6659
6660         //EmpDetails eb=new EmpDetails();
6661         Class c=EmpBean.class;
6662         EmpDetails eb=(EmpDetails)ses.load(c,new Integer(1001));
6663         String name=eb.getName();
6664
6665         Clob resume=eb.getEresume();
6666         Blob photo=eb.getEphoto();
6667
6668         String res1=FileUtility.createFileFromClob(resume, "c:\\store2\\newResume.txt");
6669
6670         String res2=FileUtility.createFileFromBlob(photo, "c:/store2/newpic.jpg");
6671         System.out.println(res1);
6672         System.out.println(res2);
6673
6674         HibernateUtil.closeSession();
6675
6676     }
6677 }
6678
6679 =====
6680 App36) One to Many Association Uni-Directional
6681 =====
6682 -----hibernate.cfg.xml-----
6683 <!DOCTYPE hibernate-configuration
6684     PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
6685     "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
6686
6687 <hibernate-configuration>
6688     <session-factory>
6689         <!-- connection properties -->
6690         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
6691         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
6692         <property name="connection.username">scott</property>
6693         <property name="connection.password">tiger</property>
6694         <!-- hibernate properties -->
6695         <property name="show_sql">>true</property>

```

```
6696      <!-- mapping file names -->
6697      <mapping resource="com/nt/cfgs/User.hbm.xml"/>
6698      <mapping resource="com/nt/cfgs/PhoneNumber.hbm.xml"/>
6699  </session-factory>
6700 </hibernate-configuration>
6701 -----User.hbm.xml-----
6702 <!DOCTYPE hibernate-mapping SYSTEM
6703 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
6704
6705 <hibernate-mapping>
6706   <class name="com.nt.model.User" table="User_table">
6707     <id name="userId" column="User_id"/> <!--Singular Id field cfg -->
6708     <property name="firstName" column="FIRST_NAME"/>
6709     <set name="phones" cascade="all-delete-orphan" >
6710       <key column="unid"/>
6711       <one-to-many class="com.nt.model.PhoneNumber"/>
6712     </set>
6713   </class>
6714 </hibernate-mapping>
6715
6716 -----PhoneNumber.hbm.xml-----
6717 <!DOCTYPE hibernate-mapping SYSTEM
6718 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
6719
6720 <hibernate-mapping>
6721   <class name="com.nt.model.PhoneNumber" table="Phone_Numbers">
6722     <id name="phone" column="Phone"/>
6723     <property name="numberType" column="number_type"/>
6724   </class>
6725 </hibernate-mapping>
6726 -----User.java-----
6727 package com.nt.model;
6728
6729 import java.util.Set;
6730
6731 public class User {
6732     private int userId;
6733     private String firstName;
6734     private Set <PhoneNumber> phones;
6735
6736     public User(){
6737         System.out.println("User:0-param constructor");
6738     }
6739
6740
6741     public User(int userId, String firstName, Set<PhoneNumber> phones) {
6742         System.out.println("User: 3-param constructor");
6743         this.userId = userId;
6744         this.firstName = firstName;
6745         this.phones = phones;
6746     }
6747
6748
6749     public int getUserId() {
6750         return userId;
6751     }
6752     public void setUserId(int userId) {
6753         this.userId = userId;
6754     }
6755     public String getFirstName() {
6756         return firstName;
6757     }
6758     public void setFirstName(String firstName) {
6759         this.firstName = firstName;
6760     }
6761 }
```

```
6761     public Set<PhoneNumber> getPhones() {
6762         return phones;
6763     }
6764     public void setPhones(Set<PhoneNumber> phones) {
6765         this.phones = phones;
6766     }
6767
6768
6769     @Override
6770     public String toString() {
6771         return "User {userId=" + userId + ", firstName=" + firstName + "}";
6772     }
6773
6774     /*-----parent table-----
6775     CREATE TABLE USER_TABLE(
6776         USER_ID NUMBER PRIMARY KEY,
6777         FIRST_NAME VARCHAR2(20));
6778
6779     -----child table-----
6780     CREATE TABLE PHONE_NUMBERS (
6781         NUMBER_TYPE VARCHAR2(20),
6782         PHONE NUMBER (10),
6783         UNID number references USER_TABLE(USER_ID)); */
6784
6785 }
6786 -----PhoneNumber.java-----
6787 package com.nt.model;
6788
6789 public class PhoneNumber {
6790     private long phone;
6791     private String numberType;
6792
6793     public PhoneNumber(){
6794         System.out.println("PhoneNumber:0-param constructor");
6795     }
6796
6797     public PhoneNumber(long phone, String numberType) {
6798         System.out.println("PhoneNumber:2-param constructor");
6799         this.phone = phone;
6800         this.numberType = numberType;
6801     }
6802
6803     public long getPhone() {
6804         return phone;
6805     }
6806
6807     public void setPhone(long phone) {
6808         this.phone = phone;
6809     }
6810
6811     public String getNumberType() {
6812         return numberType;
6813     }
6814
6815     public void setNumberType(String numberType) {
6816         this.numberType = numberType;
6817     }
6818
6819     @Override
6820     public String toString() {
6821         return "PhoneNumber [phone=" + phone + ", numberType=" + numberType
6822             + "]";
6823     }
6824
6825 }
```

```
6826 }
6827 -----HibernateUtility.java-----
6828 --->same<---
6829 -----InsertTest.java-----
6830 package com.nt.test;
6831
6832 import java.util.HashSet;
6833 import java.util.Set;
6834
6835 import org.hibernate.Transaction;
6836
6837 import org.hibernate.Session;
6838
6839 import com.nt.model.PhoneNumber;
6840 import com.nt.model.User;
6841 import com.nt.utility.HibernateUtil;
6842
6843 public class InsertTest {
6844
6845     public static void main(String[] args) {
6846         //Get Session obj
6847         Session ses=HibernateUtil.getSession();
6848         //Save objs (parent-child)
6849         //parent1
6850         User user1=new User();
6851         user1.setUserId(101);
6852         user1.setFirstName("raja");
6853         //child1
6854         PhoneNumber ph1=new PhoneNumber(9999999,"residence");
6855         //child2
6856         PhoneNumber ph2=new PhoneNumber(8888888,"office");
6857         //Line child1,child2 with parent1
6858         Set <PhoneNumber>s=new HashSet();
6859         s.add(ph1); s.add(ph2);
6860         user1.setPhones(s);
6861         Transaction tx=null;
6862         try{
6863             tx=ses.beginTransaction();
6864             ses.save(user1);
6865             tx.commit();
6866         }
6867         catch(Exception e){
6868             tx.rollback();
6869             e.printStackTrace();
6870         }
6871
6872         //close Session
6873         HibernateUtil.closeSession();
6874     } //main
6875 } //class
6876 -----SelectTest.java-----
6877 package com.nt.test;
6878
6879 import java.util.HashSet;
6880 import java.util.Set;
6881
6882 import org.hibernate.Transaction;
6883
6884 import org.hibernate.Session;
6885
6886 import com.nt.model.PhoneNumber;
6887 import com.nt.model.User;
6888 import com.nt.utility.HibernateUtil;
6889
6890 public class InsertTest {
```

```
6891
6892     public static void main(String[] args) {
6893         //Get Session obj
6894         Session ses=HibernateUtil.getSession();
6895
6896         Query q1=ses.createQuery("from User");//gives parent table records
6897         //and associated child table records
6898         List<User> l=q1.list();
6899         for(User u1:l)
6900         {
6901             System.out.println("parent:"+u1.getUserId()+" "+u1.getFirstName());
6902             Set s=u1.getPhones();
6903             Iterator<PhoneNumber> it=s.iterator();
6904             while(it.hasNext())
6905             {
6906                 PhoneNumber ph=it.next();
6907                 System.out.println("\t\t\t Child:"+ph.getPhone()+" "+ph.getNumberType()
6908                     +" "+ph.getId());
6909             }//while
6910         }//for
6911
6912         Query q1=ses.createQuery("from PhoneNumber");// gives only child table records
6913         List<PhoneNumber> l1=q1.list();
6914         for(PhoneNumber ph:l1)
6915         {
6916             System.out.println(ph.getNumberType()+" "+ph.getPhone()+
6917                 " "+ph.getId());
6918         }//for
6919         ses.close();
6920     }//main
6921 }//class
6922 -----DeleteTest.java-----
6923 package com.nt.test;
6924
6925 import java.util.Set;
6926
6927 import org.hibernate.Transaction;
6928
6929 import org.hibernate.Session;
6930
6931 import com.nt.model.PhoneNumber;
6932 import com.nt.model.User;
6933 import com.nt.utility.HibernateUtil;
6934
6935 public class DeleteTest {
6936
6937     public static void main(String[] args) {
6938         // Get Session
6939         Session ses=HibernateUtil.getSession();
6940
6941         // Load Parent obj
6942         User user=(User)ses.load(User.class,101);
6943         // get All childs
6944         Set<PhoneNumber> phones=user.getPhones();
6945         //Load the object that u want to delete
6946         PhoneNumber ph1=(PhoneNumber)ses.get(PhoneNumber.class,(long)99999999);
6947         //Delete loaded from Set
6948         Transaction tx=null;
6949         try{
6950             tx=ses.beginTransaction();
6951             phones.remove(ph1);
6952             tx.commit();
6953         }
6954         catch(Exception e){
6955             tx.rollback();
```

```

6956     }
6957 }
6958
6959 }
6960 =====
6961 App 36)One to Many Association (Uni-Directional) (java.util.List)
6962 =====
6963 -----hibernate.cfg.xml-----
6964 <!DOCTYPE hibernate-configuration
6965     PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
6966     "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
6967
6968 <hibernate-configuration>
6969     <session-factory>
6970         <!-- connection properties -->
6971         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
6972         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
6973         <property name="connection.username">scott</property>
6974         <property name="connection.password">tiger</property>
6975         <!-- hibernate properties -->
6976         <property name="show_sql">>true</property>
6977         <!-- mapping file names -->
6978         <mapping resource="com/nt/cfgs/User.hbm.xml"/>
6979         <mapping resource="com/nt/cfgs/PhoneNumber.hbm.xml"/>
6980     </session-factory>
6981 </hibernate-configuration>
6982 -----User.hbm.xml-----
6983 <!DOCTYPE hibernate-mapping SYSTEM
6984     "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
6985
6986 <hibernate-mapping>
6987     <class name="com.nt.model.User" table="User_table1">
6988         <id name="userId" column="User_id"/> <!--Singular Id field cfg -->
6989         <property name="firstName" column="FIRST_NAME"/>
6990         <list name="phones" cascade="all-delete-orphan" >
6991             <key column="unid"/>
6992             <list-index column="lst_indx"/>
6993             <one-to-many class="com.nt.model.PhoneNumber"/>
6994         </list>
6995     </class>
6996 </hibernate-mapping>
6997
6998 -----PhoneNumber.hbm.xml-----
6999 <!DOCTYPE hibernate-mapping SYSTEM
7000     "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
7001
7002 <hibernate-mapping>
7003     <class name="com.nt.model.PhoneNumber" table="Phone_Numbers1">
7004         <id name="phone" column="Phone"/>
7005         <property name="numberType" column="number_type"/>
7006     </class>
7007 </hibernate-mapping>
7008
7009 -----User.java-----
7010 package com.nt.model;
7011
7012 import java.util.List;
7013 import java.util.Set;
7014
7015 public class User {
7016     private int userId;
7017     private String firstName;
7018     private List <PhoneNumber> phones;
7019
7020     public User(){

```



```

7021         System.out.println("User:0-param constructor");
7022     }
7023
7024
7025     public User(int userId, String firstName, List<PhoneNumber> phones) {
7026         System.out.println("User: 3-param constructor");
7027         this.userId = userId;
7028         this.firstName = firstName;
7029         this.phones = phones;
7030     }
7031
7032
7033     public int getUserId() {
7034         return userId;
7035     }
7036     public void setUserId(int userId) {
7037         this.userId = userId;
7038     }
7039     public String getFirstName() {
7040         return firstName;
7041     }
7042     public void setFirstName(String firstName) {
7043         this.firstName = firstName;
7044     }
7045     public List<PhoneNumber> getPhones() {
7046         return phones;
7047     }
7048     public void setPhones(List<PhoneNumber> phones) {
7049         this.phones = phones;
7050     }
7051
7052
7053     @Override
7054     public String toString() {
7055         return "User [userId=" + userId + ", firstName=" + firstName + "]";
7056     }
7057
7058     /* -----parent table-----
7059     CREATE TABLE USER_TABLE1(
7060         USER_ID NUMBER PRIMARY KEY,
7061         FIRST_NAME VARCHAR2(20));
7062
7063     -----child table-----
7064     CREATE TABLE PHONE_NUMBERS1 (
7065         NUMBER_TYPE VARCHAR2(20),
7066         PHONE NUMBER (10),
7067         UNID number references USER_TABLE1(USER_ID),
7068         lst_indx number(6)); */
7069 }
7070 -----PhoneNumber.java-----
7071 package com.nt.model;
7072
7073 public class PhoneNumber {
7074     private long phone;
7075     private String numberType;
7076
7077     public PhoneNumber(){
7078         System.out.println("PhoneNumber:0-param constructor");
7079     }
7080
7081     public PhoneNumber(long phone, String numberType) {
7082         System.out.println("PhoneNumber:2-param constructor");
7083         this.phone = phone;
7084         this.numberType = numberType;
7085     }

```

```
7086
7087     public long getPhone() {
7088         return phone;
7089     }
7090
7091     public void setPhone(long phone) {
7092         this.phone = phone;
7093     }
7094
7095     public String getNumberType() {
7096         return numberType;
7097     }
7098
7099     public void setNumberType(String numberType) {
7100         this.numberType = numberType;
7101     }
7102
7103     @Override
7104     public String toString() {
7105         return "PhoneNumber [phone=" + phone + ", numberType=" + numberType
7106             + "]\n";
7107     }
7108 }
```

```
7109 -----InsertTest.java-----
7110 package com.nt.test;
7111
7112 import java.util.ArrayList;
7113 import java.util.List;
7114 import org.hibernate.Session;
7115 import org.hibernate.Transaction;
7116 import com.nt.model.PhoneNumber;
7117 import com.nt.model.User;
7118 import com.nt.utility.HibernateUtil;
7119
7120 public class InsertTest {
7121
7122     public static void main(String[] args) {
7123         //Get Session obj
7124         Session ses=HibernateUtil.getSession();
7125         //Save objs (parent-child)
7126         //parent1
7127         User user1=new User();
7128         user1.setUserId(101);
7129         user1.setFirstName("raja");
7130         //child1
7131         PhoneNumber ph1=new PhoneNumber(99999999,"residence");
7132         //child2
7133         PhoneNumber ph2=new PhoneNumber(88888888,"office");
7134         //Line child1,child2 with parent1
7135         List <PhoneNumber>list=new ArrayList<PhoneNumber>();
7136         list.add(ph1);list.add(ph2);
7137         user1.setPhones(list);
7138         Transaction tx=null;
7139         try{
7140             tx=ses.beginTransaction();
7141             ses.save(user1);
7142             tx.commit();
7143         }
7144         catch(Exception e){
7145             tx.rollback();
7146             e.printStackTrace();
7147         }
7148
7149         //close Session
7150         HibernateUtil.closeSession();
7151     }
7152 }
```

```

7151     }//main
7152 }//class
7153
7154 -----DeleteTest.java-----
7155 package com.nt.utility;
7156
7157 import java.util.List;
7158 import org.hibernate.Session;
7159 import org.hibernate.Transaction;
7160 import com.nt.model.PhoneNumber;
7161 import com.nt.model.User;
7162 import com.nt.utility.HibernateUtil;
7163
7164 public class DeleteTest {
7165
7166     public static void main(String[] args) {
7167         // Get Session
7168         Session ses=HibernateUtil.getSession();
7169         // Load Parent obj
7170         User user=(User)ses.load(User.class,101);
7171         // get All childs
7172         List<PhoneNumber> phones=user.getPhones();
7173         //Load the object that want to delete
7174         Transaction tx=null;
7175         try{
7176             tx=ses.beginTransaction();
7177             phones.remove(0);
7178             tx.commit();
7179         }
7180         catch(Exception e){
7181             tx.rollback();
7182         }
7183     }
7184
7185 }

```

```

7186 =====
7187 App 37) One-to-Many Uni Association mapping (Using <bag>)
7188 =====
7189 -----hibernate.cfg.xml-----
7190 <!--> same as previous App-->
7191 -----User.hbm.xml-----
7192 <!DOCTYPE hibernate-mapping SYSTEM
7193 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
7194
7195 <hibernate-mapping>
7196     <class name="com.nt.model.User" table="User_table2">
7197         <id name="userId" column="User_id"/> <!--Singular Id field cfg -->
7198         <property name="firstName" column="FIRST_NAME"/>
7199         <bag name="phones" cascade="all-delete-orphan" >
7200             <key column="unid"/>
7201             <one-to-many class="com.nt.model.PhoneNumber"/>
7202         </bag>
7203     </class>
7204 </hibernate-mapping>
7205 /* -----parent table-----
7206 CREATE TABLE USER_TABLE2(
7207     USER_ID NUMBER PRIMARY KEY,
7208     FIRST_NAME VARCHAR2(20));
7209
7210 -----child table-----
7211 CREATE TABLE PHONE_NUMBERS2 (
7212     NUMBER_TYPE VARCHAR2(20),
7213     PHONE NUMBER (10),
7214     UNID number references USER_TABLE2(USER_ID)); */
7215

```

```

7216 -----PhoneNumber.hbm.xml-----
7217 <!DOCTYPE hibernate-mapping SYSTEM
7218 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
7219
7220 <hibernate-mapping>
7221     <class name="com.nt.model.PhoneNumber" table="Phone_Numbers2">
7222         <id name="phone" column="Phone"/>
7223         <property name="numberType" column="number_type"/>
7224     </class>
7225 </hibernate-mapping>
7226 -----User.java-----
7227 <-->same as previous App
7228 -----PhoneNumber.java-----
7229 package com.nt.model;
7230
7231 public class PhoneNumber {
7232     private long phone;
7233     private String numberType;
7234
7235     public PhoneNumber(){
7236         System.out.println("PhoneNumber:0-param constructor");
7237     }
7238
7239     public PhoneNumber(long phone, String numberType) {
7240         System.out.println("PhoneNumber:2-param constructor");
7241         this.phone = phone;
7242         this.numberType = numberType;
7243     }
7244
7245     public long getPhone() {
7246         return phone;
7247     }
7248
7249     public void setPhone(long phone) {
7250         this.phone = phone;
7251     }
7252
7253     public String getNumberType() {
7254         return numberType;
7255     }
7256
7257     public void setNumberType(String numberType) {
7258         this.numberType = numberType;
7259     }
7260
7261     @Override
7262     public String toString() {
7263         return "PhoneNumber [phone=" + phone + ", numberType=" + numberType
7264             + " ]";
7265     }
7266
7267 }
7268 -----HibernateUtil.java-----
7269 <---Same--->
7270 -----InsertTest.java-----
7271 --->Same as previous App<----
7272
7273 =====
7274 App38 ) One to many Association Uni-Directional (Using java.util.Map)
7275 =====
7276 -----hibernate.cfg.xml-----
7277 <---Same previous App-->
7278 -----User.hbm.xml-----
7279 <!DOCTYPE hibernate-mapping SYSTEM
7280 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >

```

```
7281 -----User.hbm.xml-----
7282 <!DOCTYPE hibernate-mapping SYSTEM
7283 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
7284
7285 <hibernate-mapping>
7286   <class name="com.nt.model.User" table="User_table3">
7287     <id name="userId" column="User_id"/> <!--Singular Id field cfg -->
7288     <property name="firstName" column="FIRST_NAME"/>
7289
7290     <map name="phones" cascade="all-delete-orphan" >
7291       <key column="unid"/>
7292       <map-key column="map_indx" type="java.lang.String"/>
7293       <one-to-many class="com.nt.model.PhoneNumber"/>
7294     </map>
7295   </class>
7296 </hibernate-mapping>
7297 -----PhoneNumber.hbm.xml-----
7298 <!DOCTYPE hibernate-mapping SYSTEM
7299 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
7300
7301 <hibernate-mapping>
7302   <class name="com.nt.model.PhoneNumber" table="Phone_Numbers3">
7303     <id name="phone" column="Phone"/>
7304     <property name="numberType" column="number_type"/>
7305   </class>
7306 </hibernate-mapping>
7307
7308 -----User.java-----
7309 package com.nt.model;
7310
7311 import java.util.Map;
7312 import java.util.Set;
7313
7314 public class User {
7315   private int userId;
7316   private String firstName;
7317   private Map <String,PhoneNumber> phones;
7318
7319   public User(){
7320     System.out.println("User:0-param constructor");
7321   }
7322
7323
7324   public User(int userId, String firstName, Map<String,PhoneNumber> phones) {
7325     System.out.println("User: 3-param constructor");
7326     this.userId = userId;
7327     this.firstName = firstName;
7328     this.phones = phones;
7329   }
7330
7331
7332   public int getUserId() {
7333     return userId;
7334   }
7335   public void setUserId(int userId) {
7336     this.userId = userId;
7337   }
7338   public String getFirstName() {
7339     return firstName;
7340   }
7341   public void setFirstName(String firstName) {
7342     this.firstName = firstName;
7343   }
7344   public Map<String,PhoneNumber> getPhones() {
7345     return phones;
```

```
7346     }
7347     public void setPhones(Map<String,PhoneNumber> phones) {
7348         this.phones = phones;
7349     }
7350
7351
7352     @Override
7353     public String toString() {
7354         return "User [userId=" + userId + ", firstName=" + firstName + "]";
7355     }
7356
7357     /* -----parent table-----
7358     CREATE TABLE USER_TABLE3(
7359         USER_ID NUMBER PRIMARY KEY,
7360         FIRST_NAME VARCHAR2(20));
7361
7362     -----child table-----
7363     CREATE TABLE PHONE_NUMBERS3 (
7364         NUMBER_TYPE VARCHAR2(20),
7365         PHONE NUMBER (10),
7366         UNID number references USER_TABLE3(USER_ID),
7367         map_idx varchar2(10)); */
7368 }
7369
7370 -----PhoneNumber.java-----
7371 package com.nt.model;
7372
7373 public class PhoneNumber {
7374     private long phone;
7375     private String numberType;
7376
7377     public PhoneNumber(){
7378         System.out.println("PhoneNumber:0-param constructor");
7379     }
7380
7381     public PhoneNumber(long phone, String numberType) {
7382         System.out.println("PhoneNumber:2-param constructor");
7383         this.phone = phone;
7384         this.numberType = numberType;
7385     }
7386
7387     public long getPhone() {
7388         return phone;
7389     }
7390
7391     public void setPhone(long phone) {
7392         this.phone = phone;
7393     }
7394
7395     public String getNumberType() {
7396         return numberType;
7397     }
7398
7399     public void setNumberType(String numberType) {
7400         this.numberType = numberType;
7401     }
7402
7403     @Override
7404     public String toString() {
7405         return "PhoneNumber [phone=" + phone + ", numberType=" + numberType
7406             + " ]";
7407     }
7408 }
7409 -----InsertTest.java-----
7410 package com.nt.test;
```

```
7411
7412 import java.util.HashMap;
7413 import java.util.Map;
7414
7415 import org.hibernate.Session;
7416 import org.hibernate.Transaction;
7417
7418 import com.nt.model.PhoneNumber;
7419 import com.nt.model.User;
7420 import com.nt.utility.HibernateUtil;
7421
7422 public class InsertTest {
7423
7424     public static void main(String[] args) {
7425         //Get Session obj
7426         Session ses=HibernateUtil.getSession();
7427         //Save objs (parent-child)
7428         //parent1
7429         User user1=new User();
7430         user1.setUserId(101);
7431         user1.setFirstName("raja");
7432         //child1
7433         PhoneNumber ph1=new PhoneNumber(99999999,"residence");
7434         //child2
7435         PhoneNumber ph2=new PhoneNumber(88888888,"office");
7436         //Link child1,child2 with parent1
7437         Map <String,PhoneNumber>map=new HashMap<String,PhoneNumber>();
7438         map.put("phone1",ph1); map.put("phone2",ph2);
7439         user1.setPhones(map);
7440         Transaction tx=null;
7441         try{
7442             tx=ses.beginTransaction();
7443             ses.save(user1);
7444             tx.commit();
7445         }
7446         catch(Exception e){
7447             tx.rollback();
7448             e.printStackTrace();
7449         }
7450
7451         //close Session
7452         HibernateUtil.closeSession();
7453     } //main
7454 } //class
7455
7456 -----DeleteTest.java-----
7457 package com.nt.test;
7458
7459 import java.util.Map;
7460
7461 import org.hibernate.Session;
7462 import org.hibernate.Transaction;
7463
7464 import com.nt.model.PhoneNumber;
7465 import com.nt.model.User;
7466 import com.nt.utility.HibernateUtil;
7467
7468 public class DeleteTest {
7469
7470     public static void main(String[] args) {
7471         // Get Session
7472         Session ses=HibernateUtil.getSession();
7473
7474         // Load Parent obj
7475         User user=(User)ses.load(User.class,101);
```

```
7476 // get All childs
7477 Map<String,PhoneNumber> phones=user.getPhones();
7478 //Delete loaded obj from Map
7479 Transaction tx=null;
7480 try{
7481     tx=ses.beginTransaction();
7482     phones.remove("phone1");
7483     tx.commit();
7484 }
7485 catch(Exception e){
7486     tx.rollback();
7487 }
7488 }
7489 }
7490
7491 =====
7492 App39 ) Many to One Uni-Directional Association
7493 =====
7494 <!DOCTYPE hibernate-configuration
7495 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
7496 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
7497
7498 <hibernate-configuration>
7499     <session-factory>
7500         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
7501         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
7502         <property name="connection.username">scott</property>
7503         <property name="connection.password">tiger</property>
7504         <property name="show_sql">>true</property>
7505         <mapping resource="com/nt/cfgs/EmpDetails.hbm.xml"/>
7506         <mapping resource="com/nt/cfgs/Department.hbm.xml"/>
7507
7508     </session-factory>
7509 </hibernate-configuration>
7510
7511 -----Department.java-----
7512 //Department.java (parent)
7513 package com.nt.model;
7514
7515 public class Department {
7516     private int deptno;
7517     private String deptname;
7518     private String depthead;
7519
7520     public Department(){
7521         System.out.println("Department:0-param constructor");
7522     }
7523
7524     public Department(String deptname, String depthead) {
7525         System.out.println("Department:2-param constructor");
7526         this.deptname = deptname;
7527         this.depthead = depthead;
7528     }
7529
7530     public int getDeptno() {
7531         return deptno;
7532     }
7533
7534     public void setDeptno(int deptno) {
7535         this.deptno = deptno;
7536     }
7537
7538     public String getDeptname() {
7539         return deptname;
7540     }
7541 }
```



```
7541
7542     public void setDeptname(String deptname) {
7543         this.deptname = deptname;
7544     }
7545
7546     public String getDepthead() {
7547         return depthead;
7548     }
7549
7550     public void setDepthead(String depthead) {
7551         this.depthead = depthead;
7552     }
7553
7554     @Override
7555     public String toString() {
7556         return "Department [deptno=" + deptno + ", deptname=" + deptname
7557             + ", depthead=" + depthead + "]";
7558     }
7559 }
7560 -----EmpDetails.java-----
7561 //EmpDetails.java (child class)
7562 package com.nt.model;
7563
7564 public class EmpDetails {
7565     private int eno;
7566     private String ename;
7567     private float salary;
7568     private Department dept;
7569
7570     public EmpDetails(){
7571         System.out.println("EmpDetails:0-param constructor");
7572     }
7573
7574     public EmpDetails(String ename, float salary) {
7575         System.out.println("EmpDetails:2-param constructor");
7576         this.ename = ename;
7577         this.salary = salary;
7578     }
7579
7580     public int getEno() {
7581         return eno;
7582     }
7583
7584     public void setEno(int eno) {
7585         this.eno = eno;
7586     }
7587
7588     public String getEname() {
7589         return ename;
7590     }
7591
7592     public void setEname(String ename) {
7593         this.ename = ename;
7594     }
7595
7596     public float getSalary() {
7597         return salary;
7598     }
7599
7600     public void setSalary(float salary) {
7601         this.salary = salary;
7602     }
7603
7604     public Department getDept() {
7605         return dept;
7606     }
7607 }
```

```

7606     }
7607
7608     public void setDept(Department dept) {
7609         this.dept = dept;
7610     }
7611
7612     @Override
7613     public String toString() {
7614         return "EmpDetails [eno=" + eno + ", ename=" + ename + ", salary="
7615             + salary + "]\n";
7616     }
7617
7618 }
7619 /* create table Department(deptno number(5) primary key,
7620                             deptname varchar2(20),
7621                             depthead varchar2(20));
7622
7623     create table EmpDetails(eno number(5) primary key,
7624                             ename varchar2(20),
7625                             salary number(7,2),
7626                             deptno number(5) references department(deptno));
7627
7628 */
7629
7630
7631 -----Department.hbm.xml-----
7632 <!DOCTYPE hibernate-mapping SYSTEM
7633 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
7634
7635 <hibernate-mapping>
7636     <class name="com.nt.model.Department" table="Department">
7637         <id name="deptno" > <!--Singular Id field cfg -->
7638             <generator class="increment"/>
7639         </id>
7640         <property name="deptname" />
7641         <property name="depthead"/>
7642     </class>
7643 </hibernate-mapping>
7644
7645 -----EmpDetails.hbm.xml-----
7646 <!DOCTYPE hibernate-mapping SYSTEM
7647 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
7648
7649 <hibernate-mapping>
7650     <class name="com.nt.model.EmpDetails" table="EmpDetails">
7651         <id name="eno">
7652             <generator class="increment"/>
7653         </id> <!--Singular Id field cfg -->
7654         <property name="ename" />
7655         <property name="salary" />
7656         <many-to-one name="dept" class="com.nt.model.Department"
7657             cascade="all" lazy="no-proxy" column="deptno" inverse="true"/>
7658     </class>
7659 </hibernate-mapping>
7660
7661 -----M2ODAO.java-----
7662 package com.nt.dao;
7663
7664 public interface M2ODAO {
7665     public void createEmployeesWithDept();
7666     public void addNewEmployeeInDept();
7667     public void listEmployeesWithDept();
7668     public void deleteEmployeeswithDept();
7669     public void addExistingEmployeeToNewDept();
7670

```

```
7671     public void deleteEmployeeWithOutDeletingDept();
7672
7673 }
7674 -----M2ODAOImpl.java-----
7675 package com.nt.dao;
7676
7677 import java.util.List;
7678
7679 import org.hibernate.Query;
7680 import org.hibernate.Session;
7681 import org.hibernate.Transaction;
7682
7683 import com.nt.model.Department;
7684 import com.nt.model.EmpDetails;
7685 import com.nt.utility.HibernateUtil;
7686
7687 public class M2ODAOImpl implements M2ODAO{
7688
7689     @Override
7690     public void createEmployeesWithDept() {
7691         // Get Session obj
7692         Session ses=HibernateUtil.getSession();
7693         // create parent obj
7694         Department dept=new Department("Accounts","SMITH");
7695         //create Employee objs (child objs)
7696         EmpDetails emp1=new EmpDetails("raja",9000);
7697         EmpDetails emp2=new EmpDetails("ravi",8000);
7698         // Build Many 2 One relation
7699         emp1.setDept(dept);
7700         emp2.setDept(dept);
7701         //save obj
7702         Transaction tx=null;
7703         try{
7704             tx=ses.beginTransaction();
7705             ses.save(emp1); ses.save(emp2);
7706             tx.commit();
7707         }
7708         catch(Exception e){
7709             tx.rollback();
7710         }
7711     }
7712
7713 }
7714
7715 @Override
7716 public void listEmployeesWithDept() {
7717     //Get Session obj
7718     Session ses=HibernateUtil.getSession();
7719     // get chids with parents
7720     Query query=ses.createQuery("from EmpDetails");
7721     List<EmpDetails> list=query.list();
7722     for(EmpDetails emp:list){
7723         System.out.println("Child--->" + emp);
7724         // get Department of each employee
7725         Department dept=emp.getDept();
7726         System.out.println("Parent--->" + dept);
7727     }
7728
7729 }
7730
7731 }
7732
7733 @Override
7734 public void deleteEmployeeswithDept() {
7735     // Get Hb Session obj
7736     Session ses=HibernateUtil.getSession();
```

```
7736      //Load one child obj
7737      EmpDetails emp=(EmpDetails)ses.get(EmpDetails.class,2);
7738      Transaction tx=null;
7739      try{
7740          tx=ses.beginTransaction();
7741          ses.delete(emp);
7742          tx.commit();
7743      }
7744      catch(Exception e){
7745          tx.rollback();
7746      }
7747  }
7748
7749
7750
7751  @Override
7752  public void addNewEmployeeInDept() {
7753      // Get Session
7754      Session ses=HibernateUtil.getSession();
7755      //Load existing Department obj
7756      Department dept=(Department)ses.get(Department.class,1);
7757      //create new Employee
7758      EmpDetails emp=new EmpDetails("Ramesh",7000);
7759      emp.setDept(dept);
7760      Transaction tx=null;
7761      try{
7762          tx=ses.beginTransaction();
7763          ses.save(emp);
7764          tx.commit();
7765      }//try
7766      catch(Exception e){
7767          tx.rollback();
7768      }
7769
7770
7771  }
7772
7773  @Override
7774  public void addExistingEmployeeToNewDept() {
7775      // Get Session
7776      Session ses=HibernateUtil.getSession();
7777      // create new Department
7778      Department dept1=new Department("IT","ALLEN");
7779      // get Existing Employee
7780      EmpDetails emp=(EmpDetails)ses.get(EmpDetails.class, 1);
7781      emp.setDept(dept1);
7782      Transaction tx=null;
7783      try{
7784          tx=ses.beginTransaction();
7785          ses.update(emp);
7786          tx.commit();
7787      }
7788      catch(Exception e){
7789          tx.rollback();
7790      }
7791
7792
7793  }
7794
7795  @Override
7796  public void deleteEmployeeWithOutDeletingDept() {
7797      // Get session obj
7798      Session ses=HibernateUtil.getSession();
7799
7800      // To delete only child object.
```

```
7801      Query q1=ses.createQuery("delete from EmpDetails where eno=:id");
7802      q1.setInteger("id",2);
7803      Transaction tx=null;
7804      try{
7805          tx=ses.beginTransaction();
7806          int result=q1.executeUpdate();
7807          System.out.println(result+" no.of records are deleted");
7808          tx.commit();
7809      }
7810      catch(Exception e){
7811          tx.rollback();
7812      }
7813
7814
7815
7816
7817
7818      /* // Get child obj (Execute this code with cascade="none" to delete only child)
7819      EmpDetails emp=(EmpDetails)sess.get(EmpDetails.class, 1);
7820      Transaction tx=null;
7821      try{
7822          tx=ses.beginTransaction();
7823          ses.delete(emp);
7824          tx.commit();
7825      }
7826      catch(Exception e){
7827          tx.rollback();
7828      } */
7829
7830
7831
7832  }
7833
7834  }
7835  -----M2ODAOFactory.java-----
7836  package com.nt.dao;
7837
7838  public class M2ODAOFactory {
7839
7840      public static M2ODAO getDAOInstance(){
7841          return new M2ODAOImpl();
7842      }
7843
7844  }
7845  -----HibernateUtil.java-----
7846  ---->same<----
7847  -----ClinetApp.java-----
7848  package com.nt.test;
7849
7850  import com.nt.dao.M2ODAO;
7851  import com.nt.dao.M2ODAOFactory;
7852
7853  public class ClientApp {
7854
7855      public static void main(String[] args) {
7856          // get DAO obj
7857          M2ODAO dao=M2ODAOFactory.getDAOInstance();
7858          //call methods
7859          //dao.createEmployeesWithDept();
7860          //dao.addNewEmployeeInDept();
7861          // dao.addExistingEmployeeToNewDept();
7862          //dao.listEmployeesWithDept();
7863          //dao.deleteEmployeeswithDept();
7864          dao.deleteEmployeeWithOutDeletingDept();
7865      }
```

```

7866 }
7867 =====
7868 App40) One to Many Bi-Directional Association
7869 =====
7870 -----hibernate.cfg.xml-----
7871 <!DOCTYPE hibernate-configuration
7872 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
7873 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
7874
7875 <hibernate-configuration>
7876     <session-factory>
7877         <!-- connection properties -->
7878         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
7879         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
7880         <property name="connection.username">scott</property>
7881         <property name="connection.password">tiger</property>
7882         <!-- hibernate properties -->
7883         <property name="show_sql">>true</property>
7884         <!-- mapping file names -->
7885         <mapping resource="com/nt/cfgs/User.hbm.xml"/>
7886         <mapping resource="com/nt/cfgs/PhoneNumber.hbm.xml"/>
7887     </session-factory>
7888 </hibernate-configuration>
7889 -----User.hbm.xml-----
7890 <!DOCTYPE hibernate-mapping SYSTEM
7891 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
7892
7893 <hibernate-mapping>
7894     <class name="com.nt.model.User" table="User_table">
7895         <id name="userId" column="User_id"/> <!--Singular Id field cfg -->
7896         <property name="firstName" column="FIRST_NAME"/>
7897         <set name="phones" cascade="all-delete-orphan" lazy="true" inverse="true" fetch="join" >
7898             <key column="unid"/>
7899             <one-to-many class="com.nt.model.PhoneNumber"/>
7900         </set>
7901     </class>
7902 </hibernate-mapping>
7903 -----PhoneNumber.hbm.xml-----
7904 <!DOCTYPE hibernate-mapping SYSTEM
7905 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
7906
7907 <hibernate-mapping>
7908     <class name="com.nt.model.PhoneNumber" table="Phone_Numbers">
7909         <id name="phone" column="Phone"/>
7910         <property name="numberType" column="number_type"/>
7911     <many-to-one name="parent" class="com.nt.model.User" cascade="all" lazy="proxy" column="unid" />
7912 </class>
7913 </hibernate-mapping>
7914 -----User.java-----
7915 package com.nt.model;
7916
7917 import java.util.Set;
7918
7919 public class User {
7920     private int userId;
7921     private String firstName;
7922     private Set <PhoneNumber> phones;
7923
7924     public User(){
7925         System.out.println("User:0-param constructor");
7926     }
7927
7928
7929     public User(int userId, String firstName) {
7930         System.out.println("User: 2-param constructor");

```

```
7931         this.userId = userId;
7932         this.firstName = firstName;
7933     }
7934
7935
7936     public int getUserId() {
7937         return userId;
7938     }
7939     public void setUserId(int userId) {
7940         this.userId = userId;
7941     }
7942     public String getFirstName() {
7943         return firstName;
7944     }
7945     public void setFirstName(String firstName) {
7946         this.firstName = firstName;
7947     }
7948     public Set<PhoneNumber> getPhones() {
7949         return phones;
7950     }
7951     public void setPhones(Set<PhoneNumber> phones) {
7952         this.phones = phones;
7953     }
7954
7955
7956     @Override
7957     public String toString() {
7958         return "User [userId=" + userId + ", firstName=" + firstName + "];";
7959     }
7960
7961     /* -----parent table-----
7962     CREATE TABLE USER_TABLE(
7963         USER_ID NUMBER PRIMARY KEY,
7964         FIRST_NAME VARCHAR2(20));
7965
7966     -----child table-----
7967     CREATE TABLE PHONE_NUMBERS (
7968         NUMBER_TYPE VARCHAR2(20),
7969         PHONE NUMBER (10),
7970         UNID number references USER_TABLE(USER_ID)); */
7971
7972 }
7973 -----PhoneNumber.java-----
7974 package com.nt.model;
7975
7976 public class PhoneNumber {
7977     private long phone;
7978     private String numberType;
7979     private User parent;
7980
7981
7982     public PhoneNumber(){
7983         System.out.println("PhoneNumber:0-param constructor");
7984     }
7985
7986     public PhoneNumber(long phone, String numberType) {
7987         System.out.println("PhoneNumber:2-param constructor");
7988         this.phone = phone;
7989         this.numberType = numberType;
7990     }
7991
7992     public long getPhone() {
7993         return phone;
7994     }
7995 }
```

```
7996     public void setPhone(long phone) {
7997         this.phone = phone;
7998     }
7999
8000     public String getNumberType() {
8001         return numberType;
8002     }
8003
8004     public void setNumberType(String numberType) {
8005         this.numberType = numberType;
8006     }
8007
8008     public User getParent() {
8009         return parent;
8010     }
8011
8012     public void setParent(User parent) {
8013         this.parent = parent;
8014     }
8015
8016     @Override
8017     public String toString() {
8018         return "PhoneNumber [phone=" + phone + ", numberType=" + numberType
8019             + "]";
8020     }
8021 }
8022 }
8023 -----O2MDAO.java-----
8024 package com.nt.dao;
8025
8026 public interface O2MDAO {
8027     public void InsertDataUsingUser();
8028     public void InsertDataUsingPhoneNumber();
8029     public void ListRecordsUsingUser();
8030     public void ListRecordsUsingPhoneNumber();
8031     public void DeleteRecordsUsingUser();
8032     public void DeleteRecordsUsingPhoneNumber();
8033     public void ListRecordUsingUserWithJoins();
8034     public void ListRecordUsingPhoneNumberWithJoins();
8035     public void ListRecordsUsingCriteria();
8036 }
8037 }
8038 -----O2MDAOImpl.java-----
8039 package com.nt.dao;
8040
8041 import java.util.HashSet;
8042 import java.util.List;
8043 import java.util.Set;
8044
8045 import org.hibernate.Criteria;
8046 import org.hibernate.FetchMode;
8047 import org.hibernate.Query;
8048 import org.hibernate.Session;
8049 import org.hibernate.Transaction;
8050
8051 import com.nt.model.PhoneNumber;
8052 import com.nt.model.User;
8053 import com.nt.utility.HibernateUtil;
8054
8055 public class O2MDAOImpl implements O2MDAO {
8056
8057     @Override
8058     public void InsertDataUsingUser() {
8059         // Get HB session obj
8060         Session ses=HibernateUtil.getSession();
```



```
8061 //save objs (parent-to-child)
8062
8063 //parent obj
8064 User user1=new User(101,"raja");
8065 //2 child objs
8066 PhoneNumber ph1=new PhoneNumber(999999,"office");
8067 PhoneNumber ph2=new PhoneNumber(888888,"residence");
8068 //set Parent to childs
8069 ph1.setParent(user1); ph2.setParent(user1);
8070 // set 2 child obj to a parent
8071 Set<PhoneNumber> phones=new HashSet<PhoneNumber>();
8072 phones.add(ph1); phones.add(ph2);
8073 user1.setPhones(phones);
8074
8075
8076 Transaction tx=null;
8077 try{
8078     tx=ses.beginTransaction();
8079     ses.save(user1);
8080     tx.commit();
8081 }
8082 catch(Exception e){
8083     tx.rollback();
8084 }
8085 }//method
8086
8087 @Override
8088 public void InsertDataUsingPhoneNumber() {
8089     //Get Sesssion obj
8090     Session ses=HibernateUtil.getSession();
8091     //Save objs (child to parent)
8092     //prepare child objs
8093     PhoneNumber ph3=new PhoneNumber(7777777,"office");
8094     PhoneNumber ph4=new PhoneNumber(6666666,"residence");
8095     //prepare Parent obj
8096     User user2=new User(102,"Rajesh");
8097     // set parent to childs
8098     ph3.setParent(user2); ph4.setParent(user2);
8099     // set Child to Parent
8100     Set<PhoneNumber> phones=new HashSet<PhoneNumber>();
8101     phones.add(ph4); phones.add(ph3);
8102     user2.setPhones(phones);
8103     Transaction tx=null;
8104     try{
8105         tx=ses.beginTransaction();
8106         ses.save(ph3);ses.save(ph4);
8107         tx.commit();
8108     }
8109     catch(Exception e){
8110         tx.rollback();
8111     }
8112 }//method
8113
8114 @Override
8115 public void ListRecordsUsingUser() {
8116     //Get Session obj
8117     Session ses=HibernateUtil.getSession();
8118     // Load obj (parent to child)
8119     Query q=ses.createQuery("from User");
8120     List <User> list=q.list();
8121     for(User user:list){
8122         System.out.println("parent---->"+user);
8123         //get childs of each parent
8124         Set <PhoneNumber> set=user.getPhones();
8125         for(PhoneNumber phone: set){
```

```
8126         System.out.println("child---->" + phone);
8127     } //for
8128
8129     } //for
8130 }
8131
8132 @Override
8133 public void ListRecordsUsingPhoneNumber() {
8134     //Get Session obj
8135     Session ses=HibernateUtil.getSession();
8136     //Load objs (child to parent)
8137     Query q=ses.createQuery("from PhoneNumber");
8138     List <PhoneNumber> list=q.list();
8139     for(PhoneNumber phone:list){
8140         System.out.println("child---->" + phone);
8141         // get parent of each child
8142         User user=phone.getParent();
8143         System.out.println("parent---->" + user);
8144     } //for
8145 } //method
8146
8147 @Override
8148 public void DeleteRecordsUsingUser() {
8149     //Get Session
8150     Session ses=HibernateUtil.getSession();
8151     //Delete objs (Parent to Child objs)
8152     //Load User obj
8153     User user=(User)ses.get(User.class,101);
8154     Transaction tx=null;
8155     try{
8156         tx=ses.beginTransaction();
8157         ses.delete(user);
8158         tx.commit();
8159     }
8160     catch(Exception e){
8161         tx.rollback();
8162     }
8163 }
8164
8165 @Override
8166 public void DeleteRecordsUsingPhoneNumber() {
8167     //Get Session
8168     Session ses=HibernateUtil.getSession();
8169     //Delete objs (Parent to Child objs)
8170     //Load all PhoneNumber objs
8171     Query q=ses.createQuery("from PhoneNumber");
8172     List <PhoneNumber> list=q.list();
8173     Transaction tx=null;
8174     try{
8175         tx=ses.beginTransaction();
8176         for(PhoneNumber phone:list){
8177             ses.delete(phone);
8178         } //for
8179         tx.commit();
8180     } //try
8181     catch(Exception e){
8182         tx.rollback();
8183     }
8184
8185
8186
8187 }
8188
8189 @Override
8190 public void ListRecordUsingUserWithJoins() {
```

```

8191 // Get Session obj
8192 Session ses=HibernateUtil.getSession();
8193 //Load objs using joins
8194 Query q1=ses.createQuery("select u1 from User u1 inner join fetch u1.phones");
8195 //Query q1=ses.createQuery("select u1 from User u1 left join u1.phones");
8196 //Query q1=ses.createQuery("select u1 from User u1 right join u1.phones");
8197 //Query q1=ses.createQuery("select u1 from User u1 full join u1.phones");
8198 List<User> list =q1.list();
8199 for(User user:list){
8200     System.out.println("Parent====>"+user);
8201     //get child of each user
8202     Set<PhoneNumber> childs=user.getPhones();
8203     for(PhoneNumber phone:childs){
8204         System.out.println("Child====>"+phone);
8205     }
8206 }
8207 }
8208
8209 @Override
8210 public void ListRecordUsingPhoneNumberWithJoins() {
8211     // Get HB Session
8212     Session ses=HibernateUtil.getSession();
8213     // Load child obj using joins
8214     Query q1=ses.createQuery("select ph from PhoneNumber ph inner join ph.parent");
8215     List <PhoneNumber>list=q1.list();
8216     for(PhoneNumber phone:list){
8217         System.out.println("child---->"+phone);
8218         //get parent of each child
8219         User user=phone.getParent();
8220         System.out.println("Parent---->"+user);
8221     }
8222
8223
8224 }
8225
8226 @Override
8227 public void ListRecordsUsingCriteria() {
8228     // Get Hb Session obj
8229     Session ses=HibernateUtil.getSession();
8230     //Load User obj using Criteria API.
8231     Criteria ct=ses.createCriteria(User.class);
8232     ct.setFetchMode("phones",FetchMode.DEFAULT);
8233     List<User> list =ct.list();
8234     for(User user:list){
8235         System.out.println("Parent====>"+user);
8236         //get child of each user
8237         Set<PhoneNumber> childs=user.getPhones();
8238         for(PhoneNumber phone:childs){
8239             System.out.println("Child====>"+phone);
8240         }
8241     }
8242 }
8243
8244 }
8245 -----O2MDAOFactory.java-----
8246 package com.nt.dao;
8247
8248 public class O2MDAOFactory {
8249
8250     public static O2MDAO getDAOInstance(){
8251         return new O2MDAOImpl();
8252     }
8253 }
8254 -----HibernateUtil.java-----
8255 <---- same ---->

```

```
8256 -----ClientApp.java-----
8257 package com.nt.test;
8258
8259 import com.nt.dao.O2MDAO;
8260
8261 import com.nt.dao.O2MDAOFactory;
8262
8263 public class ClientApp {
8264
8265     public static void main(String[] args) {
8266         //Get DAO
8267         O2MDAO dao=O2MDAOFactory.getDAOInstance();
8268         //call B.methods
8269         //dao.InsertDataUsingUser();
8270         //dao.InsertDataUsingPhoneNumber();
8271         //dao.ListRecordsUsingUser();
8272         //dao.ListRecordsUsingPhoneNumber();
8273         //dao.DeleteRecordsUsingUser();
8274         //dao.DeleteRecordsUsingPhoneNumber();
8275         //dao.ListRecordUsingUserWithJoins();
8276         //dao.ListRecordUsingPhoneNumberWithJoins();
8277         dao.ListRecordsUsingCriteria();
8278     }
8279 }
8280
8281
8282 =====
8283 App41) Many to Many Association mapping
8284 =====
8285 -----hibernate.cfg.xml-----
8286 <!DOCTYPE hibernate-configuration
8287 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
8288 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
8289
8290 <hibernate-configuration>
8291     <session-factory>
8292         <!-- connection properties -->
8293         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
8294         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
8295         <property name="connection.username">scott</property>
8296         <property name="connection.password">tiger</property>
8297         <!-- hibernate properties -->
8298         <property name="show_sql">>true</property>
8299         <!-- mapping file names -->
8300         <mapping resource="com/nt/cfgs/programmer.hbm.xml"/>
8301         <mapping resource="com/nt/cfgs/project.hbm.xml"/>
8302     </session-factory>
8303 </hibernate-configuration>
8304 -----Programmer.java-----
8305 package com.nt.model;
8306
8307 import java.util.HashSet;
8308 import java.util.Set;
8309
8310 public class Programmer {
8311     private int pid;
8312     private String pname;
8313     private float salary;
8314     private Set<Project> projects=new HashSet<Project>();
8315
8316     public Programmer(){
8317         System.out.println("Programmer:0-param constructor");
8318     }
8319     public Programmer(int pid, String pname, float salary) {
8320         System.out.println("Programmer:3-param constructor");
```

```
8321         this.pid = pid;
8322         this.pname = pname;
8323         this.salary = salary;
8324     }
8325     public int getPid() {
8326         return pid;
8327     }
8328     public String getPname() {
8329         return pname;
8330     }
8331     public float getSalary() {
8332         return salary;
8333     }
8334     public Set<Project> getProjects() {
8335         return projects;
8336     }
8337     public void setPid(int pid) {
8338         this.pid = pid;
8339     }
8340     public void setPname(String pname) {
8341         this.pname = pname;
8342     }
8343     public void setSalary(float salary) {
8344         this.salary = salary;
8345     }
8346     public void setProjects(Set<Project> projects) {
8347         this.projects = projects;
8348     }
8349     @Override
8350     public String toString() {
8351         return "Programmer [pid=" + pid + ", pname=" + pname + ", salary="
8352             + salary + "];"
8353     }
8354 }
8355
8356 /*
8357 create table programmers
8358 (pid number(5) primary key,
8359  pname varchar2(20),
8360  salary number(8,2)
8361 );
8362
8363 create table projects
8364 ( proid number(5) primary key,
8365  prname varchar2(20)
8366 );
8367
8368 create table programmers_projects
8369 ( programmer_id number(5) references programmers(pid),
8370   project_id number(5) references projects(proid),
8371   primary key(programmer_id,project_id)
8372 );
8373 */
8374 -----Project.java-----
8375 package com.nt.model;
8376
8377 import java.util.HashSet;
8378 import java.util.Set;
8379
8380 public class Project {
8381
8382
8383     private int proid;
8384     private String prname;
8385     private Set<Programmer> programmers=new HashSet<Programmer>();
```

```

8386
8387     public Project(){
8388         System.out.println("Project:0-param constructor");
8389     }
8390
8391     public Project(int proid, String proname) {
8392         System.out.println("Project:2-param constructor");
8393         this.proid = proid;
8394         this.proname = proname;
8395     }
8396
8397     public int getProid() {
8398         return proid;
8399     }
8400
8401     public String getProname() {
8402         return proname;
8403     }
8404
8405     public Set<Programmer> getProgrammers() {
8406         return programmers;
8407     }
8408
8409     public void setProid(int proid) {
8410         this.proid = proid;
8411     }
8412
8413     public void setProname(String proname) {
8414         this.proname = proname;
8415     }
8416
8417     public void setProgrammers(Set<Programmer> programmers) {
8418         this.programmers = programmers;
8419     }
8420
8421     @Override
8422     public String toString() {
8423         return "Project [proid=" + proid + ", proname=" + proname + "]";
8424     }
8425 }
8426 -----programmer.hbm.xml-----
8427 <!DOCTYPE hibernate-mapping SYSTEM
8428 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
8429
8430 <hibernate-mapping>
8431     <class name="com.nt.model.Programmer" table="programmers">
8432         <id name="pid"/> <!--Singular Id field cfg -->
8433         <property name="pname" />
8434         <property name="salary" />
8435         <set name="projects" cascade="all-delete-orphan" table="programmers_projects">
8436             <key column="programmer_id"/>
8437             <many-to-many class="com.nt.model.Project" column="project_id"/>
8438         </set>
8439     </class>
8440 </hibernate-mapping>
8441
8442 -----project.hbm.xml-----
8443 <!DOCTYPE hibernate-mapping SYSTEM
8444 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
8445
8446 <hibernate-mapping>
8447     <class name="com.nt.model.Project" table="projects">
8448         <id name="proid"/> <!--Singular Id field cfg -->
8449         <property name="proname" />
8450         <set name="programmers" cascade="all-delete-orphan" table="programmers_projects">

```

```
8451     <key column="project_id"/>
8452     <many-to-many class="com.nt.model.Programmer" column="programmer_id"/>
8453     </set>
8454     </class>
8455 </hibernate-mapping>
8456 -----M2MDAO.java-----
8457 package com.nt.dao;
8458
8459 public interface M2MDAO {
8460     public void createProgrammersWithProjects();
8461     public void listProjectsWithProgrammers();
8462     public void addNewProjectToProgrammer();
8463     public void addNewProgrammerToProject();
8464 }
8465 -----M2MDAOImpl.java-----
8466 package com.nt.dao;
8467
8468 import java.util.List;
8469 import java.util.Set;
8470
8471 import org.hibernate.Query;
8472 import org.hibernate.Session;
8473 import org.hibernate.Transaction;
8474
8475 import com.nt.model.Programmer;
8476 import com.nt.model.Project;
8477 import com.nt.utility.HibernateUtil;
8478
8479 public class M2MDAOImpl implements M2MDAO{
8480
8481     @Override
8482     public void createProgrammersWithProjects() {
8483         //Get HB Session objs
8484         Session ses=HibernateUtil.getSession();
8485         //create parent objs
8486         Programmer p1=new Programmer(101,"raja",9000);
8487         Programmer p2=new Programmer(102,"ravi",9000);
8488         Programmer p3=new Programmer(103,"ramesh",9000);
8489         //create Child objs
8490         Project proj1=new Project(1001,"proj1");
8491         Project proj2=new Project(1002,"proj2");
8492         // Add programmers to projects
8493         proj1.getProgrammers().add(p1);
8494         proj1.getProgrammers().add(p2);
8495         proj1.getProgrammers().add(p3);
8496
8497         proj2.getProgrammers().add(p1);
8498         proj2.getProgrammers().add(p3);
8499
8500         Transaction tx=null;
8501         try{
8502             tx=ses.beginTransaction();
8503             ses.save(proj1); ses.save(proj2);
8504             tx.commit();
8505         }
8506         catch(Exception e){
8507             tx.rollback();
8508             e.printStackTrace();
8509         }
8510     } //method
8511
8512     @Override
8513     public void listProjectsWithProgrammers() {
8514         // Get HB Session obj
8515         Session ses=HibernateUtil.getSession();
```

```
8516 // Load all child objs along with associated parent obj
8517 Query query=ses.createQuery("from Project");
8518 List <Project> list=query.list();
8519
8520 for(Project proj: list){
8521     //get each child
8522     System.out.println(proj);
8523     //get parents of each parent
8524     Set <Programmer> allprgrms=proj.getProgrammers();
8525     for(Programmer prgmr:allprgrms){
8526         System.out.println(prgmr);
8527     }//for
8528 }//for
8529 }//method
8530
8531 @Override
8532 public void addNewProjectToProgrammer() {
8533
8534     // Get Hb Session obj
8535     Session ses=HibernateUtil.getSession();
8536     // Load existing programmer
8537     Programmer prgmr=(Programmer)sess.get(Programmer.class,101);
8538     // get all all projects of the programmer
8539     Set <Project> projects=prgmr.getProjects();
8540     //create New Object
8541     Project proj3=new Project(1003,"proj3");
8542     Transaction tx=null;
8543     try{
8544         tx=sess.beginTransaction();
8545         // add new project
8546         projects.add(proj3);
8547         tx.commit();
8548     }
8549     catch(Exception e){
8550         tx.rollback();
8551     }
8552 }//method
8553
8554 @Override
8555 public void addNewProgrammerToProject() {
8556     //Get Session obj
8557     Session ses=HibernateUtil.getSession();
8558     //Load existing Project obj
8559     Project project=(Project)sess.get(Project.class,1001);
8560     // Get all programmer of the above Project
8561     Set <Programmer>set=project.getProgrammers();
8562     //create new Programmer
8563     Programmer programmer=new Programmer(104,"anil",9889);
8564     //add new Programmer to existing project
8565     Transaction tx=null;
8566     try{
8567         tx=sess.beginTransaction();
8568         set.add(programmer);
8569         tx.commit();
8570     }
8571     catch(Exception e){
8572         tx.rollback();
8573     }
8574 }//method
8575
8576 }//class
8577 -----M2MDAOFactory.java-----
8578 package com.nt.dao;
8579
8580 public class M2MDAOFactory {
```



```

8581
8582     public static M2MDAO getDAOInstance(){
8583         return new M2MDAOImpl();
8584     }
8585
8586 }
8587 -----HibernateUtil.java-----
8588 <---- same ---->
8589 -----ClientApp.java-----
8590 package com.nt.test;
8591
8592 import com.nt.dao.M2MDAO;
8593 import com.nt.dao.M2MDAOFactory;
8594
8595
8596 public class ClientApp {
8597
8598     public static void main(String[] args) {
8599         // Get DAO
8600         M2MDAO dao=M2MDAOFactory.getDAOInstance();
8601         //call B.methods
8602         dao.createProgrammersWithProjects();
8603         dao.listProjectsWithProgrammers();
8604         dao.addNewProjectToProgrammer();
8605         dao.addNewProgrammerToProject();
8606     }
8607 }
8608 =====
8609 App42 ) Many to Many Assciation Mapping (Using IdBag)
8610 =====
8611 -----hibernate.cfg.xml-----
8612 <!DOCTYPE hibernate-configuration
8613 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
8614 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
8615
8616 <hibernate-configuration>
8617     <session-factory>
8618         <!-- connection properties -->
8619         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
8620         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
8621         <property name="connection.username">scott</property>
8622         <property name="connection.password">tiger</property>
8623         <!-- hibernate properties -->
8624         <property name="show_sql">>true</property>
8625         <!-- mapping file names -->
8626         <mapping resource="com/nt/cfgs/programmer.hbm.xml"/>
8627         <mapping resource="com/nt/cfgs/project.hbm.xml"/>
8628     </session-factory>
8629 </hibernate-configuration>
8630 -----Project.java-----
8631 package com.nt.model;
8632
8633 import java.util.ArrayList;
8634 import java.util.List;
8635 import java.util.Set;
8636
8637 public class Project {
8638
8639     private int proid;
8640     private String pronaame;
8641     private List<Programmer> programmers=new ArrayList<Programmer>();
8642
8643     public Project(){
8644         System.out.println("Project:0-param consturctor");
8645

```

```
8646     }
8647
8648     public Project(int proid, String pronaame) {
8649         System.out.println("Project:2-param consturctor");
8650         this.proid = proid;
8651         this.pronaame = pronaame;
8652     }
8653
8654     public int getProid() {
8655         return proid;
8656     }
8657
8658     public String getPronaame() {
8659         return pronaame;
8660     }
8661
8662     public List<Programmer> getProgrammers() {
8663         return programmers;
8664     }
8665
8666     public void setProid(int proid) {
8667         this.proid = proid;
8668     }
8669
8670     public void setPronaame(String pronaame) {
8671         this.pronaame = pronaame;
8672     }
8673
8674     public void setProgrammers(List<Programmer> programmers) {
8675         this.programmers = programmers;
8676     }
8677
8678     @Override
8679     public String toString() {
8680         return "Project [proid=" + proid + ", pronaame=" + pronaame + "]";
8681     }
8682 }
8683 -----Programmer.java-----
8684 package com.nt.model;
8685
8686 import java.util.HashSet;
8687 import java.util.List;
8688 import java.util.Set;
8689
8690 public class Programmer {
8691     private int pid;
8692     private String pname;
8693     private float salary;
8694     private Set<Project> projects=new HashSet<Project>();
8695
8696     public Programmer(){
8697         System.out.println("Programmer:0-param constructor");
8698     }
8699     public Programmer(int pid, String pname, float salary) {
8700         System.out.println("Programmer:3-param constructor");
8701         this.pid = pid;
8702         this.pname = pname;
8703         this.salary = salary;
8704     }
8705     public int getPid() {
8706         return pid;
8707     }
8708     public String getPname() {
8709         return pname;
8710     }
8711 }
```

```

8711     public float getSalary() {
8712         return salary;
8713     }
8714     public Set<Project> getProjects() {
8715         return projects;
8716     }
8717     public void setPid(int pid) {
8718         this.pid = pid;
8719     }
8720     public void setPname(String pname) {
8721         this.pname = pname;
8722     }
8723     public void setSalary(float salary) {
8724         this.salary = salary;
8725     }
8726     public void setProjects(Set<Project> projects) {
8727         this.projects = projects;
8728     }
8729     @Override
8730     public String toString() {
8731         return "Programmer [pid=" + pid + ", pname=" + pname + ", salary="
8732             + salary + "];"
8733     }
8734 }
8735
8736 /*
8737 create table programmers
8738 (pid number(5) primary key,
8739  pname varchar2(20),
8740  salary number(8,2)
8741 );
8742
8743 create table projects
8744 ( proid number(5) primary key,
8745  prname varchar2(20)
8746 );
8747
8748 create table programmers_projects
8749 ( programmer_id number(5) references programmers(pid),
8750   project_id number(5) references projects(proid),
8751   primary key(programmer_id,project_id)
8752 );
8753 */
8754 -----programmer.java-----
8755 <!DOCTYPE hibernate-mapping SYSTEM
8756 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
8757
8758 <hibernate-mapping>
8759     <class name="com.nt.model.Programmer" table="programmers">
8760         <id name="pid"/> <!--Singular Id field cfg -->
8761         <property name="pname" />
8762         <property name="salary"/>
8763         <set name="projects" cascade="all-delete-orphan" table="programmers_projects">
8764             <key column="programmer_id"/>
8765             <many-to-many class="com.nt.model.Project" column="project_id"/>
8766         </set>
8767     </class>
8768 </hibernate-mapping>
8769 -----project.hbm.xml-----
8770 <!DOCTYPE hibernate-mapping SYSTEM
8771 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
8772
8773 <hibernate-mapping>
8774     <class name="com.nt.model.Project" table="projects">
8775         <id name="proid"/> <!--Singular Id field cfg -->

```

```
8776 <property name="praname" />
8777 <idbag name="programmers" cascade="all-delete-orphan" table="programmers_projects">
8778 <collection-id column="proj_prgmr_index" type="int">
8779 <generator class="increment"/>
8780 </collection-id>
8781
8782 <key column="project_id"/>
8783 <many-to-many class="com.nt.model.Programmer" column="programmer_id"/>
8784 </idbag>
8785 </class>
8786 </hibernate-mapping>
8787 -----M2MDAO.java-----
8788 package com.nt.dao;
8789
8790 public interface M2MDAO {
8791     public void createProgrammersWithProjects();
8792 }
8793 -----M2MDAOImpl.java-----
8794 package com.nt.dao;
8795
8796 import java.util.List;
8797 import java.util.Set;
8798
8799 import org.hibernate.Query;
8800 import org.hibernate.Session;
8801 import org.hibernate.Transaction;
8802
8803 import com.nt.model.Programmer;
8804 import com.nt.model.Project;
8805 import com.nt.utility.HibernateUtil;
8806
8807 public class M2MDAOImpl implements M2MDAO{
8808
8809     @Override
8810     public void createProgrammersWithProjects() {
8811         //Get HB Session objs
8812         Session ses=HibernateUtil.getSession();
8813         //create parent objs
8814         Programmer p1=new Programmer(101,"raja",9000);
8815         Programmer p2=new Programmer(102,"ravi",9000);
8816         Programmer p3=new Programmer(103,"ramesh",9000);
8817         //create Child objs
8818         Project proj1=new Project(1001,"proj1");
8819         Project proj2=new Project(1002,"proj2");
8820         // Add programmers to projects
8821         proj1.getProgrammers().add(p1);
8822         proj1.getProgrammers().add(p2);
8823         proj1.getProgrammers().add(p3);
8824
8825         proj2.getProgrammers().add(p1);
8826         proj2.getProgrammers().add(p3);
8827
8828         Transaction tx=null;
8829         try{
8830             tx=ses.beginTransaction();
8831             ses.save(proj1); ses.save(proj2);
8832             tx.commit();
8833         }
8834         catch(Exception e){
8835             tx.rollback();
8836             e.printStackTrace();
8837         }
8838     } //method
8839
8840 }
```

```

8841 }//class
8842 -----M2MDAOFactory.java-----
8843 package com.nt.dao;
8844
8845 public class M2MDAOFactory {
8846
8847     public static M2MDAO getDAOInstance(){
8848         return new M2MDAOImpl();
8849     }
8850
8851 }
8852 -----HibernateUtil.java-----
8853 <--- same --->
8854 -----ClientApp.java-----
8855 package com.nt.test;
8856
8857 import com.nt.dao.M2MDAO;
8858 import com.nt.dao.M2MDAOFactory;
8859
8860
8861 public class ClientApp {
8862
8863     public static void main(String[] args) {
8864         // Get DAO
8865         M2MDAO dao=M2MDAOFactory.getDAOInstance();
8866         //call B.methods
8867         dao.createProgrammersWithProjects();
8868
8869     }
8870 }
8871 =====
8872 App43 ) One to One FK (uni-directional)
8873 =====
8874 -----hibernate.cfg.xml-----
8875 <!DOCTYPE hibernate-configuration
8876 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
8877 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
8878
8879 <hibernate-configuration>
8880     <session-factory>
8881         <!-- connection properties -->
8882         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
8883         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
8884         <property name="connection.username">scott</property>
8885         <property name="connection.password">tiger</property>
8886         <!-- hibernate properties -->
8887         <property name="show_sql">>true</property>
8888         <!-- mapping file names -->
8889         <mapping resource="com/nt/cfgs/person.hbm.xml"/>
8890         <mapping resource="com/nt/cfgs/license.hbm.xml"/>
8891     </session-factory>
8892 </hibernate-configuration>
8893 -----Person.java-----
8894 package com.nt.model;
8895
8896 public class Person {
8897     private long id;
8898     private byte age;
8899     private String firstName;
8900     private String lastName;
8901
8902     public Person(){
8903         System.out.println("Person:0-param constructor");
8904     }
8905

```

```
8906     public Person(byte age, String firstName, String lastName) {
8907         System.out.println("Person:3-param constructor");
8908         this.age = age;
8909         this.firstName = firstName;
8910         this.lastName = lastName;
8911     }
8912
8913     public long getId() {
8914         return id;
8915     }
8916
8917     public byte getAge() {
8918         return age;
8919     }
8920
8921     public String getFirstName() {
8922         return firstName;
8923     }
8924
8925     public String getLastName() {
8926         return lastName;
8927     }
8928
8929     public void setId(long id) {
8930         this.id = id;
8931     }
8932
8933     public void setAge(byte age) {
8934         this.age = age;
8935     }
8936
8937     public void setFirstName(String firstName) {
8938         this.firstName = firstName;
8939     }
8940
8941     public void setLastName(String lastName) {
8942         this.lastName = lastName;
8943     }
8944
8945     @Override
8946     public String toString() {
8947         return "Person [id=" + id + ", age=" + age + ", firstName=" + firstName
8948             + ", lastName=" + lastName + "];"
8949     }
8950 }
8951
8952 /* create table Person(person_id number(5) primary key,
8953     age number (5),
8954     firstName varchar2(20),
8955     lastName varchar2(20)
8956 );
8957
8958 create table License(license_id number(5) primary key,
8959     type varchar2(20),
8960     valid_from date,
8961     valid_to date,
8962     license_holder number(5) references Person(person_id)
8963 );
8964 create sequence lic_seq increment by 1 start with 1000;
8965 */
8966
8967 -----License.java-----
8968 package com.nt.model;
8969
8970 import java.util.Date;
```

```
8971
8972 public class License {
8973     private long id;
8974     private String type;
8975     private Date dateFrom,dateTo;
8976     private Person licenseHolder ;
8977
8978     public License(){
8979         System.out.println("License:0-param constructor");
8980     }
8981
8982     public License(String type, Date dateFrom, Date dateTo){
8983         System.out.println("License:3-param constructor");
8984         this.type = type;
8985         this.dateFrom = dateFrom;
8986         this.dateTo = dateTo;
8987     }
8988
8989     public long getId() {
8990         return id;
8991     }
8992
8993     public String getType() {
8994         return type;
8995     }
8996
8997     public Date getDateFrom() {
8998         return dateFrom;
8999     }
9000
9001     public Date getDateTo() {
9002         return dateTo;
9003     }
9004
9005     public Person getLicenseHolder() {
9006         return licenseHolder;
9007     }
9008
9009     public void setId(long id) {
9010         this.id = id;
9011     }
9012
9013     public void setType(String type) {
9014         this.type = type;
9015     }
9016
9017     public void setDateFrom(Date dateFrom) {
9018         this.dateFrom = dateFrom;
9019     }
9020
9021     public void setDateTo(Date dateTo) {
9022         this.dateTo = dateTo;
9023     }
9024
9025     public void setLicenseHolder(Person licenseHolder) {
9026         this.licenseHolder = licenseHolder;
9027     }
9028
9029     @Override
9030     public String toString() {
9031         return "License [id=" + id + ", type=" + type + ", dateFrom="
9032             + dateFrom + ", dateTo=" + dateTo + "];"
9033     }
9034 }
9035 -----person.hbm.xml-----
```

```
9036 <!DOCTYPE hibernate-mapping SYSTEM
9037 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
9038
9039 <hibernate-mapping>
9040     <class name="com.nt.model.Person" table="Person">
9041         <id name="id" column="person_id"> <!--Singular Id field cfg -->
9042             <generator class="increment"/>
9043         </id>
9044         <property name="age" />
9045         <property name="firstName"/>
9046         <property name="lastName"/>
9047     </class>
9048 </hibernate-mapping>
```

```
9049
9050 -----license.hbm.xml-----
9051 <!DOCTYPE hibernate-mapping SYSTEM
9052 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
9053
9054 <hibernate-mapping>
9055     <class name="com.nt.model.License" table="License">
9056         <id name="id" column="license_id"> <!--Singular Id field cfg -->
9057             <generator class="sequence">
9058                 <param name="sequence">Lic_seq</param>
9059             </generator>
9060         </id>
9061         <property name="type" />
9062         <property name="dateFrom" column="valid_from" type="date" />
9063         <property name="dateTo" column="valid_to" type="date" />
9064         <many-to-one name="licenseHolder"
9065             class="com.nt.model.Person"
9066             column="license_holder"
9067             unique="true"
9068             not-null="true"
9069             cascade="all"
9070             lazy="false"/>
9071     </class>
9072 </hibernate-mapping>
```

```
9073 -----O2ODAO.java-----
9074 package com.nt.dao;
9075
9076 public interface O2ODAO {
9077     public void createPersonWithLicense();
9078     public void createPersonWithOutLicense();
9079     public void listLicenseAndPersonDetails();
9080     public void CancelPersonLicense();
9081     public void deletePersonWithLicense();
9082 }
```

```
9083 -----O2ODAOImpl.java-----
9084 package com.nt.dao;
9085
9086 import java.util.Date;
9087 import java.util.List;
9088
9089 import org.hibernate.Query;
9090 import org.hibernate.Session;
9091 import org.hibernate.Transaction;
9092
9093 import com.nt.model.License;
9094 import com.nt.model.Person;
9095 import com.nt.utility.HibernateUtil;
9096
9097 public class O2ODAOImpl implements O2ODAO {
9098
9099     @Override
9100     public void createPersonWithLicense() {
```



```
9101 //Get HB session obj
9102 Session ses=HibernateUtil.getSession();
9103 //create Parent obj
9104 Person person=new Person((byte)24,"Ramesh","rao");
9105 //create Child obj
9106 License license=new License("2-wheeler",new Date(),new Date(144,10,20));
9107 license.setLicenseHolder(person);
9108 Transaction tx=null;
9109 try{
9110     tx=ses.beginTransaction();
9111     ses.save(license);
9112     tx.commit();
9113 }
9114 catch(Exception e){
9115     tx.rollback();
9116 }
9117 }//method
9118
9119 @Override
9120 public void createPersonWithOutLicense() {
9121     //Get HB session obj
9122     Session ses=HibernateUtil.getSession();
9123     //create Parent obj
9124     Person person=new Person((byte)25,"rakesh","chari");
9125     Transaction tx=null;
9126     try{
9127         tx=ses.beginTransaction();
9128         ses.save(person);
9129         tx.commit();
9130     }
9131     catch(Exception e){
9132         tx.rollback();
9133     }
9134 }//method
9135
9136 @Override
9137 public void listLicenseAndPersonDetails() {
9138     //Get HB session obj
9139     Session ses=HibernateUtil.getSession();
9140     // Load child objs
9141     Query query=ses.createQuery("from License");
9142     List <License>list=query.list();
9143     for(License license:list){
9144         System.out.println("Child ---->" +license);
9145         // get Parent of each child
9146         Person person=license.getLicenseHolder();
9147         System.out.println("Parent---->" +person);
9148     }//for
9149 }//method
9150
9151 @Override
9152 public void CancelPersonLicense() {
9153     //Get HB session obj
9154     Session ses=HibernateUtil.getSession();
9155     //Delete License with out deleting Person
9156     Query query=ses.createQuery("delete from License where id=:lid");
9157     query.setInteger("lid",1020);
9158     Transaction tx=null;
9159     try{
9160         tx=ses.beginTransaction();
9161         int result=query.executeUpdate();
9162         System.out.println(result+"no. of records deleted");
9163         tx.commit();
9164     }//try
9165     catch(Exception e){
```

```

9166         tx.rollback();
9167     }
9168 }
9169
9170 @Override
9171 public void deletePersonWithLicense() {
9172     //Get HB session obj
9173     Session ses=HibernateUtil.getSession();
9174     //Load child obj
9175     License license=(License)ses.load(License.class,(long)1000);
9176     //Delete child obj
9177     Transaction tx=null;
9178     try{
9179         tx=ses.beginTransaction();
9180         ses.delete(license);
9181         tx.commit();
9182     }//try
9183     catch(Exception e){
9184         tx.rollback();
9185     }
9186
9187
9188 }
9189
9190 }
9191 -----O2ODAOFactroy.java-----
9192 package com.nt.dao;
9193
9194 public class O2ODAOFactory {
9195
9196     public static O2ODAO getDAOInstance(){
9197         return new O2ODAOImpl();
9198     }
9199
9200 }
9201 -----HibernateUtil.java-----
9202 <----Same ---->
9203 -----ClientApp.java-----
9204 package com.nt.test;
9205
9206 import com.nt.dao.O2ODAO;
9207 import com.nt.dao.O2ODAOFactory;
9208
9209
9210 public class ClientApp {
9211
9212     public static void main(String[] args) {
9213         // Get DAO
9214         O2ODAO dao=O2ODAOFactory.getDAOInstance();
9215         //call B.methods
9216         //dao.createPersonWithLicense();
9217         //dao.createPersonWithOutLicense();
9218         //dao.listLicenseAndPersonDetails();
9219         //dao.deletePersonWithLicense();
9220         dao.CancelPersonLicense();
9221     }
9222 }
9223
9224 =====
9225 App44) One to One (PK) Bi-Directional Association
9226 =====
9227 -----hibernate.cfg.xml-----
9228 <!DOCTYPE hibernate-configuration
9229     PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
9230     "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">

```

```

9231
9232 <hibernate-configuration>
9233     <session-factory>
9234         <!-- connection properties -->
9235         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
9236         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
9237         <property name="connection.username">scott</property>
9238         <property name="connection.password">tiger</property>
9239         <!-- hibernate properties -->
9240         <property name="show_sql">true</property>
9241         <!-- mapping file names -->
9242         <mapping resource="com/nt/cfgs/student.hbm.xml"/>
9243         <mapping resource="com/nt/cfgs/Library.hbm.xml"/>
9244     </session-factory>
9245 </hibernate-configuration>
9246 -----Student.java-----
9247 package com.nt.model;
9248
9249 public class Student {
9250     private int id;
9251     private String name;
9252     private String address;
9253     private LibraryMembership libraryDetails;
9254
9255     public Student(){
9256         System.out.println("Student:0-param constructor");
9257     }
9258
9259     public Student(String name,String address){
9260         System.out.println("Student:2-param constructor");
9261         this.name=name;
9262         this.address=address;
9263     }
9264
9265     public int getId() {
9266         return id;
9267     }
9268
9269     public String getName() {
9270         return name;
9271     }
9272
9273     public String getAddress() {
9274         return address;
9275     }
9276
9277     public LibraryMembership getLibraryDetails() {
9278         return libraryDetails;
9279     }
9280
9281     public void setId(int id) {
9282         this.id = id;
9283     }
9284
9285     public void setName(String name) {
9286         this.name = name;
9287     }
9288
9289     public void setAddress(String address) {
9290         this.address = address;
9291     }
9292
9293     public void setLibraryDetails(LibraryMembership libraryDetails) {
9294         this.libraryDetails = libraryDetails;
9295     }
9296
9297     @Override
9298     public String toString() {
9299         return "Student [id=" + id + ", name=" + name + ", address=" + address
9300             + "]";
9301     }
9302 }

```

```
9296 }
9297
9298 /* create table student(sid number(5) primary key,
9299                        name varchar2(20),
9300                        address varchar2(20)
9301                        );
9302 create table Lib_Membership(lid number(5) primary key,
9303                        doj date);
9304
9305
9306
9307 */
9308 -----LibraryMembership.java-----
9309 package com.nt.model;
9310
9311 import java.util.Date;
9312
9313 public class LibraryMembership {
9314     private int id;
9315     private Date joiningDate;
9316     private Student studentDetails;
9317
9318     public LibraryMembership(){
9319         System.out.println("LibraryMembership:0-param constructor");
9320     }
9321
9322     public LibraryMembership(Date joiningDate){
9323         this.joiningDate=joiningDate;
9324     }
9325
9326     public int getId() {
9327         return id;
9328     }
9329
9330     public Date getJoiningDate() {
9331         return joiningDate;
9332     }
9333
9334     public Student getStudentDetails() {
9335         return studentDetails;
9336     }
9337
9338     public void setId(int id) {
9339         this.id = id;
9340     }
9341
9342     public void setJoiningDate(Date joiningDate) {
9343         this.joiningDate = joiningDate;
9344     }
9345
9346     public void setStudentDetails(Student studentDetails) {
9347         this.studentDetails = studentDetails;
9348     }
9349
9350     @Override
9351     public String toString() {
9352         return "LibraryMembership [id=" + id + ", joiningDate=" + joiningDate
9353             + ", studentDetails=" + studentDetails + "]";
9354     }
9355
9356 }
9357
9358 -----student.hbm.xml-----
9359 <!DOCTYPE hibernate-mapping SYSTEM
9360 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
```

```

9361
9362 <hibernate-mapping>
9363   <class name="com.nt.model.Student" table="Student">
9364     <id name="id" column="sid"> <!--Singular Id field cfg -->
9365       <generator class="increment"/>
9366     </id>
9367     <property name="name" />
9368     <property name="address"/>
9369     <one-to-one name="libraryDetails"
9370       class="com.nt.model.LibraryMembership" cascade="all-delete-orphan"/>
9371   </class>
9372 </hibernate-mapping>
9373
9374
9375 -----Library.hbm.xml-----
9376 <!DOCTYPE hibernate-mapping SYSTEM
9377   "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
9378
9379 <hibernate-mapping>
9380   <class name="com.nt.model.LibraryMembership" table="Lib_Membership">
9381     <id name="id" column="lid"> <!--Singular Id field cfg -->
9382       <generator class="foreign">
9383         <param name="property">studentDetails</param>
9384       </generator>
9385     </id>
9386     <property name="joiningDate" column="doj"/>
9387     <one-to-one name="studentDetails" class="com.nt.model.Student" cascade="all"/>
9388   </class>
9389 </hibernate-mapping>
9390 -----O2ODAO.java-----
9391 package com.nt.dao;
9392
9393 public interface O2ODAO {
9394   public void createDataUsingStudent();
9395   public void createDataUsingLibraryMembership();
9396   public void listDetailsUsingStudent();
9397   public void listDetailsUsingLibraryMembership();
9398 }
9399 -----O2ODAOImpl.java-----
9400 package com.nt.dao;
9401
9402 import java.util.Date;
9403 import java.util.List;
9404
9405 import org.hibernate.Query;
9406 import org.hibernate.Session;
9407 import org.hibernate.Transaction;
9408
9409 import com.nt.model.LibraryMembership;
9410 import com.nt.model.Student;
9411 import com.nt.utility.HibernateUtil;
9412
9413 public class O2ODAOImpl implements O2ODAO {
9414
9415   @Override
9416   public void createDataUsingStudent() {
9417     //Get HB session
9418     Session ses=HibernateUtil.getSession();
9419     // save objs (parent to child)
9420     //parent obj
9421     Student student=new Student("raja","hyd");
9422     //child obj
9423     LibraryMembership lib=new LibraryMembership(new Date());
9424     student.setLibraryDetails(lib);
9425     lib.setStudentDetails(student);

```

```

9426 Transaction tx=null;
9427 try{
9428     tx=ses.beginTransaction();
9429     ses.save(student);
9430     tx.commit();
9431 }
9432 catch(Exception e){
9433     tx.rollback();
9434 }
9435
9436
9437 }
9438
9439 @Override
9440 public void createDataUsingLibraryMembership() {
9441     //Get HB session
9442     Session ses=HibernateUtil.getSession();
9443     // save objs (child to parent)
9444     //parent obj
9445     Student student=new Student("ravi","delhi");
9446     //child obj
9447     LibraryMembership lib=new LibraryMembership(new Date());
9448     student.setLibraryDetails(lib);
9449     lib.setStudentDetails(student);
9450
9451     Transaction tx=null;
9452     try{
9453         tx=ses.beginTransaction();
9454         ses.save(lib);
9455         tx.commit();
9456     }
9457     catch(Exception e){
9458         tx.rollback();
9459     }
9460 }
9461
9462 @Override
9463 public void listDetailsUsingStudent() {
9464     //get HB Session obj
9465     Session ses=HibernateUtil.getSession();
9466     // List (parent to child)
9467     Query query=ses.createQuery("from Student");
9468     List <Student> list=query.list();
9469     System.out.println("=====Parent to child =====");
9470     for(Student student:list){
9471         System.out.println("parent---->" + student);
9472         //get each child
9473         LibraryMembership lib=student.getLibraryDetails();
9474         System.out.println("child----->" + lib);
9475     }
9476
9477
9478
9479 }
9480
9481 @Override
9482 public void listDetailsUsingLibraryMembership() {
9483     //get HB Session obj
9484     Session ses=HibernateUtil.getSession();
9485     // List (parent to child)
9486     Query query=ses.createQuery("from LibraryMembership");
9487     List <LibraryMembership> list=query.list();
9488     System.out.println("=====child to parent =====");
9489     for(LibraryMembership lib:list){
9490         System.out.println("child---->" + lib);

```

```

9491         //get each child
9492         Student student=lib.getStudentDetails();
9493         System.out.println("parent----->" +student);
9494     }//for
9495 }//method
9496
9497 }//class
9498
9499 -----O2ODAOFactory.java-----
9500 package com.nt.dao;
9501
9502 public class O2ODAOFactory {
9503
9504     public static O2ODAO getDAOInstance(){
9505         return new O2ODAOImpl();
9506     }
9507
9508 }
9509 -----HibernateUtil.java-----
9510 <----same ---->
9511 -----ClientApp.java-----
9512 package com.nt.test;
9513
9514 import com.nt.dao.O2ODAO;
9515 import com.nt.dao.O2ODAOFactory;
9516
9517
9518 public class ClientApp {
9519
9520     public static void main(String[] args) {
9521         // Get DAO
9522         O2ODAO dao=O2ODAOFactory.getDAOInstance();
9523         // Call methods
9524         //dao.createDataUsingLibraryMembership();
9525         dao.createDataUsingStudent();
9526         //dao.listDetailsUsingStudent();
9527         //dao.listDetailsUsingLibraryMembership();
9528     }
9529 }
9530 =====
9531 App45) Annotations based One to Many Association
9532 =====
9533 -----User.java-----
9534 <!DOCTYPE hibernate-configuration
9535 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
9536 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
9537
9538 <hibernate-configuration>
9539     <session-factory>
9540         <!-- connection properties -->
9541         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
9542         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
9543         <property name="connection.username">scott</property>
9544         <property name="connection.password">tiger</property>
9545         <!-- hibernate properties -->
9546         <property name="show_sql">true</property>
9547         <!-- mapping file names -->
9548         <mapping class="com.nt.model.User"/>
9549         <mapping class="com.nt.model.PhoneNumber"/>
9550     </session-factory>
9551 </hibernate-configuration>
9552 -----User.java-----
9553 package com.nt.model;
9554
9555 import java.util.Set;

```

```
9556
9557 import javax.persistence.CascadeType;
9558 import javax.persistence.Column;
9559 import javax.persistence.Entity;
9560 import javax.persistence.FetchType;
9561 import javax.persistence.Id;
9562 import javax.persistence.JoinColumn;
9563 import javax.persistence.OneToOne;
9564 import javax.persistence.Table;
9565
9566 import org.hibernate.annotations.Cascade;
9567 import org.hibernate.annotations.Fetch;
9568 import org.hibernate.annotations.FetchMode;
9569
9570 @Table(name="user_table")
9571 @Entity
9572 public class User{
9573     @Id
9574     @Column(name="user_id")
9575     private int userId;
9576     @Column(name="first_Name")
9577     private String firstName;
9578
9579     @OneToOne(targetEntity=PhoneNumber.class,
9580             fetch=FetchType.LAZY,
9581             cascade=CascadeType.ALL,
9582             orphanRemoval=true)
9583     @JoinColumn(name="unid",referencedColumnName="user_id")
9584     @Fetch(FetchMode.SELECT)
9585     @Cascade(org.hibernate.annotations.CascadeType.REMOVE)
9586     private Set <PhoneNumber> phones;
9587
9588     public User(){
9589         System.out.println("User:0-param constructor");
9590     }
9591
9592     public User(int userId, String firstName, Set<PhoneNumber> phones) {
9593         System.out.println("User: 3-param constructor");
9594         this.userId = userId;
9595         this.firstName = firstName;
9596         this.phones = phones;
9597     }
9598
9599
9600
9601     public int getUserId() {
9602         return userId;
9603     }
9604     public void setUserId(int userId) {
9605         this.userId = userId;
9606     }
9607     public String getFirstName() {
9608         return firstName;
9609     }
9610     public void setFirstName(String firstName) {
9611         this.firstName = firstName;
9612     }
9613     public Set<PhoneNumber> getPhones() {
9614         return phones;
9615     }
9616     public void setPhones(Set<PhoneNumber> phones) {
9617         this.phones = phones;
9618     }
9619
9620
```



```
9621     @Override
9622     public String toString() {
9623         return "User [userId=" + userId + ", firstName=" + firstName + "]";
9624     }
9625
9626     /* -----parent table-----
9627     CREATE TABLE USER_TABLE(
9628         USER_ID NUMBER PRIMARY KEY,
9629         FIRST_NAME VARCHAR2(20));
9630
9631     -----child table-----
9632     CREATE TABLE PHONE_NUMBERS (
9633         NUMBER_TYPE VARCHAR2(20),
9634         PHONE NUMBER (10),
9635         UNID number references USER_TABLE(USER_ID)); */
9636
9637 }
9638 -----PhoneNumber.java-----
9639 package com.nt.model;
9640
9641 import javax.persistence.Column;
9642 import javax.persistence.Entity;
9643 import javax.persistence.Id;
9644 import javax.persistence.Table;
9645
9646 @Entity
9647 @Table(name="Phone_numbers")
9648 public class PhoneNumber{
9649     @Id
9650     @Column(name="phone")
9651     private long phone;
9652     @Column(name="number_Type")
9653     private String numberType;
9654
9655     public PhoneNumber(){
9656         System.out.println("PhoneNumber:0-param constructor");
9657     }
9658
9659     public PhoneNumber(long phone, String numberType) {
9660         System.out.println("PhoneNumber:2-param constructor");
9661         this.phone = phone;
9662         this.numberType = numberType;
9663     }
9664
9665     public long getPhone() {
9666         return phone;
9667     }
9668
9669     public void setPhone(long phone) {
9670         this.phone = phone;
9671     }
9672
9673     public String getNumberType() {
9674         return numberType;
9675     }
9676
9677     public void setNumberType(String numberType) {
9678         this.numberType = numberType;
9679     }
9680
9681     @Override
9682     public String toString() {
9683         return "PhoneNumber [phone=" + phone + ", numberType=" + numberType
9684             + "]";
9685     }
}
```

```
9686
9687
9688 }
9689 -----HibernateUtil.java-----
9690 <---same--->
9691 -----InsertTest.java-----
9692 package com.nt.test;
9693 import java.util.HashSet;
9694 import java.util.Set;
9695
9696 import org.hibernate.Transaction;
9697
9698 import org.hibernate.Session;
9699
9700 import com.nt.model.PhoneNumber;
9701 import com.nt.model.User;
9702 import com.nt.utility.HibernateUtil;
9703
9704 public class InsertTest {
9705
9706     public static void main(String[] args) {
9707         //Get Session obj
9708         Session ses=HibernateUtil.getSession();
9709         //Save objs (parent-child)
9710         //parent1
9711         User user1=new User();
9712         user1.setUserId(101);
9713         user1.setFirstName("raja");
9714         //child1
9715         PhoneNumber ph1=new PhoneNumber(99999999,"residence");
9716         //child2
9717         PhoneNumber ph2=new PhoneNumber(88888888,"office");
9718         //Line child1,child2 with parent1
9719         Set <PhoneNumber>s=new HashSet();
9720         s.add(ph1); s.add(ph2);
9721         user1.setPhones(s);
9722         Transaction tx=null;
9723         try{
9724             tx=ses.beginTransaction();
9725             ses.save(user1);
9726             tx.commit();
9727         }
9728         catch(Exception e){
9729             tx.rollback();
9730             e.printStackTrace();
9731         }
9732
9733         //close Session
9734         HibernateUtil.closeSession();
9735     } //main
9736 } //class
9737
9738 -----SelectTest.java-----
9739 //SelectTest.java (ClentApp)
9740 package com.nt.test;
9741 import java.util.List;
9742 import java.util.Set;
9743
9744 import org.hibernate.Criteria;
9745 import org.hibernate.Session;
9746
9747 import com.nt.model.PhoneNumber;
9748 import com.nt.model.User;
9749 import com.nt.utility.HibernateUtil;
9750
```

```
9751 public class SelectTest {
9752
9753 public static void main(String[] args)
9754 {
9755     //Get HB Session obj
9756     Session ses=HibernateUtil.getSession();
9757     // Load Parent objs
9758     /* Query query=sess.createQuery("from User");
9759     List <User>list =query.list();
9760     for(User user:list){
9761         System.out.println("parent ---->" +user);
9762         // get all childs of each parent
9763         Set <PhoneNumber> phset=user.getPhones();
9764         for(PhoneNumber phone:phset){
9765             System.out.println("Child---->" +phone);
9766         }
9767     }*/
9768
9769     //Using Criteria API
9770     Criteria ct=sess.createCriteria(User.class);
9771     List <User>list =ct.list();
9772     for(User user:list){
9773         System.out.println("parent ---->" +user);
9774         // get all childs of each parent
9775         Set <PhoneNumber> phset=user.getPhones();
9776         for(PhoneNumber phone:phset){
9777             System.out.println("Child---->" +phone);
9778         }
9779     }
9780
9781     //close Session
9782     HibernateUtil.closeSession();
9783 }
9784 }
```

```
9785 -----DeleteTest.java-----
9786 package com.nt.test;
9787
9788 import java.util.Set;
9789
9790 import org.hibernate.Transaction;
9791
9792 import org.hibernate.Session;
9793
9794 import com.nt.model.PhoneNumber;
9795 import com.nt.model.User;
9796 import com.nt.utility.HibernateUtil;
9797
9798 public class DeleteTest {
9799
9800     public static void main(String[] args) {
9801         // Get Session
9802         Session ses=HibernateUtil.getSession();
9803
9804         // Load Parent obj
9805         User user=(User)sess.load(User.class,101);
9806         // get All childs
9807         Set<PhoneNumber> phones=user.getPhones();
9808         //Load the object that u want to delete
9809         PhoneNumber ph1=(PhoneNumber)sess.get(PhoneNumber.class,(long)88888888);
9810         //Delete loaded from Set
9811         Transaction tx=null;
9812         try{
9813             tx=sess.beginTransaction();
9814             phones.remove(ph1);
9815             tx.commit();
9816         }
```

```
9816     }
9817     catch(Exception e){
9818         tx.rollback();
9819     }
9820
9821
9822
9823     }
9824
9825 }
9826
9827 =====
9828 App47) Annotations based One to Many Association using java.util.Map
9829 =====
9830 -----hibernate.cfg.xml-----
9831 <!DOCTYPE hibernate-configuration
9832     PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
9833     "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
9834
9835 <hibernate-configuration>
9836     <session-factory>
9837         <!-- connection properties -->
9838         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
9839         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
9840         <property name="connection.username">scott</property>
9841         <property name="connection.password">tiger</property>
9842         <property name="hbm2ddl.auto">update</property>
9843         <!-- hibernate properties -->
9844         <property name="show_sql">true</property>
9845         <!-- mapping file names -->
9846         <mapping class="com.nt.model.User"/>
9847         <mapping class="com.nt.model.PhoneNumber"/>
9848     </session-factory>
9849 </hibernate-configuration>
9850 -----User.java-----
9851 package com.nt.model;
9852
9853 import java.util.Map;
9854
9855 import javax.persistence.CascadeType;
9856 import javax.persistence.Column;
9857 import javax.persistence.Entity;
9858 import javax.persistence.FetchType;
9859 import javax.persistence.Id;
9860 import javax.persistence.JoinColumn;
9861 import javax.persistence.MapKeyColumn;
9862 import javax.persistence.OneToOne;
9863 import javax.persistence.Table;
9864
9865 import org.hibernate.annotations.Fetch;
9866 import org.hibernate.annotations.FetchMode;
9867
9868 @Table(name="user_table")
9869 @Entity
9870 public class User{
9871     @Id
9872     @Column(name="user_id")
9873     private int userId;
9874     @Column(name="first_Name")
9875     private String firstName;
9876
9877     @OneToOne(targetEntity=PhoneNumber.class,
9878         fetch=FetchType.LAZY,
9879         cascade=CascadeType.ALL,
9880         orphanRemoval=true)
```

```

9881 @JoinColumn(name="unid",referencedColumnName="user_id")
9882 @MapKeyColumn(name="map_index")
9883 @Fetch(FetchMode.SELECT)
9884 private Map <String,PhoneNumber>phones;
9885
9886 public User(){
9887     System.out.println("User:0-param constructor");
9888 }
9889
9890
9891 public User(int userId, String firstName) {
9892     System.out.println("User: 2-param constructor");
9893     this.userId = userId;
9894     this.firstName = firstName;
9895 }
9896
9897
9898
9899 public int getUserId() {
9900     return userId;
9901 }
9902 public void setUserId(int userId) {
9903     this.userId = userId;
9904 }
9905 public String getFirstName() {
9906     return firstName;
9907 }
9908 public void setFirstName(String firstName) {
9909     this.firstName = firstName;
9910 }
9911 public Map<String,PhoneNumber> getPhones() {
9912     return phones;
9913 }
9914 public void setPhones(Map<String,PhoneNumber> phones) {
9915     this.phones = phones;
9916 }
9917
9918
9919 @Override
9920 public String toString() {
9921     return "User [userId=" + userId + ", firstName=" + firstName + "];";
9922 }
9923
9924 /* -----parent table-----
9925 CREATE TABLE USER_TABLE(
9926     USER_ID NUMBER PRIMARY KEY,
9927     FIRST_NAME VARCHAR2(20));
9928
9929 -----child table-----
9930 CREATE TABLE PHONE_NUMBERS (
9931     NUMBER_TYPE VARCHAR2(20),
9932     PHONE NUMBER (10),
9933     UNID number references USER_TABLE(USER_ID)); */
9934 }
9935
9936
9937 -----PhoneNumber.java-----
9938 package com.nt.model;
9939
9940 import javax.persistence.Column;
9941 import javax.persistence.Entity;
9942 import javax.persistence.Id;
9943 import javax.persistence.Table;
9944
9945 @Entity

```

```
946 @Table(name="Phone_numbers")
947 public class PhoneNumber{
948     @Id
949     @Column(name="phone")
950     private long phone;
951     @Column(name="number_Type")
952     private String numberType;
953
954     public PhoneNumber(){
955         System.out.println("PhoneNumber:0-param constructor");
956     }
957
958     public PhoneNumber(long phone, String numberType) {
959         System.out.println("PhoneNumber:2-param constructor");
960         this.phone = phone;
961         this.numberType = numberType;
962     }
963
964     public long getPhone() {
965         return phone;
966     }
967
968     public void setPhone(long phone) {
969         this.phone = phone;
970     }
971
972     public String getNumberType() {
973         return numberType;
974     }
975
976     public void setNumberType(String numberType) {
977         this.numberType = numberType;
978     }
979
980     @Override
981     public String toString() {
982         return "PhoneNumber [phone=" + phone + ", numberType=" + numberType
983             + "]\n";
984     }
985
986
987 }
988 -----HibernateUtil.java-----
989 <---- same ---->
990 -----InsertTest.java-----
991 package com.nt.test;
992
993 import java.util.HashMap;
994 import java.util.Map;
995
996 import org.hibernate.Session;
997 import org.hibernate.Transaction;
998
999 import com.nt.model.PhoneNumber;
0000 import com.nt.model.User;
0001 import com.nt.utility.HibernateUtil;
0002
0003 public class InsertTest {
0004
0005     public static void main(String[] args) {
0006         //Get Session obj
0007         Session ses=HibernateUtil.getSession();
0008         //Save objs (parent-child)
0009         //parent1
0010         User user1=new User();
```

```

10011     user1.setUserId(101);
10012     user1.setFirstName("raja");
10013     //child1
10014     PhoneNumber ph1=new PhoneNumber(9999999,"residence");
10015     //child2
10016     PhoneNumber ph2=new PhoneNumber(8888888,"office");
10017     //Line child1,child2 with parent1
10018     Map <String,PhoneNumber>map=new HashMap();
10019     map.put("phone1",ph1);map.put("phone2",ph2);
10020     user1.setPhones(map);
10021     Transaction tx=null;
10022     try{
10023         tx=ses.beginTransaction();
10024         ses.save(user1);
10025         tx.commit();
10026     }
10027     catch(Exception e){
10028         tx.rollback();
10029         e.printStackTrace();
10030     }
10031
10032     //close Session
10033     HibernateUtil.closeSession();
10034 }//main
10035 }//class
10036 =====
10037 App48) Annotations based Many to One Association (Uni-Directional)
10038 =====
10039 -----hibernate.cfg.xml-----
10040 <!DOCTYPE hibernate-configuration
10041     PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
10042     "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
10043
10044     <hibernate-configuration>
10045         <session-factory>
10046             <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
10047             <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
10048             <property name="connection.username">scott</property>
10049             <property name="connection.password">tiger</property>
10050             <property name="show_sql">true</property>
10051             <mapping class="com.nt.model.EmpDetails"/>
10052             <mapping class="com.nt.model.Department"/>
10053
10054         </session-factory>
10055     </hibernate-configuration>
10056
10057 -----Department.java-----
10058 //Department.java (parent)
10059 package com.nt.model;
10060
10061 import javax.persistence.Entity;
10062 import javax.persistence.GeneratedValue;
10063 import javax.persistence.Id;
10064
10065 import org.hibernate.annotations.GenericGenerator;
10066
10067
10068 @Entity
10069 public class Department {
10070     @Id
10071     @GenericGenerator(name="gen1",strategy="increment")
10072     @GeneratedValue(generator="gen1")
10073     private int deptno;
10074     private String deptname;
10075     private String depthead;

```

```
10076
10077     public Department(){
10078         System.out.println("Department:0-param constructor");
10079     }
10080
10081     public Department(String deptname, String depthead) {
10082         System.out.println("Department:2-param constructor");
10083         this.deptname = deptname;
10084         this.depthead = depthead;
10085     }
10086
10087     public int getDeptno() {
10088         return deptno;
10089     }
10090
10091     public void setDeptno(int deptno) {
10092         this.deptno = deptno;
10093     }
10094
10095     public String getDeptname() {
10096         return deptname;
10097     }
10098
10099     public void setDeptname(String deptname) {
10100         this.deptname = deptname;
10101     }
10102
10103     public String getDepthead() {
10104         return depthead;
10105     }
10106
10107     public void setDepthead(String depthead) {
10108         this.depthead = depthead;
10109     }
10110
10111     @Override
10112     public String toString() {
10113         return "Department [deptno=" + deptno + ", deptname=" + deptname
10114             + ", depthead=" + depthead + "]";
10115     }
10116 }
10117 -----EmpDetails.java-----
10118 //EmpDetails.java (child class)
10119 package com.nt.model;
10120
10121 import javax.persistence.CascadeType;
10122 import javax.persistence.Entity;
10123 import javax.persistence.FetchType;
10124 import javax.persistence.GeneratedValue;
10125 import javax.persistence.Id;
10126 import javax.persistence.JoinColumn;
10127 import javax.persistence.ManyToOne;
10128
10129 import org.hibernate.annotations.GenericGenerator;
10130 import org.hibernate.annotations.Parameter;
10131
10132 @Entity
10133 public class EmpDetails {
10134     @Id
10135     @GenericGenerator(name="gen1",strategy="sequence",parameters={@Parameter(name="sequence")
10136     @GeneratedValue(generator="gen1")
10137     private int eno;
10138     private String ename;
10139     private float salary;
10140     @ManyToOne(targetEntity=Department.class,cascade=CascadeType.ALL,fetch=FetchType.LAZY)
```



```
10141 @JoinColumn(name="deptno",referencedColumnName="deptno")
10142 private Department dept;
10143
10144 public EmpDetails(){
10145     System.out.println("EmpDetails:0-param constructor");
10146 }
10147
10148 public EmpDetails(String ename, float salary) {
10149     System.out.println("EmpDetails:2-param constructor");
10150     this.ename = ename;
10151     this.salary = salary;
10152 }
10153
10154 public int getEno() {
10155     return eno;
10156 }
10157
10158 public void setEno(int eno) {
10159     this.eno = eno;
10160 }
10161
10162 public String getEname() {
10163     return ename;
10164 }
10165
10166 public void setEname(String ename) {
10167     this.ename = ename;
10168 }
10169
10170 public float getSalary() {
10171     return salary;
10172 }
10173
10174 public void setSalary(float salary) {
10175     this.salary = salary;
10176 }
10177
10178 public Department getDept() {
10179     return dept;
10180 }
10181
10182 public void setDept(Department dept) {
10183     this.dept = dept;
10184 }
10185
10186 @Override
10187 public String toString() {
10188     return "EmpDetails [eno=" + eno + ", ename=" + ename + ", salary="
10189         + salary + "]";
10190 }
10191
10192 }
10193 /* create table Department(deptno number(5) primary key,
10194                             deptname varchar2(20),
10195                             depthead varchar2(20));
10196
10197 create table EmpDetails(eno number(5) primary key,
10198                             ename varchar2(20),
10199                             salary number(7,2),
10200                             deptno number(5) references department(deptno));
10201
10202 create Sequence Seq_EmpNo increment by 1 start with 1000
10203 */
10204 -----M2ODAO.java-----
10205 package com.nt.dao;
```

```
10206
10207 public interface M20DAO {
10208     public void createEmployeesWithDept();
10209     public void addNewEmployeeInDept();
10210     public void listEmployeesWithDept();
10211     public void deleteEmployeesWithDept();
10212     public void addExistingEmployeeToNewDept();
10213     public void deleteEmployeeWithOutDeletingDept();
10214
10215 }
10216 -----M20DAOImpl.java-----
10217 package com.nt.dao;
10218
10219 import java.util.List;
10220
10221 import org.hibernate.Query;
10222 import org.hibernate.Session;
10223 import org.hibernate.Transaction;
10224
10225 import com.nt.model.Department;
10226 import com.nt.model.EmpDetails;
10227 import com.nt.utility.HibernateUtil;
10228
10229 public class M20DAOImpl implements M20DAO{
10230
10231     @Override
10232     public void createEmployeesWithDept() {
10233         // Get Session obj
10234         Session ses=HibernateUtil.getSession();
10235         // create parent obj
10236         Department dept=new Department("Accounts","SMITH");
10237         //create Employee objs (child objs)
10238         EmpDetails emp1=new EmpDetails("raja",9000);
10239         EmpDetails emp2=new EmpDetails("ravi",8000);
10240         // Build Many 2 One relation
10241         emp1.setDept(dept);
10242         emp2.setDept(dept);
10243         //save obj
10244         Transaction tx=null;
10245         try{
10246             tx=ses.beginTransaction();
10247             ses.save(emp1); ses.save(emp2);
10248             tx.commit();
10249         }
10250         catch(Exception e){
10251             tx.rollback();
10252         }
10253
10254
10255     }
10256
10257     @Override
10258     public void listEmployeesWithDept() {
10259         //Get Session obj
10260         Session ses=HibernateUtil.getSession();
10261         // get chids with parents
10262         Query query=ses.createQuery("from EmpDetails");
10263         List<EmpDetails> list=query.list();
10264         /* for(EmpDetails emp:list){
10265             System.out.println("Child--->" +emp);
10266             // get Department of each employee
10267             Department dept=emp.getDept();
10268             System.out.println("Parent--->" +dept);
10269         }*/
10270
10271     }
```

```
10271
10272     }
10273
10274     @Override
10275     public void deleteEmployeeswithDept() {
10276         // Get Hb Session obj
10277         Session ses=HibernateUtil.getSession();
10278         //Load one child obj
10279         EmpDetails emp=(EmpDetails)ses.get(EmpDetails.class,2);
10280         Transaction tx=null;
10281         try{
10282             tx=ses.beginTransaction();
10283             ses.delete(emp);
10284             tx.commit();
10285         }
10286         catch(Exception e){
10287             tx.rollback();
10288         }
10289     }
10290
10291
10292
10293     @Override
10294     public void addNewEmployeeInDept() {
10295         // Get Session
10296         Session ses=HibernateUtil.getSession();
10297         //Load existing Department obj
10298         Department dept=(Department)ses.get(Department.class,1);
10299         //create new Employee
10300         EmpDetails emp=new EmpDetails("Ramesh",7000);
10301         emp.setDept(dept);
10302         Transaction tx=null;
10303         try{
10304             tx=ses.beginTransaction();
10305             ses.save(emp);
10306             tx.commit();
10307         }//try
10308         catch(Exception e){
10309             tx.rollback();
10310         }
10311
10312
10313     }
10314
10315     @Override
10316     public void addExistingEmployeeToNewDept() {
10317         // Get Sesssion
10318         Session ses=HibernateUtil.getSession();
10319         // create new Department
10320         Department dept1=new Department("IT","ALLEN");
10321         // get Exsting Employee
10322         EmpDetails emp=(EmpDetails)ses.get(EmpDetails.class, 1);
10323         emp.setDept(dept1);
10324         Transaction tx=null;
10325         try{
10326             tx=ses.beginTransaction();
10327             ses.update(emp);
10328             tx.commit();
10329         }
10330         catch(Exception e){
10331             tx.rollback();
10332         }
10333
10334
10335     }
```

```
10336
10337     @Override
10338     public void deleteEmployeeWithOutDeletingDept() {
10339         // Get session obj
10340         Session ses=HibernateUtil.getSession();
10341
10342         // To delete only child object.
10343         Query q1=ses.createQuery("delete from EmpDetails where eno=:id");
10344         q1.setInteger("id",2);
10345         Transaction tx=null;
10346         try{
10347             tx=ses.beginTransaction();
10348             int result=q1.executeUpdate();
10349             System.out.println(result+" no.of records are deleted");
10350             tx.commit();
10351         }
10352         catch(Exception e){
10353             tx.rollback();
10354         }
10355
10356
10357
10358
10359
10360         /* // Get child obj (Execute this code with cascade="none" to delete only child)
10361         EmpDetails emp=(EmpDetails)ses.get(EmpDetails.class, 1);
10362         Transaction tx=null;
10363         try{
10364             tx=ses.beginTransaction();
10365             ses.delete(emp);
10366             tx.commit();
10367         }
10368         catch(Exception e){
10369             tx.rollback();
10370         } */
10371
10372     }
10373 }
10374 }
10375 }
10376 }
10377 -----M2ODAOFactory.java-----
10378 package com.nt.dao;
10379
10380 public class M2ODAOFactory {
10381
10382     public static M2ODAO getDAOInstance(){
10383         return new M2ODAOImpl();
10384     }
10385 }
10386 }
10387 -----HibernateUtil.java-----
10388 <---same --->
10389 -----ClientApp.java-----
10390
10391 package com.nt.test;
10392
10393 import com.nt.dao.M2ODAO;
10394 import com.nt.dao.M2ODAOFactory;
10395
10396 public class ClientApp {
10397
10398     public static void main(String[] args) {
10399         // get DAO obj
10400         M2ODAO dao=M2ODAOFactory.getDAOInstance();
```

```

10401         //call methods
10402         //dao.createEmployeesWithDept();
10403         //dao.addNewEmployeeInDept();
10404         // dao.addExistingEmployeeToNewDept();
10405         dao.listEmployeesWithDept();
10406         //dao.deleteEmployeesWithDept();
10407         //dao.deleteEmployeeWithOutDeletingDept();
10408     }
10409 }
10410 =====
10411 App 49) Annotations based One To Many Association (Bi-Directional)
10412 =====
10413 -----hibernate.cfg.xml-----
10414 <!DOCTYPE hibernate-configuration
10415     PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
10416     "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
10417
10418 <hibernate-configuration>
10419     <session-factory>
10420         <!-- connection properties -->
10421         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
10422         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
10423         <property name="connection.username">scott</property>
10424         <property name="connection.password">tiger</property>
10425         <!-- hibernate properties -->
10426         <property name="show_sql">>true</property>
10427         <!-- mapping file names -->
10428         <mapping class="com.nt.model.User"/>
10429         <mapping class="com.nt.model.PhoneNumber"/>
10430     </session-factory>
10431 </hibernate-configuration>
10432 -----User.java-----
10433 package com.nt.model;
10434
10435 import java.util.Set;
10436
10437 import javax.persistence.CascadeType;
10438 import javax.persistence.Column;
10439 import javax.persistence.Entity;
10440 import javax.persistence.FetchType;
10441 import javax.persistence.Id;
10442 import javax.persistence.OneToOne;
10443 import javax.persistence.Table;
10444
10445 @Entity
10446 @Table(name="user_Table")
10447 public class User{
10448     @Id
10449     @Column(name="user_id")
10450     private int userId;
10451
10452     @Column(name="first_name")
10453     private String firstName;
10454
10455     @OneToOne(targetEntity=PhoneNumber.class,cascade=CascadeType.ALL,
10456             fetch=FetchType.LAZY,mappedBy="parent")
10457     private Set <PhoneNumber> phones;
10458
10459     public User(){
10460         System.out.println("User: 0-param constructor");
10461     }
10462
10463     public User(int userId, String firstName) {
10464         System.out.println("User: 2-param constructor");
10465     }

```

```

10466         this.userId = userId;
10467         this.firstName = firstName;
10468     }
10469
10470
10471     public int getUserId() {
10472         return userId;
10473     }
10474     public void setUserId(int userId) {
10475         this.userId = userId;
10476     }
10477     public String getFirstName() {
10478         return firstName;
10479     }
10480     public void setFirstName(String firstName) {
10481         this.firstName = firstName;
10482     }
10483     public Set<PhoneNumber> getPhones() {
10484         return phones;
10485     }
10486     public void setPhones(Set<PhoneNumber> phones) {
10487         this.phones = phones;
10488     }
10489
10490
10491     @Override
10492     public String toString() {
10493         return "User [userId=" + userId + ", firstName=" + firstName + "]";
10494     }
10495
10496     /* -----parent table-----
10497     CREATE TABLE USER_TABLE(
10498         USER_ID NUMBER PRIMARY KEY,
10499         FIRST_NAME VARCHAR2(20));
10500
10501     -----child table-----
10502     CREATE TABLE PHONE_NUMBERS (
10503         NUMBER_TYPE VARCHAR2(20),
10504         PHONE NUMBER (10),
10505         UNID number references USER_TABLE(USER_ID)); */
10506
10507 }
10508 -----PhoneNumber.java-----
10509 package com.nt.model;
10510
10511 import javax.persistence.CascadeType;
10512 import javax.persistence.Column;
10513 import javax.persistence.Entity;
10514 import javax.persistence.FetchType;
10515 import javax.persistence.Id;
10516 import javax.persistence.JoinColumn;
10517 import javax.persistence.ManyToOne;
10518 import javax.persistence.Table;
10519
10520 @Entity
10521 @Table(name="Phone_numbers")
10522 public class PhoneNumber{
10523     @Id
10524     private long phone;
10525     @Column(name="number_type")
10526     private String numberType;
10527     @ManyToOne(targetEntity=User.class,cascade=CascadeType.ALL,
10528         fetch=FetchType.LAZY)
10529     @JoinColumn(name="unid",referencedColumnName="user_id")
10530     private User parent;

```

```
10531
10532     public PhoneNumber(){
10533         System.out.println("PhoneNumber:0-param constructor");
10534     }
10535
10536     public PhoneNumber(long phone, String numberType) {
10537         System.out.println("PhoneNumber:2-param constructor");
10538         this.phone = phone;
10539         this.numberType = numberType;
10540     }
10541
10542     public long getPhone() {
10543         return phone;
10544     }
10545
10546     public void setPhone(long phone) {
10547         this.phone = phone;
10548     }
10549
10550     public String getNumberType() {
10551         return numberType;
10552     }
10553
10554     public void setNumberType(String numberType) {
10555         this.numberType = numberType;
10556     }
10557
10558     public User getParent() {
10559         return parent;
10560     }
10561
10562     public void setParent(User parent) {
10563         this.parent = parent;
10564     }
10565
10566     @Override
10567     public String toString() {
10568         return "PhoneNumber [phone=" + phone + ", numberType=" + numberType
10569             + "]";
10570     }
10571
10572 }
10573
10574 -----O2MDAO.java-----
10575 package com.nt.dao;
10576
10577 public interface O2MDAO {
10578     public void InsertDataUsingUser();
10579     public void InsertDataUsingPhoneNumber();
10580     public void ListRecordsUsingUser();
10581     public void ListRecordsUsingPhoneNumber();
10582     public void DeleteRecordsUsingUser();
10583     public void DeleteRecordsUsingPhoneNumber();
10584     public void ListRecordUsingUserWithJoins();
10585     public void ListRecordUsingPhoneNumberWithJoins();
10586     public void ListRecordsUsingCriteria();
10587
10588 }
10589 -----O2MDAOImpl.java-----
10590 package com.nt.dao;
10591
10592 import java.util.HashSet;
10593 import java.util.List;
10594 import java.util.Set;
10595
```

```
10596 import org.hibernate.Criteria;
10597 import org.hibernate.FetchMode;
10598 import org.hibernate.Query;
10599 import org.hibernate.Session;
10600 import org.hibernate.Transaction;
10601
10602 import com.nt.model.PhoneNumber;
10603 import com.nt.model.User;
10604 import com.nt.utility.HibernateUtil;
10605
10606 public class O2MDAOImpl implements O2MDAO {
10607
10608     @Override
10609     public void InsertDataUsingUser() {
10610         // Get HB session obj
10611         Session ses=HibernateUtil.getSession();
10612         //save objs (parent-to-child)
10613
10614         //parent obj
10615         User user1=new User(101,"raja");
10616         //2 child objs
10617         PhoneNumber ph1=new PhoneNumber(999999,"office");
10618         PhoneNumber ph2=new PhoneNumber(888888,"residence");
10619         //set Parent to childs
10620         ph1.setParent(user1); ph2.setParent(user1);
10621         // set 2 child obj to a parent
10622         Set<PhoneNumber> phones=new HashSet<PhoneNumber>();
10623         phones.add(ph1); phones.add(ph2);
10624         user1.setPhones(phones);
10625
10626
10627         Transaction tx=null;
10628         try{
10629             tx=ses.beginTransaction();
10630             ses.save(user1);
10631             tx.commit();
10632         }
10633         catch(Exception e){
10634             tx.rollback();
10635         }
10636     } //method
10637
10638     @Override
10639     public void InsertDataUsingPhoneNumber() {
10640         //Get Sesssion obj
10641         Session ses=HibernateUtil.getSession();
10642         //Save objs (child to parent)
10643         //prepare child objs
10644         PhoneNumber ph3=new PhoneNumber(7777777,"office");
10645         PhoneNumber ph4=new PhoneNumber(6666666,"residence");
10646         //prepare Parent obj
10647         User user2=new User(102,"Rajesh");
10648         // set parent to childs
10649         ph3.setParent(user2); ph4.setParent(user2);
10650         // set Child to Parent
10651         Set<PhoneNumber> phones=new HashSet<PhoneNumber>();
10652         phones.add(ph4); phones.add(ph3);
10653         user2.setPhones(phones);
10654         Transaction tx=null;
10655         try{
10656             tx=ses.beginTransaction();
10657             ses.save(ph3);ses.save(ph4);
10658             tx.commit();
10659         }
10660         catch(Exception e){
```



```
10661         tx.rollback();
10662     }
10663 }//method
10664
10665 @Override
10666 public void ListRecordsUsingUser() {
10667     //Get Session obj
10668     Session ses=HibernateUtil.getSession();
10669     // Load obj (parent to child)
10670     Query q=ses.createQuery("from User");
10671     List <User> list=q.list();
10672     for(User user:list){
10673         System.out.println("parent---->" +user);
10674         //get childs of each parent
10675         Set <PhoneNumber> set=user.getPhones();
10676         for(PhoneNumber phone: set){
10677             System.out.println("chird---->" +phone);
10678         }//for
10679     }
10680 }//for
10681 }
10682
10683 @Override
10684 public void ListRecordsUsingPhoneNumber() {
10685     //Get Session obj
10686     Session ses=HibernateUtil.getSession();
10687     // Load objs (child to parent)
10688     Query q=ses.createQuery("from PhoneNumber");
10689     List <PhoneNumber> list=q.list();
10690     for(PhoneNumber phone:list){
10691         System.out.println("child---->" +phone);
10692         // get parent of each child
10693         User user=phone.getParent();
10694         System.out.println("parent---->" +user);
10695     }//for
10696 }//method
10697
10698 @Override
10699 public void DeleteRecordsUsingUser() {
10700     //Get Session
10701     Session ses=HibernateUtil.getSession();
10702     //Delete objs (Parent to Child objs)
10703     //Load User obj
10704     User user=(User)ses.get(User.class,101);
10705     Transaction tx=null;
10706     try{
10707         tx=ses.beginTransaction();
10708         ses.delete(user);
10709         tx.commit();
10710     }
10711     catch(Exception e){
10712         tx.rollback();
10713     }
10714 }
10715
10716 @Override
10717 public void DeleteRecordsUsingPhoneNumber() {
10718     //Get Session
10719     Session ses=HibernateUtil.getSession();
10720     //Delete objs (Parent to Child objs)
10721     //Load all PhoneNumber objs
10722     Query q=ses.createQuery("from PhoneNumber");
10723     List <PhoneNumber>list=q.list();
10724     Transaction tx=null;
10725     try{
```

```

10726         tx=ses.beginTransaction();
10727         for(PhoneNumber phone:list){
10728             ses.delete(phone);
10729         }//for
10730         tx.commit();
10731     }//try
10732     catch(Exception e){
10733         tx.rollback();
10734     }
10735
10736
10737
10738 }
10739
10740 @Override
10741 public void ListRecordUsingUserWithJoins() {
10742     // Get Session obj
10743     Session ses=HibernateUtil.getSession();
10744     //Load objs using joins
10745     Query q1=ses.createQuery("select u1 from User u1 inner join fetch u1.phones");
10746     //Query q1=ses.createQuery("select u1 from User u1 left join u1.phones");
10747     //Query q1=ses.createQuery("select u1 from User u1 right join u1.phones");
10748     //Query q1=ses.createQuery("select u1 from User u1 full join u1.phones");
10749     List<User> list =q1.list();
10750     for(User user:list){
10751         System.out.println("Parent==>:"+user);
10752         //get childs of each user
10753         Set<PhoneNumber> childs=user.getPhones();
10754         for(PhoneNumber phone:childs){
10755             System.out.println("Child==>"+phone);
10756         }//for
10757     }//for
10758 }//method
10759
10760 @Override
10761 public void ListRecordUsingPhoneNumberWithJoins() {
10762     // Get HB Session
10763     Session ses=HibernateUtil.getSession();
10764     // Load child obj using joins
10765     Query q1=ses.createQuery("select ph from PhoneNumber ph inner join ph.parent");
10766     List <PhoneNumber>list=q1.list();
10767     for(PhoneNumber phone:list){
10768         System.out.println("child---->"+phone);
10769         //get parent of each child
10770         User user=phone.getParent();
10771         System.out.println("Parent---->"+user);
10772     }
10773
10774
10775 }
10776
10777 @Override
10778 public void ListRecordsUsingCriteria() {
10779     // Get Hb Session obj
10780     Session ses=HibernateUtil.getSession();
10781     //Load User obj using Criteria API.
10782     Criteria ct=ses.createCriteria(User.class);
10783     ct.setFetchMode("phones",FetchMode.DEFAULT);
10784     List<User> list =ct.list();
10785     for(User user:list){
10786         System.out.println("Parent==>:"+user);
10787         //get childs of each user
10788         Set<PhoneNumber> childs=user.getPhones();
10789         for(PhoneNumber phone:childs){
10790             System.out.println("Child==>"+phone);

```

```

10791         }//for
10792     }//for
10793 }//method
10794
10795 }
10796 -----O2MDAOFactory.java-----
10797 package com.nt.dao;
10798
10799 public class O2MDAOFactory {
10800
10801     public static O2MDAO getDAOInstance(){
10802         return new O2MDAOImpl();
10803     }
10804 }
10805 -----HibernateUtil.java-----
10806 <---- same ---->
10807 -----ClientApp.java-----
10808 package com.nt.test;
10809
10810 import com.nt.dao.O2MDAO;
10811
10812 import com.nt.dao.O2MDAOFactory;
10813
10814 public class ClientApp {
10815
10816     public static void main(String[] args) {
10817         // Get DAO
10818         O2MDAO dao=O2MDAOFactory.getDAOInstance();
10819         //call B.methods
10820         dao.InsertDataUsingUser();
10821         dao.InsertDataUsingPhoneNumber();
10822         //dao.ListRecordsUsingUser();
10823         //dao.ListRecordsUsingPhoneNumber();
10824         //dao.DeleteRecordsUsingUser();
10825         //dao.DeleteRecordsUsingPhoneNumber();
10826         //dao.ListRecordUsingUserWithJoins();
10827         //dao.ListRecordUsingPhoneNumberWithJoins();
10828         //dao.ListRecordsUsingCriteria();
10829
10830
10831     }
10832
10833 }
10834
10835 =====
10836 App 50) Annotations based Many to Many Association
10837 =====
10838 -----hibernate.cfg.xml-----
10839 <!DOCTYPE hibernate-configuration
10840 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
10841 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
10842
10843 <hibernate-configuration>
10844     <session-factory>
10845         <!-- connection properties -->
10846         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
10847         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
10848         <property name="connection.username">scott</property>
10849         <property name="connection.password">tiger</property>
10850         <!-- hibernate properties -->
10851         <property name="show_sql">>true</property>
10852         <!-- mapping file names -->
10853         <mapping class="com.nt.model.Programmer"/>
10854         <mapping class="com.nt.model.Project"/>
10855     </session-factory>

```

```
1856 </hibernate-configuration>
1857 -----Programmer.java-----
1858 package com.nt.model;
1859
1860 import java.util.HashSet;
1861 import java.util.Set;
1862
1863 import javax.persistence.CascadeType;
1864 import javax.persistence.Entity;
1865 import javax.persistence.FetchType;
1866 import javax.persistence.Id;
1867 import javax.persistence.JoinColumn;
1868 import javax.persistence.JoinTable;
1869 import javax.persistence.ManyToMany;
1870 import javax.persistence.Table;
1871
1872 @Entity
1873 @Table(name="programmers")
1874 public class Programmer{
1875     @Id
1876     private int pid;
1877     private String pname;
1878     private float salary;
1879
1880     @ManyToMany(targetEntity=Project.class, cascade=CascadeType.ALL,
1881                 fetch=FetchType.LAZY,mappedBy="programmers")
1882     private Set<Project> projects=new HashSet<Project>();
1883     public Programmer(){
1884         System.out.println("Programmer:0-param constructor");
1885     }
1886     public Programmer(int pid, String pname, float salary) {
1887         System.out.println("Programmer:3-param constructor");
1888         this.pid = pid;
1889         this.pname = pname;
1890         this.salary = salary;
1891     }
1892     public int getPid() {
1893         return pid;
1894     }
1895     public String getPname() {
1896         return pname;
1897     }
1898     public float getSalary() {
1899         return salary;
1900     }
1901     public Set<Project> getProjects() {
1902         return projects;
1903     }
1904     public void setPid(int pid) {
1905         this.pid = pid;
1906     }
1907     public void setPname(String pname) {
1908         this.pname = pname;
1909     }
1910     public void setSalary(float salary) {
1911         this.salary = salary;
1912     }
1913     public void setProjects(Set<Project> projects) {
1914         this.projects = projects;
1915     }
1916     @Override
1917     public String toString() {
1918         return "Programmer [pid=" + pid + ", pname=" + pname + ", salary="
1919             + salary + "];"
1920     }
}
```

```

10921 }
10922
10923 /*
10924 create table programmers
10925 (pid number(5) primary key,
10926  pname varchar2(20),
10927  salary number(8,2)
10928 );
10929
10930 create table projects
10931 ( proid number(5) primary key,
10932  prname varchar2(20)
10933 );
10934
10935 create table programmers_projects
10936 ( programmer_id number(5) references programmers(pid),
10937   project_id number(5) references projects(proid),
10938   primary key(programmer_id,project_id)
10939 );
10940 */
10941 -----Project.java-----
10942 package com.nt.model;
10943
10944 import java.util.HashSet;
10945 import java.util.Set;
10946
10947 import javax.persistence.CascadeType;
10948 import javax.persistence.Entity;
10949 import javax.persistence.FetchType;
10950 import javax.persistence.Id;
10951 import javax.persistence.JoinColumn;
10952 import javax.persistence.JoinTable;
10953 import javax.persistence.ManyToMany;
10954 import javax.persistence.Table;
10955
10956 @Entity
10957 @Table(name="projects")
10958 public class Project{
10959     @Id
10960     private int proid;
10961     private String prname;
10962
10963     @ManyToMany(targetEntity=Programmer.class, cascade=CascadeType.ALL,
10964                 fetch=FetchType.LAZY)
10965     @JoinTable(name="programmers_projects",
10966               joinColumns=@JoinColumn(name="project_id", referencedColumnName="proid"),
10967               inverseJoinColumns=@JoinColumn(name="programmer_id", referencedColumnName="pid"))
10968
10969
10970     private Set<Programmer> programmers=new HashSet<Programmer>();
10971     public Project(){
10972         System.out.println("Project:0-param constructor");
10973     }
10974
10975     public Project(int proid, String prname) {
10976         System.out.println("Project:2-param constructor");
10977         this.proid = proid;
10978         this.prname = prname;
10979     }
10980
10981     public int getProid() {
10982         return proid;
10983     }
10984
10985     public String getPrname() {

```

```
10986         return pronomame;
10987     }
10988
10989     public Set<Programmer> getProgrammers() {
10990         return programmers;
10991     }
10992
10993     public void setProid(int proid) {
10994         this.proid = proid;
10995     }
10996
10997     public void setProname(String pronomame) {
10998         this.proname = pronomame;
10999     }
11000
11001     public void setProgrammers(Set<Programmer> programmers) {
11002         this.programmers = programmers;
11003     }
11004
11005     @Override
11006     public String toString() {
11007         return "Project [proid=" + proid + ", pronomame=" + pronomame + "]";
11008     }
11009 }
11010 }
```

```
11011
11012 -----M2MDAO.java-----
```

```
11013 package com.nt.dao;
11014
11015 public interface M2MDAO {
11016     public void createProgrammersWithProjects();
11017     public void listProjectsWithProgrammers();
11018     public void addNewProjectToProgrammer();
11019     public void addNewProgrammerToProject();
11020 }
```

```
11021 -----M2MDAOImpl.java-----
```

```
11022 package com.nt.dao;
11023
11024 import java.util.List;
11025 import java.util.Set;
11026
11027 import org.hibernate.Query;
11028 import org.hibernate.Session;
11029 import org.hibernate.Transaction;
11030
11031 import com.nt.model.Programmer;
11032 import com.nt.model.Project;
11033 import com.nt.utility.HibernateUtil;
11034
11035 public class M2MDAOImpl implements M2MDAO{
11036
11037     @Override
11038     public void createProgrammersWithProjects() {
11039         //Get HB Session objs
11040         Session ses=HibernateUtil.getSession();
11041         //create parent objs
11042         Programmer p1=new Programmer(101,"raja",9000);
11043         Programmer p2=new Programmer(102,"ravi",9000);
11044         Programmer p3=new Programmer(103,"ramesh",9000);
11045         //create Child objs
11046         Project proj1=new Project(1001,"proj1");
11047         Project proj2=new Project(1002,"proj2");
11048         // Add programmers to projects
11049         proj1.getProgrammers().add(p1);
11050         proj1.getProgrammers().add(p2);
```

```
11051         proj1.getProgrammers().add(p3);
11052
11053         proj2.getProgrammers().add(p1);
11054         proj2.getProgrammers().add(p3);
11055         //Add Projects to Programmers
11056         p1.getProjects().add(proj1); p1.getProjects().add(proj2);
11057         p2.getProjects().add(proj1);
11058         p3.getProjects().add(proj1); p3.getProjects().add(proj2);
11059
11060         Transaction tx=null;
11061         try{
11062             tx=ses.beginTransaction();
11063             .ses.save(p1); ses.save(p2); ses.save(p3);
11064             tx.commit();
11065         }
11066         catch(Exception e){
11067             tx.rollback();
11068             e.printStackTrace();
11069         }
11070     } //method
11071
11072     @Override
11073     public void listProjectsWithProgrammers() {
11074         // Get HB Session obj
11075         Session ses=HibernateUtil.getSession();
11076         // Load all child objs along with associated parent obj
11077         Query query=ses.createQuery("from Project");
11078         List <Project> list=query.list();
11079
11080         for(Project proj: list){
11081             //get each child
11082             System.out.println(proj);
11083             //get parents of each parent
11084             Set <Programmer> allprgrms=proj.getProgrammers();
11085             for(Programmer prgmr:allprgrms){
11086                 System.out.println(prgmr);
11087             } //for
11088         } //for
11089     } //method
11090
11091     @Override
11092     public void addNewProjectToProgrammer() {
11093
11094         // Get Hb Session obj
11095         Session ses=HibernateUtil.getSession();
11096         // Load existing programmer
11097         Programmer prgmr=(Programmer)ses.get(Programmer.class,101);
11098         // get all all projects of the programmer
11099         Set <Project> projects=prgmr.getProjects();
11100         //create New Object
11101         Project proj3=new Project(1003,"proj3");
11102         Transaction tx=null;
11103         try{
11104             tx=ses.beginTransaction();
11105             // add new project
11106             projects.add(proj3);
11107             tx.commit();
11108         }
11109         catch(Exception e){
11110             tx.rollback();
11111         }
11112     } //method
11113
11114     @Override
11115     public void addNewProgrammerToProject() {
```

```

11116 //Get Session obj
11117 Session ses=HibernateUtil.getSession();
11118 //Load existing Project obj
11119 Project project=(Project)ses.get(Project.class,1001);
11120 // Get all programmer of the above Project
11121 Set <Programmer>set=project.getProgrammers();
11122 //create new Programmer
11123 Programmer programmer=new Programmer(104,"anil",9889);
11124 //add new Programmer to existing project
11125 Transaction tx=null;
11126 try{
11127     tx=ses.beginTransaction();
11128     set.add(programmer);
11129     tx.commit();
11130 }
11131 catch(Exception e){
11132     tx.rollback();
11133 }
11134 }//method
11135

```

```

11136 }//class

```

```

11137 -----M2MDAOFactory.java-----

```

```

11138 package com.nt.dao;

```

```

11139 public class M2MDAOFactory {

```

```

11140     public static M2MDAO getDAOInstance(){
11141         return new M2MDAOImpl();
11142     }
11143 }

```

```

11144 }

```

```

11145 -----HibernateUtil.java-----

```

```

11146 <---same--->

```

```

11147 -----ClientApp.java-----

```

```

11148 package com.nt.test;

```

```

11149 import com.nt.dao.M2MDAO;

```

```

11150 import com.nt.dao.M2MDAOFactory;

```

```

11151 public class ClientApp {

```

```

11152     public static void main(String[] args) {
11153         // Get DAO
11154         M2MDAO dao=M2MDAOFactory.getDAOInstance();
11155         //call B.methods
11156         dao.createProgrammersWithProjects();
11157         //dao.listProjectsWithProgrammers();
11158         //dao.addNewProjectToProgrammer();
11159         //dao.addNewProgrammerToProject();
11160     }

```

```

11161 }

```

```

11162 App 56 Pagination

```

```

11163 -----hibernate.cfg.xml-----

```

```

11164 <!DOCTYPE hibernate-configuration

```

```

11165 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"

```

```

11166 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">

```

```

11167 <hibernate-configuration>

```

```

11168     <session-factory>

```

```

11169         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>

```

```

11170         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>

```

```

11171         <property name="connection.username">scott</property>

```



```

11181     <property name="connection.password">tiger</property>
11182     <property name="show_sql">true</property>
11183     <property name="hibernate.hbm2ddl.auto">validate</property>
11184     <mapping resource="com/nt/cfgs/Employee.hbm.xml"/>
11185 </session-factory>
11186 </hibernate-configuration>
11187
11188 -----Employee.hbm.xml-----
11189 <!DOCTYPE hibernate-mapping SYSTEM
11190 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
11191
11192 <hibernate-mapping>
11193     <class name="com.nt.model.EmpDetails" table="Employee">
11194         <id name="no" column="EID"/> <!--Singular Id field cfg -->
11195         <property name="fname" column="FIRSTNAME" length="20"/>
11196         <property name="lname" column="LASTNAME" length="30"/>
11197         <property name="mail" column="EMAIL" length="20" unique="true" not-null="true"/>
11198     </class>
11199     <query name="REC_COUNT" >
11200         select count(*) from EmpDetails
11201     </query>
11202     <query name="PAGE_RECORDS">
11203         from EmpDetails
11204     </query>
11205 </hibernate-mapping>
11206
11207 -----EmpBean.java-----
11208 // EmpDetails.java (Domain class/model class/Persistence class)
11209 package com.nt.model;
11210 public class EmpDetails {
11211     //Properties
11212     private int no;
11213     private String fname, lname, mail;
11214
11215     public EmpDetails(){
11216         System.out.println("EmpDetails:0-param constructor");
11217     }
11218
11219     //write setXxx(-) and getXxx() methods
11220     public int getNo() {
11221         return no;
11222     }
11223     public void setNo(int no) {
11224         this.no = no;
11225     }
11226     public String getFname() {
11227         return fname;
11228     }
11229     public void setFname(String fname) {
11230         this.fname = fname;
11231     }
11232     public String getLname() {
11233         return lname;
11234     }
11235     public void setLname(String lname) {
11236         this.lname = lname;
11237     }
11238     public String getMail() {
11239         return mail;
11240     }
11241     public void setMail(String mail) {
11242         this.mail = mail;
11243     }
11244 }
11245

```

```
11246     @Override
11247     public String toString() {
11248         return "EmpDetails [no=" + no + ", fname=" + fname + ", lname=" + lname
11249             + ", mail=" + mail + "]\n";
11250     }
11251
11252
11253
11254
11255 }
11256 -----index.html-----
11257 <!-- input.html -->
11258
11259 <center> <b> <a href="pageurl">Generate Report</a></b></center>
11260
11261 -----web.xml-----
11262 <?xml version="1.0" encoding="ISO-8859-1"?>
11263
11264 <!DOCTYPE web-app
11265     PUBLIC "-//Sun Microsystems, Inc.//DTD Web Application 2.3//EN"
11266     "http://java.sun.com/dtd/web-app_2_3.dtd">
11267
11268 <web-app>
11269     <servlet>
11270         <servlet-name>ABC</servlet-name>
11271         <servlet-class>com.nt.web.PageSrv</servlet-class>
11272     </servlet>
11273     <servlet-mapping>
11274         <servlet-name>ABC</servlet-name>
11275         <url-pattern>/pageurl</url-pattern>
11276     </servlet-mapping>
11277 </web-app>
11278
11279
11280 -----EmpDao.java-----
11281 package com.nt.dao;
11282 import java.util.List;
11283 import com.nt.model.EmpDetails;
11284
11285 public interface EmpDao {
11286     public long getRecordsCount();
11287     public List<EmpDetails> getPageRecords(int first,int max);
11288
11289 }
11290
11291 -----EmpDaoImpl.java-----
11292 package com.nt.dao;
11293
11294 import java.util.List;
11295
11296 import org.hibernate.Session;
11297 import com.nt.model.EmpDetails;
11298 import com.nt.utility.HibernateUtil;
11299
11300 public class EmpDaoImpl implements EmpDao{
11301
11302     @Override
11303     public long getRecordsCount() {
11304         //get HB Session
11305         Session ses=HibernateUtil.getSession();
11306         //get records count
11307         return (Long)ses.getNamedQuery("REC_COUNT").list().get(0);
11308     }
11309
11310     @Override
```

```
11311     public List<EmpDetails> getPageRecords(int first, int max) {
11312         //get HB Session
11313         Session ses=HibernateUtil.getSession();
11314
11315         List<EmpDetails> list = ses.getNamedQuery("PAGE_RECORDS")
11316         .setFirstResult(first).setMaxResults(max).list();
11317         return list;
11318
11319     }
11320
11321 }
11322 -----EmpDaoFactory.java-----
11323 package com.nt.dao;
11324
11325 public class EmpDaoFactory {
11326
11327     public static EmpDao getInstance(){
11328         return new EmpDaoImpl();
11329     }
11330
11331 }
11332 -----HibernateUtil.java-----
11333 <--same -->
11334 -----PageSrv.java-----
11335 //PageSrv.java
11336 package com.nt.web;
11337 import java.io.IOException;
11338 import java.io.PrintWriter;
11339 import java.util.List;
11340
11341 import javax.servlet.ServletException;
11342 import javax.servlet.http.HttpServlet;
11343 import javax.servlet.http.HttpServletRequest;
11344 import javax.servlet.http.HttpServletResponse;
11345 import javax.servlet.http.HttpSession;
11346
11347 import org.hibernate.Session;
11348
11349 import com.nt.dao.EmpDao;
11350 import com.nt.dao.EmpDaoFactory;
11351 import com.nt.model.EmpDetails;
11352 import com.nt.utility.HibernateUtil;
11353
11354 public class PageSrv extends HttpServlet
11355 {
11356     Session ses=null;
11357     int initval;
11358     long totrecords=0;
11359     private static int PAGE_SIZE=2;
11360
11361
11362     public void doGet(HttpServletRequest req,HttpServletResponse res)
11363     throws ServletException,IOException
11364     {
11365         // get DAO
11366         EmpDao dao=EmpDaoFactory.getInstance();
11367         totrecords=dao.getRecordsCount();
11368
11369         //General settings
11370         PrintWriter pw=res.getWriter();
11371         res.setContentType("text/html");
11372
11373         // creates /locates Session for Client
11374         HttpSession session=req.getSession();
11375     }
```

```

11376     if(session.getAttribute("counter")==null)
11377     {
11378         //logic of first request
11379         initval=0;
11380         session.setAttribute("counter",initval);
11381     }
11382     else
11383     {
11384         //logic for other than 1st request
11385         int i1=(Integer)session.getAttribute("counter");
11386         initval=i1+2;
11387         session.setAttribute("counter",initval);
11388     }
11389
11390     // logic to get records to display through pagination
11391     List <EmpDetails> list=dao.getPageRecords(initval,PAGE_SIZE);
11392
11393     pw.println("<table border='1'><tr><th>NO</th>
11394     <th>FIRSTNAME</th><th>LASTNAME</th><th>EMAIL</th></tr>");
11395     for(EmpDetails eb:list)
11396     {
11397         pw.println("<tr><td>" +eb.getNo()+"</td><td>"
11398         +eb.getFname()+"</td><td>" +eb.getLname()+"</td><td>" +eb.getMail()+"</td></tr>");
11399     }
11400     pw.println("</table><br>");
11401
11402     // logic to display home or next hyperlinks in the page
11403     if ((initval+2)<totrecords)
11404     {
11405         pw.println("<a href='pageurl'>next</a>");
11406     }
11407     else
11408     {
11409         pw.println("<a href='index.html'>home</a>");
11410         session.invalidate();
11411     }
11412
11413     pw.close();
11414 }//doGet(-,-)
11415
11416 public void doPost(HttpServletRequest req,HttpServletResponse res)throws ServletException,IOExcept
11417 {
11418     doGet(req,res);
11419 }
11420
11421 public void destroy()
11422 {
11423     HibernateUtil.closeSession();
11424 }//destroy
11425
11426 }//class
11427
11428 =====
11428 App51) Annotations based Many to Many Association Using IdBag
11429 =====
11430 -----hibernate.cfg.xml-----
11431 <!DOCTYPE hibernate-configuration
11432 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
11433 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
11434
11435 <hibernate-configuration>
11436     <session-factory>
11437         <!-- connection properties -->
11438         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
11439         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
11440         <property name="connection.username">system</property>

```

```
11441     <property name="connection.password">manager</property>
11442     <!-- hibernate properties -->
11443     <property name="show_sql">true</property>
11444     <!-- mapping class names -->
11445     <mapping class="com.nt.model.Programmer"/>
11446     <mapping class="com.nt.model.Project"/>
11447 </session-factory>
11448 </hibernate-configuration>
11449 -----Programmer.java-----
11450 package com.nt.model;
11451
11452 import java.util.HashSet;
11453 import java.util.List;
11454 import java.util.Set;
11455
11456 import javax.persistence.CascadeType;
11457 import javax.persistence.Entity;
11458 import javax.persistence.FetchType;
11459 import javax.persistence.Id;
11460 import javax.persistence.ManyToMany;
11461 import javax.persistence.Table;
11462
11463 @Entity
11464 @Table(name="programmers")
11465 public class Programmer{
11466     @Id
11467     private int pid;
11468     private String pname;
11469     private float salary;
11470
11471     @ManyToMany(targetEntity=Project.class, cascade=CascadeType.ALL,
11472                 fetch=FetchType.LAZY,mappedBy="programmers")
11473     private Set<Project> projects=new HashSet<Project>();
11474     public Programmer(){
11475         System.out.println("Programmer:0-param constructor");
11476     }
11477     public Programmer(int pid, String pname, float salary) {
11478         System.out.println("Programmer:3-param constructor");
11479         this.pid = pid;
11480         this.pname = pname;
11481         this.salary = salary;
11482     }
11483     public int getPid() {
11484         return pid;
11485     }
11486     public String getPname() {
11487         return pname;
11488     }
11489     public float getSalary() {
11490         return salary;
11491     }
11492     public Set<Project> getProjects() {
11493         return projects;
11494     }
11495     public void setPid(int pid) {
11496         this.pid = pid;
11497     }
11498     public void setPname(String pname) {
11499         this.pname = pname;
11500     }
11501     public void setSalary(float salary) {
11502         this.salary = salary;
11503     }
11504     public void setProjects(Set<Project> projects) {
11505         this.projects = projects;
```

```

11506     }
11507     @Override
11508     public String toString() {
11509         return "Programmer [pid=" + pid + ", pname=" + pname + ", salary="
11510             + salary + "];"
11511     }
11512 }
11513
11514 /*
11515 create table programmers
11516 (pid number(5) primary key,
11517  pname varchar2(20),
11518  salary number(8,2)
11519 );
11520
11521 create table projects
11522 ( proid number(5) primary key,
11523  proname varchar2(20)
11524 );
11525
11526 create table programmers_projects
11527 ( programmer_id number(5) references programmers(pid),
11528   project_id number(5) references projects(proid),
11529   primary key(programmer_id,project_id)
11530 );
11531 */
11532 -----Project.java-----
11533 package com.nt.model;
11534
11535 import java.util.ArrayList;
11536 import java.util.List;
11537 import java.util.Set;
11538
11539 import javax.persistence.CascadeType;
11540 import javax.persistence.Column;
11541 import javax.persistence.Entity;
11542 import javax.persistence.FetchType;
11543 import javax.persistence.Id;
11544 import javax.persistence.JoinColumn;
11545 import javax.persistence.JoinTable;
11546 import javax.persistence.ManyToMany;
11547 import javax.persistence.Table;
11548
11549 import org.hibernate.annotations.CollectionId;
11550 import org.hibernate.annotations.GenericGenerator;
11551 import org.hibernate.annotations.Type;
11552
11553 @Entity
11554 @Table(name="projects")
11555 public class Project{
11556     @Id
11557     private int proid;
11558     private String proname;
11559
11560     @ManyToMany(targetEntity=Programmer.class, cascade=CascadeType.ALL,
11561                 fetch=FetchType.LAZY)
11562     @JoinTable(name="programmers_projects",
11563               joinColumns=@JoinColumn(name="project_id", referencedColumnName="proid"),
11564               inverseJoinColumns=@JoinColumn(name="programmer_id", referencedColumnName="pid")
11565     )
11566     @GenericGenerator(name="gen1",strategy="increment")
11567     @CollectionId(columns={@Column(name="prgmr_pro_id")},
11568                  type=@Type(type="int"),
11569                  generator="gen1")
11570     private List<Programmer> programmers=new ArrayList<Programmer>();

```

```
11571     public Project(){
11572         System.out.println("Project:0-param consturctor");
11573     }
11574
11575     public Project(int proid, String pronaame) {
11576         System.out.println("Project:2-param consturctor");
11577         this.proid = proid;
11578         this.pronaame = pronaame;
11579     }
11580
11581     public int getProid() {
11582         return proid;
11583     }
11584
11585     public String getPronaame() {
11586         return pronaame;
11587     }
11588
11589     public List<Programmer> getProgrammers() {
11590         return programmers;
11591     }
11592
11593     public void setProid(int proid) {
11594         this.proid = proid;
11595     }
11596
11597     public void setPronaame(String pronaame) {
11598         this.pronaame = pronaame;
11599     }
11600
11601     public void setProgrammers(List<Programmer> programmers) {
11602         this.programmers = programmers;
11603     }
11604
11605     @Override
11606     public String toString() {
11607         return "Project [proid=" + proid + ", pronaame=" + pronaame + "]";
11608     }
11609
11610 }
11611 -----M2MDAO.java-----
11612 package com.nt.dao;
11613
11614 public interface M2MDAO {
11615     public void createProgrammersWithProjects();
11616 }
11617 -----M2MDAOImpl.java-----
11618 package com.nt.dao;
11619
11620 import java.util.List;
11621 import java.util.Set;
11622
11623 import org.hibernate.Query;
11624 import org.hibernate.Session;
11625 import org.hibernate.Transaction;
11626
11627 import com.nt.model.Programmer;
11628 import com.nt.model.Project;
11629 import com.nt.utility.HibernateUtil;
11630
11631 public class M2MDAOImpl implements M2MDAO{
11632
11633     @Override
11634     public void createProgrammersWithProjects() {
11635         //Get HB Session objs
```

```

11636     Session ses=HibernateUtil.getSession();
11637     //create parent objs
11638     Programmer p1=new Programmer(101,"raja",9000);
11639     Programmer p2=new Programmer(102,"ravi",9000);
11640     Programmer p3=new Programmer(103,"ramesh",9000);
11641     //create Child objs
11642     Project proj1=new Project(1001,"proj1");
11643     Project proj2=new Project(1002,"proj2");
11644     // Add programmers to projects
11645     proj1.getProgrammers().add(p1);
11646     proj1.getProgrammers().add(p2);
11647     proj1.getProgrammers().add(p3);
11648
11649     proj2.getProgrammers().add(p1);
11650     proj2.getProgrammers().add(p3);
11651     //Add Projects to Programmers
11652     p1.getProjects().add(proj1); p1.getProjects().add(proj2);
11653     p2.getProjects().add(proj1);
11654     p3.getProjects().add(proj1); p3.getProjects().add(proj2);
11655
11656     Transaction tx=null;
11657     try{
11658         tx=ses.beginTransaction();
11659         ses.save(p1); ses.save(p2); ses.save(p3);
11660         tx.commit();
11661     }
11662     catch(Exception e){
11663         tx.rollback();
11664         e.printStackTrace();
11665     }
11666 }//method

```

```

11667
11668 }//class

```

```

11669 -----M2MDAOFactory.java-----

```

```

11670 package com.nt.dao;

```

```

11671 public class M2MDAOFactory {

```

```

11672     public static M2MDAO getDAOInstance(){
11673         return new M2MDAOImpl();
11674     }

```

```

11675 }

```

```

11676 -----HibernateUtil.java-----

```

```

11677 <---same -->

```

```

11678 -----ClientApp.java-----

```

```

11679 package com.nt.test;

```

```

11680 import com.nt.dao.M2MDAO;
11681 import com.nt.dao.M2MDAOFactory;

```

```

11682 public class ClientApp {

```

```

11683     public static void main(String[] args) {
11684         // Get DAO
11685         M2MDAO dao=M2MDAOFactory.getDAOInstance();
11686         //call B.methods
11687         dao.createProgrammersWithProjects();

```

```

11688     }

```

```

11689 }

```

```

11690 =====
11691

```



```

11701 App52 (Annotations based One to One PK)
11702 =====
11703 -----hibernate.cfg.xml-----
11704 <!DOCTYPE hibernate-configuration
11705 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
11706 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
11707
11708 <hibernate-configuration>
11709     <session-factory>
11710         <!-- connection properties -->
11711         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
11712         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
11713         <property name="connection.username">system</property>
11714         <property name="connection.password">manager</property>
11715         <!-- hibernate properties -->
11716         <property name="show_sql">true</property>
11717         <!-- mapping classes -->
11718         <mapping class="com.nt.model.Student"/>
11719         <mapping class="com.nt.model.LibraryMembership"/>
11720     </session-factory>
11721 </hibernate-configuration>
11722 -----Student.java-----
11723 package com.nt.model;
11724 import javax.persistence.CascadeType;
11725 import javax.persistence.Column;
11726 import javax.persistence.Entity;
11727 import javax.persistence.GeneratedValue;
11728 import javax.persistence.Id;
11729 import javax.persistence.OneToOne;
11730
11731 import org.hibernate.annotations.GenericGenerator;
11732
11733 import com.nt.model.LibraryMembership;
11734
11735 @Entity
11736 public class Student{
11737     @Id
11738     @Column(name="sid")
11739     @GenericGenerator(name="g1",strategy="increment")
11740     @GeneratedValue(generator="g1")
11741     private int id;
11742     private String name;
11743     private String address;
11744     @OneToOne(targetEntity=LibraryMembership.class,cascade=CascadeType.ALL,
11745             mappedBy="studentDetails")
11746
11747     private LibraryMembership libraryDetails;
11748     public Student(){
11749         System.out.println("Student:0-param constructor");
11750     }
11751
11752     public Student(String name,String address){
11753         System.out.println("Student:2-param constructor");
11754         this.name=name;
11755         this.address=address;
11756     }
11757
11758
11759     public int getId() {
11760         return id;
11761     }
11762     public String getName() {
11763         return name;
11764     }
11765     public String getAddress() {

```

```
766         return address;
767     }
768     public LibraryMembership getLibraryDetails() {
769         return libraryDetails;
770     }
771     public void setId(int id) {
772         this.id = id;
773     }
774     public void setName(String name) {
775         this.name = name;
776     }
777     public void setAddress(String address) {
778         this.address = address;
779     }
780     public void setLibraryDetails(LibraryMembership libraryDetails) {
781         this.libraryDetails = libraryDetails;
782     }
783
784     @Override
785     public String toString() {
786         return "Student [id=" + id + ", name=" + name + ", address=" + address
787             + "]";
788     }
789 }
790
791 /* create table student(sid number(5) primary key,
792                        name varchar2(20),
793                        address varchar2(20)
794                        );
795     create table Lib_Membership(lid number(5) primary key,
796                                doj date);
797
798
799
800 */
801 -----LibraryMembership.java-----
802 package com.nt.model;
803
804 import java.util.Date;
805
806 import javax.persistence.CascadeType;
807 import javax.persistence.Column;
808 import javax.persistence.Entity;
809 import javax.persistence.GeneratedValue;
810 import javax.persistence.Id;
811 import javax.persistence.OneToOne;
812 import javax.persistence.PrimaryKeyJoinColumn;
813 import javax.persistence.Table;
814
815 import org.hibernate.annotations.GenericGenerator;
816 import org.hibernate.annotations.Parameter;
817
818 @Entity
819 @Table(name="Lib_membership")
820 public class LibraryMembership{
821     @Id
822     @Column(name="lid")
823     @GenericGenerator(name="g1",strategy="foreign",
824         parameters={@Parameter(name="property",value="studentDetails")})
825     @GeneratedValue(generator="g1")
826     private int id;
827
828     @Column(name="doj")
829     private Date joiningDate;
830 }
```

```

11831 @OneToOne(targetEntity=Student.class,cascade=CascadeType.ALL)
11832 @PrimaryKeyJoinColumn(name="lid",referencedColumnName="sid")
11833 private Student studentDetails;
11834
11835     public LibraryMembership(){
11836         System.out.println("LibraryMembership:0-param constructor");
11837     }
11838
11839     public LibraryMembership(Date joiningDate){
11840         this.joiningDate=joiningDate;
11841     }
11842
11843     public int getId() {
11844         return id;
11845     }
11846
11847     public Date getJoiningDate() {
11848         return joiningDate;
11849     }
11850
11851     public Student getStudentDetails() {
11852         return studentDetails;
11853     }
11854
11855     public void setId(int id) {
11856         this.id = id;
11857     }
11858
11859     public void setJoiningDate(Date joiningDate) {
11860         this.joiningDate = joiningDate;
11861     }
11862
11863     public void setStudentDetails(Student studentDetails) {
11864         this.studentDetails = studentDetails;
11865     }
11866
11867     @Override
11868     public String toString() {
11869         return "LibraryMembership [id=" + id + ", joiningDate=" + joiningDate
11870             + ", studentDetails=" + studentDetails + "]";
11871     }
11872
11873 }
11874

```

-----O2ODAO.java-----

```

11876 package com.nt.dao;
11877
11878 public interface O2ODAO {
11879     public void createDataUsingStudent();
11880     public void createDataUsingLibraryMembership();
11881     public void listDetailsUsingStudent();
11882     public void listDetailsUsingLibraryMembership();
11883 }
11884

```

-----O2ODAOImpl.java-----

```

11886 package com.nt.dao;
11887
11888 import java.util.Date;
11889 import java.util.List;
11890
11891 import org.hibernate.Query;
11892 import org.hibernate.Session;
11893 import org.hibernate.Transaction;
11894
11895 import com.nt.model.LibraryMembership;

```

```
11896 import com.nt.model.Student;
11897 import com.nt.utility.HibernateUtil;
11898
11899 public class O2ODAOImpl implements O2ODAO {
11900
11901     @Override
11902     public void createDataUsingStudent() {
11903         //Get HB session
11904         Session ses=HibernateUtil.getSession();
11905         // save objs (parent to child)
11906         //parent obj
11907         Student student=new Student("raja","hyd");
11908         //child obj
11909         LibraryMembership lib=new LibraryMembership(new Date());
11910         student.setLibraryDetails(lib);
11911         lib.setStudentDetails(student);
11912         Transaction tx=null;
11913         try{
11914             tx=ses.beginTransaction();
11915             ses.save(student);
11916             tx.commit();
11917         }
11918         catch(Exception e){
11919             tx.rollback();
11920         }
11921
11922     }
11923
11924
11925     @Override
11926     public void createDataUsingLibraryMembership() {
11927         //Get HB session
11928         Session ses=HibernateUtil.getSession();
11929         // save objs (child to parent)
11930         //parent obj
11931         Student student=new Student("ravi","delhi");
11932         //child obj
11933         LibraryMembership lib=new LibraryMembership(new Date());
11934         student.setLibraryDetails(lib);
11935         lib.setStudentDetails(student);
11936
11937         Transaction tx=null;
11938         try{
11939             tx=ses.beginTransaction();
11940             ses.save(lib);
11941             tx.commit();
11942         }
11943         catch(Exception e){
11944             tx.rollback();
11945         }
11946
11947     }
11948
11949     @Override
11950     public void listDetailsUsingStudent() {
11951         //get HB Session obj
11952         Session ses=HibernateUtil.getSession();
11953         // List (parent to child)
11954         Query query=ses.createQuery("from Student");
11955         List <Student>list=query.list();
11956         System.out.println("=====Parent to child =====");
11957         for(Student student:list){
11958             System.out.println("parent---->" +student);
11959             //get each child
11960             LibraryMembership lib=student.getLibraryDetails();
```

```

11961         System.out.println("child----->" + lib);
11962     }
11963 }
11964 }
11965
11966 @Override
11967 public void listDetailsUsingLibraryMembership() {
11968     //get HB Session obj
11969     Session ses=HibernateUtil.getSession();
11970     // List (parent to child)
11971     Query query=ses.createQuery("from LibraryMembership");
11972     List <LibraryMembership>list=query.list();
11973     System.out.println("=====child to parent =====");
11974     for(LibraryMembership lib:list){
11975         System.out.println("child----->" + lib);
11976         //get each child
11977         Student student=lib.getStudentDetails();
11978         System.out.println("parent----->" + student);
11979     }//for
11980 }//method
11981
11982 }//class
11983
11984 -----ClientApp.java-----
11985 package com.nt.test;
11986
11987 import com.nt.dao.O2ODAO;
11988 import com.nt.dao.O2ODAOFactory;
11989
11990
11991 public class ClientApp {
11992
11993     public static void main(String[] args) {
11994         // Get DAO
11995         O2ODAO dao=O2ODAOFactory.getDAOInstance();
11996         // Call methods
11997         dao.createDataUsingLibraryMembership();
11998         //dao.createDataUsingStudent();
11999         //dao.listDetailsUsingStudent();
12000         //dao.listDetailsUsingLibraryMembership();
12001     }
12002 }
12003 -----HibernateUtil.java-----
12004 <----Same ---->
12005 -----ClientApp.java-----
12006 package com.nt.test;
12007
12008 import com.nt.dao.O2ODAO;
12009 import com.nt.dao.O2ODAOFactory;
12010
12011
12012 public class ClientApp {
12013
12014     public static void main(String[] args) {
12015         // Get DAO
12016         O2ODAO dao=O2ODAOFactory.getDAOInstance();
12017         // Call methods
12018         dao.createDataUsingLibraryMembership();
12019         //dao.createDataUsingStudent();
12020         //dao.listDetailsUsingStudent();
12021         //dao.listDetailsUsingLibraryMembership();
12022     }
12023 }
12024 =====
12025 App 53) Annotatations based one to one FK

```

```
2026 =====
2027 -----hibernate.cfg.xml-----
2028 <!DOCTYPE hibernate-configuration PUBLIC
2029 "-//Hibernate/Hibernate Configuration DTD//EN"
2030 "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
2031
2032 <hibernate-configuration>
2033   <session-factory>
2034     <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
2035     <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
2036     <property name="connection.username">scott</property>
2037     <property name="connection.password">tiger</property>
2038
2039     <property name="hibernate.hbm2ddl.auto">update</property>
2040     <property name="show_sql">true</property>
2041     <property name="hibernate.dialect">org.hibernate.dialect.Oracle10gDialect</property>
2042
2043     <mapping class="com.nt.domain.Person"/>
2044     <mapping class="com.nt.domain.License"/>
2045   </session-factory>
2046 </hibernate-configuration>
2047
2048 -----Person.java-----
2049 package com.nt.domain;
2050
2051 import javax.persistence.Column;
2052 import javax.persistence.Entity;
2053 import javax.persistence.GeneratedValue;
2054 import javax.persistence.Id;
2055 import javax.persistence.Table;
2056
2057 import org.hibernate.annotations.GenericGenerator;
2058
2059 @Entity
2060 @Table
2061 public class Person {
2062   @Id
2063   @Column(name="person_id")
2064   @GenericGenerator(name="gen1",strategy="increment")
2065   @GeneratedValue(generator="gen1")
2066   private int id;
2067   private String firstName;
2068   private String lastName;
2069   private byte age;
2070
2071   public Person(int id, String firstName, String lastName, byte age) {
2072     System.out.println("Person:4-param constructor");
2073     this.id = id;
2074     this.firstName = firstName;
2075     this.lastName = lastName;
2076     this.age = age;
2077   }
2078
2079   public Person() {
2080     System.out.println("Person:0-param constructor");
2081   }
2082
2083   public int getId() {
2084     return id;
2085   }
2086
2087   public void setId(int id) {
2088     this.id = id;
2089   }
2090 }
```

```
12091     public String getFirstName() {
12092         return firstName;
12093     }
12094
12095     public void setFirstName(String firstName) {
12096         this.firstName = firstName;
12097     }
12098
12099     public String getLastName() {
12100         return lastName;
12101     }
12102
12103     public void setLastName(String lastName) {
12104         this.lastName = lastName;
12105     }
12106
12107     public byte getAge() {
12108         return age;
12109     }
12110
12111     public void setAge(byte age) {
12112         this.age = age;
12113     }
12114
12115     @Override
12116     public String toString() {
12117         return "Person [id=" + id + ", firstName=" + firstName + ", lastName="
12118             + lastName + ", age=" + age + "];"
12119     }
12120 }
12121 -----License.java-----
12122 package com.nt.domain;
12123
12124 import java.util.Date;
12125
12126 import javax.persistence.CascadeType;
12127 import javax.persistence.Column;
12128 import javax.persistence.Entity;
12129 import javax.persistence.GeneratedValue;
12130 import javax.persistence.Id;
12131 import javax.persistence.JoinColumn;
12132 import javax.persistence.OneToOne;
12133 import javax.persistence.Table;
12134
12135 import org.hibernate.annotations.GenericGenerator;
12136 import org.hibernate.annotations.Parameter;
12137
12138 @Entity
12139 @Table
12140 public class License {
12141     @Id
12142     @Column(name="license_id")
12143     @GenericGenerator(name="gen1",
12144         strategy="sequence",
12145         parameters={@Parameter(name="sequence",value="myseq1")})
12146     @GeneratedValue(generator="gen1")
12147     private int id;
12148     private String type;
12149     @Column(name="valid_from")
12150     private Date dateFrom;
12151     @Column(name="valid_to")
12152     private Date dateTo;
12153     @OneToOne(targetEntity=Person.class,cascade=CascadeType.ALL)
12154     @JoinColumn(name="license_holder",referencedColumnName="person_id")
12155     private Person licenseHolder;
```

```
2156
2157
2158
2159     public License() {
2160         System.out.println("License:0-param constructor");
2161     }
2162
2163
2164
2165     public License(int id, String type, Date dateFrom, Date dateTo) {
2166         System.out.println("License:4-param constructor");
2167         this.id = id;
2168         this.type = type;
2169         this.dateFrom = dateFrom;
2170         this.dateTo = dateTo;
2171     }
2172
2173
2174
2175     public int getId() {
2176         return id;
2177     }
2178
2179
2180
2181     public void setId(int id) {
2182         this.id = id;
2183     }
2184
2185
2186
2187     public String getType() {
2188         return type;
2189     }
2190
2191
2192
2193     public void setType(String type) {
2194         this.type = type;
2195     }
2196
2197
2198
2199     public Date getDateFrom() {
2200         return dateFrom;
2201     }
2202
2203
2204
2205     public void setDateFrom(Date dateFrom) {
2206         this.dateFrom = dateFrom;
2207     }
2208
2209
2210
2211     public Date getDateTo() {
2212         return dateTo;
2213     }
2214
2215
2216
2217     public void setDateTo(Date dateTo) {
2218         this.dateTo = dateTo;
2219     }
2220
```



```
12221
12222
12223     public Person getLicenseHolder() {
12224         return licenseHolder;
12225     }
12226
12227
12228
12229     public void setLicenseHolder(Person licenseHolder) {
12230         this.licenseHolder = licenseHolder;
12231     }
12232
12233
12234
12235     @Override
12236     public String toString() {
12237         return "License [id=" + id + ", type=" + type + ", dateFrom="
12238             + dateFrom + ", dateTo=" + dateTo + "];"
12239     }
12240
12241 }
12242
12243 /* create table Person(person_id number(5) primary key,
12244     firstName varchar2(20),
12245     lastName varchar2(20),
12246     age number(3));
12247 create table License(license_id number(5) primary key,
12248     type varchar2(20),
12249     valid_From date,
12250     valid_To date,
12251     license_holder number (5) references person(person_id));
12252
12253 create sequence my_seq1 increment by 1 start with 1001
12254
12255
12256 */
12257
12258 -----O2ODAO.java-----
12259 package com.nt.dao;
12260
12261 public interface O2ODAO {
12262     public void insertDataUsingLicense();
12263 }
12264
12265 -----O2ODAOImpl.java-----
12266 package com.nt.dao;
12267
12268 import java.util.Date;
12269 import java.util.List;
12270
12271 import org.hibernate.Query;
12272 import org.hibernate.Session;
12273 import org.hibernate.Transaction;
12274
12275 import com.nt.domain.License;
12276 import com.nt.domain.Person;
12277 import com.nt.utility.HibernateUtil;
12278
12279 public class O2ODAOImpl implements O2ODAO {
12280
12281     @Override
12282     public void insertDataUsingLicense() {
```

```
.2286 //Get Session
.2287 Session ses=HibernateUtil.getSession();
.2288
.2289 //Save objs (child to parent)
.2290 //parent obj
.2291 Person p=new Person();
.2292 p.setFirstName("Raja");
.2293 p.setLastName("Rao");
.2294 p.setAge((byte)20);
.2295
.2296
.2297 //child obj
.2298 License l=new License();
.2299 l.setType("2-wheeler");
.2300 l.setDateFrom(new Date());
.2301 l.setDateTo(new Date(145,6,30));
.2302 //set Parent to child
.2303 l.setLicenseHolder(p);
.2304
.2305 //Only Parent obj
.2306 Person p1=new Person();
.2307 p1.setFirstName("Ravi");
.2308 p1.setLastName("chari");
.2309 p1.setAge((byte)26);
.2310
.2311
.2312 Transaction tx=null;
.2313 try{
.2314     tx=ses.beginTransaction();
.2315     //save objs
.2316     ses.save(l);
.2317     ses.save(p1);
.2318     tx.commit();
.2319 }
.2320 catch(Exception e){
.2321     tx.rollback();
.2322     e.printStackTrace();
.2323 }
.2324
.2325 }
.2326
.2327 }//class
.2328
.2329 -----O2ODAOFactory.java-----
.2330 package com.nt.dao;
.2331
.2332 public class O2ODAOFactory {
.2333
.2334     public static O2ODAO getInstance(){
.2335         return new O2ODAOImpl();
.2336     }
.2337
.2338
.2339 }
.2340 -----ClientApp.java-----
.2341 package com.nt.test;
.2342
.2343 import com.nt.dao.O2ODAO;
.2344 import com.nt.dao.O2ODAOFactory;
.2345
.2346 public class ClientApp {
.2347     public static void main(String[] args) {
.2348         // get DAO
.2349         O2ODAO dao=O2ODAOFactory.getInstance();
.2350         dao.insertDataUsingLicense();
```

```

12351
12352     }
12353
12354 }
12355
12356
12357 =====
12358 App53) Second Level Cache Cfg (EH Cache)
12359 =====
12360 -----hibernate.cfg.xml-----
12361 <!DOCTYPE hibernate-configuration
12362 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
12363 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
12364
12365 <hibernate-configuration>
12366     <session-factory>
12367         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
12368         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
12369         <property name="connection.username">system</property>
12370         <property name="connection.password">manager</property>
12371         <property name="show_sql">>true</property>
12372         <property name="hibernate.hbm2ddl.auto">validate</property>
12373
12374         <property name="cache.use_second_level_cache">>true</property>
12375         <property name="cache.region.factory_class">org.hibernate.cache.EhCacheRegionFactory<
12376         <property name="net.sf.ehcache.configurationResourceName">/com/nt/cfgs/ehcache.xml</
12377
12378         <mapping resource="com/nt/cfgs/Employee.hbm.xml"/>
12379
12380     </session-factory>
12381 </hibernate-configuration>
12382 -----Employee.hbm.xml-----
12383 <!DOCTYPE hibernate-mapping SYSTEM
12384 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" >
12385
12386 <hibernate-mapping>
12387     <class name="com.nt.model.EmpDetails" table="Employee">
12388         <cache usage="read-only"/>
12389         <id name="no" column="EID"/> <!--Singular Id field cfg -->
12390         <property name="fname" column="FIRSTNAME" length="20"/>
12391         <property name="lname" column="LASTNAME" length="30"/>
12392         <property name="mail" column="EMAIL" length="20" unique="true" not-null="true"/>
12393
12394     </class>
12395 </hibernate-mapping>
12396
12397 -----EmpDetails.java-----
12398 // EmpDetails.java (Domain class/model class/Persistence class)
12399 package com.nt.model;
12400 public class EmpDetails {
12401     //Properties
12402     private int no;
12403     private String fname, lname, mail;
12404
12405     public EmpDetails(){
12406         System.out.println("EmpDetails:0-param constructor");
12407     }
12408
12409     //write setXxx(-) and getXxx() methods
12410     public int getNo() {
12411         return no;
12412     }
12413     public void setNo(int no) {
12414         this.no = no;
12415     }

```

```
2416 public String getFname() {
2417     return fname;
2418 }
2419 public void setFname(String fname) {
2420     this.fname = fname;
2421 }
2422 public String getLname() {
2423     return lname;
2424 }
2425 public void setLname(String lname) {
2426     this.lname = lname;
2427 }
2428 public String getMail() {
2429     return mail;
2430 }
2431 public void setMail(String mail) {
2432     this.mail = mail;
2433 }
2434
2435 @Override
2436 public String toString() {
2437     return "EmpDetails[no=" + no + ", fname=" + fname + ", lname=" + lname
2438         + ", mail=" + mail + "];"
2439 }
2440
2441 }
```

```
2442
2443 -----ehcache.xml-----
2444 <ehcache>
2445     <diskStore path="java.io.tmpdir"/>
2446     <defaultCache
2447         maxElementsInMemory="100"
2448         eternal="false"
2449         timeToIdleSeconds="20"
2450         timeToLiveSeconds="60"
2451         overflowToDisk="true"
2452     />
2453 </ehcache>.
```

```
2454 -----TestClient.java-----
2455 package com.nt.test;
2456
2457 import org.hibernate.Session;
2458 import org.hibernate.SessionFactory;
2459 import org.hibernate.Transaction;
2460 import org.hibernate.cfg.Configuration;
2461
2462 import com.nt.model.EmpDetails;
2463
2464 public class TestClient {
2465
2466     public static void main(String[] args) throws Exception{
2467
2468         //Activate HB framework
2469         Configuration cfg=new Configuration();
2470         // read and verify cfg file, mapping file entries
2471         cfg=cfg.configure("/com/nt/cfgs/hibernate.cfg.xml");
2472         //Get SessionFactory obj
2473         SessionFactory factory=cfg.buildSessionFactory();
2474         // Get Session
2475         Session ses=factory.openSession();
2476
2477
2478         //Loads from DB and keeps in L2,L1 caches
2479         EmpDetails eb=(EmpDetails)ses.load(EmpDetails.class,101);
2480         System.out.println(eb);
```

```

12481
12482      //Loads from L1 cache
12483      EmpDetails eb1=(EmpDetails)ses.load(EmpDetails.class,101);
12484      System.out.println(eb1);
12485
12486      ses.clear(); // Removes objs from L1 cache
12487
12488      //Loads from L2 cache and keeps in L1 cache
12489      EmpDetails eb2=(EmpDetails)ses.load(EmpDetails.class,101);
12490      System.out.println(eb2);
12491
12492      ses.evict(eb2); //removes EmpDetails obj from L1 cache
12493      Thread.sleep(30000); // Removes EmpDetails obj from L2 cahce (Idle timeout:20 secs)
12494
12495      //Loads from DB and keeps L1,L2 caches
12496      EmpDetails eb3=(EmpDetails)ses.load(EmpDetails.class,101);
12497      System.out.println(eb3);
12498
12499      ses.close();
12500      factory.close();
12501  }//main
12502 }//class
12503
12504 =====
12505 App54) Annoatations based Second Level Cache + Query Cache
12506 =====
12507 -----hibernate.cfg.xml-----
12508 <!DOCTYPE hibernate-configuration
12509 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
12510 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
12511
12512 <hibernate-configuration>
12513   <session-factory>
12514     <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
12515     <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
12516     <property name="connection.username">system</property>
12517     <property name="connection.password">manager</property>
12518     <property name="show_sql">>true</property>
12519     <property name="hibernate.hbm2ddl.auto">validate</property>
12520
12521     <property name="cache.use_second_level_cache">>true</property>
12522     <property name="cache.region.factory_class">org.hibernate.cache.EhCacheRegionFactory</proper
12523 <property name="net.sf.ehcache.configurationResourceName">/com/nt/cfgs/ehcache.xml</propert
12524     <property name="cache.use_query_cache">>true</property>
12525     <mapping class="com.nt.model.EmpDetails"/>
12526
12527   </session-factory>
12528 </hibernate-configuration>
12529
12530 -----ehcache.xml-----
12531 <ehcache>
12532   <diskStore path="java.io.tmpdir"/>
12533   <defaultCache
12534     maxElementsInMemory="100"
12535     eternal="false"
12536     timeToIdleSeconds="20"
12537     timeToLiveSeconds="60"
12538     overflowToDisk="true"
12539   />
12540 </ehcache>
12541 -----EmpDetails.java-----
12542 // EmpDetails.java (Domain class/model class/Persistence class)
12543 package com.nt.model;
12544
12545 import javax.persistence.Column;

```

```
12546 import javax.persistence.Entity;
12547 import javax.persistence.Id;
12548 import javax.persistence.Table;
12549
12550 import org.hibernate.annotations.Cache;
12551 import org.hibernate.annotations.CacheConcurrencyStrategy;
12552
12553 @Table(name="Employee")
12554 @Entity
12555 @Cache(usage=CacheConcurrencyStrategy.READ_ONLY)
12556 public class EmpDetails {
12557     //Properties
12558     @Id
12559     @Column(name="EID")
12560     private int no;
12561     @Column(name="FIRSTNAME")
12562     private String fname;
12563     @Column(name="LASTNAME")
12564     private String lname;
12565     @Column(name="EMAIL")
12566     private String mail;
12567
12568
12569
12570     public EmpDetails(){
12571         System.out.println("EmpDetails:0-param constructor");
12572     }
12573
12574     //write setXxx(-) and getXxx() methods
12575     public int getNo() {
12576         return no;
12577     }
12578     public void setNo(int no) {
12579         this.no = no;
12580     }
12581     public String getFname() {
12582         return fname;
12583     }
12584     public void setFname(String fname) {
12585         this.fname = fname;
12586     }
12587     public String getLname() {
12588         return lname;
12589     }
12590     public void setLname(String lname) {
12591         this.lname = lname;
12592     }
12593     public String getMail() {
12594         return mail;
12595     }
12596     public void setMail(String mail) {
12597         this.mail = mail;
12598     }
12599
12600     @Override
12601     public String toString() {
12602         return "EmpDetails [no=" + no + ", fname=" + fname + ", lname=" + lname
12603             + ", mail=" + mail + "];"
12604     }
12605 }
12606
12607 -----TestClient.java-----
12608 package com.nt.test;
12609
12610 import java.util.List;
```

```

12611
12612 import org.hibernate.CacheMode;
12613 import org.hibernate.Query;
12614 import org.hibernate.Session;
12615 import org.hibernate.SessionFactory;
12616 import org.hibernate.cfg.Configuration;
12617
12618 import com.nt.model.EmpDetails;
12619
12620 public class TestClient {
12621
12622     public static void main(String[] args) throws Exception{
12623
12624         //Activate HB framework
12625         Configuration cfg=new Configuration();
12626         // read and verify cfg file, mapping file entries
12627         cfg=cfg.configure("/com/nt/cfgs/hibernate.cfg.xml");
12628         //Get SessionFactory obj
12629         SessionFactory factory=cfg.buildSessionFactory();
12630         // Get Session
12631         Session ses=factory.openSession();
12632
12633         /* For L1,L2 caches Demonstration
12634            //Loads from DB and keeps in L2,L1 caches
12635            EmpDetails eb=(EmpDetails)ses.load(EmpDetails.class,101);
12636            System.out.println(eb);
12637
12638            //Loads from L1 cache
12639            EmpDetails eb1=(EmpDetails)ses.load(EmpDetails.class,101);
12640            System.out.println(eb1);
12641
12642            ses.clear(); // Removes objs from L1 cache
12643
12644            //Loads from L2 cache and keeps in L1 cache
12645            EmpDetails eb2=(EmpDetails)ses.load(EmpDetails.class,101);
12646            System.out.println(eb2);
12647
12648            ses.evict(eb2); //removes EmpDetails obj from L1 cache
12649            Thread.sleep(30000); // Removes EmpDetails obj from L2 cahce (Idle timeout:20 secs)
12650
12651            //Loads from DB and keeps L1,L2 caches
12652            EmpDetails eb3=(EmpDetails)ses.load(EmpDetails.class,101);
12653            System.out.println(eb3); */
12654
12655            //For Query Cache.
12656            Query query=ses.createQuery("from EmpDetails");
12657            query.setCacheable(true);
12658            query.setCacheRegion("qcache1");
12659
12660            List <EmpDetails> list=query.list(); // Get from DB and keeps in Query cache
12661            for(EmpDetails eb:list){
12662                System.out.println(eb);
12663            }
12664
12665            list=query.list(); // Get from Query cache
12666            for(EmpDetails eb:list){
12667                System.out.println(eb);
12668            }
12669
12670            ses.close();
12671            factory.close();
12672        } //main
12673    } //class
12674    =====
12675    App55 ) Transaction Management in Hibernate App

```

```

676 =====
677 -----hibernate.cfg.xml-----
678 <!DOCTYPE hibernate-configuration
679 PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
680 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
681
682 <hibernate-configuration>
683     <session-factory>
684         <!-- connection properties -->
685         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
686         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
687         <property name="connection.username">system</property>
688         <property name="connection.password">manager</property>
689         <!-- hibernate properties -->
690         <property name="show_sql">true</property>
691         <!-- mapping class -->
692         <mapping class="com.nt.model.Account"/>
693     </session-factory>
694 </hibernate-configuration>
695 -----Account.java-----
696 package com.nt.model;
697
698 import javax.persistence.Entity;
699 import javax.persistence.Id;
700
701 @Entity
702 public class Account {
703     @Id
704     private int acno;
705     private String holder;
706     private double balance;
707
708     public Account(){
709         System.out.println("Account:0-param constructor");
710     }
711
712     public Account(int acno, String holder, double balance) {
713         System.out.println("Account:3-param constructor");
714         this.acno = acno;
715         this.holder = holder;
716         this.balance = balance;
717     }
718
719     public int getAcno() {
720         return acno;
721     }
722
723     public String getHolder() {
724         return holder;
725     }
726
727     public double getBalance() {
728         return balance;
729     }
730
731     public void setAcno(int acno) {
732         this.acno = acno;
733     }
734
735     public void setHolder(String holder) {
736         this.holder = holder;
737     }
738
739     public void setBalance(double balance) {
740         this.balance = balance;

```



```
12741     }
12742
12743     @Override
12744     public String toString() {
12745         return "Account [acno=" + acno + ", holder=" + holder + ", balance="
12746             + balance + "];"
12747     }
12748
12749 }
12750 -----BankDAO.java-----
12751 package com.nt.dao;
12752
12753 public interface BankDAO {
12754
12755     public boolean transferMoney(int srcno,int destno, double amt);
12756
12757 }
12758 -----BankDAOImpl.java-----
12759 package com.nt.dao;
12760
12761 import org.hibernate.Query;
12762 import org.hibernate.Session;
12763 import org.hibernate.Transaction;
12764
12765 import com.nt.utility.HibernateUtil;
12766
12767 public class BankDAOImpl implements BankDAO {
12768
12769     @Override
12770     public boolean transferMoney(int srcno, int destno, double amt) {
12771         //get Session
12772         Session ses=HibernateUtil.getSession();
12773         Transaction tx=null;
12774         try{
12775             tx=ses.beginTransaction();
12776             //for withdraw operation
12777             Query q1=ses.createQuery("update Account set balance=balance-:amt where acno=:no");
12778             q1.setDouble("amt",amt);
12779             q1.setInteger("no",srcno);
12780             int res1=q1.executeUpdate();
12781
12782             //for Deposit operation
12783             Query q2=ses.createQuery("update Account set balance=balance+ :amt where acno=:no");
12784             q2.setDouble("amt",amt);
12785             q2.setInteger("no",destno);
12786             int res2=q2.executeUpdate();
12787
12788             if(res1==0 || res2==0){
12789                 tx.rollback();
12790                 return false;
12791             }
12792             else{
12793                 tx.commit();
12794                 return true;
12795             }
12796         }//try
12797         catch(Exception e){
12798             tx.rollback();
12799             return false;
12800         }
12801     }//method
12802 }//class
12803
12804 -----BankDAOFactory.java-----
12805 package com.nt.dao;
```

```

06
07 public class BankDAOFactory {
08
09     public static BankDAO getDAOInstance()
10     {
11         return new BankDAOImpl();
12     }
13
14 }
15 -----HibernateUtil.java-----
16 <-----same-->
17 -----ClientApp.java-----
18 package com.nt.test;
19
20 import com.nt.dao.BankDAO;
21 import com.nt.dao.BankDAOFactory;
22
23
24 public class ClientApp {
25
26     public static void main(String[] args) {
27         // Get DAO
28         BankDAO dao=BankDAOFactory.getDAOInstance();
29         // Call methods
30         System.out.println("Moeny Tranffered????"+dao.transferMoney(101, 105, 5000));
31     }
32 }
33
34 App 56)Example App on Pagination
35
36 -----index.html-----
37
38 <a href="pageurl?pageNo=1"> Employee Report</a>
39 -----web.xml-----
40
41 <web-app>
42     <servlet>
43         <servlet-name>page</servlet-name>
44         <servlet-class>com.nt.web.PaginationSrv</servlet-class>
45     </servlet>
46     <servlet-mapping>
47         <servlet-name>page</servlet-name>
48         <url-pattern>/pageurl</url-pattern>
49     </servlet-mapping>
50 </web-app>
51 -----hibernate.cfg.xml-----
52 <!DOCTYPE hibernate-configuration PUBLIC
53 "-//Hibernate/Hibernate Configuration DTD//EN"
54 "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
55
56 <hibernate-configuration>
57     <session-factory>
58         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
59         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
60         <property name="connection.username">scott</property>
61         <property name="connection.password">tiger</property>
62
63         <property name="hibernate.hbm2ddl.auto">update</property>
64         <property name="show_sql">true</property>
65         <property name="hibernate.dialect">org.hibernate.dialect.Oracle10gDialect</property>
66
67         <mapping class="com.nt.domain.EmpDetails"/>
68     </session-factory>
69 </hibernate-configuration>
70 -----EmpDetails.java-----

```

```
12871 package com.nt.domain;
12872
12873 import javax.persistence.Entity;
12874 import javax.persistence.Id;
12875 import javax.persistence.Table;
12876
12877 @Entity
12878 @Table(name="Employee")
12879 public class EmpDetails {
12880     @Id
12881     private int eid;
12882     private String firstname;
12883     private String lastname;
12884     private String email;
12885
12886     public EmpDetails() {
12887         System.out.println("EmpDetails:0-param constructor");
12888     }
12889
12890
12891
12892     public int getEid() {
12893         return eid;
12894     }
12895     public void setEid(int eid) {
12896         this.eid = eid;
12897     }
12898     public String getFirstname() {
12899         return firstname;
12900     }
12901     public void setFirstname(String firstname) {
12902         this.firstname = firstname;
12903     }
12904     public String getLastname() {
12905         return lastname;
12906     }
12907     public void setLastname(String lastname) {
12908         this.lastname = lastname;
12909     }
12910     public String getEmail() {
12911         return email;
12912     }
12913     public void setEmail(String email) {
12914         this.email = email;
12915     }
12916
12917     @Override
12918     public String toString() {
12919         return "EmpDetails [eid=" + eid + ", firstname=" + firstname
12920             + ", lastname=" + lastname + ", email=" + email + "];"
12921     }
12922 }
12923 }
12924 -----PaginationSrv.java-----
12925 package com.nt.web;
12926
12927 import java.io.IOException;
12928 import java.io.PrintWriter;
12929 import java.util.List;
12930
12931 import javax.servlet.ServletException;
12932 import javax.servlet.http.HttpServlet;
12933 import javax.servlet.http.HttpServletRequest;
12934 import javax.servlet.http.HttpServletResponse;
12935
```

Naresh i Technologies, Opp. Satyam Theatre, Ameerpet, Hyd, Ph: 040-23746666, www.nareshit.com ::420::

```
13001 import com.nt.domain.EmpDetails;
13002
13003 public interface EmpDAO {
13004     public int getEmployeeCount();
13005     public List<EmpDetails> getEmpDetails(int startPosition,int pageSize);
13006
13007
13008 }
13009 -----EmpDAOImpl.java-----
13010 package com.nt.dao;
13011
13012 import java.util.List;
13013
13014 import org.hibernate.Criteria;
13015 import org.hibernate.Session;
13016 import org.hibernate.criterion.Projection;
13017 import org.hibernate.criterion.Projections;
13018
13019 import com.nt.domain.EmpDetails;
13020 import com.nt.utility.HibernateUtil;
13021
13022 public class EmpDAOImpl implements EmpDAO {
13023
13024     @Override
13025     public int getEmployeeCount() {
13026         // Get Session
13027         Session ses=HibernateUtil.getSession();
13028         //Use QBC to get record count
13029         Criteria ct=s ses.createCriteria(EmpDetails.class);
13030         Projection p1=Projections.rowCount();
13031         ct.setProjection(p1);
13032         List l=ct.list();
13033         long cnt=(Long)l.get(0);
13034
13035
13036         return (int)cnt;
13037     }//list
13038
13039     @Override
13040     public List<EmpDetails> getEmpDetails(int startPosition,int pageSize) {
13041         // get Session
13042         Session ses=HibernateUtil.getSession();
13043         // get Records
13044         Criteria ct=s ses.createCriteria(EmpDetails.class);
13045         ct.setFirstResult(startPosition);
13046         ct.setMaxResults(pageSize);
13047         List<EmpDetails> list=ct.list();
13048
13049         return list;
13050
13051     }//getEmpDetails(-,-)
13052 }//class
13053
13054 -----EmpDAOFactory.java-----
13055 package com.nt.dao;
13056
13057 public class EmpDAOFactory {
13058
13059     public static EmpDAO getInstance(){
13060         return new EmpDAOImpl();
13061     }
13062
13063
13064 }
13065
```

```
066 =====
067 App57) App on Optimistic Locking
068 =====
069 -----table-----
070
071 SQL>
072
073 create table product5(
074   pid number(5),
075   pname varchar2(10),
076   price number(10),
077   ver number(4)
078 );
079
080 -----hibernate.cfg.xml-----
081 <!DOCTYPE hibernate-configuration PUBLIC
082   "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
083   "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
084
085 <hibernate-configuration>
086   <session-factory>
087     <property name="hibernate.connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
088     <property name="hibernate.connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
089     <property name="hibernate.connection.username">scott</property>
090     <property name="hibernate.connection.password">tiger</property>
091     <property name="hibernate.dialect">org.hibernate.dialect.Oracle10gDialect</property>
092     <property name="show_sql">>true</property>
093     <mapping class="com.nt.domain.Product"/>
094   </session-factory>
095 </hibernate-configuration>
096
097 -----Product.java-----
098 package com.nt.domain;
099 import javax.persistence.Column;
100 import javax.persistence.Entity;
101 import javax.persistence.Id;
102 import javax.persistence.Table;
103 import javax.persistence.Version;
104
105 @Entity
106 @Table(name="product5")
107 public class Product{
108
109     @Id
110     @Column(name="pid")
111     private int productId;
112
113     @Column(name="pname")
114     private String proName;
115
116     private double price;
117     @Version
118     @Column(name="ver")
119     private int v;
120
121     public void setProductId(int productId)
122     {
123         this.productId = productId;
124     }
125     public int getProductId()
126     {
127         return productId;
128     }
129 }
130
```

```
13131 public void setProName(String proName)
13132 {
13133     this.proName = proName;
13134 }
13135 public String getProName()
13136 {
13137     return proName;
13138 }
13139
13140 public void setPrice(double price)
13141 {
13142     this.price = price;
13143 }
13144 public double getPrice()
13145 {
13146     return price;
13147 }
13148
13149 public void setV(int v)
13150 {
13151     this.v = v;
13152 }
13153 public int getV()
13154 {
13155     return v;
13156 }
13157 }
13158
13159 -----TestClient.java-----
13160 package com.nt.test;
13161 import org.hibernate.*;
13162 import org.hibernate.cfg.*;
13163
13164 public class TestClient {
13165
13166     public static void main(String[] args)
13167     {
13168         Session session=Hibernate.getSession();
13169
13170         Product p=(Product)session.get(Product.class,101);
13171
13172         Transaction tx=session.beginTransaction();
13173
13174         System.out.println("Version before sleep"+p.getV());
13175         try{Thread.sleep(30000);}catch(Exception e){}
13176         System.out.println("Version after sleep"+p.getV());
13177
13178         p.setPrice(355);
13179         session.update(p);
13180         System.out.println("Object updated successfully.....(Client1)!!");
13181         tx.commit();
13182
13183         System.out.println("record modified for"+p.getV()+"times");
13184
13185
13186         session.close();
13187         factory.close();
13188     }
13189 }
13190
13191 -----TestClient2.java-----
13192 package com.nt.test;
13193 import org.hibernate.*;
13194 import org.hibernate.cfg.*;
13195
```

```
3196 public class TestClient2 {
3197
3198     public static void main(String[] args)
3199     {
3200
3201         Session session = HibernateUtil.getSession();
3202
3203         Product p=(Product)session.get(Product.class,101);
3204
3205         Transaction tx=session.beginTransaction();
3206         p.setPrice(976);
3207         session.update(p);
3208         System.out.println("Object updated successfully.....(Client2)!!");
3209         tx.commit();
3210
3211         System.out.println("record modified for"+p.getV()+"times");
3212         session.close();
3213         factory.close();
3214     }
3215 }
3216 }
3217 =====
3218 App 58) App on Pessimistic Locking
3219 =====
3220 -----hibernate.cfg.xml-----
3221 <!DOCTYPE hibernate-configuration PUBLIC
3222     "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
3223     "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
3224
3225 <hibernate-configuration>
3226     <session-factory>
3227         <property name="hibernate.connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
3228         <property name="hibernate.connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
3229         <property name="hibernate.connection.username">scott</property>
3230         <property name="hibernate.connection.password">tiger</property>
3231
3232         <property name="hibernate.dialect">org.hibernate.dialect.Oracle10gDialect</property>
3233         <property name="show_sql">>true</property>
3234         <mapping resource="com/nt/cfgs/product.hbm.xml" />
3235     </session-factory>
3236 </hibernate-configuration>
3237
3238 -----product.hbm.xml-----
3239 <!DOCTYPE hibernate-mapping PUBLIC
3240     "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
3241     "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
3242
3243 <hibernate-mapping>
3244     <class name="com.nt.domain.Product" table="product">
3245
3246         <id name="productId" column="pid" />
3247
3248         <property name="proName" column="pname" length="10" />
3249         <property name="price" />
3250
3251     </class>
3252 </hibernate-mapping>
3253 -----Product.java-----
3254 package com.nt.domain;
3255 public class Product
3256 {
3257     private int productId;
3258     private String proName;
3259     private double price;
3260 }
```



```
13261     public void setProductId(int productId)
13262     {
13263         this.productId = productId;
13264     }
13265
13266     public int getProductId()
13267     {
13268         return productId;
13269     }
13270
13271     public void setProName(String proName)
13272     {
13273         this.proName = proName;
13274     }
13275
13276     public String getProName()
13277     {
13278         return proName;
13279     }
13280
13281     public void setPrice(double price)
13282     {
13283         this.price = price;
13284     }
13285
13286     public double getPrice()
13287     {
13288         return price;
13289     }
13290
13291 }
13292 -----HibernateUtil.java-----
13293 ----->same <-----
13294
13295 -----TestClient1.java-----
13296 import org.hibernate.LockMode;
13297 import org.hibernate.Session;
13298 import org.hibernate.SessionFactory;
13299 import org.hibernate.Transaction;
13300 import org.hibernate.cfg.Configuration;
13301 import org.hibernate.service.ServiceRegistry;
13302 import org.hibernate.service.ServiceRegistryBuilder;
13303
13304 public class TestClient1
13305 {
13306
13307     public static void main(String[] args)
13308     {
13309
13310         Session session = HibernateUtil.getSession();
13311
13312         Product p = (Product) session.get(Product.class, 101, LockMode.UPGRADE_NOWAIT);
13313
13314         Transaction tx = session.beginTransaction();
13315         p.setPrice(166);
13316         try {
13317             Thread.sleep(15000);
13318         } catch (Exception e) {
13319         }
13320         session.saveOrUpdate(p);
13321
13322         System.out.println("Object saved successfully.....!!");
13323         tx.commit();
13324
13325         System.out.println("record modified @TestClient1");
```

```
13326         session.close();
13327         factory.close();
13328     }
13329
13330 }
13331
13332 -----TestClient2.java-----
13333 import org.hibernate.*;
13334 import org.hibernate.cfg.*;
13335 import org.hibernate.service.ServiceRegistry;
13336 import org.hibernate.service.ServiceRegistryBuilder;
13337
13338 public class TestClient2
13339 {
13340
13341     public static void main(String[] args)
13342     {
13343         Session ses=HibernateUtil.getSession();
13344
13345         Product p = (Product) session.get(Product.class, 101,LockMode.UPGRADE_NOWAIT);
13346         Transaction tx = session.beginTransaction();
13347         p.setPrice(195);
13348         session.saveOrUpdate(p);
13349         System.out.println("Object saved successfully.....!!");
13350         tx.commit();
13351
13352         session.close();
13353         factory.close();
13354     }
13355
13356 }
13357
13358
13359
13360
13361
```

Caching?

What is Caching?

- Used for optimizing database applications.
- Cache is designed to reduce traffic between your application and the database by conserving data already loaded from the database.
- Database access is necessary only when retrieving data that is not currently available in the cache.
- The application may need to empty (invalidate) the cache from time to time if the database is updated or modified in some way, because it has no way of knowing whether the cache is up to date

Hibernate has two different Caches (for Caching Objects)

- First-level cache – associated with the *Session* object
- Second-level cache – associated with the *SessionFactory* object

First-level Cache

Hibernate uses first-level cache on a per transaction basis (within a single transaction boundary)

- Used mainly to reduce the number of SQL queries it needs to generate within a given transaction.
- For example, if an object is modified several times within the same transaction, Hibernate will generate only one SQL UPDATE statement at the end of the transaction, containing all the modifications

Second-level Cache

Second-level cache keeps loaded objects at the Session Factory level across transaction boundaries

- The objects in the second-level cache are available to the whole application, not just to the user running the query
- This way, each time a query returns an object that is already loaded in the cache, one or more database transactions potentially are avoided.

Second-level Cache Implementations

Hibernate supports these open-source cache implementations out of the box

- EHCACHE (org.hibernate.cache.EhCacheProvider)
- OSCache (org.hibernate.cache.OSCacheProvider)
- SwarmCache (org.hibernate.cache.SwarmCacheProvider)
- JBoss TreeCache (org.hibernate.cache.TreeCacheProvider)

Commercial cache implementation

- Tangosol Coherence cache

EHCACHE

- Fast, lightweight, and easy-to-use in-process cache
- Supports read-only and read/write caching, and memory- and disk-based caching.
- However, it does not support clustering

OSCache

- Part of a larger package, which also provides caching functionalities for JSP pages or arbitrary objects.

- It is a powerful and flexible package, which, like EHCache, supports read-only and read/write caching, and memory- and disk-based caching.
- It also provides basic support for clustering via either JavaGroups or JMS

SwarmCache

- Is a simple cluster-based caching solution based on JavaGroups.
- Supports read-only or nonstrict read/write caching (the next section explains this term).
- This type of cache is appropriate for applications that typically have many more read operations than write operations.

TreeCache

- Is a powerful replicated (synchronous or asynchronous) and transactional cache.
- Use this solution if you really need a true transaction-capable caching architecture

Cache	Provider Class	Type	Cluster Safe	Query Cache Supported
EHCache	net.sf.hibernate.ehcache.hibernate.Provider	Memory, disk	No	Yes
OSCache	net.sf.hibernate.cache.OSCacheProvider	Memory, disk	No	Yes
SwarmCache	net.sf.hibernate.cache.SwarmCacheProvider	Clustered (IP multicast)	Yes (clustered invalidation)	No
TreeCache	net.sf.hibernate.cache.TreeCacheProvider	Clustered (IP multicast), transactional	Yes (replication)	No

Caching Strategies for Second-level Cache

- Read-only
- Read/write
- Nonstrict read/write
- Transactional

Caching Strategy: Read-only

- Useful for data that is read frequently but never updated.
- Simplest and best-performing cache strategy.

hibernate-mapping>

```

<class name="Country" table="COUNTRY" dynamic-update="true">
    <meta attribute="implement-equals">true</meta>
    <cache usage="read-only"/>
    <id name="id" type="long" unsaved-value="null">
        <column name="cn_id" not-null="true"/>
        <generator class="increment"/>
    </id>
    <property column="cn_code" name="code" type="string"/>
    <property column="cn_name" name="name" type="string"/>
    <set name="airports">
        <cache usage="read-only"/>
        <key column="cn_id"/>
        <one-to-many class="Airport"/>
    </set>

```

```
</class>
```

```
</hibernate-mapping>
```

Caching Strategy: Read/Write

- Appropriate if your data needs to be updated.
- They carry more overhead than read-only caches.
- In non-JTA environments, each transaction should be completed when *Session.close()* or *Session.disconnect()* is called.

```
<hibernate-mapping>
```

```
<class name="Employee" table="EMPLOYEE" dynamic-update="true">
  <meta attribute="implement-equals">true</meta>
  <cache usage="read-write"/>
```

```
  <id name="id" type="long" unsaved-value="null" >
    <column name="emp_id" not-null="true"/>
    <generator class="increment"/>
  </id>
```

```
  <property column="emp_surname" name="surname" type="string"/>
  <property column="emp_firstname" name="firstname" type="string"/>
```

```
  <many-to-one name="country" column="cn_id"
    class="com.wakaleo.articles.caching.businessobjects.Country"
    not-null="true" />
```

```
  <set name="languages" table="EMPLOYEE_SPEAKS_LANGUAGE" lazy="false">
    <cache usage="read-write"/>
    <key column="emp_id"/>
    <many-to-many column="lan_id" class="Language"/>
  </set>
```

```
</class>
```

```
</hibernate-mapping>
```

Caching Strategy: Nonstrict Cache

- Does not guarantee that two transactions won't simultaneously modify the same data.
- Therefore, it may be most appropriate for data that is read often but only occasionally modified.

Caching Strategy: Transactional

- A fully transactional cache that may be used
- only in a JTA environment

Cache Configuration in Hibernate Configuration File - using EHCACHE

```
<hibernate-configuration>
```

```
  <session-factory>
```

```
    ...
```

```
    <property name="hibernate.cache.use_query_cache">
      true
    </property>
```

```
<property name="hibernate.cache.use_second_level_cache">
    true
</property>
```

```
<property name="hibernate.cache.provider_class">
    org.hibernate.cache.EhCacheProvider
</property>
```

```
...
```

```
</session-factory>
```

```
</hibernate-configuration>
```

Interceptors

Hibernate allows us to execute pre/post logic after some core event has occurred. We call this Interceptors. If you would like to perform some logic before or after load() is called, the Interceptors will be your best friend. Interceptors comes up with 18 callback methods that gives a very good control to perform a lot of pre/post logic before or after core logic.

How to Work with Interceptors

- Create a class implementing org.hibernate.Interceptor interface and override 18 methods of those interface
- Create a class extending org.hibernate.EmptyInterceptor. It is an adapter class with default implementation of those methods. Override only those methods in which you are interested in.

Here is the list of those 18 methods. Please refer API for complete prototype

1. afterTransactionBegin
2. afterTransactionCompletion
3. beforeTransactionCompletion
4. findDirty
5. getEntity
6. getEntityName
7. instantiate
8. isTransient
9. onCollectionRecreate
10. onCollectionRemove
11. onCollectionUpdate
12. onDelete
13. onFlushDirty
14. onLoad
15. onPrepareStatement
16. onSave
17. postFlush
18. preFlush

Configure Your Interceptor class Object while creating session object.

```
Session ses=sessionfactory.openSession(new MyInterceptor());
```

The callback methods which you have configured will be called in a flow

onSave

```
public boolean onSave(Object obj,Serializable id, Object[] state,String[] propertyNames,Type[]
types)
```

This callback method gets called just before save operation gets executed.

First Argument obj,is a reference to the object which you are going to save into session.

If Code is `session.save(emp);`

Obj holds emp object,which we can get by typecasting and you can perform some sort of business logic before the real save operation takes place.

If this method is returning false,then the updated property value of obj, will be stored in database. I mean if you have written some logic inside onSave() like increment the salary of employee if his salary is less than 10,000,and if you are returnig false,then the latest and modified value will be updated in the database.

If this method is returning true, then any updates that is made on obj will be discarded.

Second Argument holds the Unique identity value.

Third Argument holds property values

Fourth argument holds property Name

Fifth Argument holds property type

EmpBean.java

```
public class EmpBean {
    private int empld;
    private String empName;
    private float empSal;
    // getXXX(),setXXX(-)
}
```

MyInterceptor.java

```
public class MyInterceptor extends EmptyInterceptor {

    // This method is called before saving an object
    public boolean onSave(Object obj, Serializable id, Object[] state,String[] propertyNames, Type[]
types) {

        System.out.println("MyInterceptor.onSave(-,-,-,-)");
        EmpBean emp=(EmpBean)obj;
        System.out.println("Emp Details before saving"+emp);
        System.out.println("Identity Value="+id);

        System.out.println("Property Names:");
        for(String names:propertyNames)
            System.out.println(names);

        System.out.println("Property Type");
        for(Type t:types)
            System.out.println(t);
```



```

System.out.println("Property Value:");
for(Object ob:state)
    System.out.println(ob);

//If Employee salary not greater than 10,000 ,20% hike
//else 10% hike

if(emp.getEmpSal()<10000) emp.setEmpSal(emp.getEmpSal()*1.2f);
else emp.setEmpSal(emp.getEmpSal()*1.1f);

    return true;//modification will be discarded
}
} // class

```

TestClient.java

```

public class TestClient {
    public static void main(String args[])throws Exception {
        //Create Hibernate Session Factory
        SessionFactory sf = new Configuration().configure().buildSessionFactory();
        Session ses=sf.openSession(new MyInterceptor());

        //create anew transaction
        Transaction tx=ses.beginTransaction();

        EmpBean eb=new EmpBean(); //eb transient state
        eb.setEmpId(117);
        eb.setEmpName("Raja");
        eb.setEmpSal(10000f);//will enable 10% hike if greater than
        //10000 else 20% hike

        Integer id=(Integer)ses.save(eb);//this line
        System.out.println(eb);
        tx.commit();//your object data will be inserted into the databqse
        System.out.println("Emp with Id"+id+"saved successfully");
        ses.close();
    } // main
} // class

```

will Output

```

MyInterceptor.onSave(-,-,-,-)
Emp Details before savingEmpId=118 EmpName=Raja EmpSal=10000.0
Identity Value=118
Property Names:
empName
empSal
Property Type
org.hibernate.type.StringType@15005d
org.hibernate.type.FloatType@cb03c4

```

Property Value:

Raja

10000.0

EmpId=118 EmpName=Raja EmpSal=10000.0

Hibernate: insert into EMPLOYEE_TAB (emp_name, emp_sal, emp_no) values (?, ?, ?)

Emp with Id118 saved successfully

5 mismatch problems between Java and RDBMS

Mismatch	Description
Granularity	Sometimes you will have an object model which has more classes than the number of corresponding tables in the database.
Inheritance	RDBMSs do not define anything similar to Inheritance which is a natural paradigm in object-oriented programming languages.
Identity	A RDBMS defines exactly one notion of 'sameness': the primary key. Java, however, defines both object identity (<code>a==b</code>) and object equality (<code>a.equals(b)</code>).
Associations	Object-oriented languages represent associations using object references whereas an RDBMS represents an association as a foreign key column.
Navigation	The ways you access objects in Java and in a RDBMS are fundamentally different.

The **Object-Relational Mapping (ORM)** is the solution to handle all the above impedance mismatches.

Hibernate 4.3

I. Provide support for initial multi-tenancy

Multi-tenancy is the ability to virtually partition clients/customers (also called tenants) in a big enterprise application instead of having all data in a shared space. This principle allows for ~~improvements in management, monitoring and even security and is very useful for big service providers.~~ Companies that offer cloud infrastructures can benefit from multi-tenancy as well. There are several ways to implement this principle which include:

1. A different database and/or schema for each client/tenant
2. The same database/schema for all clients but with an extra column (e.g. tenant_id) in all tables that can be used to filter data

Hibernate supports the first method in the 4.0 release. Support for the second method (i.e. discriminator multi-tenancy) is planned in the next version. (May be Hibernate 5.0 or 5.1)

II. Working with Service Registry

Initial move to ServiceRegistry. Not all "services" have been migrated to this model yet. The main ones (JDBC and transaction stuff) as well as lower level one (classloading and such) have been migrated. The rest will be moved during Alpha2 development.

III. Improvized ByteCode

Continued work on new bytecode enhancement support within Hibernate, adding support for inline dirty checking

IV. Package Re-Structure

Some packages are restructured in order to identify the classes which are intended as

1. public API, which are fully expected to be used in application code.
2. internal implementation details, which are only intended for Hibernate use.
3. SPI contracts, which define
 - extension contracts
 - contracts with Hibernate internals exposed to these extensions

Example : org.hibernate.dialect.resolver.DialectResolver->
org.hibernate.service.jdbc.dialect.spi.DialectResolver

V. Deprecated Classes and Interfaces are Removed

1. org.hibernate.classic.Session
2. org.hibernate.classic.Validatable
3. org.hibernate.classic.ValidationException

VI. Deprecated Methods have been removed

1. Deprecated methods that have been removed:

1. References to `org.hibernate.type.AbstractSingleColumnStandardBasicType` and `org.hibernate.type.SingleColumnType` methods should be changed as indicated:
 1. `nullSafeGet(ResultSet rs, String name)` should be changed to `nullSafeGet(ResultSet rs, String name, SessionImplementor session)`
 2. `get(ResultSet rs, String name)` should be changed to `get(ResultSet rs, String name, SessionImplementor session)`
 3. `nullSafeSet(PreparedStatement st, T value, int index)` should be changed to `nullSafeSet(PreparedStatement st, Object value, int index, SessionImplementor session)`
 4. `set(PreparedStatement st, T value, int index)` should be changed to `set(PreparedStatement st, T value, int index, SessionImplementor session)`
2. References to `org.hibernate.usertype.UserType` methods should be changed as indicated:
 1. `nullSafeGet(ResultSet rs, String[] names, Object owner)` should be changed to `nullSafeGet(ResultSet rs, String[] names, SessionImplementor session, Object owner)`
 2. `nullSafeSet(PreparedStatement st, Object value, int index)` should be changed to `nullSafeSet(PreparedStatement st, Object value, int index, SessionImplementor session)`
3. `Session.reconnect()` - manual disconnection and reconnection is now only supported for user-supplied-connection scenarios (JDBC Connection passed in while opening the Session)
4. `Session.connection()` - use `Session.doWork()`, `Session.doReturningWork()` or `Session.sessionWithOptions()...openSession()` as replacement depending on need
5. Most of the overloaded `SessionFactory.openSession` methods. Use `SessionFactory.withOptions()...openSession()` instead

VII. Change in Configuring Listener

`hibernate.cfg.xml` no longer supported as means of specifying listeners. New approach involves using an `org.hibernate.integrator.spi.Integrator` which works based on "service discovery".

VII. Logging Framework changed to JBoss Logging

Developers wanted to have i18N support in the logging. Presently, JBoss Logging is the only logging library with full i18n support including parameterization. JBoss Logging also supports i18n in exception messages through a common approach.

VIII. Added JPA 2.1 specification support which includes

- 1) Support for stored procedures. Now you can call stored procedure in two ways
 - A) Hibernate way
 - B) JPA way in your hibernate App

@PersistenceContext

```
EntityManager entityManager;
```

```
StoredProcedureQuery query = entityManager.createStoredProcedureQuery(  
    "getEmployeeDetails", Employee.class );  
query.registerStoredProcedureParameter( "salary", String.class, ParameterMode.IN );  
query.setParameter( "salary", "9000" );  
List salaryGT10000 = query.getResultList();
```

- 2) *CriteriaUpdate* and *CriteriaDelete* allow definition and execution of UPDATE and DELETE queries in Criterion Style
- 3) Entity listeners can now take advantage of dependency injection through CDI.(Context and Dependency Injection)
- 4) **AttributeConverters**, to apply conversions on basic values between their database representation and their representation in your domain model. similar to Hibernate's Type

Some Other Changes

1. package org.hibernate.connection was removed
2. Hibernate.INTEGER and other similar instances were removed and now IntegerType.INSTANCE should be used; another example is Hibernate.BINARY -> BitaryType.INSTANCE and etc.)
3. org.hibernate.util.EqualsHelper moved to internal package org.hibernate.internal.util.compare.EqualsHelper
4. SessionImplementor have lost getEntityMode() and to get it I use SessionImplementor.getEntityPersister().getEntityMode()
5. org.hibernate.persister.entity.EntityPersister got new methods to implement
6. EntityMode.MAP was removed
7. Hibernate.entity(Class) was removed (didn't find really good way to migrate it to new version)

org.hibernate.jdbc.BatchFactory, Batch, and their implementations have been replaced by org.hibernate.engine.jdbc.batch.spi.BatchBuilder and Batch, with default implementations in org.hibernate.engine.jdbc.batch.internal. You can override the default BatchBuilder by defining the "hibernate.jdbc.batch.builder" property as the name of a BatchBuilder implementation, or by providing a BatchBuilder in a custom ServiceRegistry.

Hibernate FAQs

1. What is ORM ?

ORM stands for object/relational mapping. ORM is the automated persistence of objects in a Java application to the tables in a relational database.

2. What does ORM consists of ?

An ORM solution consists of the following four pieces:

- API for performing basic CRUD operations
- API to express queries referring to classes
- Facilities to specify metadata
- Optimization facilities : dirty checking, lazy associations fetching

3. What are the ORM levels ?

The ORM levels are:

- Pure relational (stored procedure.)
- Light objects mapping (JDBC)
- Medium object mapping
- Full object Mapping (composition, inheritance, polymorphism, persistence by reachability)

4. What is Hibernate?

Hibernate is a pure Java object-relational mapping (ORM) and persistence framework that allows you to map plain old Java objects to relational database tables using (XML) configuration files. Its purpose is to relieve the developer from a significant amount of relational data persistence-related programming tasks.

5. Why do you need ORM tools like hibernate?

The main advantage of ORM like hibernate is that it shields developers from messy SQL. Apart from this, ORM provides following benefits:

- **Improved productivity**
 - High-level object-oriented API
 - Less Java code to write
 - No SQL to write
- **Improved performance**
 - Sophisticated caching
 - Lazy loading
 - Eager loading
- **Improved maintainability**
 - A lot less code to write
- **Improved portability**
 - ORM framework generates database-specific SQL for you

6. What Does Hibernate Simplify?

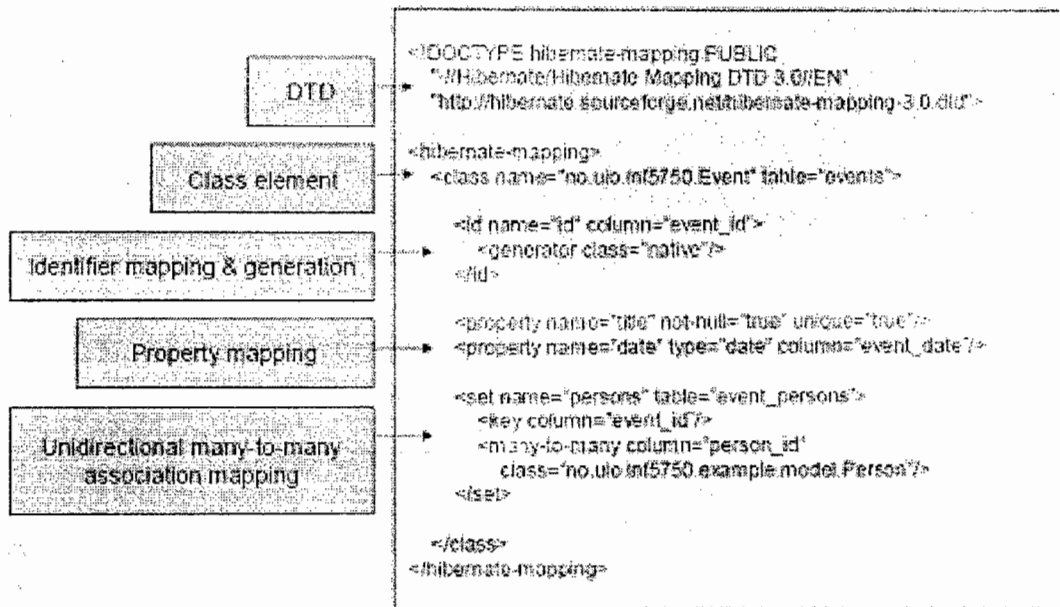
Hibernate simplifies:

- Saving and retrieving your domain objects
- Making database column and table name changes
- Centralizing pre save and post retrieve logic
- Complex joins for retrieving related items
- Schema creation from object model

7. What is the need for Hibernate xml mapping file?

Hibernate mapping file tells Hibernate which tables and columns to use to load and store objects.

Typical mapping file look as follows:



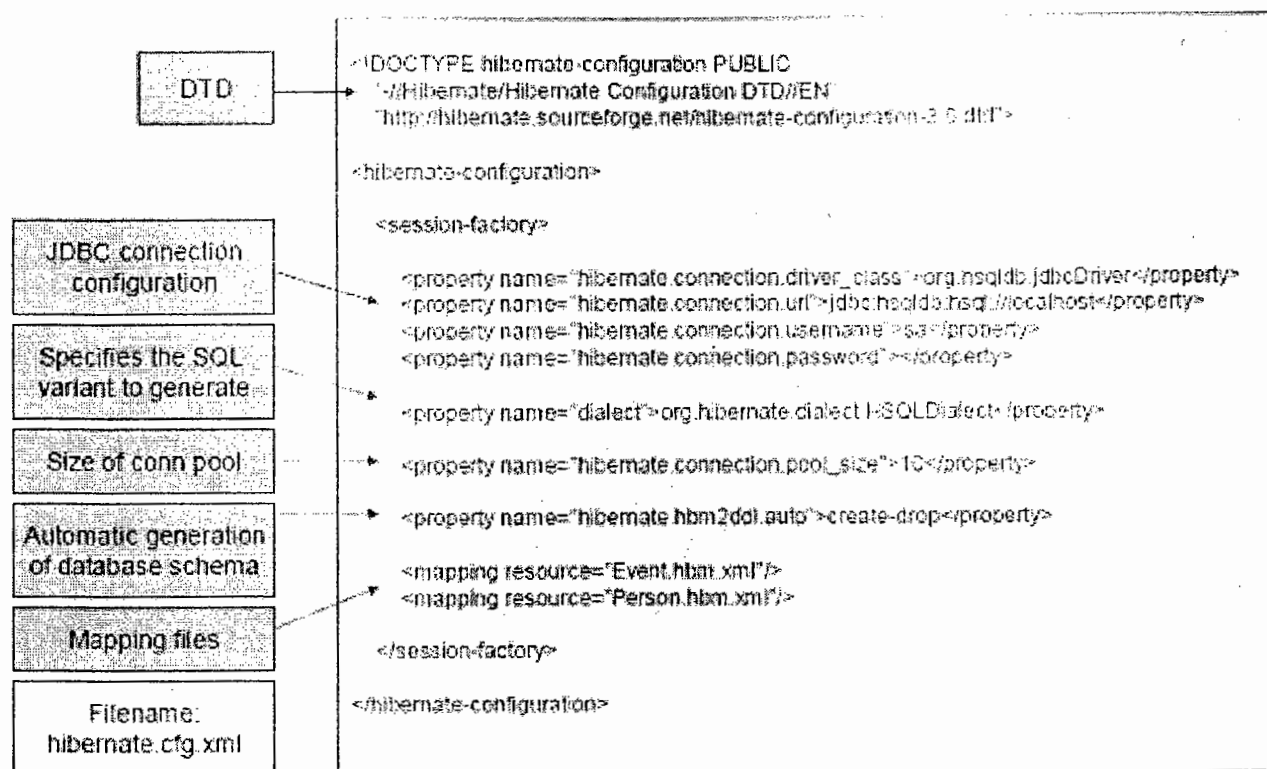
8. What are the most common methods of Hibernate configuration?

The most common methods of Hibernate configuration are:

- Programmatic configuration
- XML configuration (hibernate.cfg.xml)

9. What are the important tags of hibernate.cfg.xml?

Following are the important tags of hibernate.cfg.xml:



10. What are the Core interfaces are of Hibernate framework?

The five core interfaces are used in just about every Hibernate application. Using these interfaces, you can store and retrieve persistent objects and control transactions.

- Session interface
- SessionFactory interface
- Configuration interface
- Transaction interface
- Query and Criteria interfaces

11. What role does the Session interface play in Hibernate?

The Session interface is the primary interface used by Hibernate applications. It is a single-threaded, short-lived object representing a conversation between the application and the persistent store. It allows you to create query objects to retrieve persistent objects.

```
Session session = sessionFactory.openSession();
```

Session interface role:

- Wraps a JDBC connection
- Factory for Transaction
- Holds a mandatory (first-level) cache of persistent objects, used when navigating the object graph or looking up objects by identifier

12. What role does the SessionFactory interface play in Hibernate?

The application obtains Session instances from a SessionFactory. There is typically a single SessionFactory for the whole application created during application initialization. The

SessionFactory caches generate SQL statements and other mapping metadata that Hibernate uses at runtime. It also holds cached data that has been read in one unit of work and may be reused in a future unit of work

SessionFactory sessionFactory = configuration.buildSessionFactory();

13. What is the general flow of Hibernate communication with RDBMS?

The general flow of Hibernate communication with RDBMS is :

- Load the Hibernate configuration file and create configuration object. It will automatically load all hbm mapping files
- Create session factory from configuration object
- Get one session from this session factory
- Create HQL Query
- Execute query to get list containing Java objects

14. What is Hibernate Query Language (HQL)?

Hibernate offers a query language that embodies a very powerful and flexible mechanism to query, store, update, and retrieve objects from a database. This language, the Hibernate query Language (HQL), is an object-oriented extension to SQL.

15. How do you map Java Objects with Database tables?

- First we need to write Java domain objects (beans with setter and getter).
- Write hbm.xml, where we map java class to table and database columns to Java class variables.

Example :

```
<hibernate-mapping>
<class name="com.test.User" table="user">
  <property column="USER_NAME" length="255"
    name="userName" not-null="true" type="java.lang.String"/>
  <property column="USER_PASSWORD" length="255"
    name="userPassword" not-null="true" type="java.lang.String"/>
</class>
</hibernate-mapping>
```

16. What is the difference between load() and get()?

load() vs. get() :-

load()	get()
Only use the load() method if you are sure that the object exists.	If you are not sure that the object exists, then use one of the get() methods.
load() method will throw an exception if the unique id is not found in the database.	get() method will return null if the unique id is not found in the database.

load() just returns a proxy by default and database won't be hit until the proxy is first invoked. get() will hit the database immediately.

17. What is the difference between merge and update ?

Use update() if you are sure that the session does not contain an already persistent instance with the same identifier, and merge() if you want to merge your modifications at any time without consideration of the state of the session.

18. How do you define sequence generated primary key in hibernate?

Using <generator> tag.

Example:-

```
<id column="USER_ID" name="id" type="java.lang.Long">
  <generator class="sequence">
    <param name="table">SEQUENCE_NAME</param>
  </generator>
</id>
```

19. Define cascade and inverse option in one-many mapping?

cascade - enable operations to cascade to child entities.

cascade="all|none|save-update|delete|all-delete-orphan"

inverse - mark this collection as the "inverse" end of a bidirectional association.

inverse="true|false"

Essentially "inverse" indicates which end of a relationship should be ignored, so when persisting a parent who has a collection of children, should you ask the parent for its list of children, or ask the children who the parents are?

20. What do you mean by Named SQL query?

Named SQL queries are defined in the mapping xml document and called wherever required.

Example:

```
<sql-query name = "empdetails">
  <return alias="emp" class="com.test.Employee"/>
  SELECT emp.EMP_ID AS {emp.empid},
         emp.EMP_ADDRESS AS {emp.address},
         emp.EMP_NAME AS {emp.name}
  FROM Employee EMP WHERE emp.NAME LIKE :name
</sql-query>
```

Invoke Named Query :

```
List people = session.getNamedQuery("empdetails")
                        .setString("TomBrady", name)
                        .setMaxResults(50)
                        .list();
```

21. How do you invoke Stored Procedures?

```
<sql-query name="selectAllEmployees_SP" callable="true">
  <return alias="emp" class="employee">
    <return-property name="empid" column="EMP_ID"/>

    <return-property name="name" column="EMP_NAME"/>
    <return-property name="address" column="EMP_ADDRESS"/>
    { ? = call selectAllEmployees() }
  </return>
</sql-query>
```

22. Explain Criteria API

Criteria is a simplified API for retrieving entities by composing Criterion objects. This is a very convenient approach for functionality like "search" screens where there is a variable number of conditions to be placed upon the result set.

Example :

```
List employees = session.createCriteria(Employee.class)
                        .add(Restrictions.like("name", "a%"))
                        .add(Restrictions.like("address", "Boston"))
                        .addOrder(Order.asc("name"))
                        .list();
```

23. Define HibernateTemplate?

org.springframework.orm.hibernate.HibernateTemplate is a helper class which provides different methods for querying/retrieving data from the database. It also converts checked HibernateExceptions into unchecked DataAccessExceptions.

24. What are the benefits does HibernateTemplate provide?

The benefits of HibernateTemplate are :

- HibernateTemplate, a Spring Template class simplifies interactions with **Hibernate Session**.
- Common functions are simplified to single method calls.
- Sessions are automatically closed.
- Exceptions are automatically caught and converted to runtime exceptions.

25. How do you switch between relational databases without code changes?

Using Hibernate SQL Dialects ; we can switch databases. Hibernate will generate appropriate hql queries based on the dialect defined.

26. If you want to see the Hibernate generated SQL statements on console, what should we do?

In Hibernate configuration file set as follows:

```
<property name="show_sql">true</property>
```

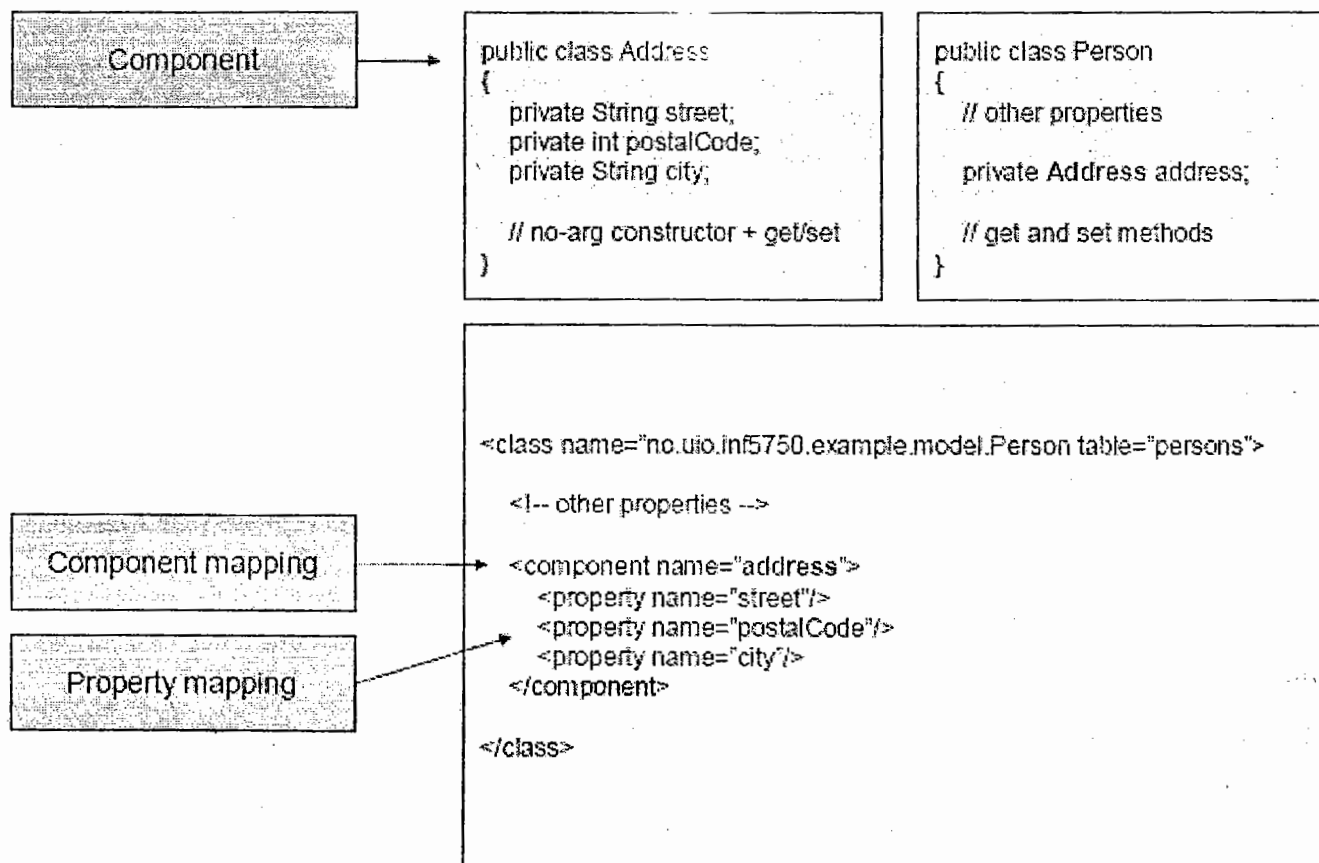
27. What are derived properties?

The properties that are not mapped to a column, but calculated at runtime by evaluation of an expression are called derived properties. The expression can be defined using the formula attribute of the element.

28. What is component mapping in Hibernate?

- A component is an object saved as a value, not as a reference
- A component can be saved directly without needing to declare interfaces or identifier properties
- Required to define an empty constructor
- Shared references not supported

Example:



9. What is the difference between sorted and ordered collection in hibernate?

orted collection vs. order collection :-

sorted collection

order collection

A sorted collection is sorting a collection by utilizing the sorting features provided by the Java collections framework. The sorting occurs in the memory of JVM which running Hibernate, after the data being read from database using java comparator.

If your collection is not large, it will be more efficient way to sort it.

Order collection is sorting a collection by specifying the order-by clause for sorting this collection when retrieval.

If your collection is very large, it will be more efficient way to sort it .

31. What is the advantage of Hibernate over jdbc?

Hibernate Vs. JDBC :-

JDBC	Hibernate
With JDBC, developer has to write code to map an object model's data representation to a relational data model and its corresponding database schema.	Hibernate is flexible and powerful ORM solution to map Java classes to database tables. Hibernate itself takes care of this mapping using XML files so developer does not need to write code for this.
With JDBC, the automatic mapping of Java objects with database tables and vice versa conversion is to be taken care of by the developer manually with lines of code.	Hibernate provides transparent persistence and developer does not need to write code explicitly to map database tables tuples to application objects during interaction with RDBMS.
JDBC supports only native Structured Query Language (SQL). Developer has to find out the efficient way to access database, i.e. to select effective query from a number of queries to perform same task.	Hibernate provides a powerful query language Hibernate Query Language (independent from type of database) that is expressed in a familiar SQL like syntax and includes full support for polymorphic queries. Hibernate also supports native SQL statements. It also selects an effective way to perform a database manipulation task for an application.
Application using JDBC to handle persistent data (database tables) having database specific code in large amount. The code written to map table data to application objects and vice versa is actually to map table fields to object properties. As table changed or database changed then it's essential to change object structure as well as to change code written to map table-to-object/object-to-table.	Hibernate provides this mapping itself. The actual mapping between tables and application objects is done in XML files. If there is change in Database or in any table then the only need to change XML file properties.

With JDBC, it is developer's responsibility to handle JDBC result set and convert it to Java objects through code to use this persistent data in application. So with JDBC, mapping between Java objects and database tables is done manually.

Hibernate reduces lines of code by maintaining object-table mapping itself and returns result to application in form of Java objects. It relieves programmer from manual handling of persistent data, hence reducing the development time and maintenance cost.

With JDBC, caching is maintained by hand-coding.

Hibernate, with Transparent Persistence, cache is set to application work space. Relational tuples are moved to this cache as a result of query. It improves performance if client application reads same data many times for same write. Automatic Transparent Persistence allows the developer to concentrate more on business logic rather than this application code.

In JDBC there is no check that always every user has updated data. This check has to be added by the developer.

Hibernate enables developer to define version type field to application, due to this defined field Hibernate updates version field of database table every time relational tuple is updated in form of Java class object to that table. So if two users retrieve same tuple and then modify it and one user save this modified tuple to database, version is automatically updated for this tuple by Hibernate. When other user tries to save updated tuple to database then it does not allow saving it because this user does not have updated data.

1. What are the Collection types in Hibernate ?

- Bag
- Set
- List
- Array
- Map

2. What are the ways to express joins in HQL?

SQL provides four ways of expressing (inner and outer) joins:-

- An *implicit* association join
- An ordinary join in the FROM clause
- A fetch join in the FROM clause.
- A *theta-style* join in the WHERE clause.

34. Define cascade and inverse option in one-many mapping?

cascade - enable operations to cascade to child entities.

cascade="all|none|save-update|delete|all-delete-orphan"

inverse - mark this collection as the "inverse" end of a bidirectional association.

inverse="true|false"

Essentially "inverse" indicates which end of a relationship should be ignored, so when persisting a parent who has a collection of children, should you ask the parent for its list of children, or ask the children who the parents are?

35. What is Hibernate proxy?

The proxy attribute enables lazy initialization of persistent instances of the class. Hibernate will initially return CGLIB proxies which implement the named interface. The actual persistent object will be loaded when a method of the proxy is invoked.

36. How can Hibernate be configured to access an instance variable directly and not through a setter method ?

By mapping the property with access="field" in Hibernate metadata. This forces hibernate to bypass the setter method and access the instance variable directly while initializing a newly loaded object.

37. How can a whole class be mapped as immutable?

Mark the class as mutable="false" (Default is true),. This specifies that instances of the class are (not) mutable. Immutable classes, may not be updated or deleted by the application.

38. What is the use of dynamic-insert and dynamic-update attributes in a class mapping?

Criteria is a simplified API for retrieving entities by composing Criterion objects. This is a very convenient approach for functionality like "search" screens where there is a variable number of conditions to be placed upon the result set.

- dynamic-update (defaults to false): Specifies that UPDATE SQL should be generated at runtime and contain only those columns whose values have changed
- dynamic-insert (defaults to false): Specifies that INSERT SQL should be generated at runtime and contain only the columns whose values are not null.

39. What do you mean by fetching strategy ?

A *fetching strategy* is the strategy Hibernate will use for retrieving associated objects if the application needs to navigate the association. Fetch strategies may be declared in the O/R mapping metadata, or over-ridden by a particular HQL or Criteria query.

40. What is automatic dirty checking?

Automatic dirty checking is a feature that saves us the effort of explicitly asking Hibernate to update the database when we modify the state of an object inside a transaction.

41. What is transactional write-behind?

Hibernate uses a sophisticated algorithm to determine an efficient ordering that avoids database foreign key constraint violations but is still sufficiently predictable to the user. This feature is called transactional write-behind.

42. What are Callback interfaces?

Callback interfaces allow the application to receive a notification when something interesting happens to an object—for example, when an object is loaded, saved, or deleted. Hibernate applications don't need to implement these callbacks, but they're useful for implementing certain kinds of generic functionality.

43. What are the types of Hibernate instance states ?

Three types of instance states:

- Transient -The instance is not associated with any persistence context
- Persistent -The instance is associated with a persistence context
- Detached -The instance was associated with a persistence context which has been closed – currently not associated

44. What are the differences between EJB 3.0 & Hibernate

Hibernate Vs EJB 3.0 :-

Hibernate	EJB 3.0
Session —Cache or collection of loaded objects relating to a single unit of work	Persistence Context —Set of entities that can be managed by a given EntityManager is defined by a persistence unit
XDoclet Annotations used to support Attribute Oriented Programming	Java 5.0 Annotations used to support Attribute Oriented Programming
Defines HQL for expressing queries to the database	Defines EJB QL for expressing queries
Supports Entity Relationships through mapping files and annotations in JavaDoc	Support Entity Relationships through Java 5.0 annotations
Provides a Persistence Manager API exposed via the Session, Query, Criteria, and Transaction API	Provides and Entity Manager Interface for managing CRUD operations for an Entity
Provides callback support through lifecycle, interceptor, and validatable interfaces	Provides callback support through Entity Listener and Callback methods
Entity Relationships are unidirectional. Bidirectional relationships are implemented by two unidirectional relationships	Entity Relationships are bidirectional or unidirectional

45. What are the types of inheritance models in Hibernate?

There are three types of inheritance models in Hibernate:

- Table per class hierarchy
- Table per subclass
- Table per concrete class